EPSON

EPSON RC+ 5.0 Option

Vision Guide 5.0

Rev.5 EM12XS2361F

EPSON RC+ 5.0 Option Vision Guide 5.0 Rev.5

EPSON RC+ 5.0 Option

Vision Guide 5.0

Rev.5

FOREWORD

Thank you for purchasing our robot products. This manual contains the information necessary for the correct use of the EPSON RC+ software.
Please carefully read this manual and other related manuals when using this software.
Keep this manual in a handy location for easy access at all times.

WARRANTY

The robot and its optional parts are shipped to our customers only after being subjected to the strictest quality controls, tests and inspections to certify its compliance with our high performance standards.
Product malfunctions resulting from normal handling or operation will be repaired free of charge during the normal warranty period. (Please ask your Regional Sales Office for warranty period information.)
However, customers will be charged for repairs in the following cases (even if they occur during the warranty period):

1. Damage or malfunction caused by improper use which is not described in the manual, or careless use.
2. Malfunctions caused by customers' unauthorized disassembly.
3. Damage due to improper adjustments or unauthorized repair attempts.
4. Damage caused by natural disasters such as earthquake, flood, etc.

Warnings, Cautions, Usage:

1. If the robot or associated equipment is used outside of the usage conditions and product specifications described in the manuals, this warranty is void.
2. If you do not follow the WARNINGS and CAUTIONS in this manual, we cannot be responsible for any malfunction or accident, even if the result is injury or death.
3. We cannot foresee all possible dangers and consequences. Therefore, this manual cannot warn the user of all possible hazards.

SOFTWARE LICENSE

For Compact Vision user, please read this software license agreement carefully before using this option.

Appendix A: END USER LICENSE AGREEMENT for Compact Vision
Appendix B: OPEN SOURCE SOFTWARE LICENSE for Compact Vision

TRADEMARKS

Microsoft, Windows, Windows logo, Visual Basic, and Visual C++ are either registered trademarks or trademarks of Microsoft Corporation in the United States and/or other countries.
Other brand and product names are trademarks or registered trademarks of the respective holders.

TRADEMARK NOTIFICATION IN THIS MANUAL

Microsoft® Windows® XP Operating system
Microsoft® Windows® Vista Operating system
Microsoft® Windows® 7 Operating system
Throughout this manual, Windows XP, Windows Vista, and Windows 7 refer to above respective operating systems. In some cases, Windows refers generically to Windows XP, Windows Vista, and Windows 7.

NOTICE

No part of this manual may be copied or reproduced without authorization.
The contents of this manual are subject to change without notice.
Please notify us if you should find any errors in this manual or if you have any comments regarding its contents.

INQUIRIES

Contact the following service center for robot repairs, inspections or adjustments.
If service center information is not indicated below, please contact the supplier office for your region.
Please prepare the following items before you contact us.

- Your controller model and its serial number
- Your manipulator model and its serial number
- Software and its version in your robot system
- A description of the problem

SERVICE CENTER

MANUFACTURER

SEIKO EPSON CORPORATION
Toyoshina Plant
Factory Automation Systems Dept.
6925 Toyoshina Tazawa,
Azumino-shi, Nagano, 399-8285
JAPAN
TEL : +81-(0)263-72-1530
FAX : +81-(0)263-72-1495

SUPPLIERS

North & South America **EPSON AMERICA, INC.**
Factory Automation/Robotics
18300 Central Avenue
Carson, CA 90746
USA
TEL : +1-562-290-5900
FAX : +1-562-290-5999
E-MAIL : info@robots.epson.com

Europe **EPSON DEUTSCHLAND GmbH**
Factory Automation Division
Otto-Hahn-Str.4
D-40670 Meerbusch
Germany
TEL : +49-(0)-2159-538-1391
FAX : +49-(0)-2159-538-3170
E-MAIL : robot.infos@epson.de

China **EPSON China Co., Ltd**
Factory Automation Division
7F, Jinbao Building No. 89 Jinbao Street
Dongcheng District, Beijing,
China, 100005
TEL : +86-(0)-10-8522-1199
FAX : +86-(0)-10-8522-1120

Taiwan **EPSON Taiwan Technology & Trading Ltd.**
Factory Automation Division
14F, No.7, Song Ren Road, Taipei 110
Taiwan, ROC
TEL : +886-(0)-2-8786-6688
FAX : +886-(0)-2-8786-6677

Southeast Asia India	**Epson Singapore Pte Ltd.** Factory Automation System 1 HarbourFrontPlace, #03-02 HarbourFront Tower one, Singapore 098633 TEL : +65-(0)-6586-5696 FAX : +65-(0)-6271-3182
Korea	**EPSON Korea Co, Ltd.** Marketing Team (Robot Business) 11F Milim Tower, 825-22 Yeoksam-dong, Gangnam-gu, Seoul, 135-934 Korea TEL : +82-(0)-2-3420-6692 FAX : +82-(0)-2-558-4271
Japan	**EPSON SALES JAPAN CORPORATION** Factory Automation Systems Department Nishi-Shinjuku Mitsui Bldg.6-24-1 Nishishinjuku. Shinjuku-ku. Tokyo. 160-8324 JAPAN TEL : +81-(0)3-5321-4161

SAFETY PRECAUTIONS

Installation of robots and robotic equipment should only be performed by qualified personnel in accordance with national and local codes. Please carefully read this manual and other related manuals when using this software.
Keep this manual in a handy location for easy access at all times.

 WARNING	This symbol indicates that a danger of possible serious injury or death exists if the associated instructions are not followed properly.
 CAUTION	This symbol indicates that a danger of possible harm to people or physical damage to equipment and facilities exists if the associated instructions are not followed properly.

TABLE OF CONTENTS

1. Introduction

1.1 Vision Guide Overview

1.1.1 Purpose of Vision Guide

Vision Guide is a machine vision system created to help EPSON robots to "see". Our primary goal was not just to allow our robots to see but to see better than any other robots available. While Vision Guide is more than capable for many different types of machine vision applications, its primary focus is to provide the vision tools necessary to solve robot guidance applications. Locating and moving parts, part inspections, and gauging applications are some of the typical applications for Vision Guide. In fact almost any application that requires motion and vision integrated together is a good candidate application for Vision Guide and EPSON robots.

One of the primary differences between Vision Guide and other vision systems is that Vision Guide was made as an integrated part of the EPSON robot systems. As a result you will find many features such robot to camera calibration procedures built into the Vision Guide system. The net result is a great reduction in time to complete your vision-based application.

1.1.2 Features of Vision Guide

Some of the primary features/tools provided with Vision Guide include the following:

- Integrated calibration routines that support several camera orientations and calibrations. Calibration is made much simpler than with other robot vision systems.
- Point and click interface for quick prototyping where the prototype vision sequences can actually be used in the final application.
- Complete integration into the EPSON RC+ programming and development environment.
- Blob analysis tools that measure the size, shape, and position of objects with variations. There are also tools that count the number of holes within and tell the roundness of a given blob.
- Geometric pattern search that searches for a model based on geometric part features.
- Normalized correlation search tool which locates objects under varying light conditions using an advanced template matching technique
- Edge detection tool that locates a specific edge with sub-pixel accuracy.
- Polar Search is a high-speed angular search tool that can quickly measure the rotation of complex objects. This tool is very useful for vision guided robot pick and place applications.
- Line and point tools that provide a mechanism for creating lines between points and measurement capabilities as well.
- Frame tool that allows all vision tools to be dynamically located based on a frame of reference.
- An object reference mechanism that allows one vision tool's position to be based on another vision tool's result thus saving hours of development time.
- Histogram charts provide a powerful mechanism for looking more closely at pixel data as well as setting proper thresholds for tools that require it.
- Statistics calculations are built in to provide mean, standard deviation, range, min and max values and a variety of other statistics that can be referenced for each vision tool at design-time or runtime.
- Automatic compensation for imperfections in camera lens and camera to part angular discrepancies

1.2 What's Inside This Manual

Introduction

Provides a quick reference to prepare the user for the rest of this manual. Includes information about how to use help, safety and assumptions on the users basic understanding of EPSON RC+.

Installation

Describes Vision Guide product configurations and hardware and software installation steps.

Quick Start: Your First Vision Guide Application

This chapter is useful for first time users as it guides you through a sample Vision Guide application. This section goes through everything from creating a new vision object, to calibrating the mobile camera and actually moving the robot to a part found by Vision Guide.

The Vision Guide Window

Shows the layout and gives a usage explanation for the Vision Guide window. Includes information on the Vision Guide toolbar, Image Display, Run Panel, and the Object, Sequence, and Calibration tabs.

Vision Objects

Describes the different types of vision tools available with Vision Guide and how to use them.

Histogram and Statistics Tools

Describes Histogram usage for various vision object types including Blob, Correlation, and Polar objects. Also describes the Vision Guide statistics tools from the Vision Guide window with the Statistics dialog and from the SPEL+ Language through accessing statistics properties.

Vision Sequences

Describes what vision sequences are, how to use and apply them. Also explains about debugging techniques for Vision Guide Sequences.

Calibration

Explains the usage for the various calibration types.

Using Vision Guide with SPEL+

Shows how to run vision sequences from the SPEL+ language and how to access vision properties and results. Also explains how to use Vision Guide results for robot guidance.

1.3 Robot Safety

Whenever you are working with robots or other automation equipment, safety must be the top priority. The EPSON RC+ system has many safety features built in and also provided for user access such as: E-Stop lines, Robot Pause Inputs, a Safety Guard Input, and other various system inputs and outputs. These safety features should be used when designing the robot cell.

Refer to the *Safety* chapter in the EPSON RC+ User's Guide for safety information and guidelines.

1.4 Assumptions about the User's Knowledge of EPSON RC+

Vision Guide is an optional addition to the core EPSON RC+ Environment. In order to use Vision Guide, you should be familiar with the EPSON RC+ Development and EPSON Robots. For the purposes of this manual, it is assumed that the user is already familiar with the following items:

- EPSON RC+ Project Management concepts and usage.
- Creating and editing SPEL+ programs with the EPSON RC+ program editor.
- Running SPEL programs from the Run window.
- The basic SPEL+ language constructs such as functions, variable usage, etc.

Those users not familiar with SPEL+ should attend an EPSON RC+ training class.

1.5 Related Manuals

There are other manuals in addition to this one that will prove useful when using EPSON Vision Guide. These manuals are shown below:

- ***Vision Guide Properties and Results Reference Manual:*** Contains a complete reference of all the properties and results available for vision sequences and vision objects. Each property or result has a description of the property or result and detailed information relating to its proper usage, caveats, etc.
- ***EPSON RC+ User's Guide***: Contains information on using the EPSON RC+ Robot Control System.
- ***SPEL$^+$ Language Reference Manual***: Contains a complete description of all instructions and commands for the SPEL$^+$ language.

1.6 Training

As with any product, proper training always helps make the product easier to use, and improves productivity. EPSON offers a complete set of regularly scheduled training classes to bring our customers up to speed on EPSON products.

1.7 Using On-Line Help

EPSON RC+ has a context sensitive On-Line Help system that makes finding information much easier than through using traditional manuals and books.

There are several ways to get help in EPSON RC+:

- Press the F1 function key at any time for context sensitive help. Help will be displayed for the current item you are working with. This is very useful when you don't understand a certain aspect of a screen or dialog. If you are editing a program, then help will be shown for the SPEL$^+$ keyword that the cursor is on. This is useful for getting syntax information about SPEL$^+$ language instructions.
- Click on the Help button for the current dialog box, if available.
- Select Contents from the Help menu to view the table of contents and select topics. Topics are selected by clicking on the underlined text that is highlighted in green. (This causes a jump to the topic of interest.)
- Select Contents from the Help menu, then press S or click the Search button to search for information on a specific topic.

Once you are in the Help System you will notice that some items are highlighted in green and underlined. These are hypertext links and when you click on this highlighted text, the system will jump to the area in the Help System that is related to the highlighted text. You will also notice that some text is highlighted in green with dotted underlines. Clicking on this type of text will cause a small popup window to appear with a more detailed description of the highlighted text and possibly related information that you can jump to.

Most of the information found in this manual is also available in the Vision Guide Help System although it may be arranged a little differently to provide the proper hypertext links and ease of use.

2. Installation

2.1 Product Configurations & Parts Checklist

Vision Guide 5.0 can be ordered in a variety of product configurations. Use this section as a parts checklist guide when you receive your Vision Guide product. Each of the available product configurations is described on the next few pages.

2.1.1 Compact Vision System Kit

The Compact Vision System kit includes the following parts:

- Compact Vision Controller (CV1).
- 640 × 480 resolution USB Camera with cable (5m length with standard cable).
- 24VDC 2A power supply connector.

2.1.2 Compact Vision System Options

- USB camera, 640 × 480 or 1280 × 1024 resolution.
- A cable for USB camera (5m length, standard cable or hi flex cable).
- A cable for connecting strobe trigger and strobe lamp signals to the camera (5m length with standard cable or high flex cable).
- Camera Lens Package.
- Camera Extension Tube Package.
- Ethernet Switch and cable.

2.1.3 Smart Camera Full Kit

The Camera starter kit includes the components for getting started with Vision Guide 5.0. The following parts are included:

- A fixed or mobile Smart Camera.
- Smart Camera DC Cable
- A 25 pin D-Sub connector and hood for connecting strobe trigger and strobe lamp signals.
- C-CS Adapter
- Power Connection Kit
- Camera Lens Package.
- Camera Extension Tube Package.
- Ethernet Switch and cable.

2.1.4 Smart Camera with Connector (Basic Configuration)

The camera with Connector includes the basic components for Vision Guide 5.0. The following parts are included:

- A fixed or mobile Smart Camera.
- C-CS Adapter
- Smart Camera DC Cable
- A 25 pin D-Sub connector and hood for connecting strobe trigger and strobe lamp signals.

2.1.5 Smart Camera Options

EPSON offers the following options for EPSON Smart Cameras:

- Camera Mounting Bracket to attach the mobile camera to a robot. (Specify robot type when ordering).
- Power Connection Kit, which includes a camera AC Adapter with three of AC power cables (North American, Japanese, European, UK) and a Camera Mounting Block.
- Ethernet Connection Kit, which includes an Ethernet Switch and three Ethernet Cables.
- Camera Lens Kit.
- Individual Camera Lenses.
- Camera Extension Tube Kit.

2.1.6 Part Descriptions

Compact Vision / Smart Cameras

Vision Guide 5.0 uses Compact Vision and Smart Cameras to acquire images and process vision sequences. These cameras are connected to the controller via Ethernet. The vision processing occurs on the cameras.

The following Compact Vision and Smart Camera models are available:

Compact Vision System (CV1)	For mobile or fixed mounting using USB cameras. Supports one or two cameras with 640 × 480 or 1280 x 1024 resolution.
Std. Res. Fixed Smart Camera (SC300)	For fixed mounting. Supports 640 × 480 resolution.
Std. Res. Mobile Smart Camera (SC300M)	For mobile mounting using a remote camera head. Supports 640 × 480 resolution.
High Res. Fixed Smart Camera (SC1200)	For fixed mounting. Supports 1280 × 1024 resolution.
High Res. Mobile Smart Camera (SC1200M)	For mobile mounting using a remote camera head. Supports 1280 × 1024 resolution.

Lenses and Extension Tubes

A variety of camera lenses can be used with Vision Guide 5.0. A camera lens kit or individual lenses are available from EPSON. Call your EPSON Regional Sales Manager for more details.

Extension tubes are used to offset the lens from the camera to change the field of view.

Camera Mounting Brackets

Mounting a mobile Camera to the end of the robot arm is easy if you use a camera mounting bracket. Mounting brackets are available for all robots that are controlled by EPSON RC+. The brackets provide the necessary hardware to mount a camera to the end of the arm close to the end effector.

NOTE

When you mount the camera head to a moving part such as the robot arm, make sure to secure the cable properly to keep the camera head connector and the cable from all the stress and shake. If the cable is not properly secured, it may have a cable disconnection and/or loose connection.

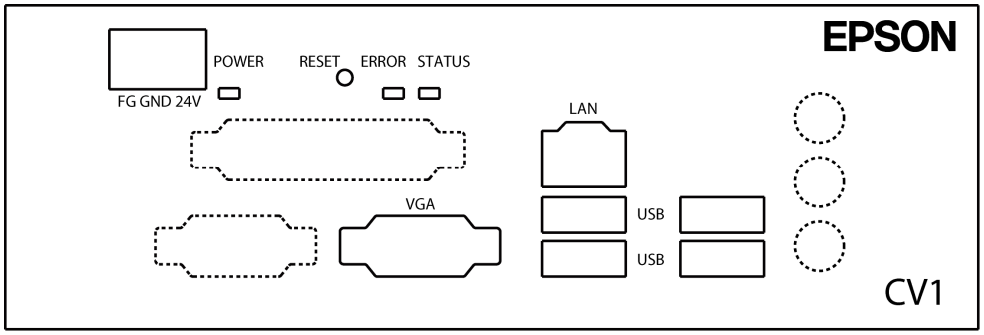
Vision Guide Software

Starting with version 5.1.0, the EPSON RC+ 5.0 installation CD includes the Vision Guide 5.0 software for Smart Cameras.

Starting with version 5.4.2, the EPSON RC+ 5.0 installation DVD includes the Vision Guide 5.0 software for both Compact Vision and Smart Cameras.

2.1.7 Compact Vision Connectors and Indicators

This section describes the connectors and indicators for all Compact Vision models.



POWER Connector

Receptacle used for connecting to an EPSON power supply or user power supply.

Pin	Signal Name	Cable color	Description
1	FG	Green	Frame ground
2	GND	Black	Ground
3	24V	Red	24 VDC Power

LAN Connector

The Ethernet communication port (10 / 100 / 1000 Base T).

USB Connectors

These four receptacles are for connecting cameras, mouse, and keyboard.

VGA Connector

Receptacle for optional VGA monitor.

STATUS LED & ERROR LED

The STATUS and ERROR LEDs together indicate status and error condition. The following table shows the status indicated by both LEDs.

STATUS LED	ERROR LED	Status
Off	Off	No power
On steady	On	Operating system starting
On blinking	Off	Ready to accept commands
Off	On	Critical error has occurred

LINK LED

Indicates network connection status as shown in the table below.

LINK LED	Status
Off	The camera is not connected to the network
On	Connected to the network
Blinking	Data is being transferred

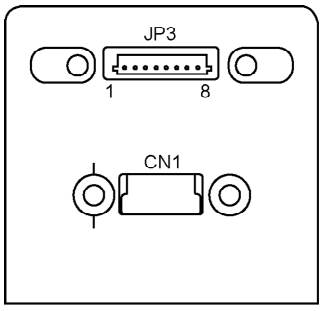
Compact Vision Cameras

There are two USB cameras available for use with the Compact Vision System.

Model	Resolution
NET 1044BU	640 × 480
NET 4133BU	1280 × 1024

Connecting a trigger input and strobe lamp to the USB cameras

The picture below shows the rear panel of the USB camera. Connect the USB cable of the CV1 to CN1. The trigger and strobe are connected to the JP3 receptacle.



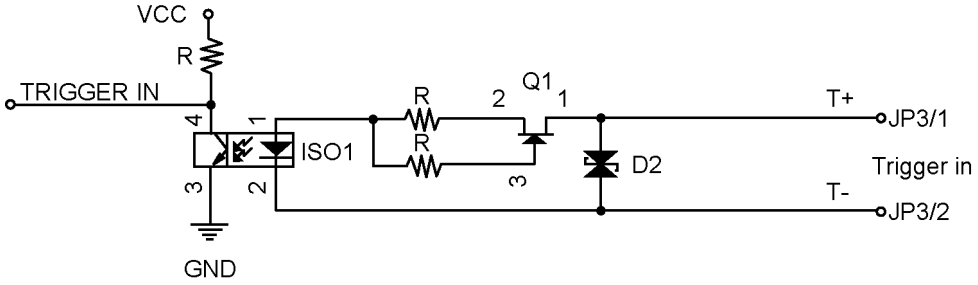
Trigger Cable

Pin #	Color	Name	Description
1	Purple	T+	Trigger signal input
2	Blue	T–	Trigger signal input
3	Green	S+	Strobe signal output
4	Yellow	S–	Strobe signal output
5	Orange	-	Not in use
6	Red	-	Not in use
7	Brown	-	Not in use
8	Black	GND	

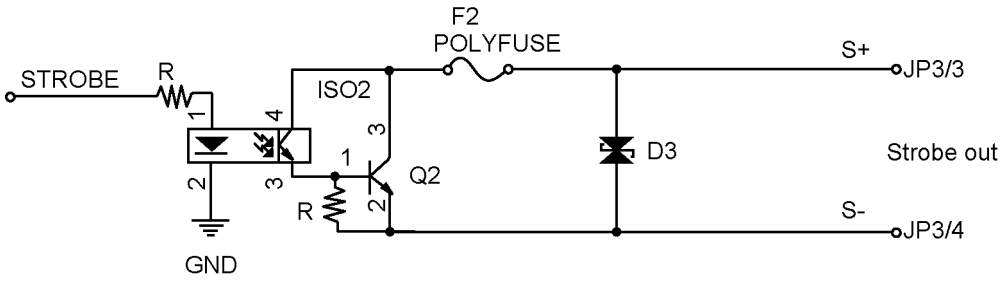
JP3 Connector

JST BM08B-SRSS-TB (Compatible plug JST 08SR-3S)

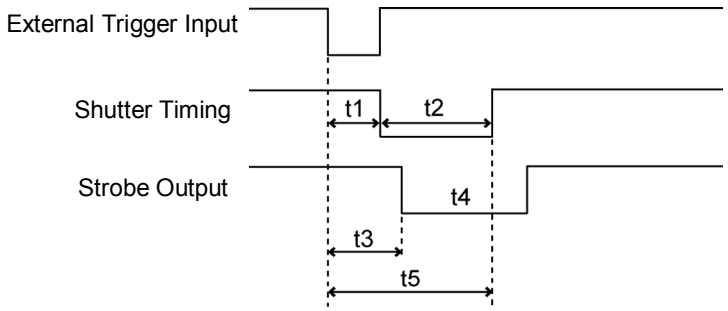
Trigger Input Circuit



Strobe Output Circuit



Trigger Timing



t1 ExposureDelay: Min. 70 microseconds [Unit of t1 to t4: microsecond]

t2 ExposureTime

t3 StrobeDelay Min 70 microseconds

t4 StrobeTime Min 140 microseconds

t5 External trigger prohibited zone

2.1.8 Smart Camera Connectors and Indicators

This section describes the connectors and indicators for all Smart Camera models.

PWR / RS-232C Connector

RJ45 receptacle used for connecting to an EPSON power supply or user power supply.

Pin	Signal Name	Cable color	Description
1	IND_RS232_TxD	Green / White	RS-232C terminal
2	IND_FRCE_BIOSRFLSH	Orange / White	Reserved. Do not connect.
3	IND_RS232_RxD	Green	RS-232C reception
4	IND_PWR_IO_0	Blue	24V DC power
5	IND_PWR_IO_1	Blue / White	24V DC power
6	IND_DIAG_MODE	Orange	Reserved. Do not connect.
7	GND	Brown / White	Ground
8	GND	Brown	Ground

10/100 BaseT Connector

The Ethernet communication port.

PWR LED & USER LED

The PWR and USER LEDs together indicate power and startup status. The following table shows the status indicated by both LEDs.

PWR LED	USER LED	Status
Off	Off	No power
Red	Red	Insufficient power.
Red	Red blinking	Errors occurred during boot load
Orange	Off	POST (power on self test) in progress
Orange	Green	Operating system is starting
Green	Off	Operating system has started
Green	Green blinking	Ready to accept commands

LINK LED

Indicates network connection status as shown in the table below.

LINK LED	Status
Off	The camera is not connected to the network
On	Connected to the network
Blinking	Data is being transferred

STAT LED

Indicates general status.

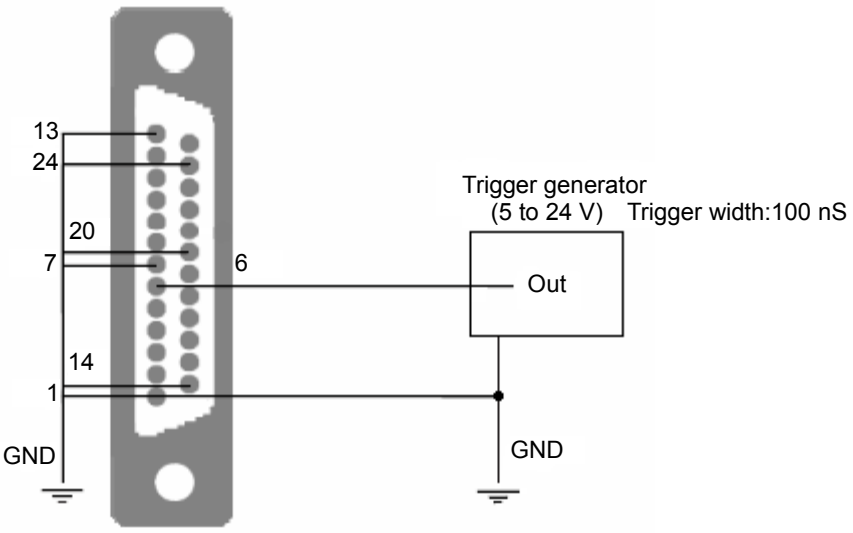
STAT LED	Status
Off	There is no power
Orange	The camera is idle
Green	Image is being acquired

I/O Connector

This DB-25 connector is used to connect the strobe trigger input and strobe lamp output.

Pin	Signal	Description
1	GND	Ground
2	STB_OUT	Strobe lamp output
6	TRIG_IN	Strobe trigger input
7	GND	Ground
13	GND	Ground
14	GND	Ground
20	GND	Ground
24	GND	Ground

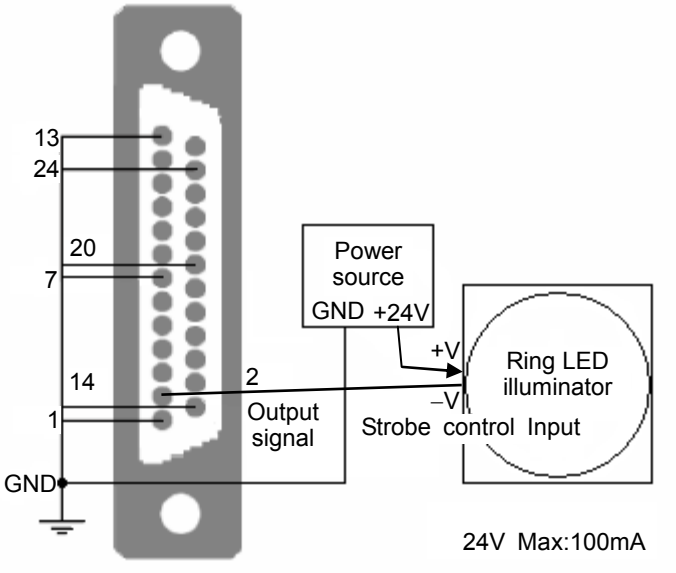
Connecting a trigger input



NOTE

Make sure that the ground of the camera and the ground of the trigger generator are connected properly.

Connecting a strobe output



NOTE

Make sure that the ground of the camera and the ground of stobe ouput are connected properly.

The stobe is a normally close negative logic output. Switches to high impedance when a strobe is requested.

2.2 Vision Hardware Installation

NOTE

If you have any problems during the following Hardware or Software installation procedures, please contact EPSON.

2.2.1 Mounting the Camera

There are two types of cameras: Fixed Smart Camera and Mobile Smart Camera. You need to decide which type to use for your application.

Fixed Smart Cameras

Use fixed Smart cameras for cameras that are stationary. Ensure that the camera is mounted such that it cannot be easily moved out of position.

NOTE

Fixed Smart Cameras are not designed for use with a robot or any other motion device.

Mobile Smart Cameras

These models are designed for mounting on a robot. They have a remote camera head that is mounted on the robot that connects with the camera main unit using a hi-flex cable.

It is recommended that you use a camera bracket supplied by EPSON for mounting the remote camera head to the robot.

NOTE

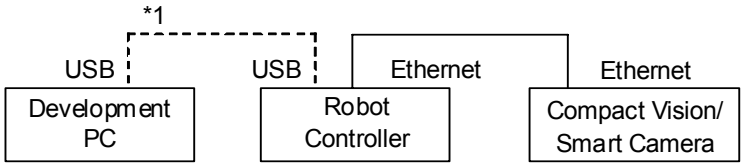
When installing the mobile camera, always make sure of the following: Using the product under unsatisfactory conditions may cause malfunction of the system and/or safety problems.

- Do not remove the cable that connects the camera head and the body (camera unit) while power is ON.
- Ensure that you properly connect and fasten both the camera head and the body (camera unit) to the cable prior to connecting power.
- Ensure that the following cables are not bind when laying cables.
  The cable for connecting the camera head and the body (camera unit)
  Cables that could be noise sources such as the power cable
- Ensure that the camera frame gound is grounded. If the camera frame ground is not grounded properly, the camera may be susceptible to noise. Use the camera mounting screw for the camera frame ground.

2.2.2 Connect Ethernet Cables

<Typical Cable Connection 1> (For only EPSON RC+ 5.0 Ver.5.4.2 or greater)

Compact Vision, Smart Camera, and Robot Controller need to be connected in the same subnet. PC network setting does not need to be set as same as Robot Controller.



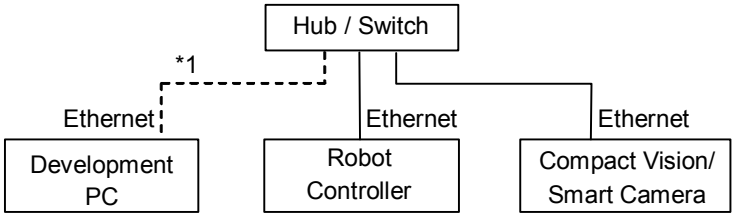
(1) Connect an ethernet cable between Robot Controller and Compact Vision or Smart Camera.

(2) This connection method can be used when you are using one Compact Vision or Smart Camera. When you are using more than two cameras, refer to <Typical Cable Connection 2>.

(3) Connect a USB cable between Robot Controller and PC.

<Typical Cable Connection 2>

Compact Vision, Smart camera, PC, and Robot Controller need to be connected in the same subnet using a switch or hub.



(1) Connect an ethernet cable between the camera and the hub.

(2) Connect an ethernet cable between Robot Controller and the hub.

(3) Connect an ethernet cable between the PC and the hub.

*1 When Vision Guide 5.0 is enabled, you must connect the controller, camera, and development PC as shown above.
When the image of the camera in not monitored at operating the robot system, you do not have to connect the PC to use the Vision Guide 5.0.

CAUTION	■ You can use the general Ethernet hub or Ethernet switch for connection. However, be sure to a use product complying with the industrial standards or noise resistant Ethernet cable (STP cable). If you use an office use product or UTP cable, it may cause communication errors and may not offer the proper performance.

2.2.3 Connect Compact Vision Cameras

The Compact Vision system uses USB cameras that connect via standard USB cables to the front panel of the Compact Vision controller. You can connect one or two cameras. Connect the cameras to any USB receptacle on the front panel.

You can only use the USB cameras provided by EPSON. The system was specifically designed for use with these cameras.

CAUTION	■ You must not use USB hub, USB repeater or USB extender cable. It may causes problem on your system.

2.2.4 Connect Compact Vision Power

Connecting your own power supply

Your power supply must be rated at 24 VDC @ 2A. The camera comes with an open ended power cable that plugs into the front panel POWER receptacle. Wire the 24V power cable to your power supply.

2.2.5 Connect Smart Camera Power

There are two methods for connecting power to your Smart Camera.

- You can connect the camera to your own power supply using a cable provided with the camera.
- You can purchase an AC power adapter from EPSON that connects directly to the camera.

Connecting your own power supply

Your power supply must be rated at 24 VDC @ 375 mA. The camera comes with an open ended power cable (SC-24V-PWR) that plugs into the camera power receptacle. Wire the 24V power cable to your power supply.

Connecting with the optional Camera AC Adapter

Connect the power cable between the AC adapter and the camera. The adapter comes with three types of adapter cables for North America (120V)*, Japan (100V)*, Europe (240V), and UK (240V).

* The cable is shared for North America and Japan.

CAUTION	■ When using an AC power adapter for camera connection, ensure that the greases do not adhere to the AC power adapter. If the greases adhere to the AC adapter, the case may deteriorate.

2.3 Vision Guide Software Installation

Vision Guide is included in EPSON RC+ 5.0 starting with version 5.1.0 for Smart Camera or version 5.4.2 for Compact Vision. Follow the instructions in the EPSON RC+ User's Guide for installing EPSON RC+.

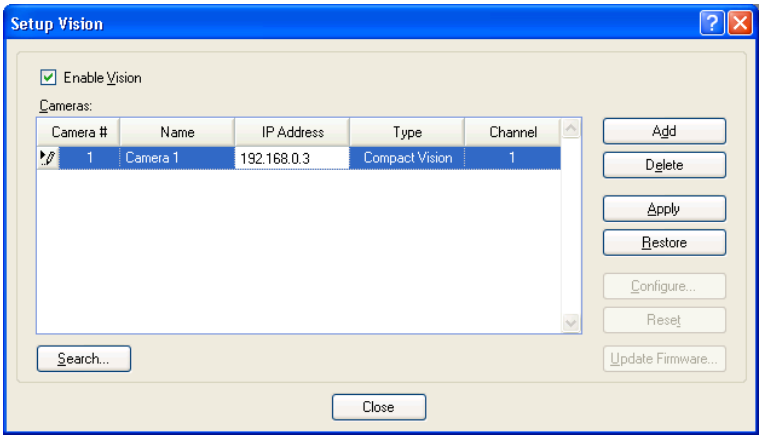
After installing EPSON RC+ 5.0, you must enable Vision Guide in the software (see the next section).

2.4 Vision Guide Software Configuration

To configure the Vision Guide software, you must open the Setup Vision dialog.

(1) Start EPSON RC+ 5.0.

(2) Select Vision from the Setup menu. The dialog shown below will be displayed.



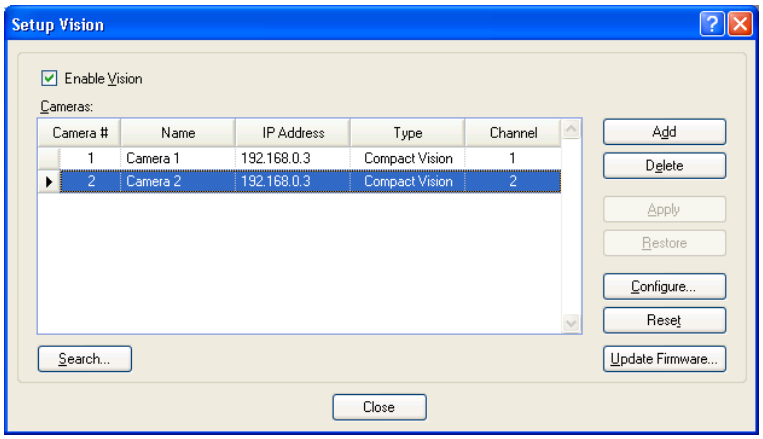
Item	Description
Enable Vision	Check this box to enable Vision Guide.
Camera #	This is the number of the camera. It is automatically assigned and cannot be changed.
Name	This is the name of the camera. The maximum length is 16.
IP Address	This is the IP address of the camera. Refer to section 2.5 for details.
Type	This is the type of the camera: Compact Vision or Smart Camera. This field is read-only and is filled by RC+ after the camera type is determined.
Channel	This is the channel of the camera. This field is read-only applies only to Compact Vision.
Add	Add a camera. Up to 8 cameras may be added. For details, refer to section 2.5.
Delete	Delete the currently selected camera.
Apply	Set current values after changes have been made. When this button is clicked, RC+ attempts to connect with the camera to verify the IP address and determine the camera type.
Restore	Revert back to the previous values.
Configure…	Click this button to open the camera configuration dialog.
Reset	Resets the currently selected camera. All project files are cleared and the camera is restarted. Restart takes approximately one minute.

Item	Description
Search…	Click this button to search for Epson Smart Cameras. You cannot search for Epson Compact Vision cameras. Refer to section 2.5.4 for details.
Update Firmware…	Click this button to update the firmware in the camera. Refer to section 2.5.5 for details.
Close	Close the Setup Vision dialog. If changes are pending, a message will be displayed asking you to click Apply or Restore before you can close the dialog.

2.4.1 Using Multiple Compact Vision Cameras

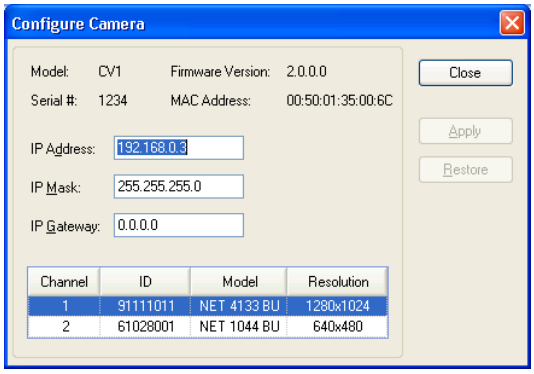
The Compact Vision system supports up to two cameras. Each camera is referenced from RC+ by a channel number. When new cameras are connected to the Compact Vision controller, they are assigned a channel.

When you add the first camera from a Compact Vision system, the [Channel] is set to 1. To add a second camera from the same Compact Vision system, click the <Add> button, and enter the IP address for the Compact Vision system. Click the <Apply> button. RC+ will set the [Channel] to 2 for the second camera.



If desired, you can change the channel number used for a camera in RC+ by clicking in the Channel field and selecting the channel number.

To see the camera configuration in the Compact Vision controller, click the Configure… button to show the Camera Configuration dialog. This will show the USB camera that is assigned for each channel. The ID for a camera is printed on the rear panel of the camera.



2.5 Vision Guide Software Camera Configuration

Before you can use your cameras with your controller and EPSON RC+, you need to configure the camera TCP/IP settings for Ethernet communications with the Controller and PC.

The camera uses static IP addressing. You must configure the IP address, IP Mask, and optionally the IP Gateway of the camera so that the PC and Controller can communicate with it.

For example, the table below shows typical IP address assignments for the PC, Controller, and camera on subnet 192.168.0.

Device	IP Address
PC	192.168.0.10
Controller	192.168.0.1
Camera	192.168.0.2

When you receive your camera from the factory, it has default TCP/IP settings as follows:

IP Address: 192.168.0.3 for Compact Vision, 192.168.0.2 for Smart Camera

Subnet Mask: 255.255.255.0

Default Gateway: None

To change the TCP/IP settings in the camera, you must configure a PC running EPSON RC+ 5.0 to communicate with the same subnet as the camera.

2.5.1 Configure PC TCP/IP

(1) Open the Windows Control Panel.

(2) Open Network Connections.

(3) Open Local Area Connection.

(4) Select Internet Protocol (TCP/IP) and click Properties.

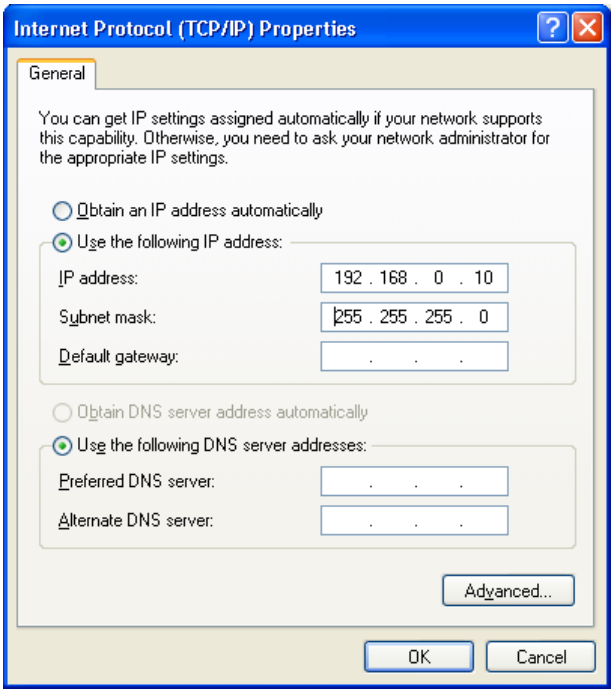
(5) Record the current settings.

(6) Select Use the following IP address

(7) Enter the following:

IP address: 192.168.0.10

Subnet mask: 255.255.255.0



(8) Click OK to save the changes.

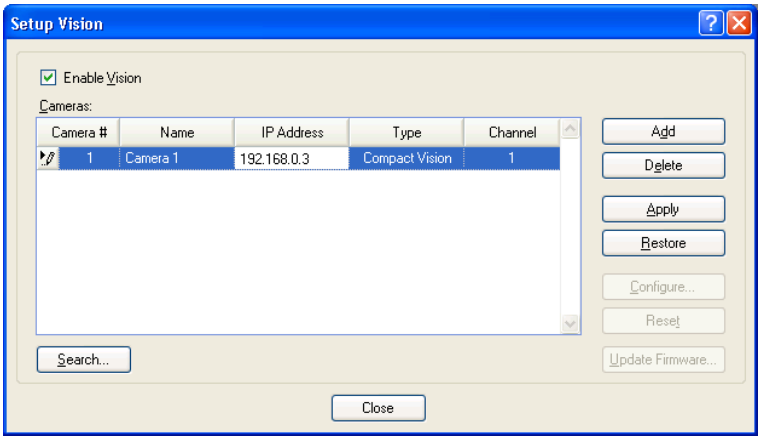
NOTE

The robot controller does not support internet protocol version 6 (TCP/IPv6). When connecting the development PC to the robot controller using Ethernet, be sure to use internet protocol version 4 (TCP/IPv4).

2.5.2 Configure Camera TCP/IP

(1) Ensure that your camera and PC are connected to the same switch or hub and that the PC TCP/IP subnet is 192.168.0 (see previous section).

(2) Start EPSON RC+ and select Setup | Vision.

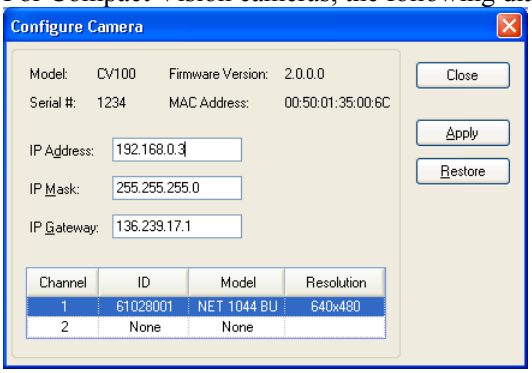


(3) Check the Enable Vision checkbox to enable Vision Guide.

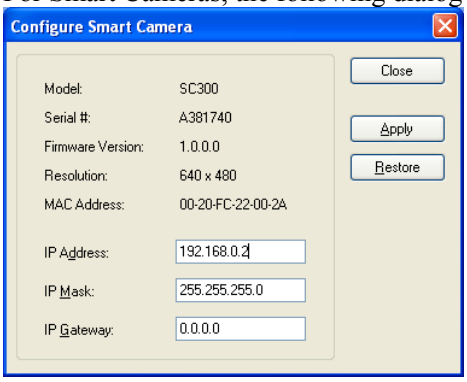
(4) Add a camera by clicking the Add button and entering the default IP address for the camera. The default address from the factory is 192.168.0.3 for Compact Vision cameras and 192.168.0.2 for Smart Cameras.

(5) Click the Apply button to save changes. EPSON RC+ will attempt to connect with the camera to determine the camera type.

(6) Click the Configure... button to open the Configure Smart Camera dialog.
For Compact Vision cameras, the following dialog is displayed:



For Smart Cameras, the following dialog is displayed:



NOTE If a communication error occurs after clicking the Configure button, check that your PC TCP/IP is configured properly as described in the previous section. Also check that the PC and camera are physically connected to the same network subnet. See the section *Searching for Cameras* later in this chapter.

(7) Change the IP Address, IP Mask, and IP Gateway to the desired settings.

(8) Click the Apply button to change the settings in the camera. The camera will be restarted and a status window will be displayed to show progress. This will take approximately one minute. If you click the Close button on the status window before the camera is finished restarting, a warning message is displayed and you can choose to continue waiting or to cancel waiting. If you cancel waiting, you will not be able to communicate with the camera until it has restarted.

(9) Click the Close button. The dialog will close and the new IP address for the camera will automatically be set in the Setup | Vision dialog.

2.5.3 Configure Controller TCP/IP

The robot Controller needs to communicate with the camera when vision commands are executed. The TCP/IP settings in the Controller must be configured to communicate on the same subnet as the camera and PC.

To configure the Controller TCP/IP settings, refer to the RC170/RC180 Controller manual.

2.5.4 Searching for Smart Cameras

In some cases, you may have lost the IP address of your camera. Vision Guide has the ability to find Smart Cameras on the same subnet as the PC running EPSON RC+ 5.0 and report their IP addresses using Universal Plug & Play.

NOTE The Compact Vision system does not support Universal Plug & Play so it cannot be searched. However, if you lose the IP address for your Compact Vision system, you can connect a monitor to the Compact Vision controller and view / change the IP address. See the chapter *Using the Compact Vision Monitor* for details.

To search for Smart Cameras, you must first enable Windows Universal Plug & Play.

To enable Windows Universal Plug & Play for Windows XP

(1) Open the Windows Control Panel in Classic View.

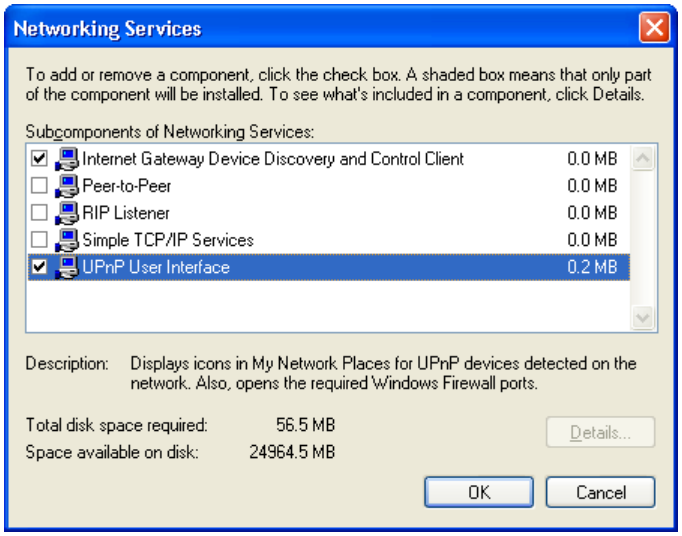
(2) Open Add or Remote Programs.

(3) Select Add/Remove Windows Components from the left side bar.

(4) Select Networking Services.

(5) Click the Details... button.

(6) Check UPnP User Interface.



(7) Click OK.

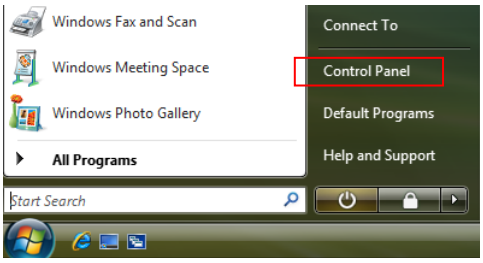
(8) Click the Next button.

To enable Windows Universal Plug & Play for Windows Vista

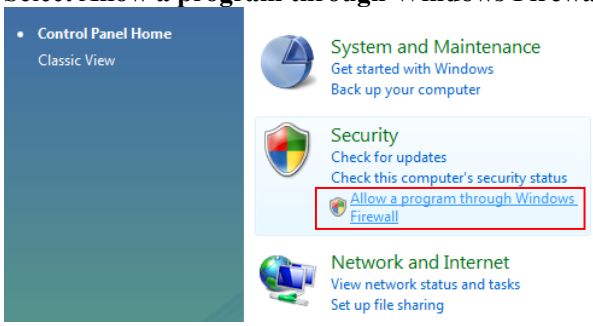
When **Network discovery** is OFF, you cannot find the camera in the network using the camera search. Change the **Network discovery** to ON using the following procedure.

Windows Vista Firewall Setting

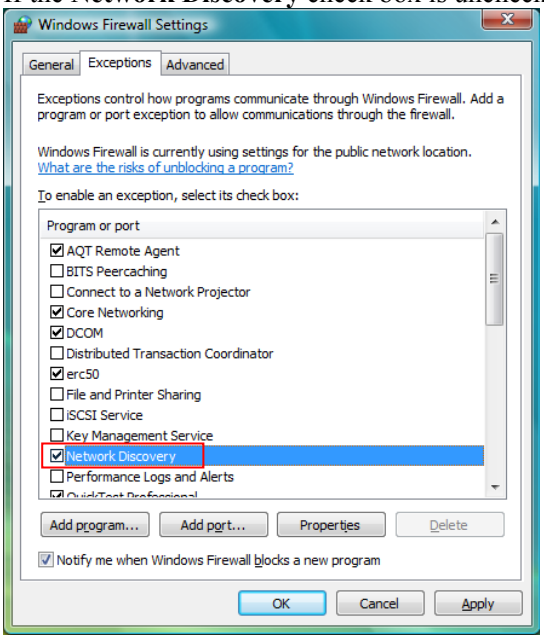
(1) Select **Control Panel** from the Windows Start menu.



(2) Select **Allow a program through Windows Firewall** in the Control Panel.



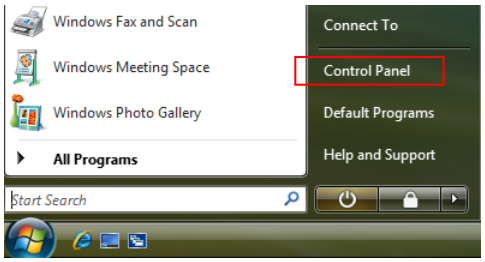
(3) If the **Network Discovery** check box is unchecked, check the box.



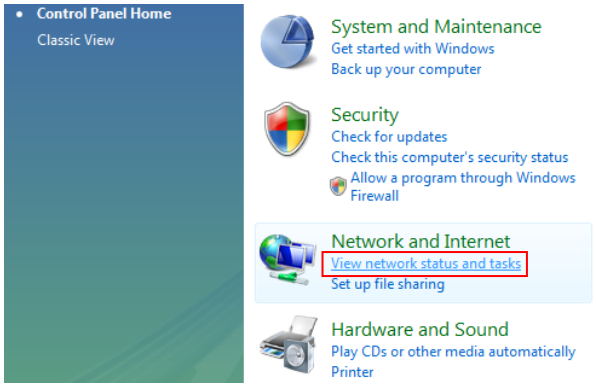
NOTE Make sure that the **Block all incoming connections** box is unchecked in the General tab of the Windows Firewall Settings dialog.

Windows Vista Network Discovery Setting

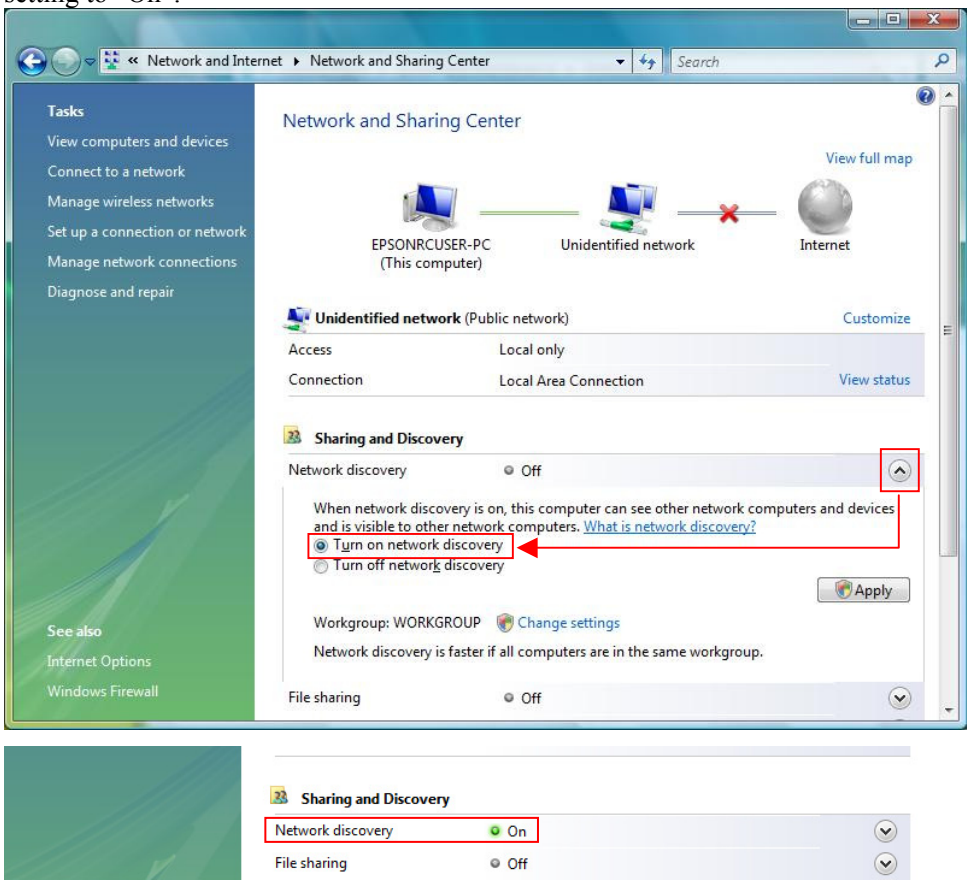
(1) Select **Control Panel** from the Windows Start menu.



(2) Select **View network status and tasks** in the Control Panel



(3) When Network discovery is "Off", click the ▲ button on the right end and change the setting to "On".



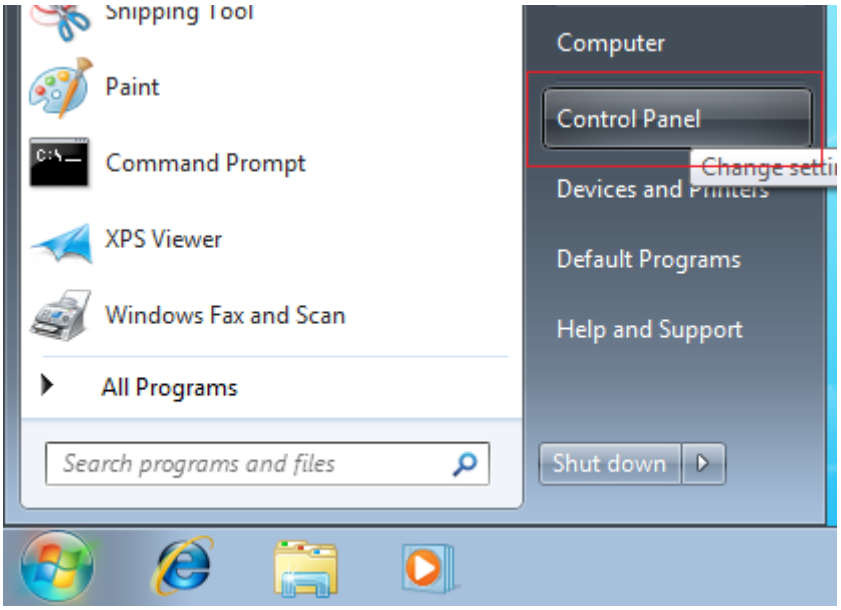
(4) When the setting is changed, the security system dialog of Windows Vista appears asking for your permission to continue or a dialog to enter the administrator password appears. Follow the instruction to allow the setting change.

To enable Windows Universal Plug & Play for Windows 7

When **Network discovery** is OFF, you cannot find the camera in the network using the camera search. Change the **Network discovery** to ON using the following procedure.

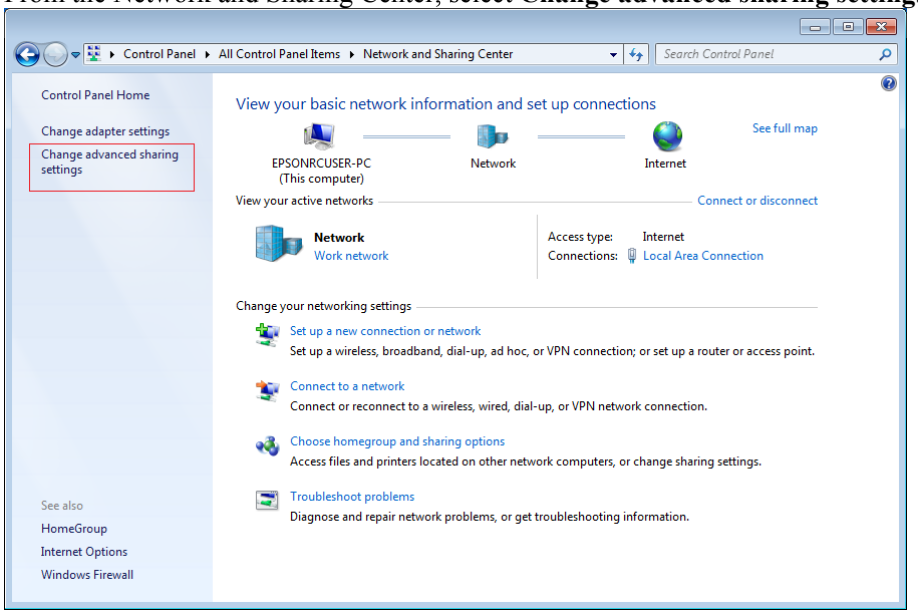
Windows 7 Network Discovery Setting

(1) Select **Control Panel** from the Windows Start menu.

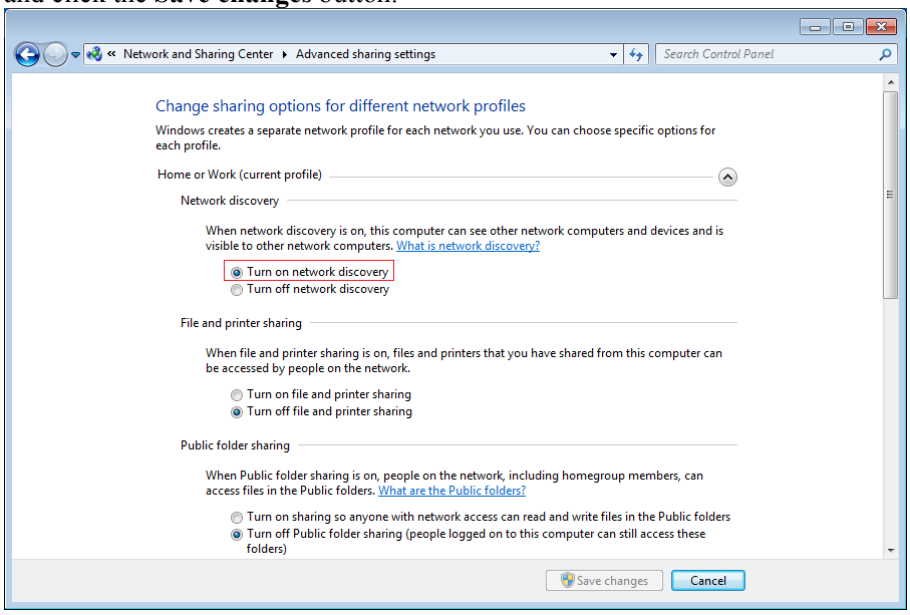


(2) Select **Network and Sharing Center** in the Control Panel

(3) From the Network and Sharing Center, select **Change advanced sharing settings**.



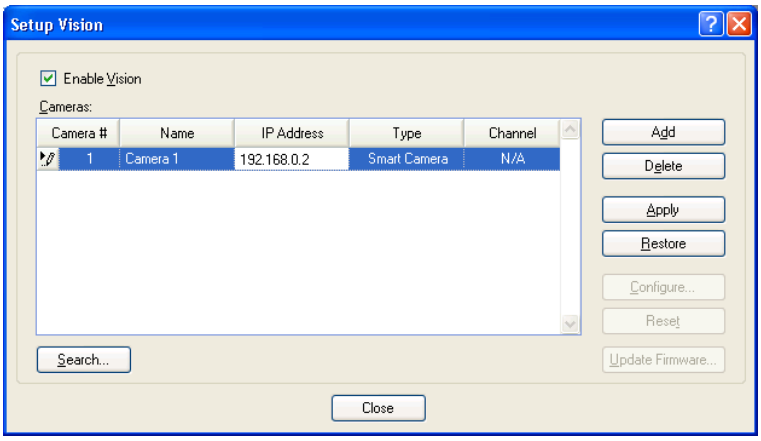
(4) When Turn on network discovery is unselected, select **Turn on network discovery** and click the **Save changes** button.



To search for Smart Cameras

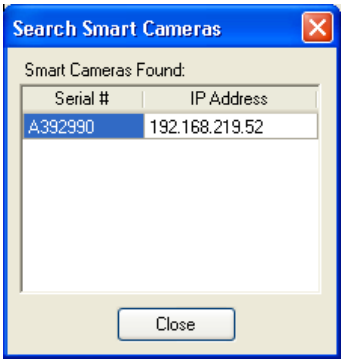
(1) Start EPSON RC+ 5.0.

(2) Select Vision from the Setup Menu.



(3) Click the Search... button. A dialog is displayed.

(4) After about 10 seconds, if any cameras are found they will be displayed in the list by serial number and IP address.



NOTE

If you no cameras were found, the subnet of the Smart Camera IP address may not match the subnet of your PC. If the camera subnet is not known, refer to Appendix C: Camera RS-232C Diagnosics to find the IP address of the camera. Otherwise, change the PC IP address so that it is on the same subnet as the camera, then execute Search again.

(5) Now you can set your PC TCP/IP settings to communicate with the camera.

(6) Click on the IP Address field for the camera you want to use and type Ctrl+C to copy the address.

(7) Click Close to close the Search Smart Cameras dialog and return to the Setup | Vision dialog.

(8) Add a camera by clicking the Add button.

(9) Paste the IP Address by typing Ctrl+V.

(10) Click Apply to save the changes.

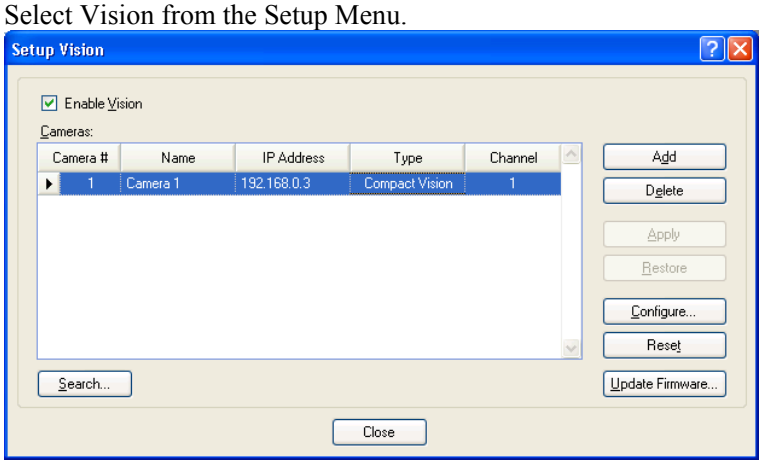
2.5.5 Updating Camera Firmware

Sometimes the firmware (non-volatile software) in the camera may need to be updated.

To update camera firmware

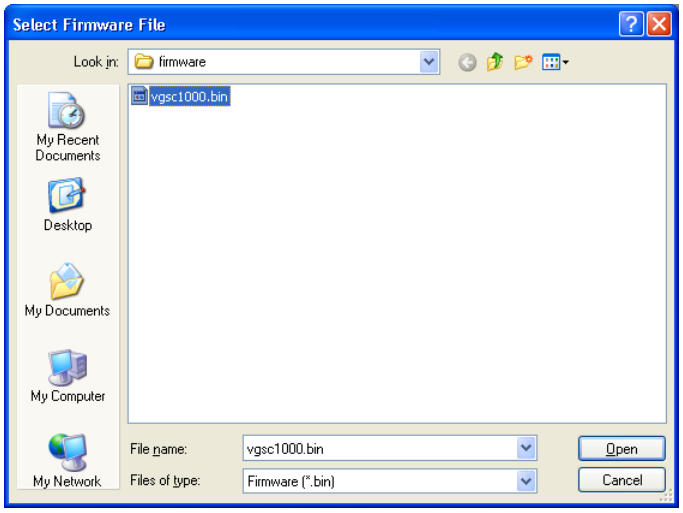
(1) Start EPSON RC+ 5.0.

(2) Select Vision from the Setup Menu.



(3) Click the Update Firmware... button.

(4) Navigate to the firmware file. The default directory is \EpsonRC50\Vision\Firmware. Firmware files have the BIN extension.
Note that you must select the correct firmware file for the camera you are upgrading. For Compact Vision cameras, the filename is vgcvxxxx.bin, and for Smart Cameras, the filename is vgscxxxx.bin, where xxxx is the version number.



(5) Click the Open button. The firmware update process will start. After the firmware is updated, the camera will be restarted.

(6) The firmware update is complete.

2.6 Testing the Vision System

2.6.1 Checking Vision Guide

Now that installation is complete, we need to make sure that everything is working properly. Follow the steps described below:

Start EPSON RC+ and create a new project

(1) Double click the EPSON RC+ icon on the Windows desktop to start EPSON RC+.

(2) Click on the Project menu from the EPSON RC+ menu bar.

(3) Click on the New Project menu entry of the Project menu. The New Project dialog will appear.

(4) Type a name for the new project. You can use any name. We suggest that you use the name "vistest" since this will be our test project to make sure everything is working OK. After you type a name in the name field, click the OK button. You have just created a new project.

Check that the Video Display is working properly

(1) Once you have created a new EPSON RC+ project, you will notice that many of the toolbar icons are now shown in color. Look for the vision toolbar button that looks like this  . Click on the vision toolbar button to open the Vision Guide window. When the vision tool bar button is not displayed, make sure that the Enable Vision checkbox is checked in step (3) of *2.5.2 Configure Camera TCP/IP*.

(2) Before we can do anything in the Vision Guide window we must first create a vision sequence. Clicking on the New Sequence  toolbar button creates a new vision sequence. (Remember that this toolbar button is located on the Vision Guide window toolbar and not on the EPSON RC+ toolbar.) The New Sequence dialog will now appear.

(3) Enter the name "vistest" for the new vision sequence and then click the OK button.

(4) Look at the area of the Sequence tab called Properties. You should see a property called Camera. This property is used to select the camera that will be used with this sequence. It should currently be set to 1 since this is default for a newly created sequence.

(5) If you are using a camera number other than 1, click on the value field of the Camera property and a down arrow will appear. Click on this down arrow and a list with choices will appear. The number of cameras available depends on the number of cameras in the system.

(6) Look at the image display of the Vision Guide window. You should see a live image from the camera. If so, then you can go on to *Checking and Adjusting for Proper Focal Distance*

(7) Is the color of the Vision Guide image display black? If it is then this is OK, we just have some minor adjustments to make. Try opening the aperture on the camera lens. This will let more light in and should cause the image display to begin lightening up a little. However, keep in mind that if the area your camera is looking at is black in color, then you will need to open the aperture more to see the transition in color of the image display. We recommend that you use a light source to help get more light into the camera just for this test. If you can see the image display change color when you open and close the aperture of the lens, then you can go on to *Checking and Adjusting for Proper Focal Distance*.

(8) If you cannot see any difference in the color of the image display when you open and close the aperture try pointing a light directly in the lens to make sure that enough light is getting into the camera. If you still see no difference in the image display color, you should review your installation again.

2.6.2 Checking and Adjusting for Proper Focal Distance

(1) Position your camera so that it is located above the area that you will be searching for parts and place an object within the camera's field of view. This will give us a target for adjusting the focus of the camera.

(2) You should now see, in the image display of the Vision Guide window, a view of the object that you put under the camera. If you cannot, try manually picking up the part and moving the target feature closer and further away from the lens. Do you see the target feature now? (We are just trying to see the part but it may be way out of focus so don't worry about that yet.) If you don't then you probably have selected the wrong lens for the distance your part is away from the camera. Try a different sized lens.

(3) A 16 or 25 mm lens should work for most mobile cameras mounted to the robot. So if you are trying to test with a Mobile camera mounted to the robot, we suggest a 16 mm lens to start with.

(4) At this point you should be able to at least see the target feature on the work surface. Adjust the focus of the lens to bring the part into good focus. If you cannot bring the part into focus, then you may need to add an extension tube between the lens and the camera. This will change the focal distance for the lens and with the proper extension tube you should be able to focus the camera.

(5) After installing an extension tube, try adjusting the focus of the camera lens. You may have to experiment with different extension tube lengths to get proper focus.

See Appendix B: Extension Tube Distance Table for a list of the common lens and extension tube configurations for various focal lengths.

Congratulations, you are now finished with installation. Go on to the chapter *Quick Start: A Vision Guide Tutorial*. This chapter will guide you through using vision guide to locate a part and then move to that part with the robot.

3. Quick Start: A Vision Guide Tutorial

3.1 Tutorial Overview

The purpose of this chapter is to walk you through a simple vision application to help introduce you to some of the Vision Guide basic usage concepts and to show you how easy it is to use. In many cases we will simply explain the steps to follow but will not explain the details behind what was done. Then in the chapters that follow you can read about the details.

We will not be using actual parts for this tutorial but instead we will use simple drawings of objects that you can then make copies of and put under a camera to follow the tutorial. This will guarantee that everyone has the same results when trying this tutorial.

This tutorial will show you how to create a simple application to use vision to find a part and then move the robot to it. It is assumed for the purposes of this tutorial that your robot is a SCARA type with a camera mounted at the end of the 2nd arm link.

This chapter will include the following sections:

- Items required for this tutorial.
- Camera lens configuration.
- Creating a new EPSON RC+ Project.
- Creating a new vision sequence.
- Using a Blob object to find a part.
- Writing a SPEL+ program to interact with the vision sequence.
- Calibrating the robot with the camera.
- Using vision to instruct the robot to move to the part.
- Finding and moving to multiple like parts.

Following the *Items Required for this Tutorial* section of this chapter, you will find two pages with printed targets for our vision tutorial. These pages will be used throughout the tutorial.

Following the target pages is a section entitled Start *EPSON RC+ and Create a New Project*. This is where we will start the actual tutorial.

3.2 Items Required for this Tutorial

This tutorial will guide you through using a EPSON robot with Vision Guide. It is assumed that you are comfortable using EPSON RC+ and EPSON Robots. If you are a little uncertain as to how to use EPSON RC+ you probably will want to spend a little time reviewing it prior to starting this tutorial. The following items will be required to work through this tutorial:

- EPSON RC+ 5.0 must be installed on your PC and Vision Guide should be enabled.
- A camera for Vision Guide 5.0 must be installed and working properly.
- An EPSON robot should be mounted to a base with a work surface attached in front of the robot so the robot can access the work surface.
- A camera should be attached to the robot at the end of the 2nd arm link. A camera mounting bracket for the particular robot you are using is available from EPSON. The camera should be mounted so that it is facing downward.

NOTE

Make sure that the PC's main power, Controller power, and camera power are turned off when attaching the Ethernet cables between the PC, camera, and controller to the Ethernet switch. Failure to turn power off could result in damage.

- Copies of the pages that contain drawings of the target parts should be made available to put under the camera. (The target part drawings can be found in the next section following this list.)
- A camera lens must be selected that will allow a field of view of about 40 mm × 30 mm. In our tests here we have been using a mobile camera (SC300M) with 16 mm lens at a distance of about 210 mm from the work surface. However, to get proper focus will require some adjustment. We'll help guide you through this.
- We won't be picking up any parts in this tutorial. However, we will be moving the robot to the part positions drawn on our target sheets. You will need to attach a gripper or rod to the Z axis shaft. A rod that comes straight down the center of the Z axis shaft will work fine. This gripper or rod will be used for calibration and also moving to the target parts on the target pages.

Figure 1: A single target feature for tutorial (a drawing of a washer)

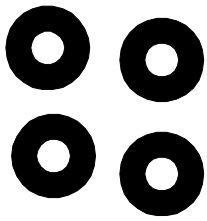


Figure 2: Multiple target features for tutorial (multiple washers)

3.3 Start EPSON RC+ and Create a New Project

(1) Turn on your EPSON RC+ PC.

(2) Double click the EPSON RC+ icon on the Windows desktop to start EPSON RC+.

(3) Click on the Project menu from the EPSON RC+ menu bar.

(4) Click on the New Project menu entry of the Project menu. The New Project dialog will appear

(5) Type a name for the new project. We suggest that you use your name with the letters "tut" attached to the end. That way each individual who tries this tutorial will have a unique name for his or her project. For our project here we will use the name "vgtut". After you type a name in the name field, click the OK button. You have just created a new project called xxxxtut.

3.4 Create a New Vision Sequence

(1) Once you have created a new EPSON RC+ project, you will notice that many of the toolbar icons are enabled (no longer grayed out). Look for the vision toolbar button that looks like this  . Click on the vision toolbar button to open the Vision Guide window.

(2) Before we can do anything in the Vision Guide window we must first create a vision sequence. This is accomplished by clicking on the New Sequence  toolbar button. Remember that this toolbar button is located on the Vision Guide window toolbar and not on the EPSON RC+ main toolbar. The New Sequence dialog will now appear.

(3) Enter the name "blobtut" for the new vision sequence and then click the OK button. We will be using the name "blobtut" later in our EPSON RC+ code so be sure to type this name in exactly as spelled above without the quote marks. We have now created a new vision sequence called "blobtut" and everything we do from now on will be for this vision sequence.

3.5 Camera Lens Configuration for Tutorial

We mentioned earlier that our target field of view for this tutorial should be about 40 mm × 30 mm. We have found that a 16mm lens and 1 mm extension tube work fine for this field of view at distances ranging from 167 mm up to 240 mm. This means that a 16 mm lens and 1 mm extension tube will probably also work for your tutorial.

If you have not attached a lens to your camera and tried to focus your lens, it would probably be a good idea to do this at this time. In the chapter Installation there is an explanation on how to determine which lens to use and focusing. You will probably want to review this information before proceeding if you are not familiar with how to select a camera lens and focus the camera to your part.

In order to check your focus, obviously you will need to view an image from Vision Guide. Since we already have the Vision Guide window open, we now only need to make sure we can properly see the target feature on the target sheet. The steps to accomplish this are described in the next section.

3.5.1 Position the mobile camera over the target sheet and focus the lens

(1) Get the copies of the target sheets that you made earlier. Select the target sheet with the single washer feature printed in the center and put it in a location on the work surface where the robot can then position the camera over it easily. The best position is normally directly in front of the robot.

(2) Now move the robot so that the camera is located over the picture of a washer. You should be able to move the robot manually without having to turn servo power on.

(3) You should be able to see the target feature (the washer) in the image display of the Vision Guide window. Bring the target feature into focus by adjusting the camera lens focus. If you cannot see the target feature or cannot bring it into proper focus go to the section titled *Checking and Adjusting for Proper Focal Distance* in the chapter *Installation*. This section will explain the details on selecting and focusing a camera lens.

The camera should now be in position directly over the target feature drawn on the target sheet. You should be able to see the target clearly on the image display of the Vision Guide window. The Vision Guide window should now appear as shown in Figure 3. Notice the target feature (the washer) is shown in the center of the image display of the Vision Guide window.

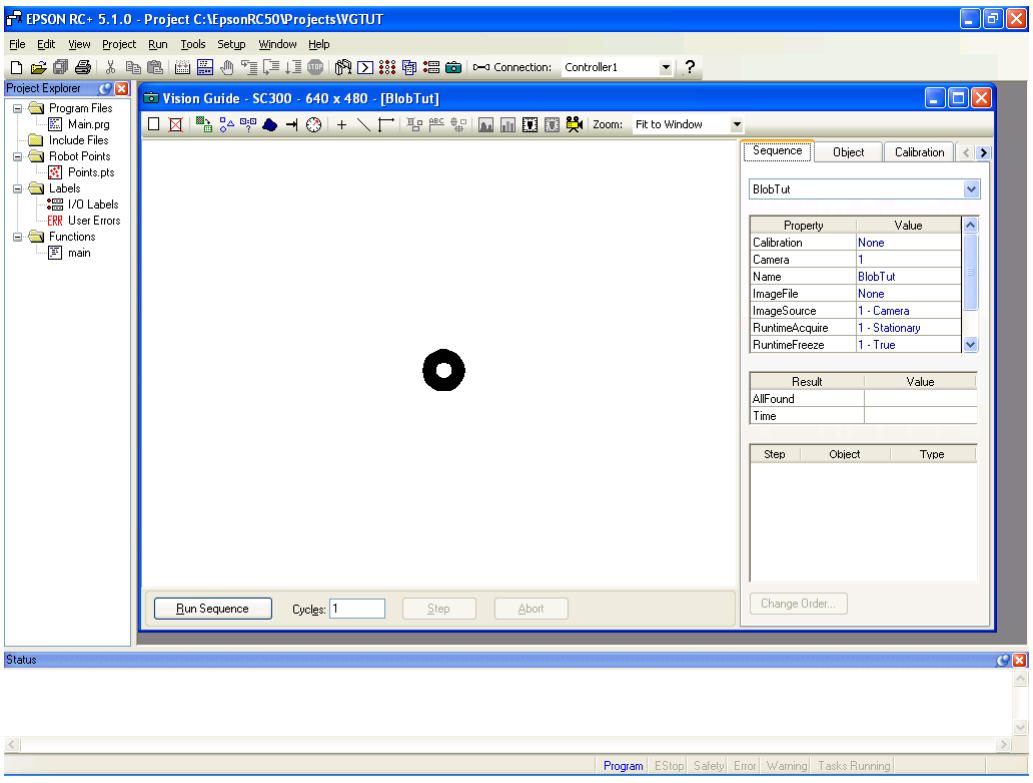


Figure 3: The Vision Guide window with target feature in center of image display

3.6 Using a Blob Object to Find a Part

Now that the target feature (washer) is located in the center of the image display of the Vision Guide window, we can create a Blob object to find the washer. The steps to create a Blob object, configure it, and then find the washer are shown next.

Step 1: Create a New Blob Object

(1) Click the New Blob toolbar button  on the Vision Guide toolbar with the left mouse button and then release the button. (Do not continue holding the left mouse button once you have clicked on the New Blob toolbar button.)

(2) Now move the mouse downward towards the image of the washer in the center of the image display. As you move off the Vision Guide toolbar the mouse pointer will change to the Blob object icon.

(3) Continue moving the mouse to the center of the image display and click the left mouse button. This will position the new Blob object onto the image display.

Your screen should look similar to the one shown below in Figure 4 below.

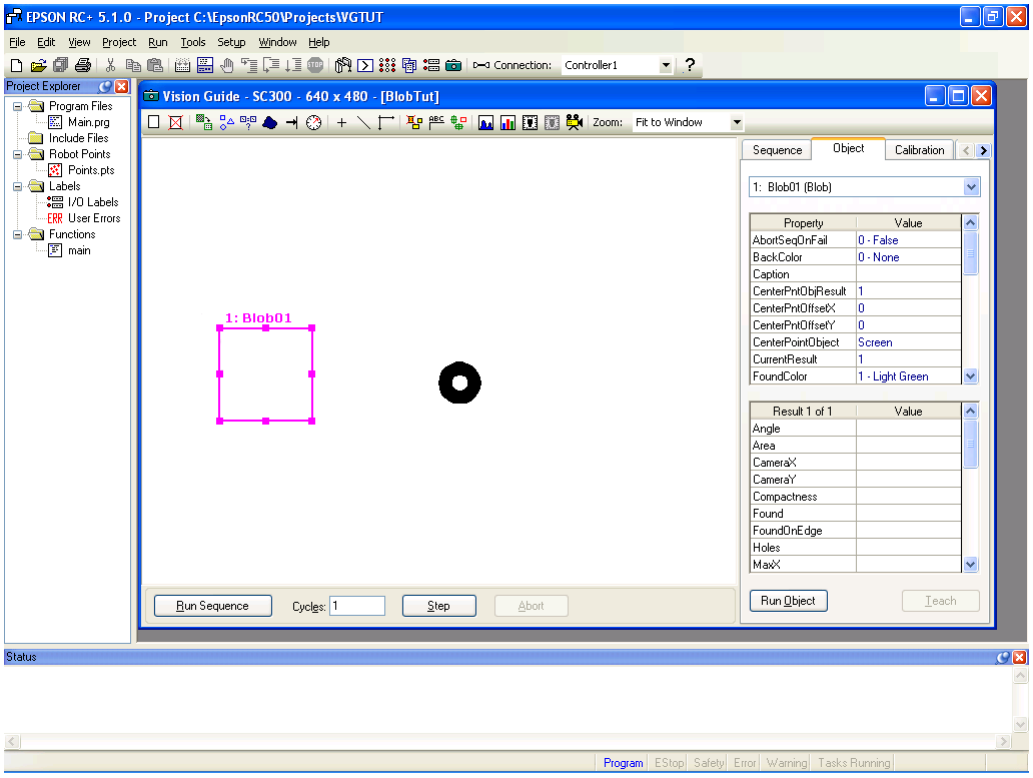


Figure 4: Vision Guide window with new Blob object "Blob01"

Step 2: Position and Size the Blob Object

We now need to set the position and size of the search window for the Blob object. The search window for the "Blob01" Blob object is the box you see to the left of the washer in Figure 5. Let's make this search window large so that we can search nearly the entire field of view for the washer.

(1) Move the mouse pointer over the name label of the Blob object and hold the left mouse button down. While continuing to hold the left mouse button drag the Blob object to the upper left corner of the image display so that the upper left corner of the search window is almost touching the upper left corner of the image display.

(2) Move the mouse pointer over the lower right sizing handle of the Blob01 search window and hold the left mouse down. While continuing to hold the left mouse down, drag the lower right sizing handle to the bottom right corner of the image display. The Blob object's search window should now cover the entire image display. This will allow us to find any blob that is located within the camera's field of view.

The Screen shot below shows the search window for the "Blob01" Blob object repositioned and sized so that it covers the entire image display.

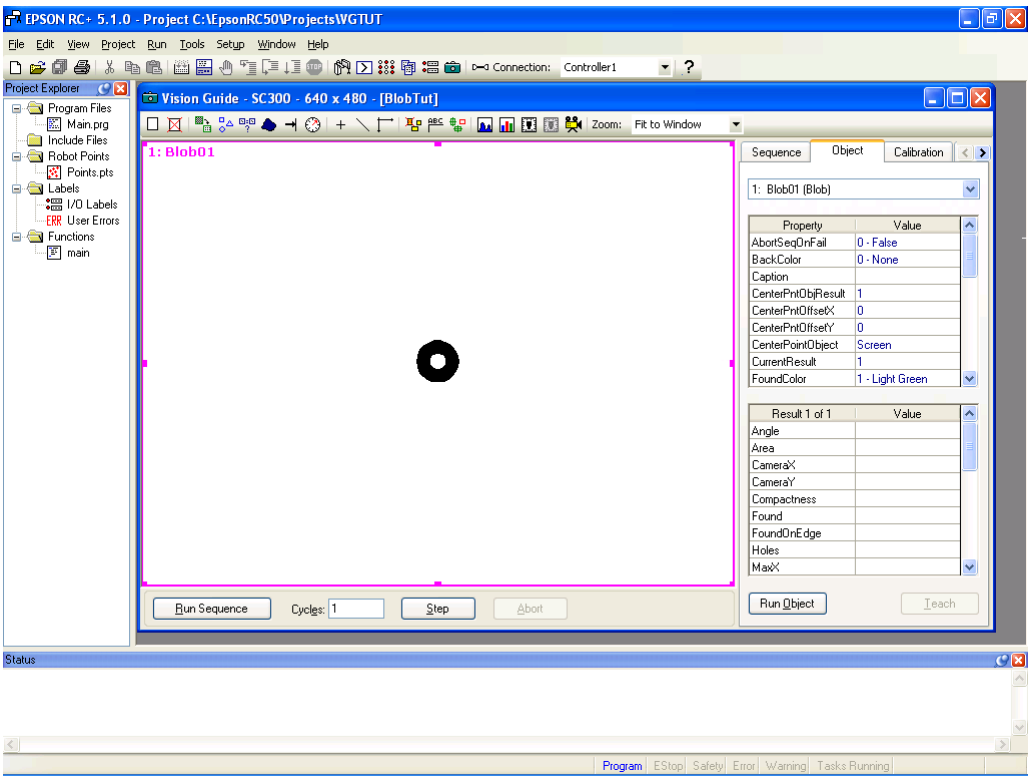


Figure 5: "Blob01" Blob object with large search window

Step 3: Set Properties and Run the Blob Object

The search window is now large enough for the washer to be seen inside of the search window. We are now ready to test our Blob object to make sure it can find the washer.

(1) Click on the Object tab located on the right side of the Vision Guide window. This will bring the Object tab forward. The Object tab will appear as shown in Figure 6.

(2) Look at the Properties list on the Object tab. Find the Name property. Double click on the Value field for the Name property to cause the current name to be highlighted. Now enter the name "washer". We have just changed the name of our Blob object to "washer". If you look at the top of the Object tab in the Name drop down list, and the name label on the search window, you will see that the name has been changed in both places.

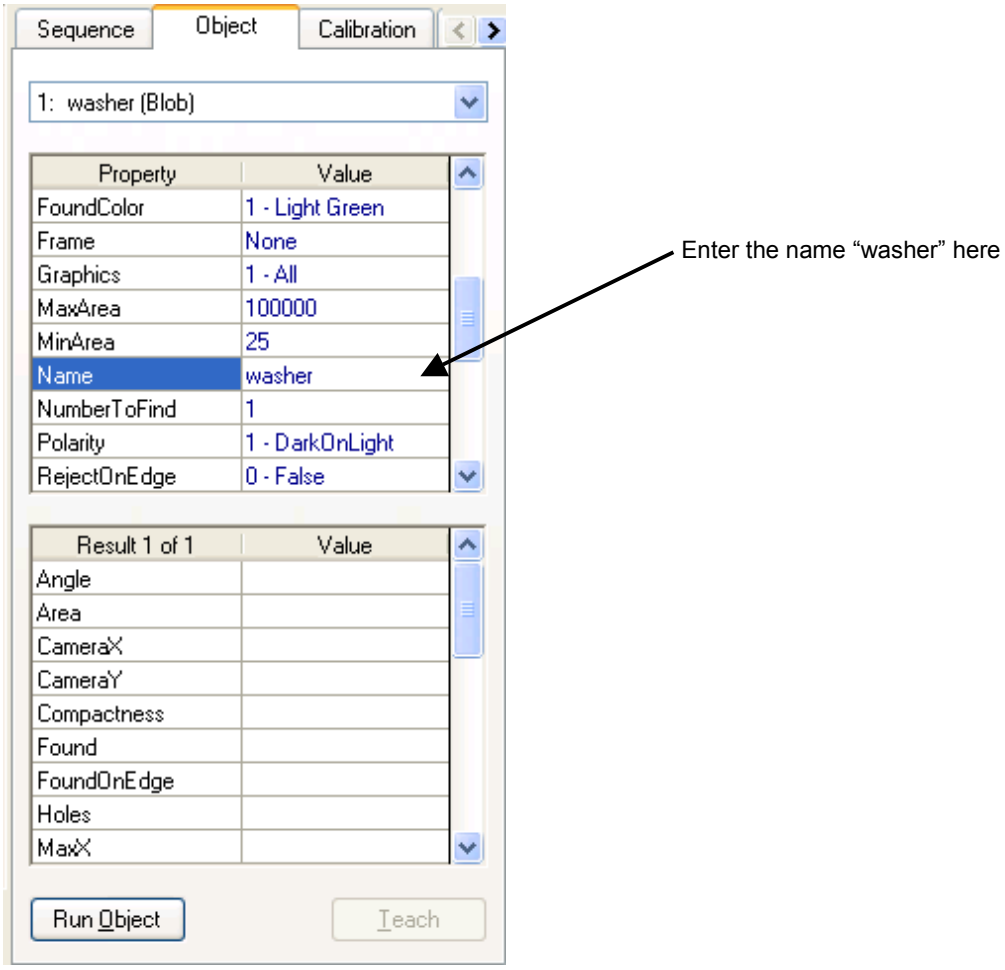


Figure 6: Object Tab showing properties for Blob object

(3) While you are looking at the Properties list you might also want to examine the Polarity property. Since we just created a new blob, the default value for Polarity is DarkOnLight, which means find a dark blob on a light background. We could also change this property to LightOnDark. But since we want to find a dark blob (the washer) on a light background, leave the Polarity property as is.

(4) We are now ready to run the Blob object "washer". To run the object click on the Run Object button located at the bottom left of the Object tab. This will cause the Blob object "washer" to execute and in this case the Blob object will find a blob that looks like a washer. You can tell that the Blob was found by examining the color of the search window after running the Blob object. The search window will be green if the Blob object finds a blob and red if it does not. (You can also examine the Results list to see if the Blob was found or not. More on that next.)

(5) Now try moving the paper with your target washer just a little and click the Run Object button on the Object tab again. Be sure to keep the washer inside the search window that you created. You can see the new position of the blob is found and highlighted in green on the image display window. (If you move the paper so that the washer is outside of the search window boundary, then the Blob object will not find the washer and you can see this because the search window will turn red when the object is not found.)

Step 4: Examining the Results

Now that you have run the Blob object called "washer" you can examine the results that are returned for this object. The results are displayed in the Results list located just below the property list on the Object tab.

(1) Look at the Results list for the result called Found. The value of the Found result should be True at this point because we just found the blob. If no blobs were found then the Found result would display False.

(2) You can also see the Area result at the top of the Results list. This shows the area of the blob that was found.

(3) Use the scroll bar to move the results list down to the bottom. At the bottom of the Blob results list you will see the Time result. This result tells us how much time it took to find this blob.

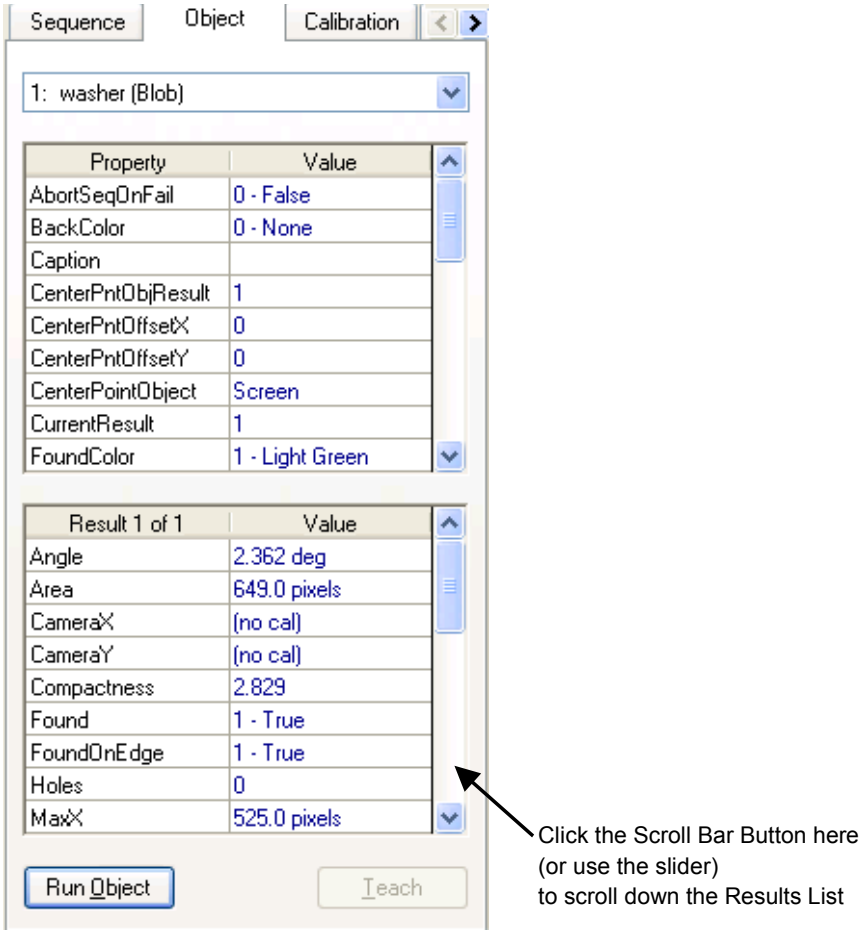


Figure 7: Object Tab with Results list scrolled down to show Time result

CAUTION	■ Ambient lighting and external equipment noise may affect vision sequence image and results. A corrupt image may be acquired and the detected position could be any position in an object's search area. Be sure to create image processing sequences with objects that use search areas that are no larger than necessary.

Step 5: Saving Your Vision Sequence

We should probably save our work at this time. As with any application, it's a good idea to periodically save your work often. The Project Management feature of EPSON RC+ causes everything related to this project to be saved at once. This is done in one easy step as shown below:

Click on the Save toolbar button  . It is located on the left side of the EPSON RC+ main toolbar. When changes have been made anywhere in your project, the Save toolbar button will be shown in blue.

We have now successfully created a new Blob object; properly positioned and sized the search window; and we ran the Blob object to find a blob. Let's take the next step and write a simple program to run our vision sequence from the SPEL+ language and retrieve some of the results for use in our application.

3.7 Writing a SPEL+ Program to Work with the Vision Sequence

One of the most powerful features of Vision Guide is that any vision sequences that are created from the point and click environment can be used from the SPEL+ language. This means that vision sequences are a core part of EPSON RC+ vision applications and are not just a prototyping tool for which you later have to rewrite everything in SPEL+. Vision sequences and the SPEL+ language are integrated together to give you the best of both worlds: the ease of use of a point and click vision development environment and the power and flexibility that a language provides. Let's write a quick program to find whether or not the blob was found, check the area of the blob, and then print a message with this information.

Open the Program File Called MAIN.PRG

You should already know how to open the main.prg program file since you are familiar with the EPSON RC+ environment. However, for those of you who may not know how to open a file we have included the basic steps below:

(1) Click on the Open File toolbar button on the EPSON RC+ toolbar. This will open the Open File dialog that contains the program called main.prg that is automatically created when you first create a new project. You will see the following Open File dialog box:

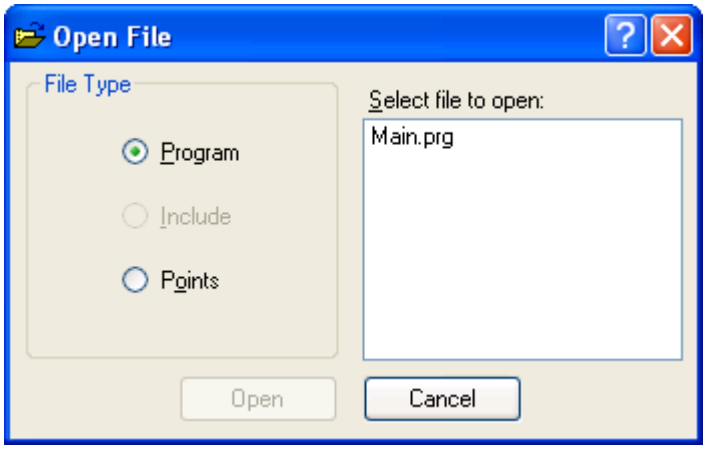


Figure 8: The EPSON RC+ Open File Dialog

(2) As you can see in the Open File dialog, the main.prg program file should already be highlighted since you haven't yet created any other programs. Go ahead and click on the Open button at the bottom left of the dialog to open the main.prg program file.

Create a SPEL+ Program to Work with our Vision Sequence

Shown below is an example program that will run the vision sequence "blobtut" and then check some of the properties associated with the Blob object called "washer". For example, we will check if the blob is found or not. If it is we will display a "The washer was found!" message and the area of the blob. If no blob is found we will display a "The washer was not found!" message.

(1) You should now see an editor window with MAIN.PRG displayed on the Title bar. The cursor will be located at the beginning of the 1st line in the edit window. (The perfect position to begin typing in our program.) Go ahead and enter the following program into the editor window. Don't worry about the capital vs. small letters. The editor automatically capitalizes all keywords.

```
Function main
  Real area
  Boolean found

  VRun blobtut
  VGet blobtut.washer.Found, found

  If found = TRUE Then
    VGet blobtut.washer.Area, area
    Print "The washer was found!"
    Print "The washer area is: ", area, " pixels"
  Else
    Print "The washer was not found!"
  EndIf
Fend
```

Running the main Function

You should be familiar with the Run window for EPSON RC+. We will use the Run window to run our sample program that was created in the previous section. The steps to accomplish this are shown below:

(1) Click on the Run toolbar button  on the EPSON RC+ toolbar. This will cause the EPSON RC+ Run Screen to appear. Notice that in Figure 9, the Run window is split into 2 parts. The left half of the Run window is the image display (note the washer shown in the center) and the right half of the Run window is the text area which is used for displaying text messages.

NOTE 

If the Run window only shows the Text Area and doesn't appear with both an image display and a text area, click on the Display Video check box on the Run window.

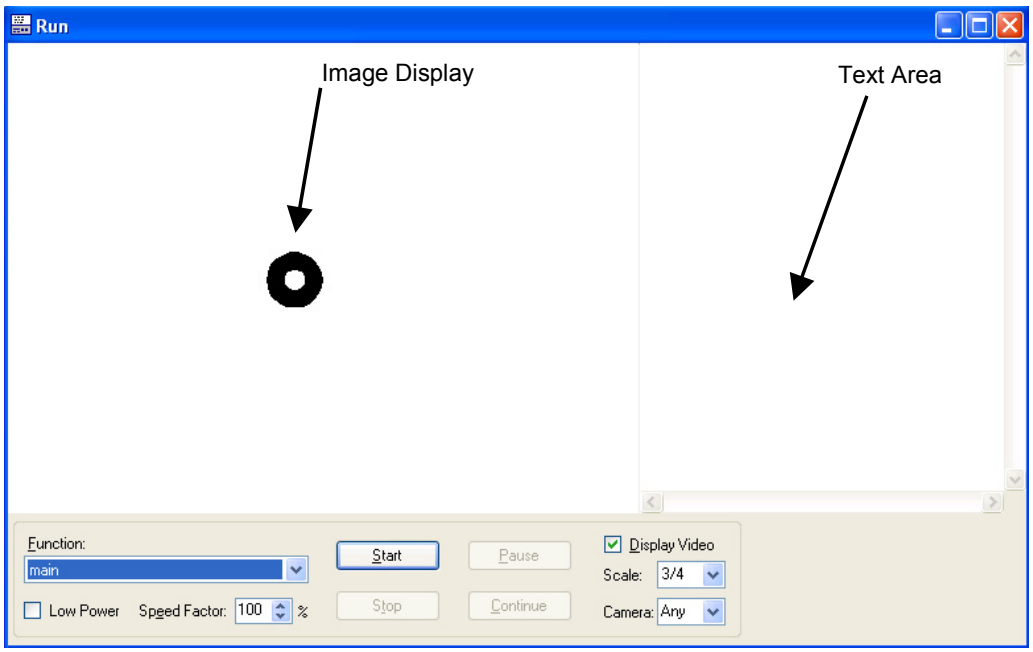


Figure 9: Run window with Image Display and Text Area

(2) Click on the Start button located at the bottom left corner of the Run window. This will cause the function called "main" to run.

(3) You can see sample results in Figure 10 from running the function "main". Notice that when the blob is found, it is highlighted in green in the image display on the left side of the Run window. The right side of the Run window displays text that states that the blob was found and the area of the blob.

(4) Double click the Control-menu box of the Run window that is located in the upper left corner of the Run window. This will close the Run window.

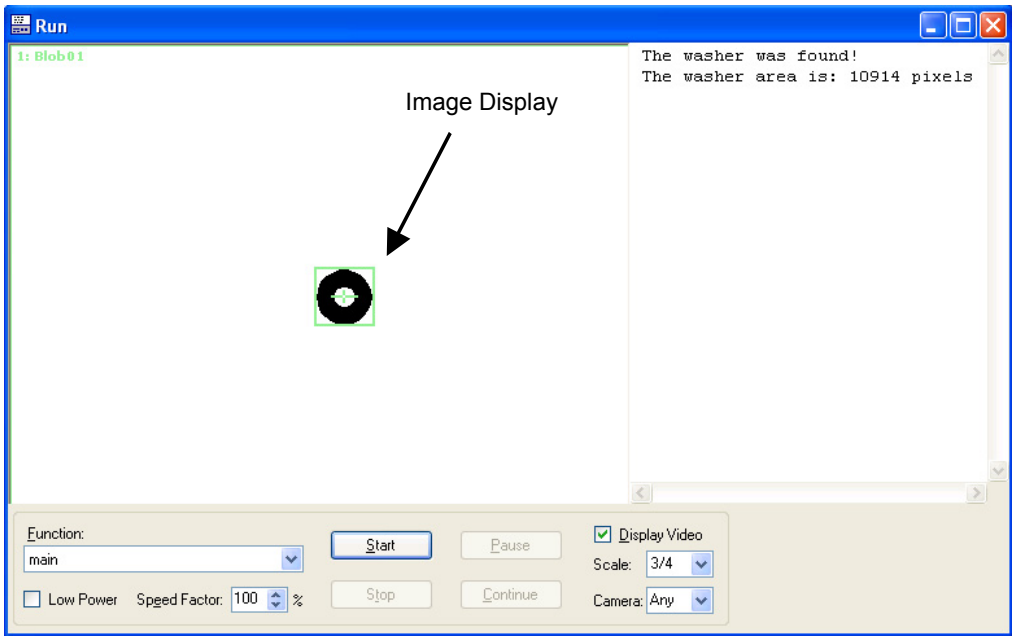


Figure 10: A sample Run window display after running "main"

3.8 Calibrating the Robot Camera

This part of the tutorial will guide you through how to calibrate the robot with the mobile camera that is mounted to the end of joint 2 on a SCARA. Calibration provides the mechanism to allow the vision system to understand the robot's world coordinate system. Once calibrated, the vision system can be used to find the robot coordinate position of parts for the robot to pick up. However, before we start calibration we need to prepare the robot for use by turning the motors on and calibrating (homing) the robot.

Step 1: Turn On Motors

(1) Click on the Robot Manager toolbar button on the EPSON RC+ main toolbar. You will see the following dialog appear.

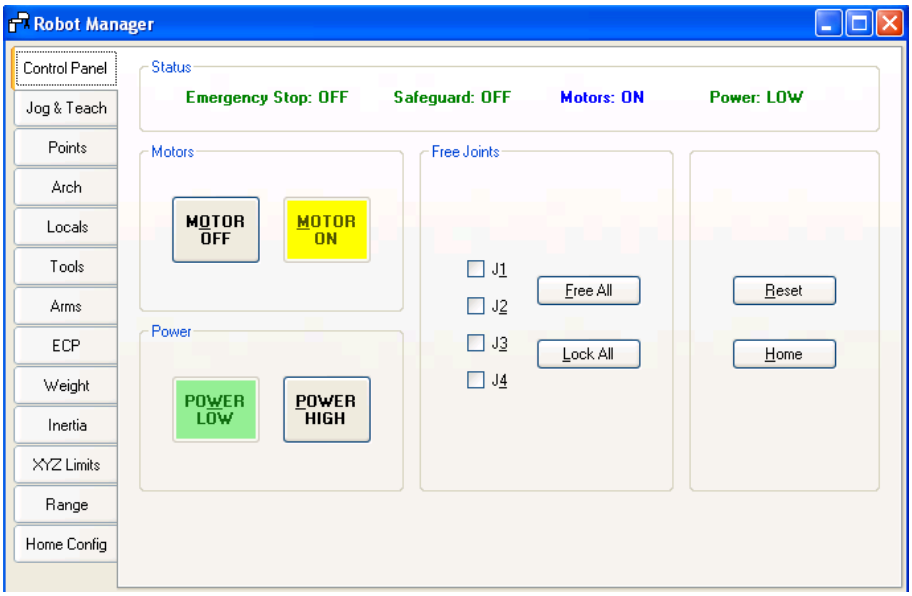


Figure 11: The Robot Control Panel

(2) Click the ON button in the Motor Group of the Robot Control Panel.

(3) A message box will appear asking if you are "Ready to turn robot motors ON". Click the Yes button. This will turn the robot's motors on.

Step 2: Use the Tool Wizard to create a new tool

In order to use the robot with a vision system for a guidance application requires that precise measurement of the position of the tool or gripper mounted at the end of the arm. Some tools are offset from the center of the Z-axis flange and other tools are mounted in line with the center of the Z-axis. Regardless of how careful we are when mounting robot tooling, you can almost always guarantee that there will be some offset from the center of the Z axis. For this reason we must use the Tools feature of SPEL+ to compensate for this adjustment.

A good understanding of tools is required to use Vision Guide for robot guidance. We will use the Tool Wizard feature to create a tool for our calibration rod (gripper).

(1) Click on the Tools page.

(2) Click the Tool Wizard button.

(3) For the first point in the Tool Wizard, jog the robot to the target position (position of washer on your paper target) so that the rod or gripper is located exactly in the center of the washer. You will need to jog the robot downward so that you are close to touching the paper but don't quite touch the paper. (Anywhere from 5-10 mm space above the paper will be fine. You just need to jog as close to the paper as is required to be able to see the rod (or gripper) well enough to center it on the washer.

(4) For the second point in the Tool Wizard, jog the U axis approximately 180 degress, then jog the rod (or gripper) until it is centered over the washer.

(5) Click the Finish button to define the new tool and exit the Tool Wizard. We have just defined the parameters for Tool 1.

The new tool values for Tool 1 for our Tutorial will appear as shown in Figure 12.

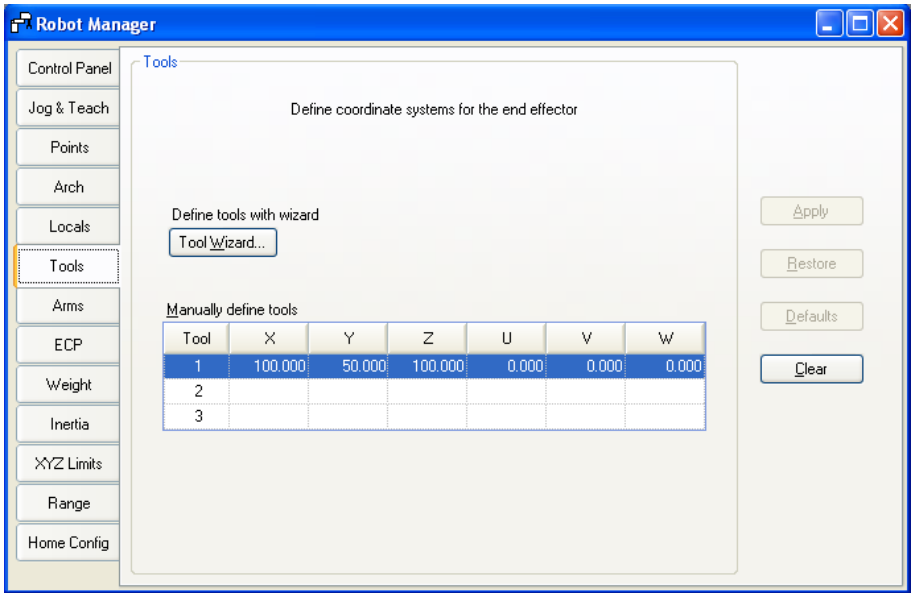


Figure 12: The Tools Tab (for defining tools)

Step 3: Test the New Tool

You should now be back in the Jog and Teach Window. And since we haven't jogged the robot since we positioned it above the target position (the washer), the robot's gripper (or rod) should still be located just above the target position.

(1) Jog the robot about 10-15 mm up in the Z direction (+Z). This will get our gripper (or rod) away from the work surface just in case there is a problem.

(2) Click on the down arrow for the Tool drop down list which is located in the upper right corner of the Jog and Teach Window.

(3) Click on "1" since we want to select Tool 1. (Remember this was the tool we just taught in Step 4.)

(4) Now try jogging the U axis in the positive or negative direction. You should see the tip of the gripper (or rod) stay in its' current XY position while the robot rotates around this point. (This is easily seen for tools that are offset from the center of the Z-axis. If you are using a rod that comes straight down from the Z-axis, this may be difficult to see.

(5) You should expect some movement of the gripper (rod) from the XY position when the U axis is moving. However, the gripper should return to the XY position of the target (the washer) after each U axis jog step is complete.

(6) If your tool doesn't appear to be working properly, go back to the beginning of step 2 and try again.

(7) Close the Robot Manager. We are now finished defining and testing the tool.

Step 4: Starting the Camera Calibration Process

We are now ready to calibrate the robot with the mobile camera.

(1) Click on the Vision toolbar button  on the EPSON RC+ toolbar to bring the Vision Development Window back to the front position on the screen.

(2) Click on the New calibration toolbar button  on the Vision Guide window. This will open the New Calibration dialog.

(3) Type in the name "downcal" without the quotes and click the OK button.

(4) The orientation will be for a mobile camera mounted on robot joint 2 so change the CameraOrientation property to Mobile J2.

(5) Change the RobotTool property to 1. This will select Tool 1 to be used when teaching the reference point for the calibration.

(6) Set the TargetSequence property to blobtut. We will use this sequence for finding the parts during calibration.

Step 5: Teaching the Calibration Points

(1) Click the Teach Points button located at the bottom of the Calibration tab.

(2) The Teach Calibration Points dialog will come up as shown below:

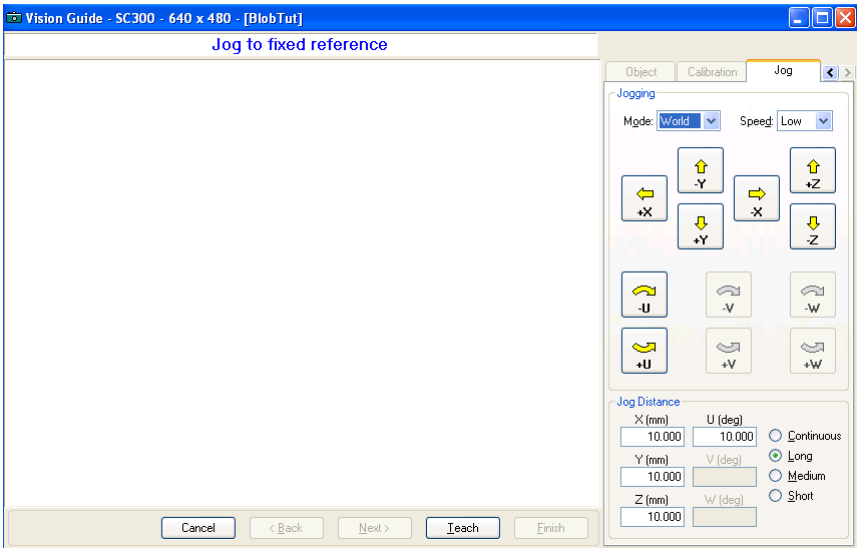


Figure 13: Teach Calibration Points dialog

(3) Notice the message shown at the top of the window that says "Jog to fixed reference". This means to jog the robot so that the gripper (rod) is centered on the calibration target part (in this case the washer). At this point you can ignore the video display, since we are aligning the robot gripper with the calibration target. Go ahead and jog the robot so that the gripper is positioned at the center of the washer. The positioning of this is very important so you will need to jog the robot down close to the washer to get a good alignment.

(4) Once you have the robot gripper positioned at the center of the washer, click the Teach button located at the bottom of the Vision Guide window.

(5) Jog the Z-axis up high enough to keep the gripper away from the work surface and any possibility to bumping into other objects during calibration because the robot gripper was too low. When we teach each of the 9 calibration points this Z height will be used for each.

(6) You will now see a message at the top of the Vision Guide window that requests that you "Jog to top left camera position", as shown in Figure 14. This means to jog the robot (and camera) to a position where the washer can be seen in the upper left corner of the image display of the Vision Guide window. Figure 14 shows the approximate position where you should jog the robot. As you can see in Figure 14, this is the 1st of 9 camera positions required for calibration.

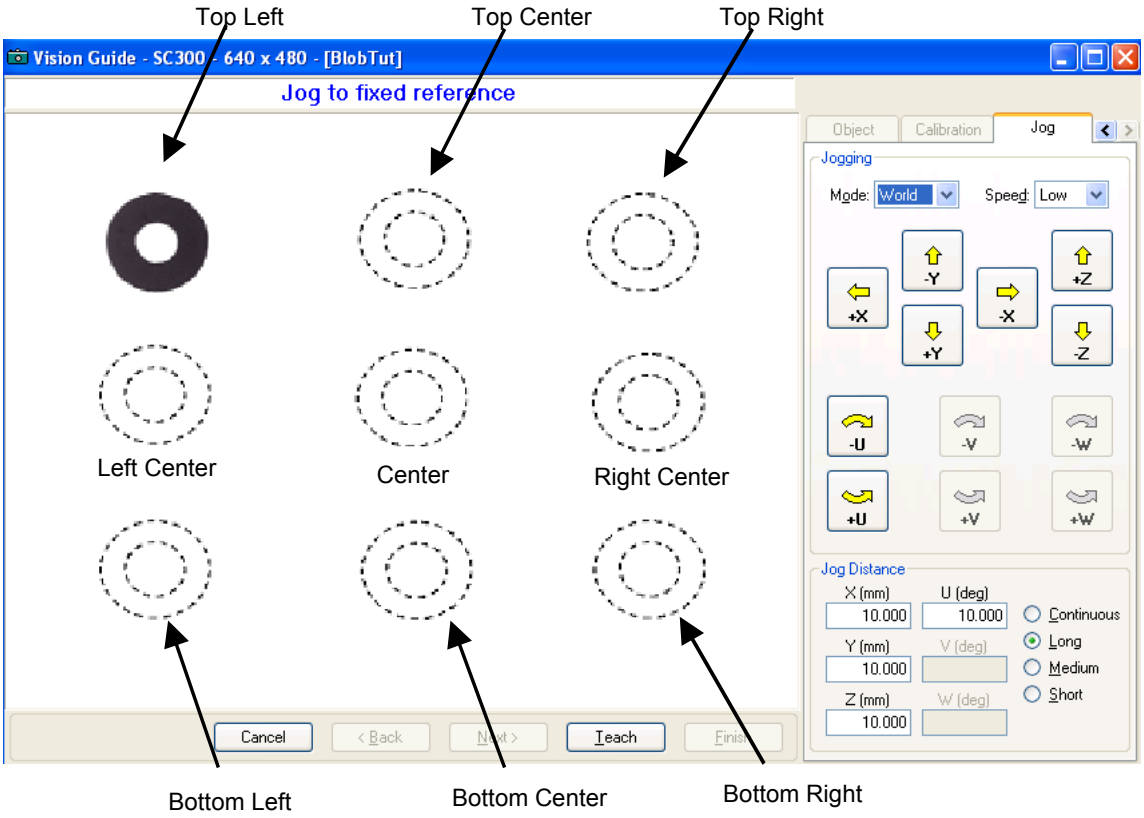


Figure 14: Teaching camera positions for calibration

(7) **Teaching the 1st Camera Position:** Jog the robot so that the camera is positioned where you can see the washer in the top left corner position of the image display of the Vision Guide window. Refer to the "top left" position of the washer in Figure 14.

(8) Click the Teach button on the Vision Guide window.

(9) **Teaching the 2nd Camera Position:** Jog the robot to the right of the 1st camera position so that the camera is positioned where you can see the washer in the top center position of the image display of the Vision Guide window. Refer to the "top center" position of the washer in Figure 14.

(10) Click the Teach button on the Vision Guide window.

(11) **Teaching the 3rd Camera Position:** Jog the robot to the right of the 2nd camera position so that the camera is positioned where you can see the washer in the top right position of the image display of the Vision Guide window. Refer to the "top right" position of the washer in Figure 14.

(12) Click the Teach button on the Vision Guide window

(13) **Teaching the 4th Camera Position:** Jog the robot down from the 3rd camera position so that the camera is positioned where you can see the washer in the center right position of the image display of the Vision Guide window. This position will be below the 3rd camera position starting a zigzag pattern when you refer to all the camera positions in order. Refer to the "right center" position of the washer in Figure 14.

(14) Click the Teach button on the Vision Guide window.

(15) **Teaching the 5th - 9th Camera Positions:** Continue jogging the robot and teaching points as instructed at the bottom of the Vision Guide window for each of the 5th through 9th camera positions. The Vision Guide window will display which position to move to for each camera position. Camera positions 5-9 are described below:

```
5 - center
6 - left center
7 - bottom left
8 - bottom center
9 - bottom right
```

(16) After the last camera position is taught a dialog will appear which says "All calibration points are now taught". Click the Finish button to continue.

Teaching the points required for Calibration is now complete.

Step 6: Running the Camera Calibration

Shown below are the final steps required to complete the camera calibration process:

(1) Click the Calibrate button on bottom of the Vision Guide window Calibration tab.

(2) A message box will appear with the message "CAUTION Robot will move during calibration at Speed 10, Accel 10 Continue?" Click on the Yes button to continue calibration. The robot will move through each of the 9 points 2 times during calibration. If you need to abort the calibration while the robot is moving around, click the Abort button at the bottom of the Calibration Cycle dialog.

(3) After the calibration cycle is complete, the Calibration Complete dialog appears and displays the calibration result data. Examine the data and then click the OK button.

(4) Notice that the calibration results are displayed in the Results list on the Calibration tab.

Step 7: Assign the "downcal" Calibration to the blobtut Sequence

Now that the "downcal" calibration has been created, we will need to assign this calibration to our vision sequence (blobtut). This will give the blobtut sequence the ability to calculate results in Robot and Camera world coordinate values.

(1) Click on the Sequence tab on the Vision Guide window to bring the Sequence tab to the front.

(2) Set the Calibration property to "downcal" by clicking once on the value field of the Calibration property, then clicking on the down arrow, and finally clicking on the calibration displayed which is called "downcal". This was the calibration we just finished creating.

(3) Click the Jog & Teach tab on the Vision Guide window. To see this tab, you may have to click the right arrow in the upper right corner of the tab control.

(4) Use the jog buttons to position the camera over the washer so that you can see the washer within the image display.

(5) Click on the Object tab.

(6) Click on the Run Object button to run the "washer" object. When the part is found look at the Result list. You will see that the CameraX, CameraY, RobotX, RobotY and RobotU results no longer show a [no cal] result. You can now see coordinate position data with respect to the Robot and Camera coordinate systems.

3.9 Teaching Points for Vision Guidance

Now we must teach a few points to define the Z height of the washer pickup position, the position where the camera will take a picture of the washer for processing, and a safe position that for this tutorial will act as a starting position.

Step 1: Define the "camshot" Position

The "camshot" position is the position where the robot must be located so that the camera is positioned over the part in a way that allows the part to be seen by the vision system. The robot must be moved towards the part until the washer can be seen in the image display on the screen. It is a good idea to position the robot so that the washer is located in the middle of the image display and not close to one of the edges of the search window. Since we just finished running the washer object, the camera should be in a good position for acquiring an image that contains the washer.

(1) Click on the Robot Manager button on the main tool bar, then click on the Jog & Teach page.

(2) Since we want the camera shot position to be taught using Tool 1, check the drop down list box labeled Tool to make sure that it is set to 1. If it is not set to 1, then go ahead and click the arrow on the drop down list box and set the Tool to 1.

(3) Verify that the current point in the Point # field is P0. We want to teach the camshot position as point P0. If it is not point P0, then select P0 in the point # field.

(4) Click on the Teach button on the Jog & Teach window. You will be prompted for a label. Enter the name "camshot". This will teach the "camshot" position.

Step 2: Define a Safe Position Away from the Washer

We will need a taught point position away from the washer that will be used as a safe position that we will move to at the start of the program.

(1) Select P1 in the point field.)

(2) Jog the Z-Axis of the robot up and then jog in X and Y to position the robot in a safe position. This will be like your start position for your program. The robot will always move to this position first before moving to the washer.

(3) Click on the Teach button on the Jog & Teach page. Enter "safept" for the label. This will teach the "safept" position.

Step 3: Calculate a Z Height for the "washer" Pickup Position

If we were doing an actual application to pick up a real washer, rather than moving to a drawing of a washer, we would need to set a Z height for the washer pickup position. Let's find a good Z height for our washer position.

CAUTION	■ Be sure to set the Z height for robot motion carefully. If the calculated Z height is incorrect, it may cause a malfunction of the system and/or safety problems.

(1) Click on the Robot Manager button on the main EPSON RC+ toolbar. Then click on the Jog & Teach page.

(2) Use the Jog Buttons to position the gripper of the robot about 5-10 mm above the washer.

(3) Write down the current Z coordinate value when the robot gripper is just above the washer. This z coordinate will be used in our program later to move the robot to this height.

(4) Select P2 in the point field.)

(5) Click the Teach button on the Jog & Teach window. Enter the label name "washpos". This will teach our initial "washpos" position. (However, the Vision System will be used to calculate a new X and Y position and then move to this point. We will also set a fixed Z height in our program based on the current Z coordinate position.)

3.10 Using Vision and Robot to Move to the Part

Now all that's left is to modify our program to cause the vision system and robot to work together to find the position of the washer and then move to it.

Step 1: Modify the SPEL+ Program

(1) Click on the Open File toolbar button on the EPSON RC+ toolbar.

(2) The MAIN.PRG program file should already be highlighted since you still haven't created any other programs. Click on the Open button at the bottom left of the dialog to open the MAIN.PRG program file. You should see the following program that we ran earlier in this tutorial.

```
Function main
  Real area
  Boolean found

  VRun blobtut
  VGet blobtut.washer.found, found

  If found = TRUE Then
    VGet blobtut.washer.area, area
    Print "The washer was found!"
    Print "The washer area is:", area, "Pixels"
  Else
    Print "The washer was not found!"
  EndIf
Fend
```

(3) Now modify the program so that it appears as shown on the next page.

NOTE

Remember that the SPEL+ language uses apostrophes ' to denote comments. Any characters that follow an apostrophe are considered comments and are not required for the program to execute properly. (This means that you can omit any characters that follow an apostrophe out to the end of the line where the apostrophe exists.)

```
Function main

  '*****************************************
  ' Very important statement below: Use the   *
  ' Z height which you wrote down earlier in  *
  ' Step 3 of "Teaching Robot Points for use  *
  ' with Vision Guidance.  Substitute the     *
  ' Z height value (a negative number) which  *
  ' you wrote down in the place of the xx     *
  ' shown below.                              *
  '*****************************************
  #define ZHeight -xx

  Real area, x, y, u
  Boolean found
  Integer answer
  String msg$, answer$

  Power Low           'Run robot at a slow speed and accel
  Tool 1              'Use Tool 1 for positioning
  Jump safept         'Move robot to safe start position

  Do                  'Continue looping until user stops
    Jump camshot      'Move robot to take picture
    VRun blobtut      'Run the vision sequence blobtut
    VGet blobtut.washer.RobotXYU, found, x, y, u

    If found = TRUE Then
      VGet blobtut.washer.area, area
      Print "The washer was found!"
      Print "The washer area is: ", area, "Pixels"
      washpos = XY(x, y, ZHeight, u)  'Set pos to move to
      Jump washpos
      msg$ = "The washer was found!"
    Else
      msg$ = "The washer was not found!"
    EndIf
    msg$ = msg$ + Chr$(&HB) + Chr$(10) + "Run another cycle(Y/N)?"
    Print msg$
    input answer$
    If Ucase$(answer$)<> "Y" Then
     Exit Do
   EndIf
  Loop
Fend
```

Step 2: Run the Program to Find the Washer and Move to it

(1) Click on the Run toolbar button on the EPSON RC+ main toolbar. This will cause the program to be compiled and then open the Run window.

(2) Click on the Start button on the Run window.

(3) Your program will now find the washer and move the robot to it. After you successfully find the washer, try moving the washer a little and then click the Yes button in the dialog that asks if you would like to run another cycle. If the washer is not found, a different dialog will appear asking whether or not you would like to try again. Clicking NO in either dialog will cause the program to stop running.

4. The Vision Guide Environment

4.1 Overview

In this chapter we will focus on some concepts and definitions so you can gain a complete understanding of Vision Guide and its components. We will cover the following topics:

- Basic definitions which you should understand to use Vision Guide.
- How to open the Vision Guide window.
- An explanation of the Vision Guide video display.
- An introduction to all the vision objects and the buttons on the toolbar to manipulate them.
- An introduction to properties and results.
- An explanation of the Sequence, Object, and Calibration tabs
- How to execute a vision object or a vision sequence and an explanation about when to use each.

4.2 Basic Concepts Required to Understand Vision Guide

A quick explanation of some of the basic concepts will help you understand this chapter much better. Please review the concepts described below before proceeding through the rest of this chapter.

What is a Vision Sequence?

A vision sequence is a grouping of vision objects in a specific order that can be executed from the Vision Guide window or from the SPEL+ language. A vision sequence has specific properties that are used to set up vision sequence execution. For example, the Camera property defines which camera will be used to acquire an image for this vision sequence and the RuntimeAcquire property defines how the image will be acquired for this vision sequence.

A vision sequence can be thought of as a type of container that holds all the necessary vision objects required to solve a specific Vision process or part of a process. In general a vision sequence is the starting point for all Vision Processing.

What is a Vision Object?

A vision object is a vision tool that can be applied to an image that is captured by the camera. Some of the vision objects supported are: Correlation Search, Blob Analysis, Polar Search, Edge Detection, Line Creation, Points, and Frames. All vision objects can be executed (applied to the current Image) and all return results such as how much time the vision object took to execute, position information, angle information, and whether or not the vision object was even found. Vision objects have properties that are used to define the characteristics of how the vision object will perform. They also have results that are the values that are returned after a vision object is executed.

What is a Property?

Properties can be thought of as parameters that are set for each vision object or vision sequence. The setting of vision properties can be done in a point and click fashion from the Vision Guide window that provides for a quick method to build and test vision applications. Vision properties can also be set and checked from the SPEL+ language. This provides the flexibility required for dynamic modifications of vision objects at runtime. Vision sequence and vision object properties are very powerful because they are easily modified and help make vision objects easier to understand and use while at the same time they don't limit the flexibility required for more complex applications.

What is a Result?

Results are the values that are returned from vision objects or vision sequences after they are executed. Examples of commonly used results include:

Time - returns how much time the vision object or sequence took to execute

RobotXYU - returns the X, Y, and U position of the found feature in robot coordinates.

Found - returns whether or not the vision object was found

Vision results can be seen from within the Vision Guide window on the Sequence and Object tabs. SPEL+ programs can also use vision results.

What is a Runtime Vision Command?

A series of Vision commands have been added to the SPEL+ robot language to provide a seamless integration of robot motion and vision guidance. Commands such as VRun allow a user to initiate a vision sequence from the SPEL+ language with just 1 function call. VGet allows a user to get the results that are returned from vision objects and Sequences. This is very powerful because vision sequences are created, modified and maintained from within the Vision Guide point and click development environment but all things created within Vision Guide are also accessible from the SPEL+ language.

How does Vision Guide Work in EPSON RC+ Projects?

EPSON RC+ is based on Projects that are containers for all the necessary programs, teach points, and robot settings required for a specific robot application. This Project configuration makes it easy to use one robot for multiple projects or test environments to check out new ideas without destroying your old working applications.

When you create a EPSON RC+ application which includes Vision Guide, all the associated vision sequences and vision objects required for that application are kept within the project along with the other items normally contained in projects. This ensures that when you open an existing project, everything relating to this project is available to you.

4.3 Coordinate Systems

The section describes the coordinate systems used in Vision Guide.

Image Coordinate System

The image coordinate system is used for the camera video buffer and video display. Units are in pixels.

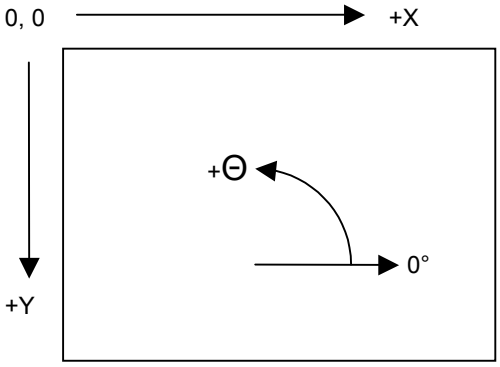


Figure 15: Image Coordinate System

Camera Coordinate System

The camera coordinate system contains physical coordinates in the camera field of view. Units are in millimeters. To obtain coordinates in this system, a camera must be calibrated using any of the camera orientations.

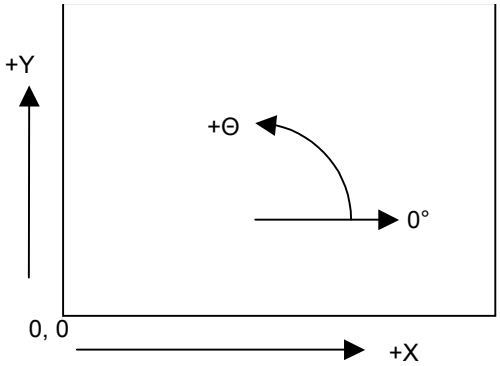


Figure 16: Camera Coordinate System

Robot Coordinate System

The robot coordinate system contains physical coordinates in the robot world. Units are in millimeters. To obtain coordinates in this system, a camera must be calibrated using camera orientation in the robot world.

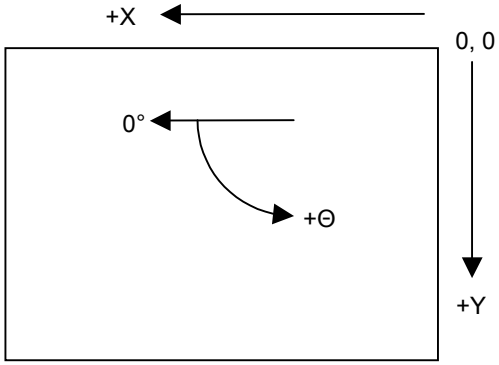


Figure 17:Robot Coordinate System

4.4 Open the Vision Guide Window

The Vision Guide window is opened from within the EPSON RC+ development environment. After EPSON RC+ has been started, the Vision Guide window can be opened in 2 different ways:

From the main Toolbar: From the main toolbar in EPSON RC+ you should see the Vision Guide icon  . Clicking on the Vision Guide icon will open the Vision Guide window.

From the Tools menu: Selecting Vision from the Tools menu will open the Vision Guide window.

Once the Vision Guide window is open we can now begin using Vision Guide. The next few pages describe the basic parts of the Vision Guide window.

4.5 The Parts of the Vision Guide Window

The Vision Guide window is the point and click environment where most of your vision development will take place. An understanding of this window and its major parts is the first thing you will need to grasp in order to use Vision Guide. The Vision Guide window can be broken into the following major sections:

- Title Bar
- Toolbar
- Image Display
- Sequence, Object and Calibration Tabs
- Run Panel

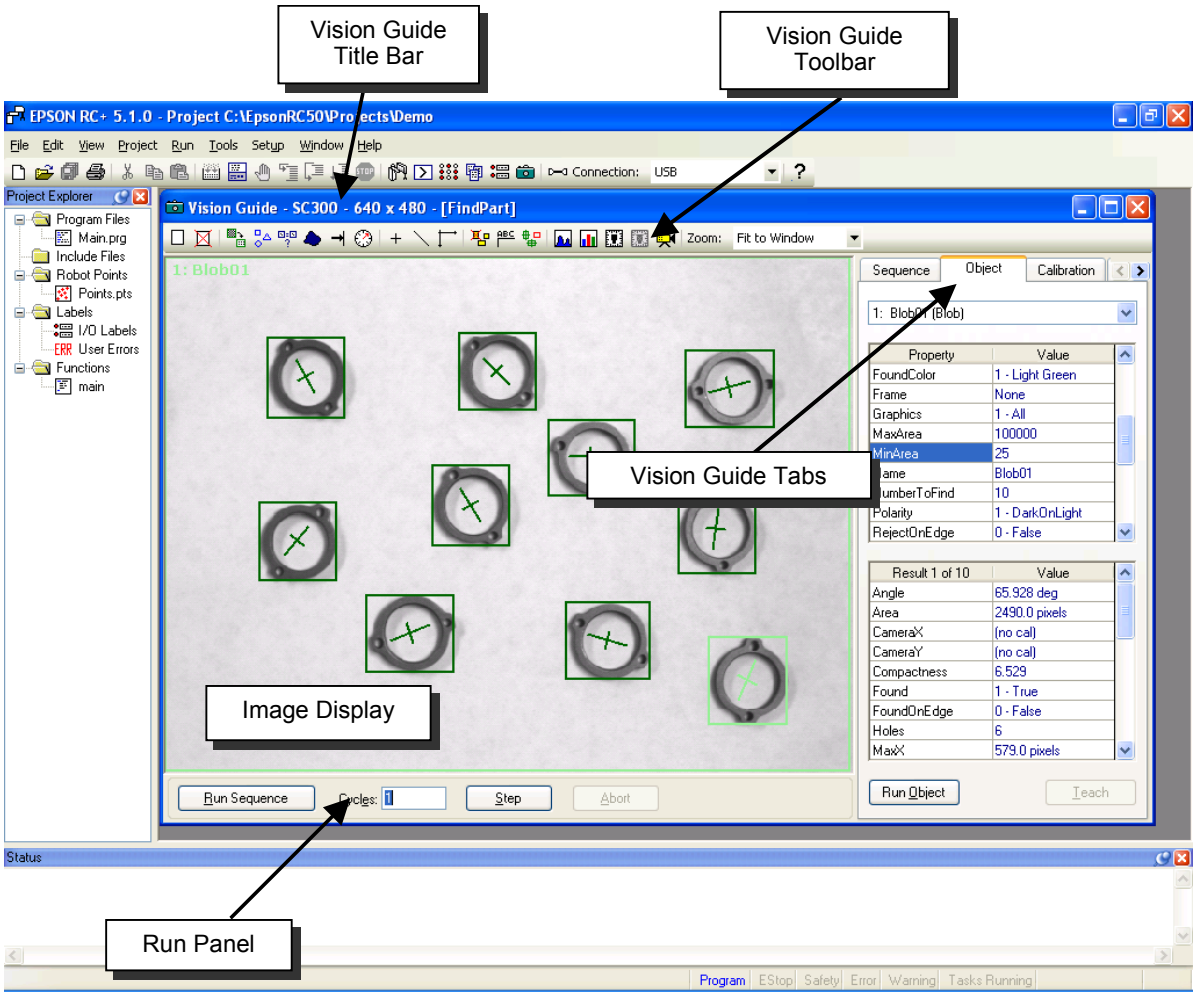


Figure 18: The Vision Guide window

4.5.1 The Title Bar

The Vision Guide window has a title bar that includes the title of the window, the camera type and resolution, and the name of the current sequence. The vision sequence name is put in brackets. A sample title bar is shown below. It shows that we are running Vision Guide with an SC300 Fixed Smart Camera and currently using the "blobtut" vision sequence.

NOTE

It is important to understand the difference between the main title bar for the EPSON RC+ environment and the Vision Guide title bar. The EPSON RC+ title bar contains the name of the current project while the Vision Guide window title bar contains the name of the current sequence.

4.5.2 The Toolbar

Toolbars are typically included with Microsoft WindowsTM applications because they provide quick access to many of the more common features of the product. Both EPSON RC+ and Vision Guide make use of toolbars. However, since we are learning about Vision Guide, let's remain focused on the Vision Guide Toolbar.

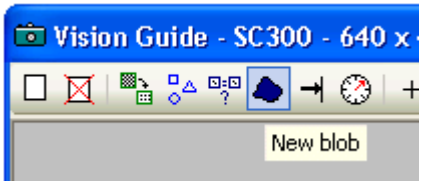
The Vision Guide Toolbar is located at the top of the Vision Guide window just below the Title Bar and appears as follows:

Figure 19: The Vision Guide Toolbar

Notice that the toolbar buttons for the Vision Guide window are separated into small groups. This is done to make them easier to find and use. A general description for each of the groups of toolbar buttons is as follows:

- The first two toolbar buttons are grouped together because they are used for creating and deleting vision sequences.
- The next group of toolbar buttons is used for creating the primary vision objects: ImageOp, Geometric, Correlation, Blob, Edge, and Polar.
- The next three toolbar buttons are used for creating the construction vision objects: Points, Lines, and Frames. We call them construction objects because they are used to tie the other vision objects together.
- The next toolbar buttons that are grouped together can be thought of as utilities for the Vision Guide Environment. They include Delete Object, Force All Labels Off, and Force All Graphics On.
- The next two toolbar buttons are for opening the Histogram and Statistics dialogs.
- The next two buttons are for creating and deleting calibrations.
- The last button is for switching between live and frozen video.

The Vision Guide toolbar includes tool tips for each toolbar button like the one shown here.



To see the description for a specific toolbar button, move the mouse pointer over the toolbar button and after about 2 seconds you will see the tool tip for that specific toolbar button.

Shown below are general descriptions for each of the Vision Guide toolbar buttons.

Button	Description
	New sequence: Creates a new vision sequence. A dialog pops up and the user is asked to enter the name for the new sequence.
	Delete sequence: Deletes a vision sequence in the current project. This button is dimmed if there are no vision sequences for the current project.
	New image operation: Creates a new ImageOp object. This button is dimmed if there are no vision sequences for the current project.
	New geometric: Creates a new Geometric object. This button is dimmed if there are no vision sequences for the current project.
	New correlation: Creates a new Correlation object to be placed in image display. This button is dimmed if there are no vision sequences for the current project.
	New blob: Creates a new Blob object to be placed somewhere in the image display. This button is dimmed if there are no vision sequences for the current project.
	New edge: Creates a new Edge object. This button is dimmed if there are no vision sequences for the current project.
	New polar search: Creates a new Polar object. This button is dimmed if there are no vision sequences for the current project.
	New point: Creates a new Point object. This button is dimmed if there are no vision sequences for the current project.
	New line: Creates a new Line object. This button is dimmed if there are no vision sequences for the current project.
	New frame: Creates a new Frame object. This button is dimmed if there are no vision sequences for the current project.
	Delete object: Deletes the current active vision object. To delete a vision object select the desired vision object and then click on this button. This button is dimmed if there are no vision sequences for the current project or if there are no vision objects for the current vision sequence.
	Force all labels off: When pressed in, this button causes the labels on the vision objects to be removed. This feature is useful when you have many

Button	Description
	vision objects close together and it's difficult to distinguish between them. This button is dimmed if there are no vision sequences for the current project.
	**Force all graphics on:** When pressed, this button causes all graphics (search window, model origin, model window, Lines, and Labels) for the vision objects to be displayed. This button overrides the setting of the Graphics property for each individual vision object making it easy to quickly see all vision objects rather than modifying the Graphics property for each vision object individually.
	**Histogram:** Clicking on this button opens the Histogram dialog. This button is dimmed if there are no vision sequences for the current project or if there are no vision objects for the current vision sequence.
	**Statistics:** Clicking on this button opens the Statistics dialog. This button is dimmed if there are no vision sequences for the current project or if there are no vision objects for the current vision sequence.
	**New calibration:** Opens the New Calibration dialog to create a new calibration. This button is dimmed if there are no vision sequences for the current project.
	**Delete calibration:** Opens the Delete Calibration dialog to delete a calibration. This button is dimmed if there are no vision calibrations for the current project.
	**Freeze image:** Toggles between live and frozen image.

4.5.3 The Image Display

The image display is located directly beneath the Vision Guide Toolbar. This is the area of the Vision Guide window where the image brought in through the camera (or from disk) is displayed. This image is overlaid on the Vision Window to fit within the area we call the image display. The image display is also used to display graphics for each of the vision objects that are used to process the images. For example, you may see boxes and cross hairs displayed at various positions in the image display. These graphics help developers use the vision object Tools. A picture of the Vision Guide window with the image display labeled is shown at the beginning of the section in this chapter called *4.5 The Parts of the Vision Guide Window*.

When high resolution cameras are used, the video display is scaled by 1/2 to fit the entire display on the screen.

4.5.4 The Vision Guide Window Tabs

The primary purpose of tabs is to provide quick access to data that is related or grouped together in some way. This proves much easier to use than additional menu items or multiple windows.

The Vision Guide window uses tabs to provide a single window vision development environment that makes the system easy to learn and remember. Located on the right side of the Vision Guide window is a set of four tabs that are labeled Sequence, Object, Calibration, and Jog. These tabs are positioned in a fixed location next to the image display. The details for each of the tabs are described later in this chapter.

4.5.5 The Run Panel

The Run Panel is located just below the image display. The purpose of the Run Panel is to run and step throuth sequences. The Run Panel is shown below:

Figure 20: The Run Panel

Once a vision sequence has been created and vision objects have been added, a vision sequence can be run by clicking on the Run Sequence button located on the left side of the Run Panel. The vision sequence will then execute as many times as specified in the Cycles input box. As the vision sequence is executing, you will see the number of cycles remaining displayed in the Cycles input box until no cycles remain. The Cycles input box has a yellow background when the number of cycles to execute is greater than 1.

If at any time you want to stop multiple vision sequence cycles, click the Abort button on the Run Panel. The Abort button is only enabled when a sequence is actually running.

The Step button allows for single stepping of a vision sequence, where each step will execute one vision object. Click on the Step button each time you want to execute the next step. The first time you click on the Step button, the vision objects are put in an inactive step mode where they are drawn with blue dashed lines. The next time you click the Step button, the first object in the sequence is executed, then the 2nd and so on until the last vision object in the sequence is executed. To reset the step execution back to the beginning, click on the Run Sequence button and run the entire vision sequence.

During step execution, you can see which vision object will execute next from the Step list in the Sequence Tab. The vision object that will execute on the next step is highlighted in blue.

To examine a specific object's result values during execution of a vision sequence, select the Object Tab and then select the vision object you are interested in. Once a vision object is selected, click the Run Sequence button on the Run Panel and the vision sequence will begin execution. When the Object Tab is displayed in front, you can see the results for the specific vision object that you selected.

If, after running the vision sequence, you would also like to examine other vision object's results select the vision object you are interested in by clicking on the drop down list box on the Object tab. You must select the vision object you are interested in via this drop down list and not by clicking on the object in the image display.

NOTE

The Run Sequence button provides a quick method to test and debug entire vision sequences without having to run them from SPEL+. You can make changes to a vision sequence, test it and if you don't like the results, you can revert back to the saved version of the vision sequence. For this reason the Run Sequence button does not automatically save your vision sequences each time you run a vision sequence. However, when you run the vision sequence from the Run window, the vision sequence is automatically saved along with the rest of the project if Auto Save is enabled.

4.6 Vision Guide Window Tabs

4.6.1 The Sequence Tab

The Sequence tab is used to:

- Select which vision sequence to work on.
- Set properties for the vision sequence.
- Examine the results for the entire vision sequence.
- Manipulate the execution order of the vision objects.

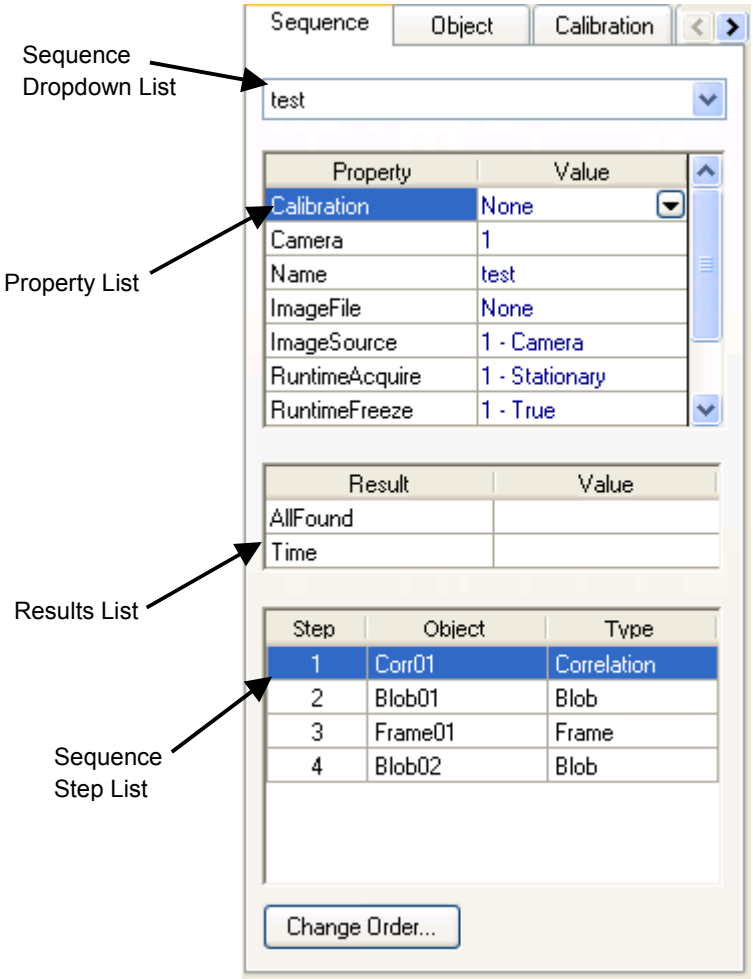


Figure 21: The Sequence Tab

Sequence Tab: Selecting a sequence

Notice the drop down box ("test" is the sequence name) shown at the top of the Sequence tab in Figure 21. Clicking on the down arrow displays a list of all the vision sequences for this project. (Some projects have only 1 vision sequence and others have many depending upon the vision application.) You can then click on any one of the vision sequences to select which one you want to work on.

Sequence Tab: Properties list

The properties for the vision sequence are shown in the Properties list that looks similar to a small spreadsheet. The name of the property is on the left side and the value is shown on the right. Vision properties are set by first clicking on the value field for a specific property and then either entering a value or selecting from a list which is displayed.

Some of the vision sequence properties include:

Camera Specifies which camera to use for this vision sequence.

RuntimeAcquire Defines the acquisition method for this sequence (i.e. strobed, none, stationary)

Calibration Defines which calibration to use for this vision sequence

For details on vision sequence properties, please see the chapter Vision Sequences or the *Vision Property and Results Reference Manual*.

Sequence Tab: Results List

The results for the vision sequence are shown in the Results list that is located just below the Properties list. The Results list will only display values in the value field after a vision sequence is executed. Prior to execution the result field will not display anything. The following results are displayed in the result column:

AllFound - displays whether or not all vision objects were found.

Time - the amount of time it took to process the vision sequence

Sequence Tab: Step List

Located at the bottom of the Sequence tab is the sequence Step list. This list displays all the vision objects that have been defined for this vision sequence. They are displayed in the order that the vision objects will execute.

4.6.2 The Object Tab

The Object tab is used to:

- Select which vision object to work with.
- Set property values for vision objects.
- Execute vision objects.
- View results after execution of vision objects.
- Teach models for vision objects that use models.

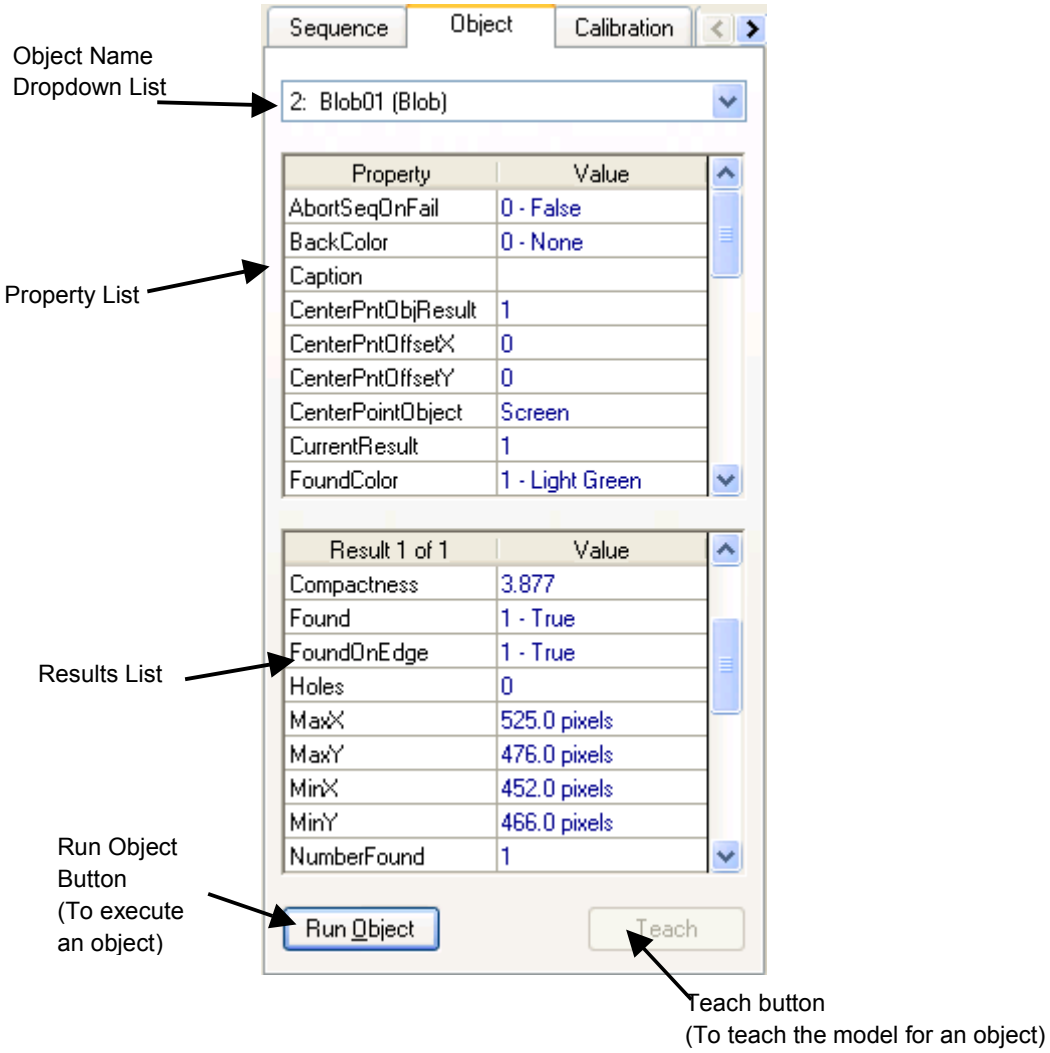


Figure 22: The Object Tab

Object Tab: Selecting an object

Notice the drop down list box located at the top of the Object tab. This drop down list box is used to select the vision object to display in the Object tab. Clicking on the down arrow displays a list of all the vision objects for the current vision sequence. You can then click on any one of the vision objects to select which one you want to work on or examine.

Selecting the current vision object to work with can also be accomplished by clicking on the label of a vision object or its search window as displayed in the image display.

Object Tab: Properties list

The properties for vision objects are shown in the Properties list that is similar to a small spreadsheet. The name of the property is on the left side and the value is shown on the right. Vision object properties can be set by clicking on the Value field for a specific property and then either entering a value or selecting from a list which is displayed.

Vision object properties vary depending upon vision object type but some of the more common vision object properties are:

AbortSeqOnFail - If set to TRUE, the entire vision sequence will abort if this object is not found during execution

Accept - The threshold level for comparing against the Score result. If the score is greater than the Accept property value, this object is considered found.

Frame - Defines which vision frame to apply to the positioning of this object.

For details on vision object properties, please see Vision Objects of this manual and the Vision Properties and Results Reference Manual.

Object Tab: Results list

The results for the vision object are shown in the Results list that is located just below the Properties list. The Results list will display values in the Value field after either this vision object or the entire vision sequence is executed. Prior to execution the Results list of a specific vision object may not display anything or it may display the results from the last time the vision object was run.

It is sometimes useful to display the Object tab when running an entire vision sequence. This allows you to see the results for a particular object each time you run the sequence.

Vision Guide allows you to run a vision sequence and then switch between vision objects to see the results for each individual vision object. You must select between the vision objects by using the vision object drop down list. If one of the vision objects is selected by clicking on the object in the image display, the object's results are cleared.

Vision Guide keeps the results available even which switching between vision objects.

The results vary depending upon vision object type but 2 of the most common vision object results are as follows:

Found - shows whether or not the vision object was found.

Time - the amount of time it took to process this vision object

Object Tab: Run Object and Teach Buttons

Located at the bottom of the Object tab are the Run Object and Teach buttons. The Teach button will be accessible only for objects that utilize models. After a vision object's model window is positioned and sized properly, the Teach button is clicked to teach the model for the vision object. Complete details for teaching vision objects is explained in Vision Objects.

The Run Object button is used to run the currently selected vision object. This is useful for testing the current vision object by itself to fine-tune the property values for that object.

If a vision object is associated with other vision objects (e.g. a Polar object whose center position is defined by the results of a Correlation object), then the required vision objects will execute first and then the current vision object will execute. For example, assume that we want to run a Polar object when that Polar object's CenterPointObject property is defined by the results of a Correlation object. In this case we select the Polar object as the current object for use in the Object tab. Then we click the Run Object button and the Correlation object will execute first, followed immediately by the Polar object.

NOTE

It is important to understand the difference between running a vision object and running a vision sequence. To run a vision object simply click on the Run object button on the Object tab. Make sure that the desired vision object is selected at the top of the Object tab before running. Running a vision object will execute only that vision object and any vision objects that may be required by that object. When executing a vision sequence, all vision objects within that sequence will execute.

4.6.3 The Calibration Tab

The Calibration tab is used for setting calibration properties and viewing calibration results.

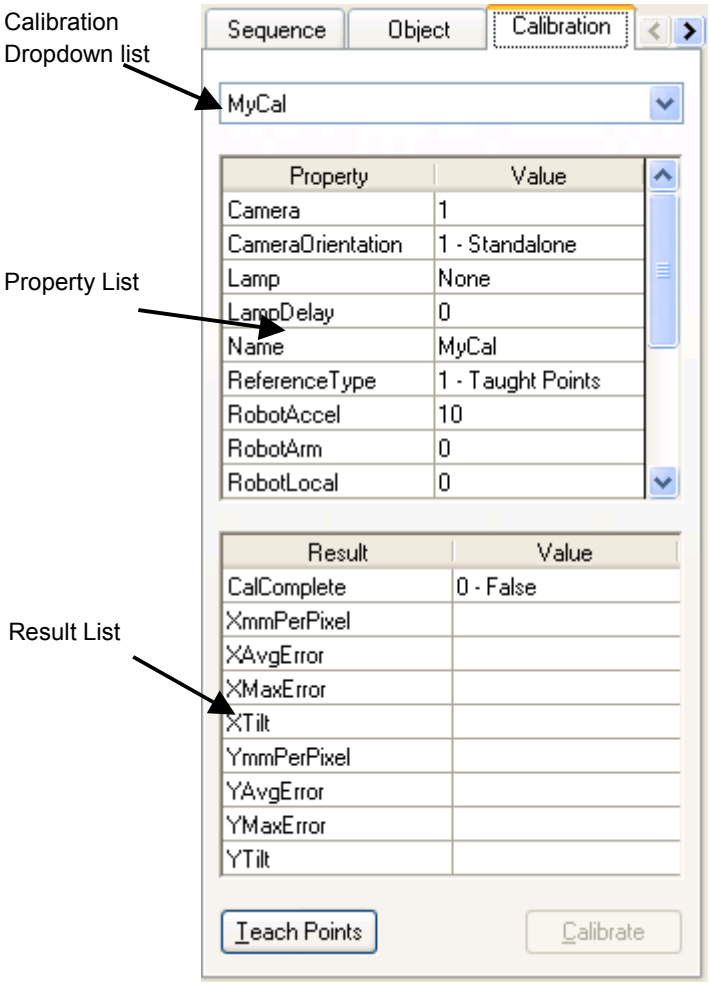


Figure 23: The Calibration Tab

Calibration Tab: Selecting a Calibration

Notice the drop down list shown at the top of the Calibration tab in Figure 23. Clicking on the down arrow displays a list of all the calibrations for this project. (Some projects have only 1 calibration and others have many depending upon the vision application.) You can then click on any one of the vision sequences to select which one you want to work on.

Calibration Tab: Properties list

The properties for the vision Calibration are shown in the Properties list, which looks similar to a small spreadsheet. The name of the property is on the left side and the value is shown on the right. Properties are set by first clicking on the Value field for a specific property and then either entering a value or selecting from a list which is displayed.

For information on Calibration properties see the chapter *Vision Calibrations* or *the Vision Property and Results Reference Manua*l.

Calibration Tab: Results list

The results for the Vision Calibration are shown in the Results list that is located just below the Properties list.

Calibration Tab: Teach Points button

The Teach Points button opens a wizard for teaching calibration points.

Calibration Tab: Calibrate button

The Calibrate button runs the calibration sequence.

For details on camera calibration, see the chapter Calibration.

4.6.4 The Jog Tab

The Jog tab is used for jogging the robot during calibration or while viewing live video.

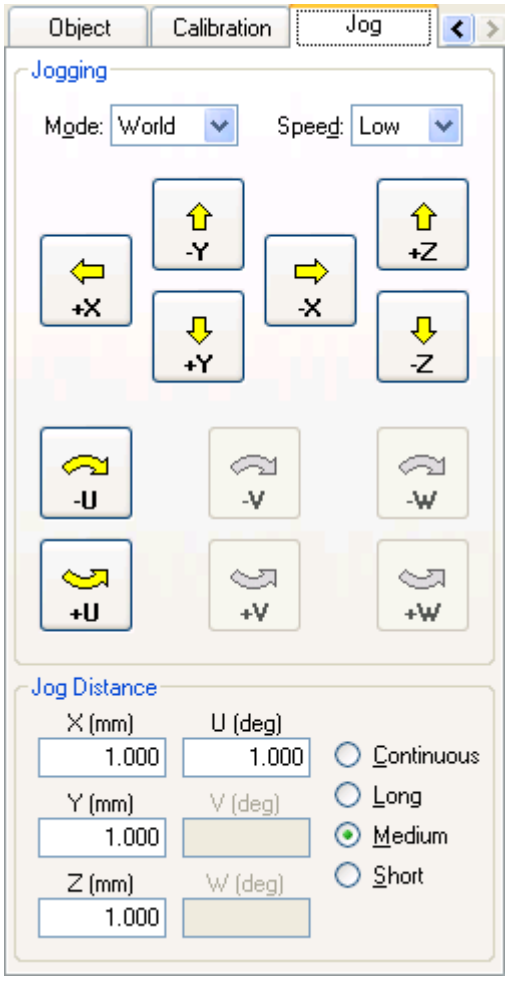


Figure 24: The Jog Tab

When the Vision Guide window is first opened, the Jog tab is not visible. Use the right arrow button in the upper right corner of the tab control to see the Jog tab, then click on the tab to select it.

When you select the Jog tab, communication is attempted with the robot Controller. If communication cannot be established, the previous tab is selected.

Before jogging, the robot motors must be turned on. You can turn on the motors from the Robot Manager or the Command Window.

Jog Tab: Selecting the Jog Mode

Select the jog mode from the Mode dropdown list. The choices available are World, Tool, Local, Joint, and ECP (if ECP is enabled). You can select the current Tool, Local, and ECP from the Robot Manager.

Jog Tab: Selecting the Jog Speed

Select the jog speed from the Speed dropdown list.

Jog Tab: Selecting the Jog Distance

Select the jog distance by clicking on the option button in the Jog Distance group. When Long, Medium, and Short distances are selected, you can change the distance by typing in the jog distance text boxes.

Jog Tab: Jogging the Robot

After selecting the jog mode, jog speed, and jog distance, click on the jogging buttons to jog the robot. When the jog distance is set to Continuous, the robot will move until you release the button. For the other distances, the robot will move the amount specified in the jog distance. If a jog button is held down, the robot will continue to move.

5. Vision Objects

5.1 Overview

Vision objects make up the heart of Vision Guide as they are the foundation upon which everything else is based. This chapter will provide the fundamental knowledge required to apply Vision Guide towards solving your vision applications. In this chapter we will cover the following items:

- Definition of a vision object and how it works.
- Explanation of the different types of vision object shapes.
- How to position and size vision objects.
- Explanation of what each vision object is used for.
- How to use each vision object.
- Review of properties and results for each vision object.
- Explanation of technical details for each vision object.
- Discuss utilities that are used with vision objects such as: histograms, statistics, vision object graphics manipulation.

5.2 Vision Object Definition

What is a Vision Object?

A vision object is a vision tool which can be applied to an image. At first glance many vision developers think vision objects are just another name for the algorithms that we support for Vision Guide. However, after using Vision Guide you will find that the vision algorithms are only one part of what defines a vision object. Vision objects are like containers that hold the information required to use a specific vision algorithm. The unique manner with which this data can be easily manipulated and examined (through properties and results) is what makes vision objects so powerful and allows users to develop new vision applications so much more quickly than with other vision systems.

Some of the vision object Tools supported include: Correlation Search, Blob Analysis, Polar Search, Edge Detection, Line Creation, Points, and Frames. All vision objects can be executed (applied to the current Image) and all return results such as how much time the vision object took to execute, position information, angle information, and whether or not the vision object was even found. Refer to each individual vision object explanation later in this section for more details.

5.3 Anatomy of a Vision Object

Before going into the details of vision objects, there are a few things you should know about vision object layouts and position/sizing methods used to manipulate them. When a new vision object is created and placed in the image display it is given specific visual characteristics that are important to understand. For example, a Correlation object is displayed with a search window, model window and model origin. All of which can be repositioned with simple dragging techniques. Shown on the next few pages are some of the basic vision objects and the methods to manipulate them.

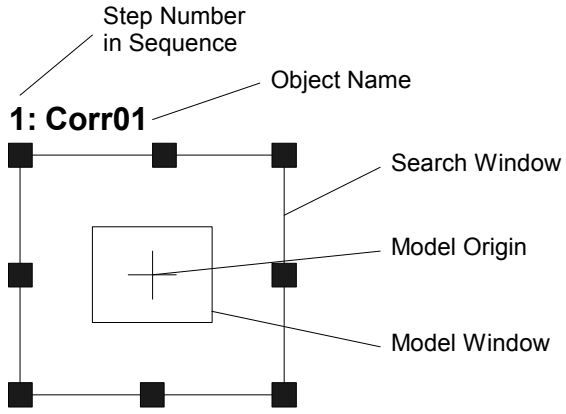


Figure 25: A Sample Vision Object Layout

5.3.1 The Search Window

Most vision objects have a search window as shown below. The search window is the outside box as shown in Figure 26 and is magenta (light purple) in color when active (the vision object you are currently working with) and blue when inactive (another vision object in the Sequence but not being worked with at the moment). At run time it changes color to green or red depending upon whether or not the object was found.

The search window is used to define the Region of Interest (or area within which we will search for a part or pattern). Moving the search window is easily accomplished by clicking down on the name of the vision object (or on the search windows itself) and then dragging it to the desired new position on the image display. ("Corr01" is the name of the vision object in the figure below.)

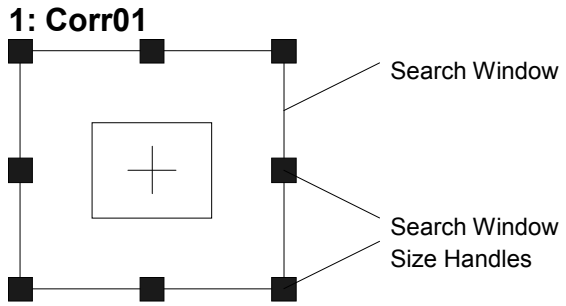


Figure 26: The Search Window

The size of the search window is very important when configuring a vision object. The larger the search window, the more time will be required to process the search. It is a good idea to make the search window large enough to handle the variation in part position

while keeping it as small as possible. Sometimes the use of a Frame object  (described in this chapter on page 162) can help reduce the size of search windows.

Positioning the Search Window

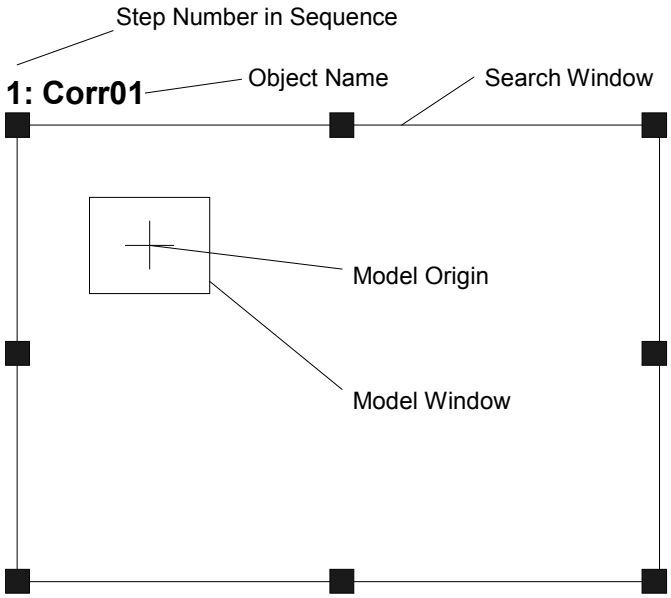
To position the search window, click on the name of the vision object, or the search window itself, and then drag the Vision object to whatever position you like. We recommend moving the vision object so that the top left corner is located in the proper position. Then use the search window Size Handles (as described in the next section) to resize the search window.

Resizing the Search Window

The search window size handles appear when a vision object is first created or when a user selects a vision object by clicking on that object. You can easily locate the search window size handles because they are the small boxes that are located on the sides and corners of the search window when the search window is selected.

Search window size handles are used to size the search window. When you move the mouse pointer over one of the search window size handles it changes into a double-ended arrow pointer. This indicates that when you click the mouse, you will be selecting the handle. You can click on any of the search window size handles and then drag the side or corner to resize the search window.

Clicking on a search window size handle located on one of the sides of the search window allows you to resize the object horizontally. Clicking on the top or bottom of the search window allows you to resize the object vertically. Clicking on one of the corner search window size handles allows you to resize the search window both horizontally and vertically at the same time.



Correlation Object After Resizing the Search Window

Figure 27: Search Window shown after resizing

CAUTION	■ Ambient lighting and external equipment noise may affect vision sequence image and results. A corrupt image may be acquired and the detected position could be any position in an object's search area. Be sure to create image processing sequences with objects that use search areas that are no larger than necessary.

5.3.2 The Model Window

Correlation, Geometric, and Polar objects contain a model window and a search window. We will use the Correlation object model window for the purpose of this discussion.

Correlation objects have a model window as shown in Figure 28. The model window is the inside box as shown in the Figure 28 and is magenta in color. (Except at run time where it changes to green when the object is found.) The model window is used for defining the boundaries of the model. (Remember that a model is an idealized representation of a feature. In general, it's what you will be searching for.)

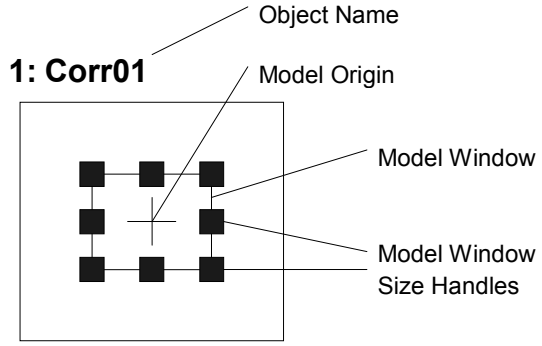


Figure 28: The Model Window

Positioning the Model Window

Since the size and position of the model is very important for teaching a new model, it is important to understand exactly how to set the position for and adjust the size of the model window. To position the model window, click on one of the 4 lines which make up the model window and then drag the model window to a new position. We recommend positioning one of the four corners of the model window first and then resizing the model window.

Just as with the search window, the model window size is very important when configuring a vision object. In general, searching for a small model in a large search region takes more time than a large model in the same large search window. It is a good idea to make the model window big enough to define the part or pattern you are looking for while keeping it as small as possible but at the same time keeping the search window small as well.

Resizing the Model Window

Then model window size handles appear when a model window is selected. However, if the search window is selected then you will not see the model window size handles. You can easily locate the model window size handles because they are the small boxes that are located on the sides and corners of the model window when the search window is selected. If you cannot see the Handles on the model window, then the search window may be the active Window. To make the model window the active Window, click on one of the sides of the model window and you should then see the model window size handles.

Model window size handles are used to size the model window. Moving the mouse pointer over one of the model window size handles causes the pointer to change to a double-ended arrow pointer. This indicates that you can click the handle and resize the model window. You can click on any of the model window size handles and then drag the side or corner to resize the search window.

Clicking on a model window size handle located on one of the sides of the model window allows you to resize the object horizontally. Clicking on the top or bottom of the model window allows you to resize the object vertically. Clicking on one of the corner model window size handles allows you to resize the model window both horizontally and vertically at the same time.

5.3.3 The Model Origin

Every model has a model origin. The model origin is the fixed reference point by which we describe the model's location in an image. It should be noted that the model origin coordinate position is referenced to the upper left position of the model window. The upper left position of the model window starts at [0,0].

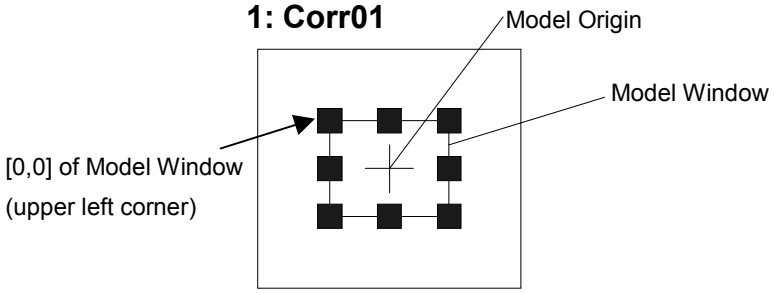


Figure 29: The Model Origin

Positioning the Model Origin

The model origin can be positioned automatically or manually. To set the model origin automatically, set the ModelOrgAutoCenter property to True for that specific object. In this case, the model origin will automatically be set to the center of the model window.

NOTE

The default setting of the ModelOrgAutoCenter property is True.

There may be cases where you want to position the model origin manually. To move the model origin manually requires that the ModelOrgAutoCenter property be set to False. This can be done from the Object tab within the Properties list for the specific object.

To move the model origin requires that the model window be the active window for the object. (As opposed to having the search window as the active Window.) If the model window is not the active window, click on one of the lines which make up the model window and you will see the model window size handles appear for the model window. Once the ModelOrgAutoCenter property is set to 0 - False, we can then move the model origin with the mouse. To move the model origin, click on the crosshair of the model origin and drag it to a new position.

It is normally a good idea to position the model origin on a position of significance for the model you are working with. For example, you may want to place the model origin at the position on the model where you want the robot grip to pick up a part.

5.4 Using Vision Objects

The remainder of this chapter will explain the details of vision objects. Each vision object is described in its own section. Since most vision applications will not require the use of all the different vision objects, you need only refer to the vision objects that pertain to your specific applications.

CAUTION	■ Ambient lighting and external equipment noise may affect vision sequence image and results. A corrupt image may be acquired and the detected position could be any position in an object's search area. Be sure to create image processing sequences with objects that use search areas that are no larger than necessary.

5.4.1 ImageOp Object

ImageOp Object Description

ImageOp objects enable you to perform morphology (including open, close, erode, dilate), convolution (including sharpen, smooth), flip, binarize, and rotate for a specified region of interest.

Other vision objects that are placed in the ImageOp region of interest will perform their operations on the output of ImageOp. For example, you can execute an erode operation on the entire video image with an ImageOp tool, and then place Blob objects inside the ImageOp search window to search the eroded image.

ImageOp Object Layout

The Image object has an image processing window, as shown below.

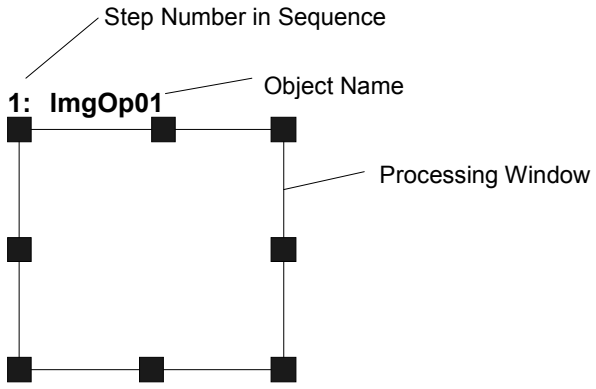


Figure 30: ImageOp Object Layout

ImageOp Object Properties

The following list is a summary of the ImageOp object properties with brief descriptions. The details for each property are explained in the *Vision Guide Properties and Results Reference Manual*.

Property	Description
AngleObject	Defines which object to perform automatic rotation from. The image will be rotated counter-clockwise using the Angle result of the specified object. The Operation property must be set to Rotation, otherwise this property is ignored. Default: None
BackColor	Sets the background color for the object's label. Default: 0 - None
Caption	Used to assign a caption to the object. Default: Empty String
FoundColor	Selects the color for an object when it is found. Default: 1 – Light Green
Frame	Specifies which positioning frame to use. Default: None
Graphics	Specifies which graphics to display. Default: 1 - All
Iterations	Defines how many times to perform the specified operation. Default: 1
Name	Used to assign a unique name to the ImageOp object. Default: ImgOpxx
Operation	Defines which image operation to perform. Default: 1 - Open
RotationAngle	Defines how many degrees to rotate the image when the Operation property is set to Rotation. Default: 0
SearchWin	Runtime only. Sets / returns the search window left, top, height, width parameters in one call.
SearchWinHeight	Defines the height of the area to be searched in pixels. Default: 100
SearchWinLeft	Defines the left most position of the area to be searched in pixels.
SearchWinTop	Defines the upper most position of the area to be searched in pixels.
SearchWinWidth	Defines the width of the area to be searched in pixels. Default: 100
ThresholdColor	Defines the color assigned to pixels withing the thresholds. Default: Black
ThresholdHigh	Defines the upper threshold hold setting to use when the Operation property is set to Binarize. Default: 100
ThresholdLow	Defines the lower threshold hold setting to use when the Operation property is set to Binarize. Default: 10

ImageOp Object Results

The following list is a summary of the ImageOp object results with brief descriptions. The details for each result are explained in the *Vision Guide Properties and Results Reference Manual*.

Result	Description
Time	Returns the amount of time in milliseconds required to process the object.

Using ImageOp Objects

Now we have set the foundation for understanding how to use Vision Guide ImageOp objects. This next section will describe the steps required to use ImageOp objects as listed below:

- How to create a new ImageOp object
- Position and size the search window
- Configure the properties associated with the ImageOp object
- Test the ImageOp object & examine the results
- Make adjustments to properties and test again

Prior to starting the steps shown below, you should have already created a new vision sequence or selected a vision sequence to use. If you have no vision sequence to work with, you can create a new vision sequence by clicking on the New Sequence  toolbar button. You can also select a sequence which was created previously by clicking on the Sequence Tab in the Vision Guide window and clicking on the drop down list box which is located towards the top of the Sequence tab. See the chapter Vision Sequences for more details on how to create a new vision sequence or select one which was previously defined.

Step 1: Create a New ImageOp Object

(1) Click on the New ImageOp toolbar button on the Vision Guide Toolbar.  (In this case click means to click and release the left mouse button)

(2) Move the mouse over the image display. You will see the mouse pointer change to the ImageOp object icon.

(3) Continue to move the mouse until the icon is at the desired position in the image display, then click the left mouse button to create the object.

(4) Notice that a name for the object is automatically created. In the example, it is called "ImgOp01" because this is the first ImageOp object created for this sequence. (We will explain how to change the name later.)

Step 2: Position and Size the Search Window

You should now see an ImageOp object similar to the one shown below:

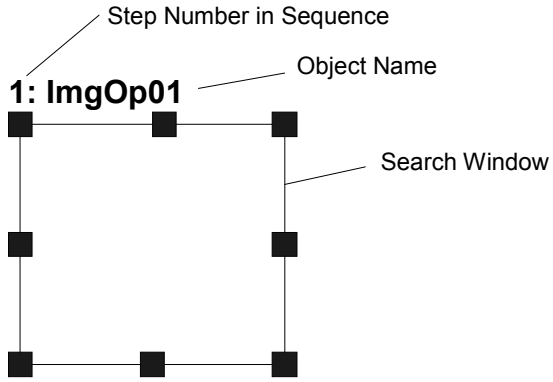


Figure 31: New ImageOp Object Layout

(1) Click on the name label of the ImageOp object and while holding the mouse down, drag the ImageOp object to the position where you would like the top left position of the search window to reside.

(2) Resize the ImageOp object search window as required using the search window size handles. Move the mouse pointer over a size handle, then while holding down the left mouse button, drag the handle to resize the window.

Step 3: Configure the ImageOp Object Properties

We can now set property values for the ImageOp object. Described below are some of the more commonly used properties that are specific to the ImageOp object. Explanations for other properties such as Graphics, etc. which are used on many of the different vision objects can be seen in the Vision Properties and Results Reference Manual or in the ImageOp properties list.

Operation property - Determines which image operation to perform. This is the most important property for the ImageOp object.

Iterations property - Determines the number of iterations to perform.

Polarity property - Would you like to perform operations on dark objects or light objects? The default setting is dark objects. If you want to change it, click on the Value field of the Polarity property and you will see a drop down list with 2 choices: DarkOnLight or LightOnDark. Click on the choice you want to use.

You can probably test the ImageOp object now and then come back and set the any other properties as required later.

Step 4: Test the ImageOp Object

To run the ImageOp object, click on the Run Object button located at the bottom left of the Object tab. You will be able to see the effect of your ImageOp tool on the image.

Step 5: Make Adjustments to Properties and Test Again

After running the ImageOp object a few times, you may have encountered problems or just want to fine-tune some of the property settings. Some common fine tuning techniques are described below:

Fine-tuning of the ImageOp object may be required for some applications. The primary properties associated with fine-tuning of an ImageOp object are described below:

Iterations - Determines how many times to perform the desired image operation.

ThresholdColor, ThresholdLow, ThresholdHigh

- These properties adjust parameters for the Binarize operation. Please refer to the descriptions these properties in the *Vision Properties and Results Reference Manual*.

Once you have completed making adjustments to properties and have tested the ImageOp until you are satisfied with the results you are finished with creating this vision object and can go on to creating other vision objects or configuring and testing an entire vision sequence.

5.4.2 Geometric Object

Geometric Description

The Geometric object finds a model, based on geometric features. It employs an algorithmic approach that finds models using edge-based geometric features instead of a pixel-to-pixel correlation. As such, the Geometric search tool offers several advantages over correlation pattern matching, including greater tolerance of lighting variations (including specular reflection), model occlusion, as well as variations in scale and angle.

The Geometric object is normally used for determining the position and orientation of a known object by locating features (e.g. registration marks) on the object. This is commonly used to find part positions to help guide the robot to pickup and placement positions.

Geometric Object Layout

The Geometric object has a search window and a model window, as shown below.

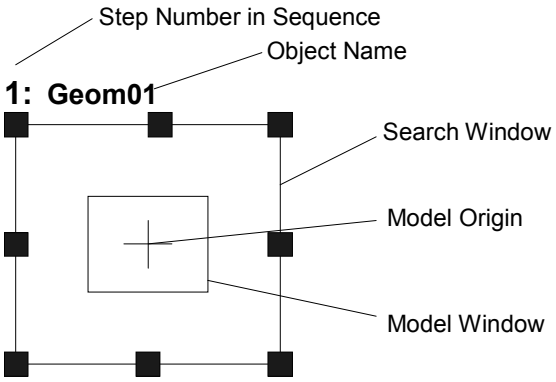


Figure 32: Geometric Object Layout

Geometric Properties

The details for each property that is used with Geometric objects are explained in the *Vision Properties and Results Reference Manual*. However, for convenience we have provided a list of the Geometric object properties and a brief explanation of each below:

Property	Description
AbortSeqOnFail	Allows the user to specify that if the specified object fails (i.e. is not found), then the entire sequence is aborted at that point and no further objects are processed. Default: 0 - False
Accept	Specifies the shape score that a feature must equal or exceed to be considered found. If the value is small, it may result in false detection. Default: 700
AngleEnable	Specifies whether or not a Geometric search will do a search with angle (rotation). Specified prior to teaching a Model of the Geometric object.
AngleRange	Specifies the range within which to train a series of rotated

Property	Description
	models.
AngleStart	Specifies the center of the angle search.
BackColor	Selects the background color for an object.
Caption	Used to assign a caption to the Geometric object.
CenterPntObjResult	Specifies which result to use from the CenterPointObject described above.
CenterPntOffsetX	Sets or returns the X offset of the center of the search window after it is positioned with the CenterPointObject.
CenterPntOffsetY	Sets or returns the Y offset of the center of the search window after it is positioned with the CenterPointObject.
CenterPointObject	Specifies the position to be used as the center point for the Geometric Search. This property is normally set to base the center of the Geometric Search on the XY position result from another vision object.
Confusion	Indicates the amount of confusion expected in the image to be searched. This is the highest shape score a feature can get that is not an instance of the feature for which you are searching.
CurrentResult	Defines which result to display in the Results list (on the Object tab) or which result to return data for when the system is requested to find more than one of a like feature within a single search window.
DetailLevel	Selects the level at which an edge is considered found during the search.
FoundColor	Selects the color for an object when it is found.
Frame	Defines the current object searching position with respect to the specified frame. (Allows the Geometric object to be positioned with respect to a frame.)
Graphics	Specifies which graphics mode to use to display the object.
ModelObject	Determines which model to use for searching.
ModelOrgAutoCenter	A model has a fixed reference point by which we describe its location in a model window. This point is referred to as the model's Origin. The ModelOrgAutoCenter property causes the model origin to be placed at the center of the model window.
ModelOrgX	Contains the X coordinate value of the model origin.
ModelOrgY	Contains the Y coordinate value of the model origin.
ModelWin	Runtime only. Sets / returns the model window left, top, height, width parameters in one call.
ModelWinHeight	Defines the height of the model window.
ModelWinLeft	Defines the left most position of the model window.
ModelWinTop	Defines the top most position of the model window.
ModelWinWidth	Defines the width of the model window.

Property	Description
Name	Used to assign a unique name to the Geometric object.
NumberToFind	Defines the number of objects to find in the current search window. (Geometric objects can find more than 1 object at once.)
RejectOnEdge	Determines whether the part will be rejected if found on the edge of the search window. Normally, this property should be set to True to avoid false detection caused by parts that are not completely within the search window. When set to False, you can use the FoundOnEdge result for each part result.
ScaleEnable	Enables scaling.
ScaleFactorMax	Sets / returns the maximum scale factor applied to the ScaleTarget value.
ScaleFactorMin	Sets / returns the minimum scale factor applied to the ScaleTarget value.
ScaleTarget	Sets / returns the expected scale of the model you are searching for.
SearchWin	Runtime only. Sets / returns the search window left, top, height, width parameters in one call.
SearchWinHeight	Defines the height of the area to be searched. (i.e. the search window height)
SearchWinLeft	Defines the left most position of the area to be searched. (i.e. the search window left side)
SearchWinTop	Defines the upper most position of the area to be searched. (i.e. the search window top side)
SearchWinWidth	Defines the width of the area to be searched. (i.e. the search window width)
SeparationAngle	Sets / returns the minimum angle allowed between found objects.
SeparationMinX	Sets / returns the minimum distance along the X axis allowed between found objects.
SeparationMinY	Sets / returns the minimum distance along the Y axis allowed between found objects.
SeparationScale	Sets / returns the minimum scale difference allowed between found objects.
SharedEdges	Sets / returns whether to allow edges to be shared between found objects.
ShowModel	Allows a previously taught model to be displayed at various zoom settings. Also changes the model orgin and don't care pixels for some models.
Smoothness	Sets / returns the smoothness level for the geometric edge extraction filter.
Sort	Selects the sort order used for the results of an object.
Timeout	Sets / returns the maximum search time for a Geometric object.

Geometric Results

The details for each result which is used with Geometric objects are explained in the *Vision Properties and Results Reference Manual*. However, for convenience we have provided a list of the Geometric object results and a brief explanation of each below:

Results	Description
Angle	Returns the amount of rotation associated with a part that was found. (i.e. This defines the amount of rotation a part may have vs. the originally taught orientation.
CameraX	Returns the X coordinate position of the found part's position (referenced by model origin) in the camera coordinate frame.
CameraY	Returns the Y coordinate position of the found part's position (referenced by model origin) in the camera coordinate frame.
CameraXYU	(Available from SPEL language only) Returns the CameraX, CameraY, and CameraU coordinates of the found part's position in the camera coordinate frame.
Found	Returns whether or not the object was found. (i.e. did the feature or part you are looking at have a shape score which is above the Accept property's current setting.)
FoundOnEdge	Returns TRUE when the Geometric object is found too close to the edge of the search window. When FoundOnEdge is TRUE the Found result is set to FALSE.
NumberFound	Returns the number of objects found within the search window. (This number can be anywhere from 0 up to the number of objects you requested the Geometric object to find with the NumberToFind property.)
PixelX	Returns the X coordinate position of the found part's position (referenced by model origin) in pixels.
PixelY	Returns the Y coordinate position of the found part's position (referenced by model origin) in pixels.
PixelXYU	Runtime only. Returns the PixelX, PixelY, and PixelU coordinates of the found part's position in pixels.
RobotX	Returns the X coordinate position of the found part's position (referenced by model origin) with respect to the Robot's Coordinate System.
RobotY	Returns the Y coordinate position of the found part's position (referenced by model origin) with respect to the Robot's Coordinate System.
RobotU	Returns the U coordinate position of the found part's position with respect to the Robot's Coordinate System.
RobotXYU	Runtime only. Returns the RobotX, RobotY, and RobotU coordinates of the found part's position with respect to the Robot's World Coordinate System.

Results	Description
Scale	Returns the scale factor. ScaleEnabled must be set to True for this result to be valid.
Score	Returns an integer value which represents the level at which the feature found at runtime matches the model for which Geometric is searching.
ShowAllResults	Displays a dialog which allows you to see all results for a specified vision object in a table form. This makes it easy to compare results when the NumberToFind property is set to 0 or greater than 1.
Time	Returns the amount of time in milliseconds required to process the object.

Understanding Geometric Search

The purpose of the Geometric object is to locate previously a trained model in a search window. Geometric objects can be used to find parts, detect presence and absence of parts or features, detect flaws and a wide variety of other things.

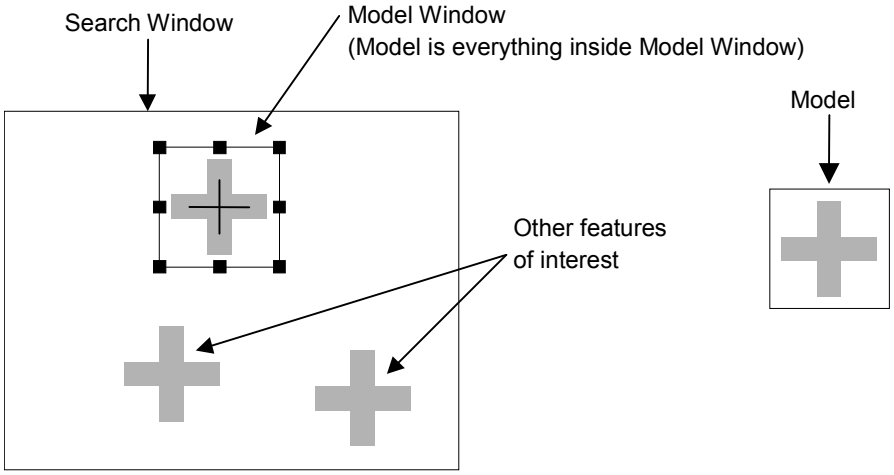
Over the next few sections we will explain the basics of the Geometric object. This information will include the following sections:

- Geometric object models and Features
- Basic Searching Concepts
- Setting the Accept and Confusion property
- Additional Search Parameters
- Using the Multiple Results Dialog to Debug Searching Problems
- Geometric objects and Rotation
- Geometric objects and Scale
- Model Training for Angle Searching
- Searching Repeatability and Accuracy
- Calibrating the Camera to Subject Distance

Geometric Object Models and Features

It is important to understand the difference between features and models when using Geometric objects. A feature is any specific pattern of edges in a search window. A feature can be anything from a simple edge a few pixels in area to a complex pattern of tens of thousands of pixels in area. The Geometric operation measures the extent to which a feature in a search window matches a previously taught Model of that feature. A feature is something within an actual search window as opposed to a model or template that is an idealized representation of a feature.

It is common to train a representative model from one search window to be used to search for similar features in that search window. The picture below shows a search window containing a feature of interest: a cross. To train a model of the cross, you define the model window and click the Teach Button on the Object tab. (Teaching Models is explained later in the section titled Using Geometric Objects.) The resulting Model is created as shown on the right side of the picture and can then be used to search for other crosses within the search window.



Search window containing several features of interest (left) and model trained from image (right)

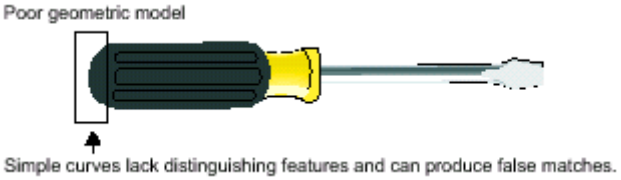
While finding models based on geometric features is a robust, reliable technique, there are a few pitfalls that you should be aware of when allocating your models so that you choose the best possible model.

Make sure your images have enough contrast

Contrast is necessary for identifying edges in your model source and target image with sub-pixel accuracy. Images with weak contrast should be avoided since the Geometric search tool uses a geometric edge-based searching algorithm; the weaker the contrast, the less the amount and accuracy of edge-based information with which to perform a search. For this reason, it is recommended that you avoid models that contain slow gradations in grayscale values. You should maintain a difference in grayscale values of at least 10 between the background and the edges in your images.

Avoid poor geometric models

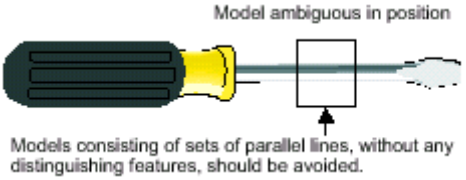
Poor geometric models suffer from a lack of clearly defined geometric characteristics, or from geometric characteristics that do not distinguish themselves sufficiently from other image features. These models can produce unreliable results.



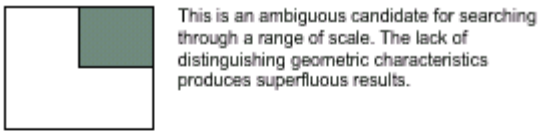
Avoid ambiguous models

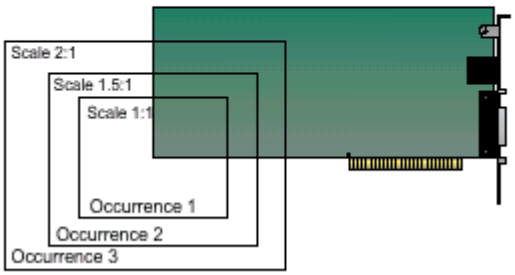
Certain types of geometric models provide ambiguous results in terms of position, angle, or scale.

Models that are ambiguous in position are usually composed of one or more sets of parallel lines only. For example, models consisting of only parallel lines should be avoided since it is impossible to establish an accurate position for them. An infinite number of matches can be found since the actual number of line segments in any particular line is theoretically limitless. Line segments should always contain some distinguishing contours that make them distinct from other image details.

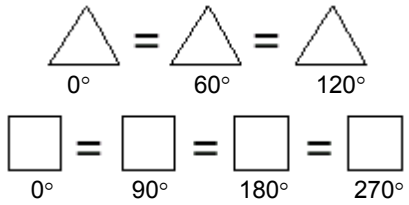


Models that consist of small portions of objects should be tested to verify that they are not ambiguous in scale. For example, models that consist of isolated corners are ambiguous in terms of scale.



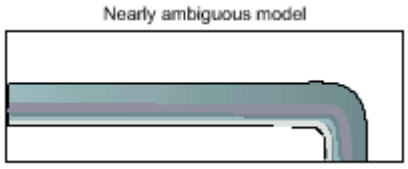


Symmetric models are often ambiguous in angle due to their similarity in features. For example, circles are completely ambiguous in terms of angle. Other simple symmetric models such as squares and triangles are ambiguous for certain angles.



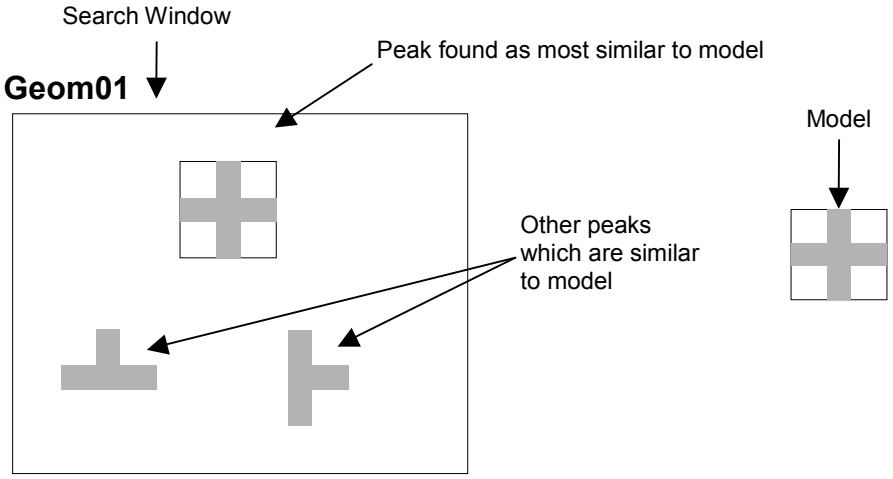
Nearly ambiguous models

When the major part of a model contains ambiguous features, false matches can occur because the percentage of the occurrence's edges involved in the ambiguous features is great enough to be considered a match. To avoid this, make sure that your models have enough distinct features to distinguish the model from other target image features. This will ensure that only correct matches are returned as occurrences. For example, the model below can produce false matches since the greater proportion of active edges in the model is composed of parallel straight lines rather than distinguishing curves.



Basic Searching Concepts

Searching locates features by finding the area of the search window to which the model is the most similar. The figure below shows a model and a search window, and the areas within the search window that are most similar to the model. A model similar to that shown below might be used to search for a feature such as a fiducial mark on a printed circuit board. A robot could then use the position data returned by the search function to find the location of the board for placing components or for positioning the board itself.



Setting the Accept and Confusion Property Thresholds

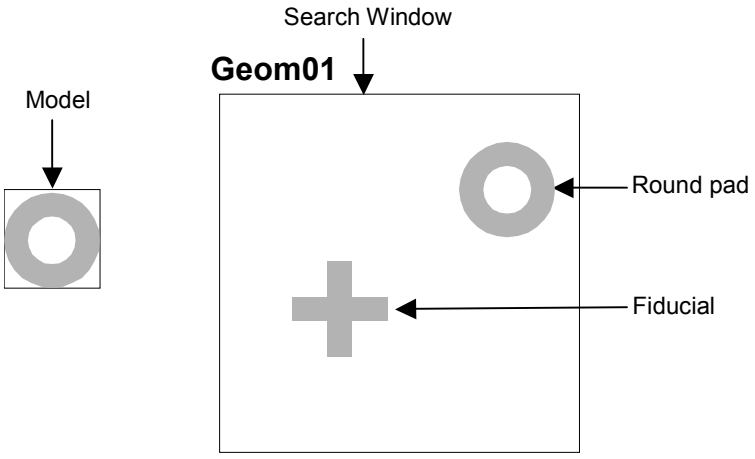
The Accept and Confusion properties are the main search parameters for Geometric objects. The Accept property influences searching speed by providing a hint as to when to pursue the search in a given region of the scene. When the Accept property is set high, features must be very similar to the model, so many regions can be ruled out by a cursory examination and not pursued further. If the Accept property is set to a low value, features that are only slightly similar to the model may exceed the Accept property threshold, so that a detailed examination of more regions in the scene is needed. Thus increasing the Accept property tends to increase speed. (i.e. higher Accept property values can make Geometric objects run faster.)

The Confusion property interacts with the number of results expected to influence searching speed. Together, the Confusion property and the number of results expected allow the system to quit the search before exploring all possible regions of the image.

Set the Accept property so that it will allow the system to find features that are examples of the "worst case degradation" you are willing to accept. The degradation may be caused by defects, scale, rotation or video noise. For the Accept property Vision Guide sets the default value to 700. This is usually a good starting point for many applications. However, experimentation and correction will help you home in on the best value for your situation. Keep in mind that you do not always have to get perfect or nearly perfect scores for an application to function well. Seiko Instruments has observed shape scores of 200 that provided good positional information for some applications, depending of course on the type of degradation to which a feature is subject.

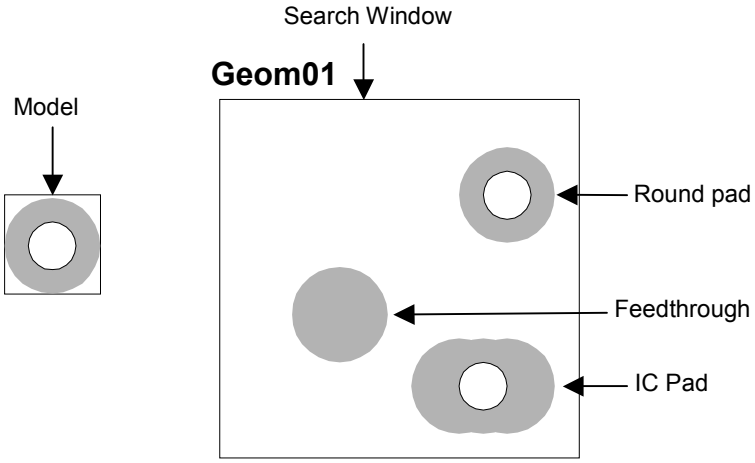
Set the Confusion property based on the highest value you expect the "wrong thing" to get (plus a margin for error). The confusion threshold should be greater than or equal to the Accept property threshold. Setting the Confusion property to a high value will increase the time of the search, but may be necessary to insure that the right features are found. The Confusion property default value is 800 but should be adjusted depending upon the specific application requirements.

The figure below shows a scene where there is little confusion: the round pad is not very similar to the fiducial (cross). The Confusion property can therefore be set to a fairly low value (around 700). The Accept property is normally set less than or equal to the Confusion property, depending upon the amount of degradation you are willing to accept. Assuming this scene has little degradation, a shape score of 920 could be expected.



A scene with little confusion

The figure below shows a scene where there is a lot of confusion; both the feedthrough and the IC pad are similar to the round pad. The Confusion property should therefore be set to a fairly high value (around 820).



A scene with a high degree of confusion

A search window that has a region of constant gray value will always get a 0 Geometric value in that region. If a scene is primarily of uniform background (e.g. a white piece of paper), so that at most places there will be not Geometric, you can set the Confusion property to a low value because if the Geometric object finds anything, it must be the feature for which you are searching.

The Accept and Confusion properties can be thought of as hints that you provide to the system to enable it to locate features more quickly. In general, these properties should be set conservatively, but need not be set precisely. The most conservative settings are a low Accept property and High Confusion property. Use very conservative settings when you know very little about a scene in which you are searching; the search will be careful but slower. (This is especially important when using Geometric property positional results to give to the robot to move to.) Use more liberal settings when you know a lot about a scene in which you are searching. For example, if you know that you are looking for one feature, and the rest of the scene is blank, a careful search is unnecessary; use more liberal settings and the search will be faster.

Additional Geometric search parameters

DetailLevel DetailLevel determines what is considered an edge during the search. Edges are defined by the transition in grayscale value between adjacent pixels. The default setting of Medium offers a robust detection of active edges from images with contrast variation, noise, and non-uniform illumination. In some cases where objects of interest have a very low contrast compared to high contrast areas in the image, some low contrast edges can be missed. If your images contain low-contrast objects, a DetailLevel setting of High should be used to ensure the detection of all important edges in the image. The VeryHigh setting performs an exhaustive edge extraction, including very low contrast edges. However, it should be noted that this mode is very sensitive to noise.

Smoothness The Smoothness property allows you to control the smoothing level of the edge extraction filter. The smoothing operation evens out rough edges and removes noise. The range of this control varies from 0 (no smooth) to 100 (a very strong smooth). The default setting is 50.

SharedEdges You can choose to allow multiple results to share edges, by setting SharedEdges to True. Otherwise, edges that can be part of more than one result are considered part of the result with the greatest score.

TimeOut In time critical applications, you can set a time limit in milliseconds for the Geometric object to find occurrences of the specified model. If the required number of occurrences is not found before the time limit is up, the search will stop. Results are still returned for those occurrences found. However, it is not possible to predict which occurrences will be found before the time limit isearched. The default value is 2000 milliseconds.

Using Multiple Results Dialog to Debug Searching Problems

Sometimes the parts that you are working with vary considerably (even within the same production lot) and sometimes there are 2 or more features on a part which are similar. This can make it very difficult to determine a good Accept property value. Just when you think you have set the Accept property to a good value, another part will come in which fools the system. In these cases it can be very difficult to see what is going on.

The Show All Results dialog was created to help solve these and other problems. While you may only be interested in 1 feature on a part, requesting multiple results can help you see why a secondary feature is sometimes being returned by Vision Guide as the primary feature you are interested in. This generally happens a few different ways:

1. When 2 or more features within the search window are very similar and as such have very close Score results.
2. When the Confusion or Accept properties are not set high enough which allow other features with lower scores than the feature you are interested in to meet the Accept property setting.

Both of the situations above can be quite confusing for the beginning Vision Guide user when searching for a single feature within a search window.

If you have a situation where sometimes the feature you are searching for is found and sometimes another feature is found instead, use the Show All Results dialog to home in on the problem. Follow the following steps to get a better view of what is happening:

(1) Set your NumberToFind property to 3 or more.

(2) Run the vision object from the Vision Guide Development Environment.

(3) Click on the ShowAllResults property button to bring up the Show All Results dialog.

(4) Examine the scores of the top 3 or more features which were found.

Once you examine the scores of the top 3 or more features which were found as described above, it should become clear to you what is happening. In most cases you will see one of these two situations.

1. Each of the features that were found have scores greater than the Accept property setting. If this is the case, simply adjust your Confusion property value up higher to force the best feature to always be found rather that allowing other features to be returned because they meet the Accept threshold. You may also want to adjust the Accept property setting.

2. Each of the features is very close in score. If this is the case, then you will need to do something to differentiate between the feature which you are primarily interested in such as:

   - Readjust the search window so that the features which are randomly returning as the found feature are not contained inside.

   - Teach the Model again for the feature which you are most interested in.

   - Adjust the lighting for your application so that the feature which you are most interested in gets a much higher score than the other features which are currently fooling the system.

See the section titled *Working with Multiple Results from a Single Object* for more information on using multiple results.

Geometric Objects and Separation

You can specify the minimum amount of separation from other occurrences (of the same model) necessary for an occurrence to be considered distinct (a match). In essence, this determines what amount of overlap by the same model occurrence is possible.

You can set the minimum separation for four criteria, which are: the X position, Y position, angle, and scale. For an occurrence to be considered distinct from another, only one of the minimum separation conditions needs to be met. For example, if the minimum separation in terms of angle is met, then the occurrence is considered distinct, regardless of the separation in position or scale. However, each of these separation criteria can be disabled so that it is not considered when determining a valid occurrence.

The minimum positional separation properties SeparationMinX and SeparationMinY determine how far apart the found positions of two occurrences of the same model must be.

This separation is specified as a percentage of the model size at the nominal scale (ScaleTarget).

The default value is 10%. A value of 0% disables the property. For example, if your model is 100 pixels wide at the ScaleTarget, setting SeparationMinX to 10% would require that occurrences be at least 10 pixels apart in the X direction to be considered distinct and separate occurrences.

The minimum angular separation (SeparationAngle) determines the minimum difference in angle between occurrences. This value is specified as an absolute angle value. The default value is 10.0 degrees. A value of 0 degrees disables the property.

The minimum scale separation (SeparationScale) determines the minimum difference in scale between occurrences, as a scale factor. The default value is 1.1. A value of 1.0 disables the property.

Geometric Objects and Scale

The scale of the model establishes the size of the model that you expect to find in the target image. If the expected occurrence is smaller or larger than that of the model, you can set the scale according to the supported scale factors. The expected scale is set using the ScaleTarget property. (range 0.5 - 2.0).

By default, searching through a scale range is disabled. If necessary, you can also enable a search through a range of scales by setting ScaleEnable to True. This allows you to find models in the target image through a range of different sizes from the specified ScaleTarget, both smaller or larger. To specify the range of scales, use ScaleMinFactor and ScaleMaxFactor. The ScaleMinFactor (0.5-1.0) and ScaleMaxFactor (1.0-2.0) together determine the scale range from the nominal ScaleTarget. These maximum and minimum factors are applied to the ScaleTarget setting as follows:

```
max scale = ScaleTarget x ScaleMaxFactor
min scale = ScaleTarget x ScaleMinFactor
```

Note that the range is defined as factors so that if you change the expected scale (ScaleTarget), you do not have to modify the range. A search through a range of scales is performed in parallel, meaning that the actual scale of an occurrence has no bearing on which occurrence will be found first. However, the greater the range of scale, the slower the search.

Geometric Objects and Rotation

Geometric objects are ideal for finding rotated parts. Since a geometric pattern of edges is being searched for, rotated parts can generally be found much more reliably than with other vision tools.

To use the search with angle capabilities of the Geometric object, the Model for the Geometric object must be taught with the AngleEnable property set to True. The AngleStart and AngleRange properties must also be set.

Model Training for Angle Searching

To Search with angle measurement, you must first configure the Geometric object for angle search. This is done by setting the AngleEnable property to True and using the AngleRange property to specify the range of angles over which models will be taught. You can also change the center of the angle search by setting the AngleStart property. This is the angle that the AngleRange is based on. For example, if AngleStart is 45 and AngleRange is 10, then the search will occur for models from 35 to 55 degrees.

Keep in mind that when training models with angle, the search window must be large enough to allow the model to be rotated without any part of the model going outside of the search window.

Searching Repeatability and Accuracy

Searching repeatability and accuracy is a function of the size and details of the model (shape, coarseness of features, and symmetry of the model), and the degradation of the features as seen in the search window (noise, defects, and rotation and scale effects).

To measure the effect of noise on position, you can perform a search in a specific search window that contains a non-degraded feature, and then perform the exact same search again (acquiring a 2nd image into the frame buffer) without changing the position of the object, and then comparing the measured positions. This can be easily done by clicking on the Run Object button of the Object tab (after a model is defined of course) 2 or more times and then clicking on the Statistics toolbar button. The Statistics dialog can then be used to see the difference in position between the 2 object searches. For a large model (30x30) on a non-degraded feature, the reported position can be repeatable to 1/20 of a pixel. However, in most cases it is more realistic to achieve results of just below a pixel. (1/2, 1/3, or 1/4 pixel)

Searching accuracy can be measured by performing a search in a specific search window that contains a non-degraded feature, moving the object an exact distance and then comparing the reported position difference with the actual difference. If you have a large model (30x30 or greater), no degradation, no rotation or scale errors, and have sufficient edges in both X and Y directions, searching can be accurate to 1/4 pixel. (Keep in mind that this searching accuracy is for the vision system only and does not take into effect the inaccuracies which are inherent with all robots. So if you try to move the part with the robot you must also consider the inaccuracies of the robot mechanism itself.)

Calibrating the Camera to Subject Distance

For optimal searching results, the size of the features in an image should be the same at search time as it was when the model was taught. Assuming the same camera and lens are used, if the camera to subject distance changes between the time the model is trained and the time the search is performed, the features in the search window will have a different apparent size. That is, if the camera is closer to the features they will appear larger; if the camera is farther away they will appear smaller.

If the camera to subject distance changes, you should retrain the Model.

Using Geometric Objects

Now that we've reviewed how normalized Geometric and searching works we have set the foundation for understanding how to use Vision Guide Geometric objects. This next section will describe the steps required to use Geometric objects as listed below:

- Create a new Geometric object
- Position and size the search window
- Position and size the model window
- Position the model origin
- Configure properties associated with the Geometric object
- Teach the Model
- Test the Geometric object and examine the results
- Make adjustments to properties and test again
- Working with Multiple Results from a Single Geometric object

Prior to starting the steps shown below, you should have already created a new vision sequence or selected a vision sequence to use. If you have no vision sequence to work with, you can create a new vision sequence by clicking on the New Sequence toolbar button  . You can also select a sequence which was created previously by clicking on the Sequence tab in the Vision Guide window and clicking on the drop down list box which is located towards the top of the Sequence tab. See vision sequences for more details on how to create a new vision sequence or select one which was previously defined.

Step 1: Create a New Geometric Object

(1) Click on the New Geometric toolbar button  on the Vision Guide Toolbar. (In this case click means to click and release the left mouse button).

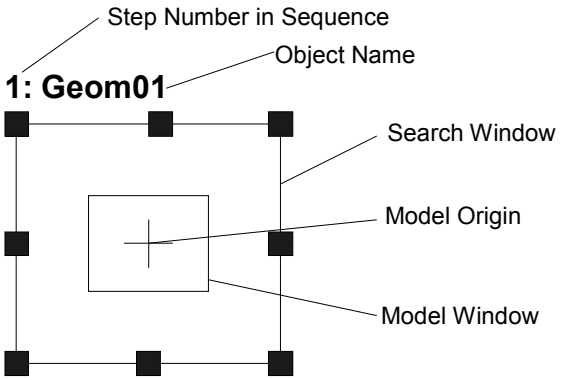
(2) You will see a Geometric icon appear above the Geometric object toolbar button.

(3) Click on the Geometric icon and drag to the image display of the Vision Guide window.

(4) Notice that a name for the object is automatically created. In the example, it is called "Geom01" because this is the first Geometric object created for this sequence. (We will explain how to change the name later.)

Step 2: Position and Size the Search Window

You should now see a Geometric object similar to the one shown below:



New Geometric Object Layout

(1) Click on the name label of the Geometric object and while holding the mouse down, drag the Geometric object to the position where you would like the top left position of the search window.

(2) Resize the Geometric object search window as required using the search window size handles. (This means click on a size handle and drag the mouse.) (The search window is the area within which we will search.)

CAUTION	■ Ambient lighting and external equipment noise may affect vision sequence image and results. A corrupt image may be acquired and the detected position could be any position in an object's search area. Be sure to create image processing sequences with objects that use search areas that are no larger than necessary.

Step 3: Position and Size the Model Window

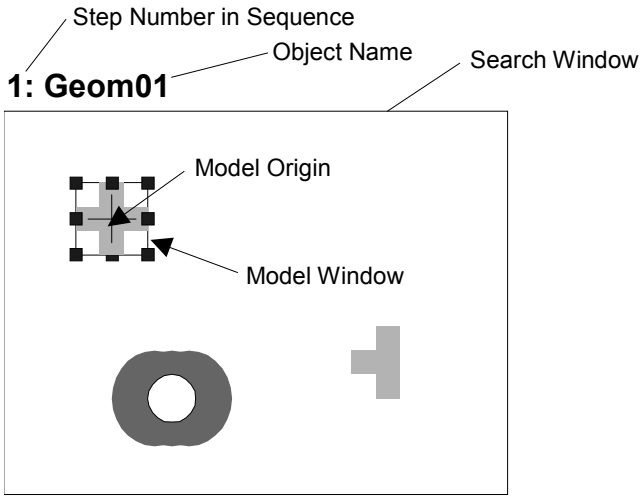
(1) The search window for the Geometric object you want to work on should be highlighted where you can see the size handles on the search window. If you cannot see the size handles click on the name field of the Geometric object. If you can see the size handles disregard this item and move on to the next item below.

(2) Click on one of the lines of the box that forms the model window. This will cause the model window to be highlighted. (You should see the size handles on the model window now.)

(3) Click on one of the lines of the box that forms the model window and while holding the mouse down, drag the model window to the position where you would like the top left position of the model window to reside.

(4) Resize the model window as required using the model window size handles. (This means click on a size handle and drag the mouse.) (The model window should now be outlining the feature that you want to teach as the model for this Geometric object.)

Your Geometric object layout should now look something like the example shown below where the search window covers the area to be searched and the model window outlines the feature you want to search for. Of course your search window and model will be different but this should give you an idea of what was expected so far.



Geometric object after search and model window positioning and resizing

Hints on Setting Proper Size and Position for the Model Window: The size and position of the model window is very important since it defines the feature to be searched for. When creating a model window for a Geometric object, there are 2 primary items you must pay attention to: (1) If you expect that your part will rotate much, smaller models are better because they require smaller search windows to search within. (2) Execution time can be reduced by making the model window as close in size to the search window as possible.

It is also sometimes a good idea to make the model window just a bit larger than the actual feature you are interested in. This will give the feature some border that may prove useful in distinguishing this object from others. Especially when 2 objects are positioned right next to each other and are touching. However, this additional border should only be a few pixels wide. Keep in mind that each vision application is different so the best model window sizing technique will vary from application to application.

Step 4: Position the Model Origin

The model origin defines the position on the model that will be returned back as the position of the feature when you run the Geometric object. This means that the model origin should be placed in a place of significance if the position data is important. For example, when using a Geometric object to find parts for a robot to pick up or place, it is important that the position of the model origin is in a location where the robot can easily grip the part because that is the position the robot will move to based on the RobotXYU, RobotX or RobotY result.

When a new Geometric object is created the ModelOrgAutoCenter property is set to True (default value). This means that the model origin is set to the center of the model window automatically and cannot be moved manually. However if you want to move the model origin manually, you must first set the ModelOrgAutoCenter property to False. The steps to do this and also actually position the model origin are shown below.

(1) Look at the Object tab that you just brought forward in Step 4 as explained on the previous page. Find the ModelOrgAutoCenter property on the properties list and click in the value field.

(2) You will see a drop down list with 2 choices: True and False. Click on the False choice. You have now set the ModelOrgAutoCenter property to False and can move the model origin with the mouse.

(3) Click on the model window to highlight the model window.

(4) Click on the model origin and keep the mouse button held down while dragging the model origin to a new position. It should be noted that the model origin can only be positioned within the bounds of the model window.

Step 5: Configure the Geometric object properties

Since the Object tab was set as the front tab in Step 4, we can now set property values for the Geometric object. To set any of the properties simply click on the associated property's value field and then either enter a new value or if a drop down list is displayed click on one of the items in the list.

Shown below are some of the more commonly used properties for the Geometric object. Explanations for other properties such as AbortSeqOnFail, Graphics, etc. which are used on many of the different vision objects can be seen in Geometric properties. We shouldn't have to set any of these properties to test our Geometric object. However, you may want to read through this information if you are working with Geometric objects for the first time.

 CAUTION	■ Ambient lighting and external equipment noise may affect vision sequence image and results. A corrupt image may be acquired and the detected position could be any position in an object's search area. Be sure to create image processing sequences with objects that use search areas that are no larger than necessary.

Name property - The default name given to a newly created Geometric object is "Geomxx" where xx is a number which is used to distinguish between multiple Geometric objects within the same vision sequence. If this is the first Geometric object for this vision sequence then the default name will be "Geom01". To change the name, click on the Value field of the Name property, type a new name and hit return. You will notice that once the name property is modified, every place where the Geometric object's name is displayed is updated to reflect the new name.

Accept property - The Accept property sets the shape score that a feature must meet or beat to be considered Found. The value returned in the Score result is compared against this Accept property Value. The default value is 700.

Confusion property - If there are many features within the search window which look similar, the Confusion property is useful to help "home in" on the exact feature you want to find. The default value is 800.

ModelOrgAutoCenter property

- If you want to change the position of the model origin you must first set the ModelOrgAutoCenter property to False. This property is originally set to True.

Frame property - Allows you to select a previously defined Frame object as a reference frame for this Geometric object. The details for Frames are defined in Frame object.

NumberToFind property

- Depending upon the number of features you want to find, you may want to set the NumberToFind property larger than 1. This will allow 1 Geometric object to find multiple features within 1 search window.

AngleEnable property - You must set this property to True if you want to use a Geometric model to search with angle. This must be set to True at the time you teach the Model so that multiple models can be configured.

RejectOnEdge property - Allows you to exclude the parts touching the boundary of the search window. Normally, this should be set to True.

You can probably go ahead with the properties at their default settings and then come back and set the any other properties as required later.

Step 6: Teach the model for the Geometric object

The Geometric object needs a model to search for and this is accomplished a process called teaching the model. You should have already positioned the model window for the Geometric object to outline the feature which you want to use as a model. Teaching the model is accomplished as follows:

(1) Make sure that the Geometric object is the currently displayed object in the Object tab. You can look at the current object drop down list at the top of the Object tab to see which object is the object you are currently working on or you can look at the image display and see which object is highlighted in magenta.

(2) Click on the Teach button at the bottom right of the Object tab. It will take only a few seconds for the Model to be taught in most cases. However, if you are teaching a model when the AngleEnable property is set to True it can take quite a few seconds to teach the model because the system is actually teaching many models each at a slight angle offset from the previous.

Step 7: Test the Geometric Object/Examine the Results

To run the Geometric object, click on the Run Object button located at the bottom left of the Object tab.

Results for the Geometric object will now be displayed. The primary results to examine at this time are:

Found result - Returns whether or not the Geometric was found. If the feature you are searching for is found this result returns as True. If the feature is not found, the Found result returns a False and is highlighted in red. If the feature was not found read on to Step 9 for some of the more common reasons why a Geometric object is not found.

FoundOnEdge result - This result will return as True if the feature was found where a part of the feature is touching the boundary of the search window. In this case the Found result will return as False.

Score result - This tells us how well we matched the model with the feature that most closely resembles the model. Score results range from 0 to 1000 with 1000 being the best match possible. Examine the Score result after running a Geometric object as this is your primary measure of how well the feature was found.

Time result - The amount of time it took for the Geometric object to execute. Remember that small search windows and small Models help speed up the search time.

NumberFound result - When searching for more than 1 Geometric object, the NumberFound result returns the number of features which matched the Geometric object's Model.

Angle result - The angle at which the Geometric is oriented. This is computed based on the original angle of the Model. However, this value may sometimes be coarse in value and not so reliable. (We strongly recommend using the Polar object for finding Angles. Especially for robot guidance.)

PixelX, PixelY results - The XY position (in pixels) of the feature. Remember that this is the position of the model origin with respect to the found feature. If you want to return a different position, you must first reposition the model origin and then re-teach the Model.

CameraX, CameraY results

- These are the XY position of the found feature in the Camera's Coordinate system. The CameraX and CameraY results will only return a value if the camera has been calibrated. If it has not, then [No Cal] will be returned.

RobotX, RobotY results

- These are the XY position of the found feature in the Robot's Coordinate system. The robot can be told to move to this XY position. (No other transformation or other steps are required.) Remember that this is the position of the model origin with respect to the found feature. If you want to return a different position, you must first reposition the model origin and then re-teach the Model. The RobotX and RobotY results will only return a value if the camera has been calibrated. If it has not then [No Cal] will be returned.

RobotU result - This is the angle returned for the found feature translated into the Robot's Coordinate system. The RobotU result will only return a value if the camera has been calibrated. If it has not then [No Cal] will be returned.

ShowAllResults - If you are working with multiple results, you may want to click on the button in the ShowAllResults value field. This will bring up a dialog to allow you to examine all the results for the current vision object.

NOTE 

The RobotX, RobotY, RobotU, RobotXYU and CameraX, CameraY, CameraU, CameraXYU results will return [no cal] at this time since we have not done a calibration in the example steps described above. This means that no calibration was performed so it is impossible for the vision system to calculate the coordinate results with respect to the Robot coordinate system or Camera coordinate system. See *Calibration* for more information.

Step 8: Make Adjustments to properties and test again

After running the Geometric object a few times, you may have encountered problems with finding a Geometric or just want to fine tune some of the property settings. Some common problems and fine tuning techniques are described in the next section called "Geometric Object Problems".

Geometric Object Problems

If the Geometric object returns a Found result of False, there are a few things to immediately examine.

- Try changing the Accept property to a lower value (for example, below the current score result) and run the Geometric object again.
- Look at the FoundOnEdge result. Does it have a return value of True? If it is True, this means that the feature was found but it was found where a part of the feature is touching the search window. This causes the Found result to be returned as False. To correct this situation, make the search window larger or if this is impossible, try changing the position of the camera, or resizing the model window.

If the Geometric object finds the wrong feature, there are a couple of things to check:

- Was the Accept property set high enough? If it is set rather low this could allow another feature to be found in place of the feature you are interested in.
- Was the Confusion property set high enough? Is it higher than the Accept property? The Confusion property should normally be set to a value equal to or higher than the Accept property. This is a good starting point. But if there are features within the search window which are similar to the feature you are interested in, then the Confusion property must be moved to a higher value to make sure that your feature is found instead of one of the others.
- Adjust the search window so that it more closely isolates the feature which you are interested in.

Geometric Object Fine Tuning

Fine tuning of the Geometric object is normally required to get the object working just right. The primary properties associated with fine tuning of a Geometric object are described below:

Accept property - After you have run the Geometric object a few times, you will become familiar with the shape score which is returned in the Score result. Use these values when determining new values to enter for the Accept property. The lower the Accept property, the faster the Geometric object can run. However, lower Accept property values can also cause features to be found which are not what you want to find. A happy medium can usually be found with a little experimentation which results in reliable feature finds which are fast to execute.

Confusion property - If there are multiple features within the search window which look similar, you will need to set the Confusion property relatively high. This will guarantee that the feature you are interested in is found rather than one of the confusing features. However, higher confusion costs execution speed. If you don't have multiple features within the search window which look similar, you can set the Confusion property lower to help reduce execution time.

Once you have completed making adjustments to properties and have tested the Geometric object until you are satisfied with the results, you are finished with creating this vision object and can go on to creating other vision objects or configuring and testing an entire vision sequence.

Other utilities for Geometric Objects

At this point you may want to consider examining the Histogram feature of Vision Guide. Histograms are useful because they graphically represent the distribution of grayscale values within the search window. The details regarding Vision Guide Histogram usage are to examine the Geometric object's results statistically.

5.4.3 Correlation Object

Correlation Description

The Correlation object is the most commonly used tool in Vision Guide. Once a Correlation object is trained it can find and measure the quality of previously trained features very quickly and reliably. The Correlation object is normally used for the following types of applications:

Alignment : For determining the position and orientation of a known object by locating features (e.g. registration marks) on the object. This is commonly used to find part positions to help guide the robot to pickup and placement positions.

Gauging : Finding features on a part such as diameters, lengths, angles and other critical dimensions for part inspection.

Inspection : Look for simple flaws such as missing parts or illegible printing.

Correlation Object Layout

The Correlation object has a search window and a model window, as shown below.

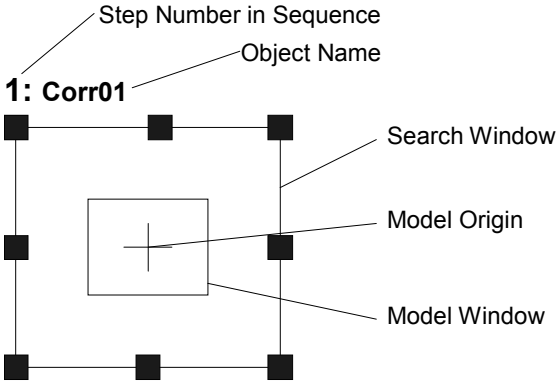


Figure 33: Correlation Object Layout

Correlation Properties

The following list is a summary of properties for the Correlation object. The details for each property are explained in the *Vision Properties and Results Reference Manual*.

Property	Description
AbortSeqOnFail	Allows the user to specify that if the specified object fails (not found), then the entire sequence is aborted at that point and no further objects in the sequence are processed. Default: 0 - False
Accept	Specifies the shape score that a feature must equal or exceed to be considered found. If the value is small, it may result in false detection. Default: 700

Property	Description
AngleAccuracy	Specifies the desired accuracy for angle search in degrees. Default: 1
AngleEnable	Specifies whether or not a correlation search will do a search with angle (rotation). Specified prior to teaching a Model of the Correlation object. Default: 0 - False
AngleMaxIncrement	Maximum angle increment amount for teaching a correlation model for searching with angle. The maximum value is 10. Default: 10
AngleRange	Specifies the range within which to train a series of rotated models. The maximum value is 45. Default: 10
AngleStart	Specifies the center of the angle search. Default: 0
BackColor	Sets the background color for the object's label. Default: 0 - None
Caption	Used to assign a caption to the Correlation object. Default: Empty String
CenterPntObjResult	Specifies which result to use from the CenterPointObject. Default: 1
CenterPntOffsetX	Specifies the X offset of the center of the search window after it is positioned with the CenterPointObject. Default: 0
CenterPntOffsetY	Specifies the Y offset of the center of the search window after it is positioned with the CenterPointObject. Default: 0
CenterPointObject	Specifies the position to be used as the center point for the Correlation Search. This property is normally set to base the center of the Correlation Search on the XY position result from another vision object. Default: Screen
Confusion	Indicates the amount of confusion expected in the image to be searched. This is the highest shape score a feature can get that is not an instance of the feature for which you are searching. Default: 800
CurrentResult	Defines which result to display in the Results list on the Object tab or which result to return data for when the system is requested to find more than one result. Default: 1
FoundColor	Selects the color for an object when it is found. Default: 1 – Light Green

Property	Description
Frame	Defines the current object searching position with respect to the specified frame. (Allows the Correlation object to be positioned with respect to a frame.) Default: None
Graphics	Specifies which graphics to display. Default: 1 - All
ModelObject	Determines which model to use for searching. Default: Self
ModelOrgAutoCenter	A model has a fixed reference point by which we describe its location in a model window. This point is referred to as the model's Origin. The ModelOrgAutoCenter property causes the model origin to be placed at the center of the model window. Default: 1 - True
ModelOrgX	Contains the X coordinate value of the model origin.
ModelOrgY	Contains the Y coordinate value of the model origin.
ModelWin	Runtime only. Sets / returns the model window left, top, height, width parameters in one call.
ModelWinLeft	Defines the left most position of the model window.
ModelWinHeight	Defines the height of the model window. Default: 50
ModelWinTop	Defines the top most position of the model window.
ModelWinWidth	Defines the width of the model window.
Name	Used to assign a unique name to the Correlation object. Default: Corrxx
NumberToFind	Defines the number of features to find in the current search window. Default: 1
RejectOnEdge	Determines whether the part will be rejected if found on the edge of the search window. Normally, this property should be set to True to avoid false detection caused by parts that are not completely within the search window. When set to False, you can use the FoundOnEdge result for each part result. Default: 0 - False
SearchWin	Runtime only. Sets / returns the search window left, top, height, width parameters in one call.
SearchWinHeight	Defines the height of the area to be searched in pixels. Default: 100
SearchWinLeft	Defines the left most position of the area to be searched in pixels.
SearchWinTop	Defines the upper most position of the area to be searched in pixels.
SearchWinWidth	Defines the width of the area to be searched in pixels. Default: 100

Property	Description
ShowModel	Allows the user to see the internal grayscale representation of a taught model.
Sort	Selects the sort order used for the results of an object. Default: 0 - None

Correlation Results

The following list is a summary of the Correlation object results with brief descriptions. The details for each result are explained in the *Vision Guide Properties and Results Reference Manual*.

Results	Description
Angle	Returns the amount of part rotation in degrees.
CameraX	Returns the X coordinate position of the found part's position (referenced by model origin) in the camera coordinate frame. Values are in millimeters.
CameraY	Returns the Y coordinate position of the found part's position (referenced by model origin) in the camera coordinate frame. Values are in millimeters.
CameraXYU	Runtime only. Returns the CameraX, CameraY, and CameraU coordinates of the found part's position in the camera coordinate frame. Values are in millimeters.
Found	Returns whether or not the object was found. (i.e. did the feature or part you are looking at have a shape score that is above the Accept property's current setting.)
FoundOnEdge	Returns TRUE when the Correlation object is found too close to the edge of the search window. If RejectOnEdge is TRUE, then the Found result is set to FALSE.
NumberFound	Returns the number of objects found within the search window. (This number can be anywhere from 0 up to the number of objects you requested the Correlation object to find with the NumberToFind property.)
PixelX	Returns the X coordinate position of the found part's position (referenced by model origin) in pixels.
PixelY	Returns the Y coordinate position of the found part's position (referenced by model origin) in pixels.
RobotX	Returns the X coordinate position of the found part's position (referenced by model origin) with respect to the Robot's Coordinate System.
RobotY	Returns the Y coordinate position of the found part's position (referenced by model origin) with respect to the Robot's Coordinate System.
RobotU	Returns the U coordinate position of the found part's position with respect to the Robot's Coordinate System.

Results	Description
RobotXYU	Runtime only. Returns the RobotX, RobotY, and RobotU coordinates of the found part's position with respect to the Robot's World Coordinate System.
Score	Returns an integer value between 0-1000 that represents the level at which the feature found at runtime matches the model for which Correlation is searching.
ShowAllResults	Displays a dialog that allows you to see all results for a specified vision object in a table form. This makes it easy to compare results when the NumberToFind property is set to 0 or greater than 1.
Time	Returns the amount of time in milliseconds required to process the object.

Understanding Normalized Correlation

The purpose of Correlation is to locate and measure the quality of one or more previously trained features in a search window. Correlation objects can be used to find parts, detect presence and absence of parts or features, detect flaws and a wide variety of other things. While Vision Guide has a wide variety of vision object tools, the Correlation object is the most commonly used tool due to its speed and overall reliability. For example, in many applications Edge Tools can be used to find the edges of an object. However, if there are many potential edges within the same area that may confuse an Edge object, then a Correlation object may be able to be used instead to find the position of the edge. There are also instances where Correlation objects are used over Blob objects (when a model can be taught) because they are more reliable.

Over the next few pages we will explain the basics behind the search tools as they apply to the Correlation object. This information will include the following sections:

- Features and Models: Explained
- Basic Searching Concepts
- Normalized Correlation
- Normalized Correlation Shape Score
- Normalized Correlation Optimization (Accept and Confusion)
- Setting the Accept and Confusion property values
- Additional Hints Regarding Accept and Confusion properties
- Correlation objects and rotation
- Model Training for angle searching
- Searching repeatability and accuracy
- Calibrating the camera to subject distance

Correlation object models and features

It is important to understand the difference between features and models when using Correlation objects. A feature is any specific pattern of gray levels in a search window. A feature can be anything from a simple edge a few pixels in area to a complex pattern of tens of thousands of pixels in area. The correlation operation measures the extent to which a feature in a search window matches a previously taught model of that feature. A feature is something within an actual search window as opposed to a model or template that is an idealized representation of a feature.

A model is a pattern of gray levels used to represent a feature. It is equivalent to a template in a template matching system. If the model has two gray levels, it is called a binary model, if it has more that two gray levels it is called a gray model. All models used with Vision Guide are gray models because gray models are more powerful in that they more closely represent the true feature than a binary model can. This helps produce more reliable results.

It is common to train a representative model from one search window to be used to search for similar features in that search window. Figure 34 shows a search window containing a feature of interest; a cross. To train a model of the cross, you define the model window and click the Teach Button on the Object tab. (Teaching Models is explained later in this chapter in the section titled Using Geometric Objects.) The resulting model is created as shown on the right side of Figure 34 and can then be used to search for other crosses within the search window.

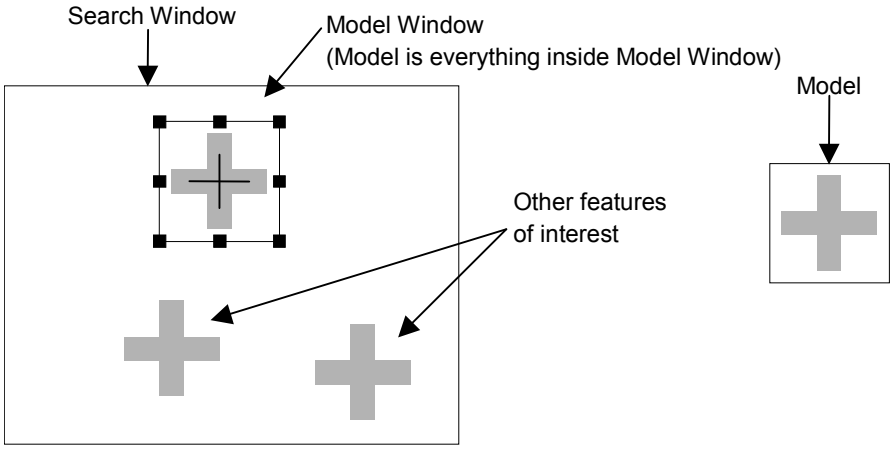


Figure 34: Search window containing several features of interest (left) and model trained from image (right)

Basic Searching Concepts

Searching locates features by finding the area of the search window to which the model is the most similar. Figure 35 shows a model and a search window, and the areas within the search window that are most similar to the model (peaks). A model similar to that shown in Figure 35 might be used to search for a feature such as a fiducial mark on a printed circuit board. A robot could then use the position data returned by the search function to find the location of the board for placing components or for positioning the board itself.

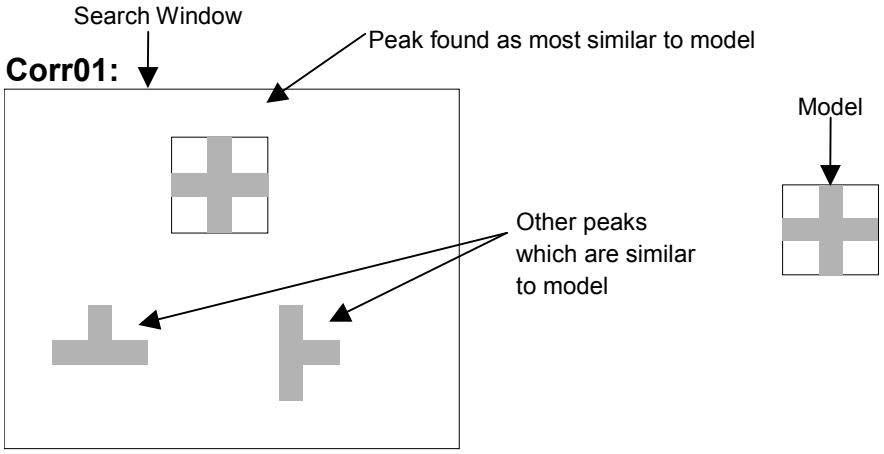


Figure 35: Search window, model, and peaks

There are a number of strategies that can be used to search for a model in a search window. The most commonly used method is the exhaustive search method for finding a match for the model. In the exhaustive search method, the model is evaluated at every possible location in the search window. The location in the search window with the greatest similarity is returned. Assume for example, a search window is 36 pixels square (36 x 36), and the model was 6 pixels square. To locate a match for the model, similarity would be assessed at all 961 possible locations. This method can return reliable results but the trade off is that it is very slow.

The Vision Guide searching method is a vast improvement over exhaustive searching. First, the search window is scanned to locate positions where a match is likely. Similarity is then assessed only for those candidate locations. The candidate of the best match is returned. This technique results in execution times that can be several thousand times faster than those achieved by an exhaustive search.

Normalized Correlation

Normalized correlation is the searching algorithm that is used with Vision Guide Correlation objects because it is both powerful and robust. Normalized correlation is a measure of the geometric similarity between an image and a model, independent of any linear differences in search window or model brightness. The normalized correlation value does not change in either of the following situations:

- If all search window or model pixels are multiplied by some constant.

- If a constant is added to all search window or model pixels.

One of the most important characteristics of normalized correlation is that the correlation value is independent of linear brightness changes in either the Search window or the Model. This is an important characteristic because the overall brightness and contrast of an image are determined by such factors as illumination intensity, scene reflectivity, camera aperture, sensor gain and offset (including possible automatic gain control circuitry), and the gain and offset of the image digitizer. All of which are hard to control in most production situations. For example, bulbs age, ambient light levels can change with time of day, cameras and digitizers may be defective and need to be replaced, and the reflectivity of objects being inspected can vary.

Another important characteristic of normalized correlation is that the shape score value (see Normalized Correlation Shape Score subheading on the next page) has absolute significance; that is, you can define a perfect match that is the same for all models and search windows. It is independent not only of image brightness and contrast but also of model brightness, contrast, and size. It is this property that allows the shape scores to be used as a measure of feature quality for inspection applications. If the shape score calculated for a search window and model is 900, they are very similar. If the value is 100, they are not similar. These statements are valid without knowing anything about the model or search window area. This means the correlation coefficient has absolute meaning.

Normalized Correlation Shape Score

For normalized correlation, the shape score of a feature (the extent to which it is similar to the model) is defined as a value between 0 and 1000. The larger the shape score, the greater the similarity between the feature and the model. (A shape score of 1000 represents a perfect match.) The shape score is the value returned by the Correlation object as the Score result. (Additional information for the Score result is available in the *Vision Guide Properties and Results Reference Manual*.)

Normalized Correlation Optimization (Accept and Confusion)

Traditional template matching is much too slow for most practical searching applications. This is because it is based on exhaustive correlation at every possible search position. The solution to this problem lies in the use of a suitable directed search. In a directed search, data derived during the search are used to direct the search toward those locations that are promising and away from those that are not.

Searching uses an adaptation of a type of directed search called hill climbing. The idea of hill climbing is that the peak of a function of one or more variables can be found entirely from local data, by always moving in the direction of steepest ascent. The peak isearched when each of the neighboring points is lower than that point.

Hill climbing alone can fail under certain circumstances:

- A false maximum can be mistaken for the peak of the hill.
- If a plateau isearched there is no way to determine the direction in which the search should resume.

Another problem with hill climbing is determining the starting points: too few expeditions might miss significant hills; too many will eliminate the speed advantage that hill climbing provides by causing the same hill to be climbed from different directions.

Certain estimates can be made about the hills based on the search window area and the Model which help to overcome these problems.

Correlation is mathematically equivalent to a filtering operation: the correlation function is the output of a specific, known filter (the model), a filter that amplifies certain spatial frequencies and attenuates others. In addition, if the frequency content of a given portion of a search window is not similar to that of the model, there is no need to start a hill climbing expedition in this region.

By examining the transfer function of the model, the system can estimate the spatial frequency content of the correlation function, and, in turn the minimum spacing and size of the hills. Knowing the minimum spacing allows the system to plan where to start the hill climbing expeditions. Knowing the minimum size allows the system to avoid getting trapped by false peaks.

In addition to information obtained from the model, a couple of Vision Guide properties are specified to control the search: the Accept property and the Confusion property.

The Accept property specifies the shape score that a feature must equal or exceed to be considered "Found" (i.e. Found result returns TRUE) by the searching software. If a rough estimate of the height of a hill does not exceed the Accept property value in a given region of a search window, hill climbing is terminated for that region.

The Confusion property indicates the amount of confusion expected in the search window. Specifically, it is the highest shape score a feature can get that is not an instance of the feature for which you are searching. The Confusion property gives the system an important hint about the scene to be searched; namely, that if a feature gets a shape score above the confusion threshold, it must be an instance of the feature for which you are searching.

The system uses the Confusion property and the number of results (specified by the NumberToFind property) to determine which hills need to be climbed and which do not. Specifically, the search can terminate as soon as the expected number of features are found whose shape score is above the Confusion property threshold and the Accept property threshold.

To start the search, a number of hill climbing expeditions are conducted in parallel, starting at positions determined from the transfer function of the Model. As hill climbing progresses, a more and more accurate estimate of each hill's height emerges until the actual peak is found. The hills are climbed in parallel, one step at a time, as long as no hill has an estimated height that exceeds the threshold set by the Confusion property. If an expedition reaches the Confusion property threshold, the hill is immediately climbed to its peak.

Setting the Accept and Confusion Property Thresholds

Both the Accept and Confusion properties affect searching speed for Correlation objects. The Accept property influences searching speed by providing a hint as to when to pursue the search in a given region of the scene. When the Accept property is set high, features must be very similar to the model, so many regions can be ruled out by a cursory examination and not pursued further. If the Accept property is set to a low value, features that are only slightly similar to the Model may exceed the Accept property threshold, so that a detailed examination of more regions in the scene is needed. Thus increasing the Accept property tends to increase speed. (i.e. higher Accept property values can make Correlation objects run faster.)

The Confusion property interacts with the number of results expected to influence searching speed. Together, the Confusion property and the number of results expected allow the system to quit the search before exploring all possible regions of the image.

Set the Accept property so that it will allow the system to find features that are examples of the "worst case degradation" you are willing to accept. The degradation may be caused by defects, scale, rotation or video noise. For the Accept property Vision Guide sets the default value to 700. This is usually a good starting point for many applications. However, experimentation and correction will help you home in on the best value for your situation. Keep in mind that you do not always have to get perfect or nearly perfect scores for an application to function well. EPSON has observed shape scores of even as low as 200 that provided good positional information for some applications, depending of course on the type of degradation to which a feature is subject. However, we normally recommend that a shape score of above 500 is used for the Accept property for most applications.

Set the Confusion property based on the highest value you expect the "wrong thing" to get (plus a margin for error). The confusion threshold should be greater than or equal to the Accept property threshold. Setting the Confusion property to a high value will increase the time of the search, but may be necessary to insure that the right features are found. The Confusion property default value is 800 but should be adjusted depending upon the specific application requirements.

Figure 36 shows a scene where there is little confusion: the round pad is not very similar to the fiducial (cross). The Confusion property can therefore be set to a fairly low value (around 500). The Accept property is normally set less than or equal to the Confusion property, depending upon the amount of degradation you are willing to accept. Assuming this scene has little degradation, a shape score of 920 could be expected.

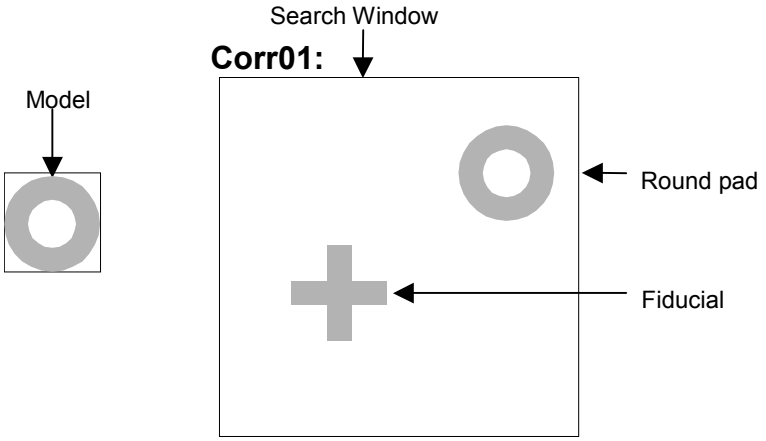


Figure 36: A scene with little confusion

Figure 37 shows a scene where there is a lot of confusion; both the feed through hole and the IC pad are similar to the round pad. The Confusion property should therefore be set to a fairly high value (around 820).

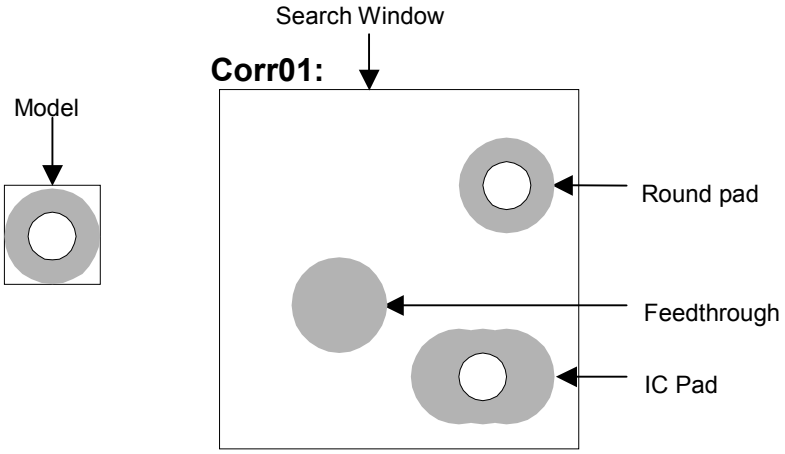


Figure 37: A scene with a high degree of confusion

Additional Hints Regarding Accept and Confusion Properties

A search window that has a region of constant gray value will always get a 0 correlation value in that region. If a scene is primarily of uniform background (e.g. a white piece of paper), so that at most places there will be not correlation, you can set the Confusion property to a low value because if the Correlation object finds anything, it must be the feature for which you are searching.

The Accept and Confusion properties can be thought of as hints that you provide to the system to enable it to locate features more quickly. In general, these properties should be set conservatively, but need not be set precisely. The most conservative settings are a low Accept property and high Confusion property. Use very conservative settings when you know very little about a scene in which you are searching; the search will be careful but slower. (This is especially important when using Correlation property positional results to give to the robot to move to.) Use more liberal settings when you know a lot about a scene in which you are searching. For example, if you know that you are looking for one feature, and the rest of the scene is blank, a careful search is unnecessary; use more liberal settings and the search will be faster.

Using the Multiple Results Dialog to Debug Searching Problems

Sometimes the parts that you are working with vary considerably (even within the same production lot) and sometimes there are 2 or more features on a part which are similar. This can make it very difficult to determine a good Accept property value. Just when you think you have set the Accept property to a good value, another part will come in which fools the system. In these cases it can be very difficult to see what is going on.

The Show All Results dialog was created to help solve these and other problems. While you may only be interested in 1 feature on a part, requesting multiple results can help you see why a secondary feature is sometimes being returned by Vision Guide as the primary feature you are interested in. This generally happens a few different ways:

1. When 2 or more features within the search window are very similar and as such have very close Score results.
2. When the Confusion or Accept properties are not set high enough which allow other features with lower scores than the feature you are interested in to meet the Accept property setting .

Both of the situations above can be quite confusing for the beginning Vision Guide user when searching for a single feature within a search window.

If you have a situation where sometimes the feature you are searching for is found and sometimes another feature is found instead, use the Show All Results dialog to home in on the problem. Follow the following steps to get a better view of what is happening:

(1) Set your NumberToFind property to 3 or more.

(2) Run the vision object from the Vision Guide Development Environment.

(3) Click on the ShowAllResults property button to bring up the Show All Results dialog.

(4) Examine the scores of the top 3 or more features that were found.

(5) If only 1 or 2 features were found (Vision Guide will only set scores for those features that are considered found) reduce your Accept property so that more than 1 feature will be found and Run the vision object again. (You can change the Accept level back after examining the ShowAllResults dialog)

(6) Click on the ShowAllResults property button to bring up the Show All Results dialog.

(7) Examine the scores of the top 3 or more features that were found.

Once you examine the scores of the top 3 or more features that were found as described above, it should become clear to you what is happening. In most cases you will see one of these two situations.

1. Each of the features that were found has a score greater than the Accept property setting. If this is the case, simply adjust your Confusion property value up higher to force the best feature to always be found rather that allowing other features to be returned because they meet the Accept threshold. You may also want to adjust the Accept property setting.
2. Each of the features is very close in score. If this is the case, then you will need to do something to differentiate between the feature which you are primarily interested in such as:

   - Readjust the search window so that the features that are randomly returning as the found feature are not contained inside.
   - Teach the Model again for the feature that you are most interested in.
   - Adjust the lighting for your application so that the feature that you are most interested in gets a much higher score than the other features that are currently fooling the system.

See the section titled *Working with Multiple Results from Single Object* later in this chapter for more information on using multiple results.

Correlation Objects and Rotation

As with any template matching procedure, shape score and location accuracy degenerate if the size or the angle of the feature differs from that of the model. If the differences are severe, the shape score will be very low, or the search operation will not find the feature.

The exact tolerance to angle and size changes depends on the Model, but typically falls between 3 and 10 degrees for angle and 2 to 5 percent for size. Exceptions include rotationally symmetric models, such as circles, which have no angle dependence; and simple edge and corner models, which have no size dependence.

Visualize two different scenes within a search window. The first scene is the picture of a human face where the nose was taught as the model. The nose is not considered XY symmetric so rotation will dramatically affect the accuracy of position for this feature. The second scene is a printed circuit board where a fiducial (similar to one in Figure 36) is the mode. In this case the fiducial mark (a cross) is XY symmetric so rotation does not have an immediate devastating effect on positional accuracy for this feature.

Basically, models that have a predominance of fine features (such as a nose, picture of a flower, tree or others) will be less tolerant to rotation. Features that are more symmetric, like the cross, will be more tolerant of rotation. However, with this in mind we strongly recommend that you use Polar objects to determine rotation angles. The Correlation object can be used to find the XY position and then the Polar object can be associated with the Correlation object such that it can use the XY position as a center to work from to find the angle of the feature. See the Polar object  section of this chapter on page 151 for more information on how to associate Polar objects with Correlation and other vision objects.

There are a variety of techniques that can be used in situations where significant angle or scale changes are expected. The primary techniques are:

- Break up complex features into smaller, simpler ones. In general, small simple models are much less sensitive to scale and angle changes than larger, more complex models.
- Use the angle related properties (AngleEnable, AngleRange, and AngleMaxIncrement) to help find rotational angles when working with Correlation objects. This is appropriate for locating complex features in complex scenes, when those features cannot be broken up into simpler ones. This capability is implemented as a series of models at various angles. It can be many times slower than a normal search.

NOTE  To use the search with angle capabilities of the Correlation object, the Model for the Correlation object must be taught with the AngleEnable property set to True. This causes the Correlation object to be taught at a variety of angles as defined by the AngleRange and AngleMaxIncrement properties.

- Use Polar objects in conjunction with Correlation objects to determine the angle of rotation of a part. (See the section titled Polar objects  on page 151.)

Model Training for Angle Searching

To Search with angle measurement, you must first direct the system to teach a series of rotated models. Setting the AngleEnable property to True and using the AngleRange property to specify the range of angles over which models will be taught does this. When you teach rotated models this way, searching automatically creates a set of models at various rotations in equal angular increments over that range.

You can also specify a maximum angle increment at which the models will be taught within the angular range. This is accomplished by setting an increment value in the AngleMaxIncrement property of a Correlation object. However, keep in mind the following regarding the AngleMaxIncrement property:

- If you provide a maximum angle increment, the model training function selects an angular increment automatically and uses the smaller of the automatically selected increment and the maximum angle increment you provide.
- If you set the AngleMaxIncrement property to 0, the model teaching function selects an angle increment automatically and uses that angle increment. In this case the system typically sets an angle increment between 2-5 degrees. This results in the smallest model storage requirements and the fastest search times, but may produce results that are more coarse than desired.

If you wish to measure angle precisely, you should set the AngleMaxIncrement property to an increment corresponding to the degree of precision you desire. Keep in mind though, that the smaller the angle increment, the more storage will be required for the model and the slower the search time will be.

NOTE

We recommend using the Polar object to determine angle whenever possible. This provides much more reliable and accurate results that are required using vision for robot guidance.

Keep in mind that when training models with angle, the search window must be large enough to allow the model to be rotated without any part of the model going outside of the search window.

Searching Repeatability and Accuracy

Searching repeatability and accuracy is a function of the size and details of the Model (shape, coarseness of features, and symmetry of the model), and the degradation of the features as seen in the search window (noise, defects, and rotation and scale effects).

To measure the effect of uncorrelated noise on position, you can perform a search in a specific search window that contains an non-degraded feature, and then perform the exact same search again (acquiring a 2nd image into the frame buffer) without changing the position of the object, and then comparing the measured positions. This can be easily done by clicking on the Run Object button of the Object tab (after a Model is defined of course) 2 or more times and then clicking on the Statistics toolbar button. The Statistics dialog can then be used to see the difference in position between the 2 object searches. For a large model (30x30) on a non-degraded feature the reported position can be repeatable to 1/20 of a pixel. However, in most cases it is more realistic to achieve results of just below a pixel. (1/2, 1/3, or 1/4 pixel)

Searching accuracy can be measured by performing a search in a specific search window that contains an undegraded feature, moving the object an exact distance and then comparing the reported position difference with the actual difference. If you have a large model (30x30 or greater), no degradation, no rotation or scale errors, and have sufficient edges in both X and Y directions, searching can be accurate to 1/4 pixel. (Keep in mind that this searching accuracy is for the vision system only and does not take into effect the inaccuracies that are inherent with all robots. So if you try to move the part with the robot you must also consider the inaccuracies of the robot mechanism itself.)

The effects of rotation and scale on searching accuracy depend on the Model:

- Models that are rotationally symmetric do well.
- Models that have fine features and no symmetry do not do well.

Calibrating the Camera to Subject Distance

For optimal searching results, the size of the features in an image should be the same at search time as it was when the model was taught. Assuming the same camera and lens are used, if the camera to subject distance changes between the time the model is trained and the time the search is performed, the features in the search window will have a different apparent size. That is, if the camera is closer to the features they will appear larger; if the camera is farther away they will appear smaller.

NOTE

If the camera to subject distance changes, you must re-train the model.

Using Correlation Objects

Now that we've reviewed how normalized correlation and searching works we have set the foundation for understanding how to use Vision Guide Correlation objects. This next section will describe the steps required to use Correlation objects as listed below:

- Create a new Correlation object
- Position and Size the search window
- Position and size the model window
- Position the model origin
- Configure properties associated with the Correlation object
- Teach the Model
- Test the Correlation object & examine the results
- Make adjustments to properties and test again
- Working with Multiple Results from a Single Correlation object

Prior to starting the steps shown below, you should have already created a new vision sequence or selected a vision sequence to use. If you have no vision sequence to work with, you can create a new vision sequence by clicking on the New Sequence  toolbar button. You can also select a sequence which was created previously by clicking on the Sequence tab in the Vision Guide window and clicking on the drop down list box which is located towards the top of the Sequence tab. See the chapter *Vision Sequences* for more details on how to create a new vision sequence or select one that was previously defined.

Step 1: Create a new Correlation object

(1) Click on the New Correlation toolbar button  on the Vision Guide Toolbar. (In this case click means to click and release the left mouse button)

(2) The mouse cursor will change to a correlation icon.

(3) Move the mouse cursor over the image display of the Vision Guide window and click the left mouse button to place the Correlation object on the image display

(4) Notice that a name for the object is automatically created. In the example, it is called "Corr01" because this is the first Correlation object created for this sequence. (We will explain how to change the name later.)

Step 2: Position and Size the Search Window

You should now see a Correlation object similar to the one shown below:

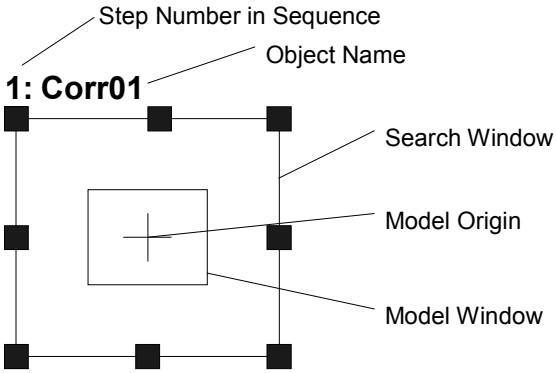


Figure 38: New Correlation object layout

(1) Click on the name label (or on one of the sides) of the Correlation object and while holding the mouse down drag the Correlation object to the position where you would like the top left position of the search window to reside.

(2) Resize the Correlation object search window as required using the search window size handles. (This means click on a size handle and drag the mouse.) The search window is the area within which we will search.

CAUTION	■ Ambient lighting and external equipment noise may affect vision sequence image and results. A corrupt image may be acquired and the detected position could be any position in an object's search area. Be sure to create image processing sequences with objects that use search areas that are no larger than necessary.

Step 3: Position and size the model window

(1) The search window for the Correlation object you want to work on should be magenta in color and the sizing handles should be visible on each corner and along the middle of each side of the search window. If you cannot see the size handles click on the name field of the Correlation object. Once you can see the sizing handles of the Correlation object you want to work on and it is magenta in color go to step 2.

(2) Click on one of the lines of the box that forms the model window (the square formed inside the search window). This will cause the model window to be highlighted. (You should see the size handles on the model window now.)

(3) Move the mouse pointer over one of the lines of the box which forms the model window and while holding the mouse down drag the model window to the position where you would like the top left position of the model window to reside.

(4) Resize the model window as required using the model window size handles. (This means click on a size handle and drag the mouse.) (The model window should now be outlining the feature that you want to teach as the model for this Correlation object.)

Your Correlation object layout should now look something like the example in Figure 39 where the search window covers the area to be searched and the model window outlines the feature you want to search for. Of course your search window and Model will be different but this should give you an idea of what was expected so far.

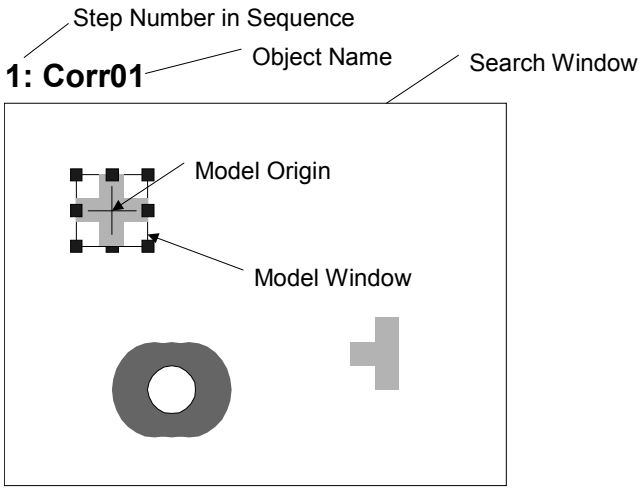


Figure 39: Correlation object after search and model window positioning and resizing

Hints on Setting Proper Size and Position for the model window: The size and position of the model window is very important since it defines the feature to be searched for. When creating a model window for a Correlation object, there are 2 primary items you must pay attention to: (1) If you expect that your part will rotate much, smaller models are better because they require smaller search windows to Search within. (2) Making the model window as close in size to the search window as possible can reduce execution time.

It is also sometimes a good idea to make the model window just a bit larger than the actual feature you are interested in. This will give the feature some border that may prove useful in distinguishing this object from others. Especially when 2 objects are positioned right next to each other and are touching. However, this additional border should only be a few pixels wide. Keep in mind that each vision application is different so the best model window sizing technique will vary from application to application.

Step 4: Position the Model Origin

The model origin defines the position on the model that will be returned back as the position of the feature when you run the Correlation object. This means that the model origin should be placed in a place of significance if the position data is important. For example, when using a Correlation object to find parts for a robot to pick up or place, it is important that the position of the model origin is in a location where the robot can easily grip the part. This is because that will be the position the robot will move to based on the RobotXYU, RobotX or RobotY result.

When a new Correlation object is created the ModelOrgAutoCenter property is set to True. (True is the default value for the ModelOrgAutoCenter property) This means that the model origin is set to the center of the model window automatically and cannot be moved manually. However if you want to move the model origin manually, you must first set the ModelOrgAutoCenter property to False. The steps to do this and also actually position the model origin are shown below.

(1) Look at the Object tab that you just brought forward in Step 4 as explained on the previous page. Find the ModelOrgAutoCenter property on the properties list and click in the value field.

(2) You will see a drop down list with 2 choices: True and False. Click on the False choice. You have now set the ModelOrgAutoCenter property to False and can move the model origin with the mouse.

(3) Click on the model window to highlight the model window.

(4) Click on the model origin and keep the mouse button held down while dragging the model origin to a new position. It should be noted that the model origin can only be positioned within the bounds of the model window.

Step 5: Configure the Correlation object properties

We can now set property values for the Correlation object. To set any of the properties simply click on the associated property's value field and then either enter a new value or if a drop down list is displayed click on one of the items in the list.

Shown below are some of the more commonly used properties for the Correlation object. Explanations for other properties such as AbortSeqOnFail, Graphics, etc. which are used on many of the different vision objects can be seen in the Vision Properties and Results Reference Manual or in the "Correlation Properties" list on page 108 earlier in this chapter. We shouldn't have to set any of these properties to test our Correlation object. However, you may want to read through this information if you are working with Correlation objects for the first time.

CAUTION

■ Ambient lighting and external equipment noise may affect vision sequence image and results. A corrupt image may be acquired and the detected position could be any position in an object's search area. Be sure to create image processing sequences with objects that use search areas that are no larger than necessary.

Name property - The default name given to a newly created Correlation object is "Corrxx" where xx is a number which is used to distinguish between multiple Correlation objects within the same vision sequence. If this is the first Correlation object for this vision sequence then the default name will be "Corr01". To change the name, click on the Value field of the Name property, type a new name and hit return. You will notice that once the name property is modified, every place where the Correlation object's name is displayed is updated to reflect the new name.

Accept property - The Accept property sets the shape score that a feature must meet or beat to be considered Found. The value returned in the Score result is compared against this Accept property Value. The default value is 700 which will probably be fine before we try running the Correlation object for the first time.

Confusion property - If there are many features within the search window which look similar, the Confusion property is useful to help "home in" on the exact feature you want to find. The default value is 800 that will probably be fine before running the Correlation object for the first time.

ModelOrgAutoCenter property

- If you want to change the position of the model origin you must first set the ModelOrgAutoCenter property to False. This property is set to True by default.

Frame property - Allows you to select a previously defined Frame object as a reference frame for this Correlation object. The details for Frames are defined in the Frame object section of this chapter.

NumberToFind property - Depending upon the number of features you want to find, you may want to set the NumberToFind property larger than 1. This will allow one Correlation object to find multiple features within one search window.

AngleEnable property - You must set this property to True if you want to use a Correlation model to search with angle. This must be set to True at the time you teach the Model so that multiple models can be configured.

AngleMaxIncrement and AngleRange properties

- These properties are used along with the AngleEnable property for using the Correlation model to search with angle.

RejectOnEdge property - Allows you to exclude the parts touching the boundary of the search window. Normally, this should be set to True.

You can probably go ahead with the properties at their default settings and then come back and set the any other properties as required later.

Step 6: Teach the model for the Correlation object

The Correlation object needs a model to search for and this is accomplished a process called teaching the model. You should have already positioned the model window for the Correlation object to outline the feature that you want to use as a model. Teaching the model is accomplished as follows:

(1) Make sure that the Correlation object is the currently displayed object in the Object tab. You can look at the current object drop down list at the top of the Object tab to see which object is the object you are currently working on or you can look at the image display and see which object is highlighted in magenta.

(2) Click on the Teach button at the bottom right of the Object tab. It will take only a few seconds for the Model to be taught in most cases. However, if you are teaching a model when the AngleEnable property is set to True it can take quite a few seconds to teach the model because the system is actually teaching many models each at a slight angle offset from the previous.

Step 7: Test the Correlation object / examine the results

To run the Correlation object, click the Run Object button located at the bottom left of the Object tab.

Results for the Correlation object will now be displayed. The primary results to examine at this time are:

Found result - Returns whether or not the Correlation was found. If the feature you are searching for is found this result returns as True. If the feature is not found, the Found result returns a False and is highlighted in red. If the feature was not found read on to Step 9 for some of the more common reasons why a Correlation object is not found.

FoundOnEdge result
- This result will return as True if the feature was found where a part of the feature is touching the boundary of the search window. In this case the Found result will return as False.

Score result - This tells us how well we matched the model with the feature that most closely resembles the model. Score results range from 0 to 1000 with 1000 being the best match possible. Examine the Score result after running a Correlation object as this is your primary measure of how well the feature was found.

Time result - The amount of time it took for the Correlation object to execute. Remember that small search windows and small Models help speed up the search time.

NumberFound result
- When searching for more than 1 Correlation object, the NumberFound result returns the number of features that matched the Correlation object's Model.

Angle result - The angle at which the Correlation is oriented. This is computed based on the original angle of the Model. However, this value may sometimes be coarse in value and not so reliable. (We strongly recommend using the Polar object for finding Angles. Especially for robot guidance.)

PixelX, PixelY results

- The XY position (in pixels) of the feature. Remember that this is the position of the model origin with respect to the found feature. If you want to return a different position, you must first reposition the model origin and then re-teach the Model.

CameraX, CameraY results

- These define the XY position of the found feature in the Camera's Coordinate system. The CameraX and CameraY results will only return a value if the camera has been calibrated. If it has not then [No Cal] will be returned.

RobotX, RobotY results

- These define the XY position of the found feature in the Robot's Coordinate system. The robot can be told to move to this XY position. (No other transformation or other steps are required.) Remember that this is the position of the model origin with respect to the found feature. If you want to return a different position, you must first reposition the model origin and then re-teach the Model. The RobotX and RobotY results will only return a value if the camera has been calibrated. If it has not then [No Cal] will be returned.

RobotU result - This is the angle returned for the found feature translated into the Robot's Coordinate system. The RobotU result will only return a value if the camera has been calibrated. If it has not then [No Cal] will be returned.

ShowAllResults - If you are working with multiple results, you may want to click on the button in the ShowAllResults value field. This will bring up a dialog to allow you to examine all the results for the current vision object.

NOTE

The RobotX, RobotY, RobotU, RobotXYU and CameraX, CameraY, CameraU, CameraXYU results will return [no cal] at this time since we have not done a calibration in the example steps described above. This means that no calibration was performed so it is impossible for the vision system to calculate the coordinate results with respect to the Robot coordinate system or Camera coordinate system. See the chapter Calibration for more information.

Step 8: Make adjustments to properties and test again

After running the Correlation object a few times, you may have encountered problems with finding a Correlation or just want to fine-tune some of the property settings. Some common problems and fine-tuning techniques are described in the next section called "Correlation object Problems".

Correlation Object Problems

If the Correlation object returns a Found result of False, there are a few places to immediately examine.

- Look at the Score result which was returned. Is the score result lower than the Accept property setting. If the Score result is lower, try changing the Accept property a little lower (for example, below the current score result) and run the Correlation object again.
- Look at the FoundOnEdge result. Does it have a return value of True? If it is True, this means that the feature was found but it was found where a part of the feature is touching the search window. This causes the Found result to be returned as False. To correct this situation, make the search window larger or if this is impossible, try changing the position of the camera, or resizing the model window.

If the Correlation object finds the wrong feature, there are a couple of places to look:

- Was the Accept property set high enough? If it is set rather low this could allow another feature to be found in place of the feature you are interested in.
- Was the Confusion property set high enough? Is it higher than the Accept property? The Confusion property should normally be set to a value equal to or higher than the Accept property. This is a good starting point. But if there are features within the search window which are similar to the feature you are interested in, then the Confusion property must be moved to a higher value to make sure that your feature is found instead of one of the others.
- Adjust the search window so that it more closely isolates the feature that you are interested in.

Correlation Object Fine Tuning

Fine-tuning of the Correlation object is normally required to get the object working just right. The primary properties associated with fine-tuning of a Correlation object are described below:

Accept property - After you have run the Correlation object a few times, you will become familiar with the shape score which is returned in the Score result. Use these values when determining new values to enter for the Accept property. The lower the Accept property, the faster the Correlation object can run. However, lower Accept property values can also cause features to be found which are not what you want to find. A happy medium can usually be found with a little experimentation that results in reliable feature finds that are fast to execute.

Confusion property - If there are multiple features within the search window which look similar, you will need to set the Confusion property relatively high. This will guarantee that the feature you are interested in is found rather than one of the confusing features. However, higher confusion costs execution speed. If you don't have multiple features within the search window that look similar, you can set the Confusion property lower to help reduce execution time.

Once you have completed making adjustments to properties and have tested the Correlation object until you are satisfied with the results, you are finished with creating this vision object and can go on to creating other vision objects or configuring and testing an entire vision sequence.

Other Useful Utilities for Use with Correlation Objects

At this point you may want to consider examining the histogram feature of Vision Guide. Histograms are useful because they graphically represent the distribution of gray-scale values within the search window. The details regarding Vision Guide histogram usage are described in *Histograms and Statistics* Tools.

You may also want to use the statistics feature of Vision Guide to examine the Correlation object's results statistically. An explanation of the Vision Guide statistics features are explained in *Histograms and Statistics* Tools.

5.4.4 Blob Object

Blob Description

Blob objects compute geometric, topological and other features of images. Blob objects are useful for determining presence/absence, size and orientation of features in an image. For example, Blob objects can be used to detect the presence, size and location of ink dots on silicon wafers, for determining the orientation of a component, or even for robot guidance.

Some of the features of the computed by Blob objects are:

- Area and perimeter
- Center of mass
- Principal axes and moments
- Connectivity
- Extrema
- Coordinate positions of the center of mass in pixel, camera world, and robot world Coordinate systems
- Holes, roughness, and compactness of blobs

Blob Object Layout

The Blob object layout is rectangular just like the correlation objects. However, Blob objects do not have models. This means there is no need for a model window or model origin for the Blob object layout. As shown below, Blob objects only have an object name and a search window. The search window defines the area within which to search for a blob. The Blob object layout is shown below:

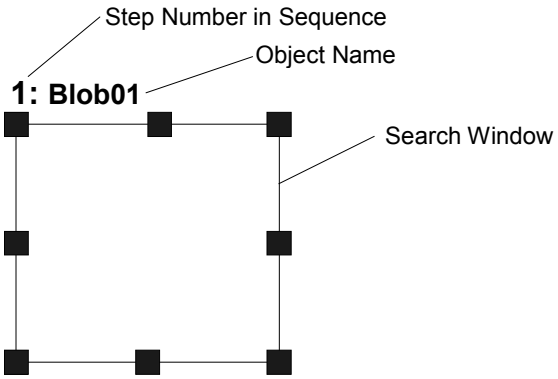


Figure 40: Blob Object Layout

Blob Properties

The following list is a summary of properties for the Blob object. The details for each property are explained in the Vision Properties and Results Reference Manual.

Property	Description
AbortSeqOnFail	Allows the user to specify that if the specified object fails (not found), then the entire sequence is aborted at that point and no further objects in the sequence are processed. Default: 0 - False
BackColor	Sets the background color for the object's label. Default: 0 - None
Caption	Used to assign a caption to the Blob object. Default: Empty String
CenterPntObjResult	Specifies which result to use from the CenterPointObject. Default: 1
CenterPntOffsetX	Sets or returns the X offset of the center of the search window after it is positioned with the CenterPointObject. Default: 0
CenterPntOffsetY	Sets or returns the Y offset of the center of the search window after it is positioned with the CenterPointObject. Default: 0
CenterPointObject	Specifies the position to be used as the center point for the Blob Search. This property is normally set to base the center of the Blob Search on the XY position result from another vision object. Default: Screen
CurrentResult	Defines which result to display in the Results list on the Object tab or which result to return data for when the system is requested to find more than one result. Default: 1
FoundColor	Selects the color for an object when it is found. Default: 1 – Light Green
Frame	Specifies which positioning frame to use. Default: None
Graphics	Specifies which graphics to display. Default: 1 - All
MaxArea	Defines the upper Area limit for the Blob object. For a Blob to be found it must have an Area result below the value set for MaxArea property. Default: 100,000
MinArea	Defines the lower Area limit for the Blob object. For a Blob to be found it must have an Area result above the value set for MinArea property. Default: 25
MinMaxArea	Runtime only. Sets or returns both MinArea and MaxArea in one statement.

Property	Description
Name	Used to assign a unique name to the Blob object. Default: Blobxx
NumberToFind	Defines the number of blobs to find in the search window. Default: 1
Polarity	Defines the differentiation between objects and background. (Either "Dark Object on Light Background" or "Light Object on Dark Background".) Default: 1 - DarkOnLight
RejectOnEdge	Determines whether the part will be rejected if found on the edge of the search window. Default: 0 - False
SearchWin	Runtime only. Sets or returns the search window left, top, height, width parameters in one call.
SearchWinHeight	Defines the height of the area to be searched in pixels. Default: 100
SearchWinLeft	Defines the left most position of the area to be searched in pixels.
SearchWinTop	Defines the upper most position of the area to be searched in pixels.
SearchWinWidth	Defines the width of the area to be searched in pixels. Default: 100
SizeToFind	Selects which size of blobs to find. Default: 0 - Any
Sort	Selects the sort order used for the results of an object. Default: 0 – None
ThresholdColor	Defines the color assigned to pixels withing the thresholds. Default: Black
ThresholdHigh	Works with the ThresholdLow property to define the gray level regions that represent the feature (or object), the background and the edges of the image. The ThresholdHigh property defines the upper bound of the gray level region for the feature area of the image. Any part of the image that falls within gray level region defined between ThresholdLow and ThresholdHigh will be assigned a pixel weight of 1. (i.e. it is part of the feature.) Default: 100
ThresholdLow	Works with the ThresholdHigh property to define the gray level regions that represent the feature (or object), the background and the edges of the image. The ThresholdLow property defines the upper bound of the gray level region for the feature area of the image. Any part of the image that falls within gray level region defined between ThresholdLow and ThresholdHigh will be assigned a pixel weight of 1. (i.e. it is part of the feature.) Default: 10

Blob Results

The following list is a summary of the Blob object results with brief descriptions. The details for each result are explained in the Vision Guide Properties and Results Reference Manual.

Results	Description
Angle	Returns the amount of part rotation in degrees.
Area	Returns the area of the blob in pixels.
CameraX	Returns the X coordinate position of the found part's position in the camera coordinate frame.
CameraY	Returns the Y coordinate position of the found part's position in the camera coordinate frame.
CameraXYU	Runtime only. Returns the CameraX, CameraY, and CameraU coordinates of the found part's position in the camera coordinate frame.
Compactness	Returns the compactness of a blob.
Extrema	Runtime only. Returns MinX, MaxX, MinY, MaxY pixel coordiates of the blob Extrema.
Found	Returns whether or not the object was found. (i.e. was a Connected Blob found which has an Area result that falls between the MinArea and MaxArea properties.)
FoundOnEdge	Returns TRUE when a Blob object is found too close to the edge of the search window.
Holes	Returns the number of holes found in the blob.
MaxX	Returns the maximum X pixel coordinate of the blob Extrema in pixels.
MaxY	Returns the maximum Y pixel coordinate of the blob Extrema in pixels.
MinX	Returns the minimum X pixel coordinate of the blob Extrema in pixels.
MinY	Returns the minimum Y pixel coordinate of the blob Extrema in pixels.
NumberFound	Returns the number of blobs found within the search window. (This number can be anywhere from 0 up to the number of blobs you requested the Blob object to find with the NumberToFind property.)
Perimeter	The number of pixels along the outer edge of the found blob.
PixelX	Returns the X coordinate position of the found part's position in pixels.
PixelY	Returns the Y coordinate position of the found part's position in pixels.
PixelXYU	Runtime only. Returns the PixelX, PixelY, and PixelU coordinates of the found part's position in pixels.

Results	Description
RobotX	Returns the X coordinate position of the found part's position with respect to the Robot's World Coordinate System.
RobotY	Returns the Y coordinate position of the found part's position with respect to the Robot's World Coordinate System.
RobotU	Returns the U coordinates of the found part's position with respect to the Robot's World Coordinate System
RobotXYU	Runtime only. Returns the RobotX, RobotY, and RobotU coordinates of the found part's position with respect to the Robot's World Coordinate System.
Roughness	Returns the roughness of a blob.
ShowAllResults	Displays a dialog which allows you to see all results for a specified vision object in a table form. This makes it easy to compare results when the NumberToFind property is set to 0 or greater than 1.
Time	Returns the amount of time in milliseconds required to process the object.
TotalArea	Returns the sum of areas of all results found.

How Blob Analysis Works

Blob analysis processing takes place in the following steps:

1. Segmentation, which consists of
   - Thresholding
   - Connectivity analysis
2. Blob results computation

Segmentation

In order to make measurements of an object's features, blob analysis must first determine where in the image an object is: that is, it must separate the object from everything else in the image. The process of splitting an image into objects and background is called **segmentation**.

The Blob object is intended for use on an image that can be segmented based on the grayscale value of each of its pixels. A simple example of this kind of segmentation is thresholding.

While the Blob object described in this chapter produces well defined results even on arbitrary gray scale images that cannot be segmented by gray-scale value, such cases are typically of limited utility because the results are influenced by the size of the window defining the image.

For whole image blob analysis, the feature of interest must be the only object in the image having a particular gray level. If another object in the image has the same gray-scale value, the image cannot be segmented successfully. Figure 41 through Figure 44 illustrate which can and cannot be segmented by gray-scale value for whole image blob analysis.

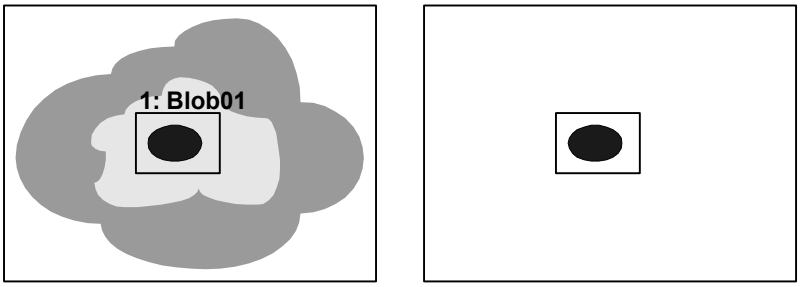


Figure 41: Scene that can be segmented by gray-scale value

Figure 41 shows the camera's field of view (left). The scene to be processed using a Blob object falls within the search window labeled "Blob01". After segmentation by gray-scale value, the object and the background are easily distinguishable as shown on the right side of Figure 41.

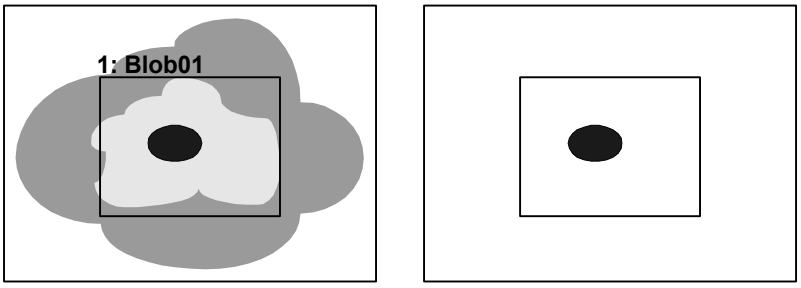


Figure 42: Scene from Figure 41 with larger search window

Changing the size of the search window as shown in Figure 42, changes only the size of the background. It has no effect on the features of the blob.

Figure 43 and Figure 44 show a similar field of view for an image. However, this scene cannot be segmented by gray-scale value because there are two objects in the image which have the same gray-scale value. Only eliminating one or the other from within the search window can separate these objects. While the image in Figure 43 could be segmented by gray-scale value, enlarging the search window as shown in Figure 44, completely changes the segmented image. For this scene, both the area of the background and the measured features of the blob vary depending on the size of the search window and the portion of the image it encloses.

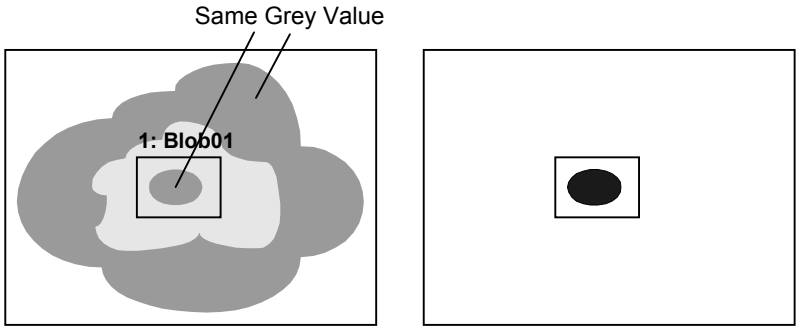


Figure 43: Scene that cannot be segmented by gray-scale value

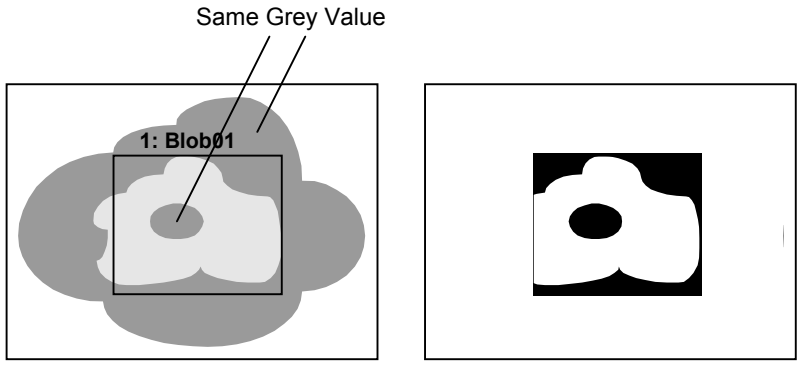


Figure 44: Enlarged search window encloses two objects with the same gray scale value

You should also watch out for situations as shown in Figure 45. In this example, the inner blob and part of the background have become connected. This forms one large blob that has very different features from the original center blob that we are trying to segment.

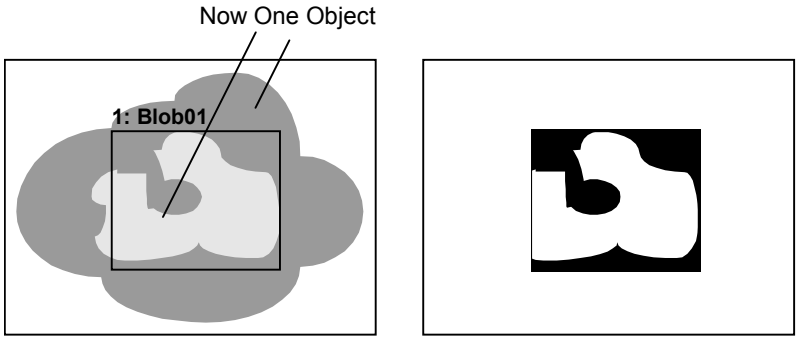


Figure 45: Object and background have been connected

Thresholding

The Blob object uses thresholding to determine the weight of each pixel in an image. Two user defined thresholds are used: ThresholdLow and ThresholdHigh. Pixels whose grayscale values are between the thresholds are assigned a pixel weight of 1 and all others 0. The ThresholdColor property is used to define the color of the pixels for weight 1. This is the color (black or white) between the thresholds.

Based on these derived pixel weights, the Blob object segments the image into feature (pixels having weight 1) and background (pixels having weight 0). The Polarity property is used to configure the Blob object to find blobs containing either black or white pixels. When Polarity is DarkOnLight, then blobs contain black pixels. When Polarity is LightOnDark, then blobs contain white pixels.

Using Histograms to Determine Thresholds

By using the Vision Guide Histogram tool, the user can determine which values to use for ThresholdLow and ThresholdHigh.

As an example, consider an ideal binary image of a black blob on a white background. Figure 46 illustrates such an image and its histogram.

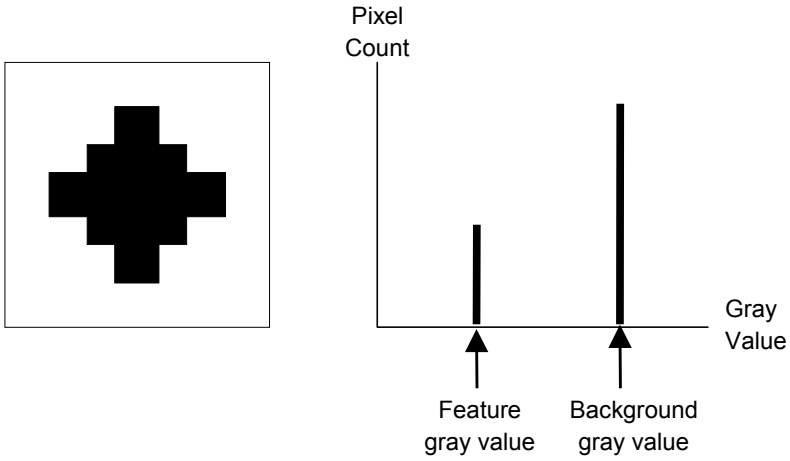


Figure 46: Ideal binary image and its histogram

Note that only two gray-scale values have non-zero contents in this histogram: the gray-scale value of the blob and the gray-scale value of the background.

Real images never have histograms such as this. The effects of noise from various sources (uneven printing, irregular lighting and electrical noise, for example) combine to spread out the peaks. A more realistic histogram is shown in Figure 47.

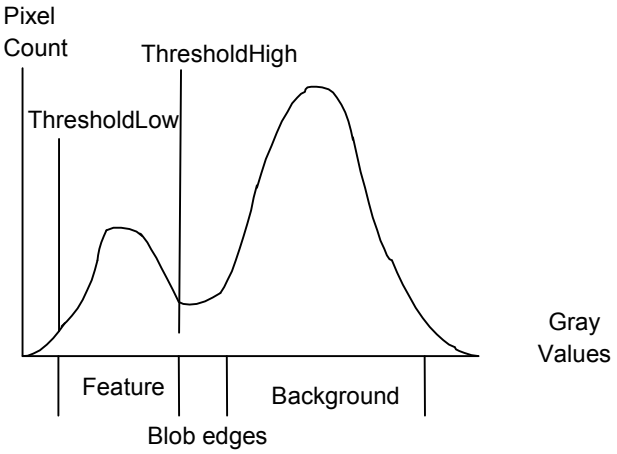


Figure 47: Setting the threshold values *using a histogram*

In the histogram in Figure 47, each of the peaks is clearly evident. The peak areas are in the same proportion as those in the previous ideal histogram, but each has now spread to involve more than one gray-scale value. The less populated grayscale values between the two principal peaks represent the edges of the blob, which are neither wholly dark nor wholly light. You should adjust the threshold values so that the blob feature will have pixels weights of 1.

Connectivity (Connected Blob Analysis)

Connectivity can be defined as analysis based on connected pixels having non zero weight. More simply stated, connectivity is used to find a group of connected pixels that are referred to as blobs. Connectivity is performed automatically by the Blob object where connectivity is performed and then measurements are computed for the blob(s) that are found. Connectivity for Blob objects returns the number of blobs found based upon the NumberToFind property that is set before running the Blob object.

Blob Results Computations

Once all the other steps for blob analysis are complete, results can be computed for the blob that was found. The list of all results returned for the Blob object are shown in the section called "Blob Results" on page 135 of this manual.

A detailed explanation of every result for all vision objects is given in the *Vision Properties and Results Reference Manual*.

Some of the results described in the *Vision Properties and Results Reference Manual* apply to many different vision objects such as the Found, Time, or PixelX results. While the Found and Time results are pretty generic and generally can be applied the same across all vision objects, some of the results such as the position related results have special meaning when applied to Blob objects. These are described below:

MinX, MinY, MaxX, MaxY Results

The MinX, MinY, MaxX and MaxY results when combined together create what is called the Extrema of the blob. The Extrema is the coordinates of the blob's minimum enclosing rectangle. The best way to understand this is to examine Figure 48.

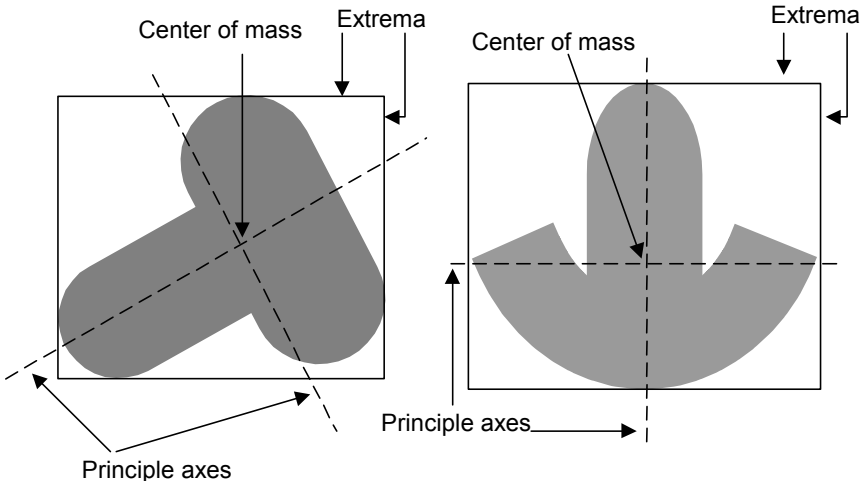


Figure 48: Principle axes, center of mass, and Extrema

Robot World, Camera World, and Pixel Position Data

Coordinate position results for the Blob object return the position of the center of mass. Keep in mind that the center of mass is not necessarily the center of the part. This can cause trouble for some parts if you try to use the center of mass to pick up the part. If you use the RobotX, RobotY and RobotU coordinate position results from a Blob object as a pick up position make sure that picking up the part at the center of mass is possible. If you don't want to pick up the part at the center of mass you will either have to calculate an offset or use another vision object such as a Correlation object to find the part and return a more useful position.

TotalArea Result

The TotalArea result is the sum of the areas for all results found. This is useful for pixel counting. By setting NumberToFind to 0, the Blob object will find all blobs with areas between MinArea and MaxArea. TotalArea will then show the total area of all results.

Angle Result Limitations for the Blob Object

Just a reminder that the Angle result for the Blob object is limited in its range. The Angle result for a Blob object returns angle values that range from +90 to -90. The Blob object is not able to return an angular result that ranges through an entire 360 degrees.

NOTE

It should be noted that a Blob object does not always return results as reliably as a Polar object. Because the range of the Blob object Angle result is limited and in some cases not reliable, we DO NOT recommend using a Blob object Angle result for robot guidance. Instead we strongly recommend using a Polar object to compute angular orientation of a part. The Polar object can use the X, Y position found as the center of mass from a Blob object and then compute an angle based from the center of mass of the Blob object. This is explained in detail later in the “Polar Object” section of this chapter.

Using Blob Objects

Now that we've reviewed how blob analysis works we have set the foundation for understanding how to use Vision Guide Blob objects. This next section will describe the steps required to use Blob objects as listed below:

- How to create a new Blob object
- Position and Size the search window
- Configure the properties associated with the Blob object
- Test the Blob object & examine the results
- Make adjustments to properties and test again

Prior to starting the steps shown below, you should have already created a new vision sequence or selected a vision sequence to use. If you have no vision sequence to work with, you can create a new vision sequence by clicking on the New Sequence  toolbar button. You can also select a sequence which was created previously by clicking on the Sequence tab in the Vision Guide window and clicking on the drop down list box which is located towards the top of the Sequence tab. See the chapter Vision Sequences for more details on how to create a new vision sequence or select one that was previously defined.

Step 1: Create a New Blob Object

(1) Click on the New Blob toolbar button on the Vision Guide Toolbar.  (In this case click means to click and release the left mouse button)

(2) Move the mouse over the image display. You will see the mouse pointer change to the Blob icon.

(3) Continue moving the mouse until the icon is at the desired position in the image display, then click the left mouse button to create the object.

(4) Notice that a name for the object is automatically created. In the example, it is called "Blob01" because this is the first Blob object created for this sequence. (We will explain how to change the name later.)

Step 2: Position and Size the Search Window

You should now see a Blob object similar to the one shown below:

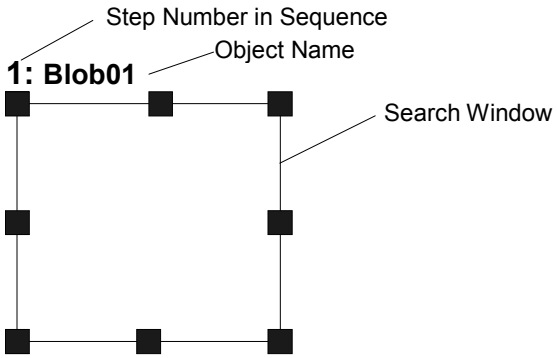


Figure 49: New Blob Object Layout

(1) Click on the name label of the Blob object and, while holding the mouse down, drag the Blob object to the position where you would like the top left position of the search window to reside.

(2) Resize the Blob object search window as required using the search window size handles. (This means click on a size handle and drag the mouse.) (The search window is the area within which we will search for Blobs.)

CAUTION	■ Ambient lighting and external equipment noise may affect vision sequence image and results. A corrupt image may be acquired and the detected position could be any position in an object's search area. Be sure to create image processing sequences with objects that use search areas that are no larger than necessary.

Step 3: Configure the Blob Object Properties

We can now set property values for the Blob object. Shown below are some of the more commonly used properties that are specific to the Blob object. Explanations for other properties such as AbortSeqOnFail, Graphics, etc. which are used on many of the different vision objects can be seen in the Vision Properties and Results Reference Manual or in the "Blob Properties" list earlier in this chapter.

CAUTION	■ Ambient lighting and external equipment noise may affect vision sequence image and results. A corrupt image may be acquired and the detected position could be any position in an object's search area. Be sure to create image processing sequences with objects that use search areas that are no larger than necessary.

Name property - The default name given to a newly created Blob object is "Blobxx" where xx is a number which is used to distinguish between multiple Blob objects within the same vision sequence. If this is the first Blob object for this vision sequence then the default name will be "Blob01". To change the name, click on the Value field of the Name property and type a new name and hit return. You will notice that every place where the Blob object's name is displayed is changed to reflect the new name.

Polarity property - Would you like to find a dark object on a light background or a light object on a dark background? Selecting between these is the fundamental purpose of the Polarity property. The default setting is for a dark object on a light background. If you want to change it, click on the Value field of the Polarity property and you will see a drop down list with 2 choices: DarkOnLight or LightOnDark. Click on the choice you want to use.

MinArea, MaxArea - These properties define the area limit for a Blob object to be considered "Found". (i.e. the Found result returned as True) The default range is set as 25-100,000 (MinArea, MaxArea) which is a very broad range. This means that most blobs will be reported as Found when you first run a new Blob object before adjusting the MinArea and MaxArea properties. Normally, you will want to modify these properties to reflect a reasonable range for the blob you are trying to find. This way if you find a blob which is outside of the range you will know it isn't the blob you wanted to find.

RejectOnEdge property - Allows you to exclude the parts touching the boundary of the search window. Normally, this should be set to True.

You can test the Blob object now and then come back and set the any other properties as required later.

Step 4: Test the Blob Object and Examine the Results

To run the Blob object, click on the Run object button located at the bottom left of the object Tab. Results for the Blob object will now be displayed. The primary results to examine at this time are shown below. There are others that you will find useful in the future as well though.

Found result - Returns whether or not the blob was found. If the blob that was found does not meet the area constraints defined by the MinArea and MaxArea properties then the Found result will return as False.

Area result - The area (in pixels) of the blob found.

Angle result - The angle at which the Blob is oriented. This is computed from the angle of the minor axis and will be a value between +/- 90 degrees.

Time result - The amount of time it took for the Blob object to execute.

PixelX, PixelY - The XY position (in pixels) of the center of mass of the found blob.

MinX, MinY, MaxX, MaxY
- Combined these 4 values define the Extrema of the blob. (A rectangle which is formed by touching the outermost points of the blob.)

NOTE

The RobotXYU, RobotX, RobotY, RobotU and CameraX, CameraY, CameraXYU results will return [no cal] at this time. This means that no calibration was performed so it is impossible for the vision system to calculate the coordinate results with respect to the Robot World or Camera World. See the chapter Calibration for more information.

Step 5: Make Adjustments to Properties and Test Again

After running the Blob object a few times, you may have encountered problems with finding a blob or just want to fine-tune some of the property settings. Some common problems and fine tuning techniques are described below:

Problems : If the Blob object returns a Found result of False, there are a few places to immediately examine.

- Look at the value defined for the Polarity property. Are you looking for a light object on a dark background or a dark object on a light background? Make sure that the Polarity property coincides with what you are looking for and with what you see within the search window.

- Look at the Area result and compare this area with the values defined in the MinArea and MaxArea properties. If the Area result does not fall between the limits defined by the MinArea and MaxArea properties, then you may want to adjust these properties and run the Blob object again.

- Use Histograms to examine the distribution of gray-scale values in an image. The Histogram tool is excellent for setting the ThresholdHigh and ThresholdLow properties. Histograms are described in detail in *Histograms and Statistics Tools*.

Fine Tuning : Fine-tuning of the Blob object may be required for some applications. The primary properties associated with fine-tuning of a Blob object are described below:

- MinArea, MaxArea - After you have run the Blob object a few times, you will become familiar with the approximate values returned for the Area result. Use these values when

determining new values to enter to the MinArea and MaxArea properties. It is generally a good idea to have MinArea And MaxArea properties set to values which constrain the Found result such that only blobs which you are interested in are returned with the Found result equal to True. (This helps eliminate unwanted blobs that are different in area from the desired blob.)

- ThresholdHigh, ThresholdLow - These properties adjust parameters for the setting the gray levels thresholds for distinguishing between what is background and what is part of the blob. These properties are best set through using the Histogram tool. Please refer to the descriptions of the ThresholdHigh and ThresholdLow properties in the *Vision Properties and Results Reference Manual*. Histograms are described in detail in *Histograms and Statistics Tools*.

Once you have completed making adjustments to properties and have tested the Blob until you are satisfied with the results you are finished with creating this vision object and can go on to creating other vision objects or configuring and testing an entire vision sequence.

Other Useful Utilities for Use with Blob Objects

At this point you may want to consider examining the Histogram  feature of Vision Guide. Histograms are useful because they graphically represent the distribution of gray-scale values within the search window. The Vision Guide Histogram tool provides a useful mechanism for setting the gray levels for the ThresholdLow and ThresholdHigh properties which then define what is considered a part of the blob and what is considered part of the background. When you are having problems with finding a blob the Histogram feature is invaluable. The details regarding the Vision Guide Histogram usage are described in *Histograms and Statistics Tools*.

You may also want to use the Statistics  feature of Vision Guide to examine the Blob object's results statistically. An explanation of the Vision Guide Statistics features are explained in the chapter *Histograms and Statistics Tools*.

Using Blob Objects as a Pixel Counter

The Blob object can be used as a pixel counter. A pixel counter counts all of the pixels in an image that fall within the blob thresholds.

Follow these steps:

(1) Create a Blob object.

(2) Set desired polarity.

(3) Set High and Low thresholds.

(4) Set NumberToFind to 0. This will cause the Blob object to find all blobs in the image.

(5) Set MinArea to 1 and MaxArea to 999999. Blobs of one pixel or more will be counted.

(6) Run the sequence.

Use the TotalArea result to read the total number of pixels that fall within the blob thresholds.

5.4.5 Edge Object

Edge Object Description

The Edge object is used to locate edges in an image. The edge of an object in an image is a change in gray value from dark to light or light to dark. This change may span several pixels. The Edge object finds the transition from Light to Dark or Dark to Light as defined by the Polarity property and defines that position as the edge position for a single edge. You can also search for edge pairs by changing the EdgeType property. With edge pairs, two opposing edges are searched for, and the midpoint is retured as the result. The Edge object supports multiple results, so you can specify how many single edges or edge pairs you want to find.

Edge objects are similar to Line objects in shape with a search length. The search length of the Edge object is the length of the Edge object. One of the powerful features of the Edge object is its ability to be positioned at any angle. This allows a user to maintain an Edge object vector that is perpendicular to the region where you want to find an edge by changing the angle of the Edge object. Normally this is done by making the Edge object relative to a Frame that moves with the region you are interested in.

Edge Object Layout

The Edge object has a different look than the Correlation and Blob objects. The Edge object's search window is the line along which the Edge object searches. The Edge object searches for a transition (light to dark or dark to light) somewhere along this line in the direction indicated by the Direction Indicator.

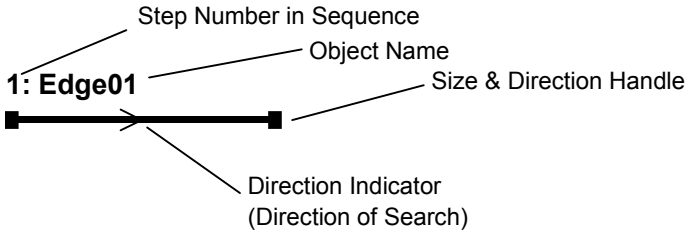


Figure 50: Edge Object Layout

The Edge object can be positioned to search in any direction (not just along the vertical and horizontal directions). This is done by using the size and direction handles of the Edge object to move the Edge object along the direction (and per a user-specified distance) required to find the edge you are interested in.

Edge Object Properties

The following list is a summary of properties for the Edge object. The details for each property are explained in the *Vision Properties and Results Reference Manual*.

Property	Description
AbortSeqOnFail	Allows the user to specify that if the specified object fails (not found), then the entire sequence is aborted at that point and no further objects in the sequence are processed. Default: 0 - False

Property	Description
Accept	Specifies the shape score that a feature must equal or exceed to be considered found. Default: 100
BackColor	Selects the background color for the object's label. Default: 0 - None
Caption	Used to assign a caption to the Edge object. Default: Empty String
ContrastTarget	Sets the desired contrast for the edge search. Default: 0 (best contrast)
ContrastVariation	Selects the allowed contrast variation for ContrastTarget. Default: 0
EdgeThreshold	Sets the threshold at which edges below this value are ignored. Default: 2
EdgeType	Select the type of edge to search for: single or pair. Default: 1 - Single
FoundColor	Selects the color for an object when it is found. Default: 1 – Light Green
Frame	Specifies which positioning frame to use. Default: none
Graphics	Specifies which graphics to display. Default: 1 - All
Name	Used to assign a unique name to the Edge object. Default: Edge01
NumberToFind	Defines the number of edges to find. Default: 1
Polarity	Defines whether the Edge object should search for a LightToDark or DarkToLight transition. Default: 1 - LightToDark
ScoreWeightContrast	Sets the percentage of the score that depends on contrast. Default: 50
ScoreWeightStrength	Sets the percentage of the score that depends on edge strength. Default: 50
SearchWidth	Defines the width of the edge search. Range is from 3 - 99. Default: 3
StrengthTarget	Sets the desired edge strength to search for. Default: 0
StrengthVariation	Sets the amount of variation for StrengthTarget. Default: 0
X1	The X coordinate position of the start point of the edge.
X2	The X coordinate position of the end point of the edge.
Y1	The Y coordinate position of the start point of the edge.
Y2	The Y coordinate position of the end point of the edge.

Edge Object Results

The following list is a summary of the Edge object results with brief descriptions. The details for each result are explained in the Vision Guide Properties and Results Reference Manual.

Results	Description
CameraX	Returns the X coordinate position of the found Edge's position in the camera coordinate frame.
CameraY	Returns the Y coordinate position of the found Edge's position in the camera coordinate frame.
CameraXYU	Runtime only. Returns the CameraX, CameraY, and CameraU coordinates of the found part's position in the camera coordinate frame.
Contrast	Returns the contrast of the found Edge.
Found	Returns whether or not the object was found. (i.e. did the feature or part you are looking at have a score that is above the Accept property's current setting.)
NumberFound	Returns the number of Edges found. (This number can be anywhere from 1 up to the number of edges you requested the Edge object to find with the NumberToFind property.)
PixelX	Returns the X coordinate position of the found Edge's position in pixels.
PixelY	Returns the Y coordinate position of the found Edge's position in pixels.
PixelXYU	Runtime only. Returns the PixelX, PixelY, and PixelU coordinates of the found part's position in pixels.
RobotX	Returns the X coordinate position of the found Edge's position with respect to the Robot's Coordinate System.
RobotY	Returns the Y coordinate position of the found Edge's position with respect to the Robot's Coordinate System
RobotXYU	Runtime only. Returns the RobotX, RobotY, and RobotU coordinates of the found Edge's position with respect to the Robot's World Coordinate System.
Score	Returns an integer value that represents the overall score of the found edge.
Strength	Returns the strength of the found edge.
Time	Returns the amount of time in milliseconds required to process the object.

Using Edge Objects

The next few sections guide you through how to create and use an Edge object.

- How to create a new Edge object
- Position and Size the search window
- Configure the properties associated with the Edge object
- Test the Edge object & examine the results
- Make adjustments to properties and test again

Prior to starting the steps shown below, you should have already created a new vision sequence or selected a vision sequence to use. If you have no vision sequence to work with, you can create a new vision sequence by clicking on the New Sequence toolbar button. You can also select a sequence which was created previously by clicking on the Sequence tab in the Vision Guide window and clicking on the drop down list box which is located towards the top of the Sequence tab. See Vision Sequences for details on how to create a new vision sequence or select one which was previously defined.

Step 1: Create a New Edge Object

(1) Click on the New Edge toolbar button on the Vision Guide Toolbar. (In this case click means to click and release the left mouse button).

(2) Move the mouse over the image display. You will see the mouse pointer change to the Edge object icon.

(3) Continue moving the mouse until the icon is at the desired position in the image display, then click the left mouse button to create the object.

(4) Notice that a name for the object is automatically created. In the example, it is called "Edge01" because this is the first Edge object created for this sequence. (We will explain how to change the name later.).

Step 2: Positioning the Edge Object

You should now see a Edge object similar to the one shown below:

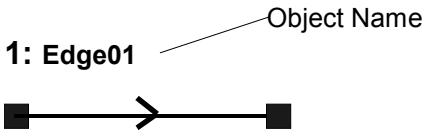


Figure 51: *New Edge Object*

Edge objects do not have a sizeable window. You can change the search length and rotation by clicking down on either size handle, and then dragging that end of the line to a new position. You can also click on the name label of the Edge object or anywhere along the edge line and while holding the mouse down drag the entire Edge object to a new location on the screen. When you find the position you like, release the mouse and the Edge object will stay in this new position on the screen.

Step 3: Configuring Properties for the Edge Object

We can now set property values for the Edge object. To set any of the properties simply click on the associated property's value field and then either enter a new value or if a drop down list is displayed click on one of the items in the list.

Shown below are some of the more commonly used properties for the Edge object. Explanations for other properties such as AbortSeqOnFail, Graphics, etc. which are used on many of the different vision objects can be seen in the *Vision Properties and Results Reference Manual*.

EdgeType (Single) - You can select whether to search for a single edge or an edge pair. For edge pairs, an edge is found from each direction and the center of the pair is reported as the position.

Name property ("Edgexx") - The default name given to a newly created Edge object is "Edgexx" where xx is a number which is used to distinguish between multiple Edge objects within the same vision sequence. If this is the first Edge object for this vision sequence then the default name will be "Edge01". To change the name, click on the Value field of the Name property, type a new name and hit return. You will notice that once the name property is modified, every place where the Edge object's name is displayed is updated to reflect the new name.

NumberToFind (1) - You can search for 1 or more edges along the search line.

Polarity (LightToDark) - You'll need to change polarity if you are looking for a DarkToLight edge.

Step 4: Running the Edge Object and Examining the Results

To run the Edge object, simply do the following:

Click on the Run Object button located at the bottom left of the Object tab.

Results for the Edge object will now be displayed. The primary results to examine at this time are:

PixelX, PixelY results - The XY position (in pixels) of the edge found along the edge search line.

CameraX, CameraY results - These define the XY position of the Edge object in the Camera's World Coordinate system. The CameraX and CameraY results will only return a value if the camera has been calibrated. If it has not then [No Cal] will be returned.

RobotX, RobotY results - These define the XY position of the Edge object in Robot World Coordinates. The robot can be told to move to this XY position. (No other transformation or other steps are required.) The RobotX and RobotY results will only return a value if the camera has been calibrated. If it has not then [no cal] will be displayed.

5.4.6 Polar Object

Polar Object Description

The Polar object provides a fast method of transforming an image having Cartesian coordinates to a corresponding image having polar coordinates. Polar objects are an excellent tool for determining object rotation. Because of the reliability and speed of the Polar object, we highly recommend using it when you need to calculate the angular rotation of an object.

The major characteristic to remember about Polar objects, is that they are highly insensitive to object rotation about their center, but intolerant of object translation in x-y space. This means that Polar objects are very good at calculating the rotation of an object, but only if that object's center position does not move. For this reason, Vision Guide has a CenterPoint property for the Polar object. The CenterPoint property allows the user to lock the Polar object's center point onto the position result from another object such as the Correlation and Blob objects. This means that if you first find the XY position with a Correlation or Blob object, the Polar object can then be centered on the XY position and then used to calculate the rotation of the object.

Polar Object Layout

The Polar object layout is quite different looking than the other object layouts described so far. However, its usage is quite similar. The Polar object has a circular basic layout and as such has a center and radius. The position of the Polar object can be moved by clicking on the name of the Polar object (or anywhere on the circle that makes up the outer perimeter) and then dragging the object to a new position.

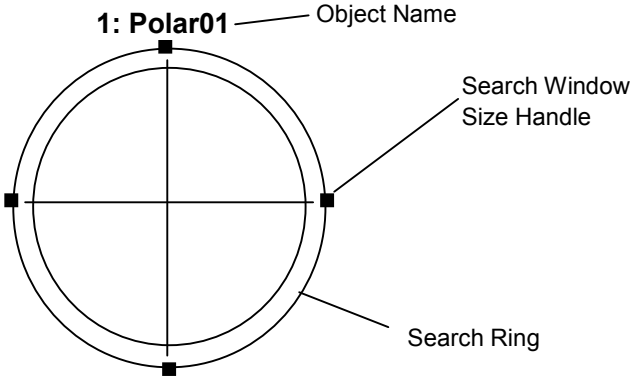


Figure 52: Polar Object Layout

NOTE

The Polar object center position (defined by the CenterPoint property) can also be based upon the position of another object. This means that even though you may reposition a Polar object, once you run the object or Sequence the Polar object center position may change. For more details see the Polar object later in this chapter.

The search window for a Polar object is circular. Its outer boundary is defined by the outer ring shown as shown below. Its inner ring is a circle that is smaller in size than the outer ring and is located n pixels inside the outer ring. The number of pixels is defined by the Thickness property. As you change the Thickness property you will notice that the "thickness" or distance between the inner and outer ring changes. This provides a visual indicator for the area in which you are searching.

To resize the search window outer boundary for the Polar object, click on one of the 4 search window size handles and drag the ring inward or outward as desired.

Polar Object Properties

The following list is a summary of properties for the Polar object. The details for each property are explained in the *Vision Properties and Results Reference Manual*.

Property	Description
AbortSeqOnFail	Allows the user to specify that if the specified object fails (not found), then the entire sequence is aborted at that point and no further objects in the sequence are processed. Default: 0 - False
Accept	Specifies the shape score that a feature must equal or exceed to be considered found. If the value is small, it may result in false detection. Default: 700
AngleOffset	Defines an offset angle which is applied to the Polar object after teaching to adjust the angle indicator graphic with the feature you are searching for. Default: 0
BackColor	Sets the background color for the object's label. Default: 0 - None
Caption	Used to assign a caption to the object. Default: Empty String
CenterPntObjResult	Specifies which result to use from the CenterPointObject. Default: 1
CenterPntOffsetX	Sets or returns the X offset of the center of the search window after it is positioned with the CenterPointObject. Default: 0
CenterPntOffsetY	Sets or returns the Y offset of the center of the search window after it is positioned with the CenterPointObject. Default: 0
CenterPointObject	Specifies the position to be used as the center point for the Polar Search. This property is normally set to base the center of the Polar Search on the XY position result from another vision object. Default: Screen
CenterX	Specifies the X coordinate position to be used as the center point for the Polar Search Tool. This property is filled in automatically when the CenterPoint property is set to another vision object.
CenterY	Specifies the Y coordinate position to be used as the center point for the Polar Search Tool. This property is filled in automatically when the CenterPoint property is set to another vision object.

Property	Description
Confusion	Indicates the amount of confusion expected in the image to be searched. This is the highest shape score a feature can get that is not an instance of the feature for which you are searching. Default: 800
FoundColor	Selects the color for an object when it is found. Default: 1 – Light Green
Frame	Specifies which positioning frame to use. Default: none
Graphics	Specifies which graphics to display. Default: 1 - All
ModelObject	Determines which model to use for searching. Default: Self
Name	Used to assign a unique name to the Polar object. Default: Polar01
Radius	Defines the distance from the CenterPoint of the Polar object to the outer most search ring of the object. Default: 50
ShowModel	Allows the user to see the internal grayscale representation of a taught model.
Thickness	Defines the thickness of the search ring which is the area searched for the Polar object. The thickness is measured in pixels starting from the outer ring defined by the Radius of the Polar object. Default: 5

Polar Object Results

The following list is a summary of the Polar object results with brief descriptions. The details for each result are explained in the *Vision Guide Properties and Results Reference Manual*.

Results	Description
Angle	Returns the amount of part rotation in degrees.
CameraX	Returns the X coordinate position of the Polar object's CenterPoint position in the camera coordinate frame.
CameraY	Returns the Y coordinate position of the Polar object's CenterPoint position in the camera coordinate frame.
CameraXYU	Runtime only. Returns the CameraX, CameraY, and CameraU coordinates of the found part's position in the camera coordinate frame. Values are in millimeters.
Found	Returns whether or not the object was found. (i.e. did the feature or part you are looking at have a shape score that is above the Accept property's current setting.)
PixelX	Returns the X coordinate position of the found part's position (referenced by model origin) in pixels.
PixelY	Returns the Y coordinate position of the found part's position (referenced by model origin) in pixels.
RobotX	Returns the X coordinate position of the Polar object's CenterPoint position with respect to the Robot's Coordinate System.
RobotY	Returns the Y coordinate position of the Polar object's CenterPoint position with respect to the Robot's Coordinate System.
RobotU	Returns the U coordinate position of the Polar object's Angle result with respect to the Robot's Coordinate System.
RobotXYU	Runtime only. Returns the RobotX, RobotY, and RobotU coordinates of the found part's position with respect to the Robot's World Coordinate System.
Score	Returns an integer value that represents the level at which the feature found at runtime matches the model for which Polar object is searching.
Time	Returns the amount of time in milliseconds required to process the object.

NOTE

All results for the Polar object which return X and Y position coordinates (CameraX, CameraY, PixelX, PixelY, RobotX, RobotY....) receive those coordinate values from the CenterPoint property. Whatever the CenterPoint property is set to before running the Polar object is then passed through as results for the Polar object. The Polar object does not calculate an X and Y center position. However, the X and Y position results are provided to make it easy to get X, Y and U results all from the Polar object rather than getting the X and Y results from one vision object and then the U results from the Polar object.

Understanding Polar Objects

The purpose of Polar objects is to find the amount of rotation of a specific object or pattern. This is done by first teaching a polar model which is basically a circular ring with a specified thickness. After teaching we can then run the Polar object and the circular ring is compared against the current rotation of the part we are interested in and an angular displacement is calculated and returned as one of the results for Polar objects.

Let's take a look at an example to help make it easier to see. Consider a part that rotates around a center point and needs to be picked up and placed on a pallet by a robot. Since this part is always oriented in a different position, it is difficult to pick up unless we can determine the rotation of the object. If this object looked something like the part shown in black & gray in Figure 53, we could define a Polar Model which intersects the outer part of the object. Notice the area within the inner and outer rings. This area defines the model of the Polar object that was taught for this part. You should be able to see that some areas within the ring are very distinctive because they have strong feature differences from the rest of the ring. These areas will be key in finding the amount of rotation the part will have.

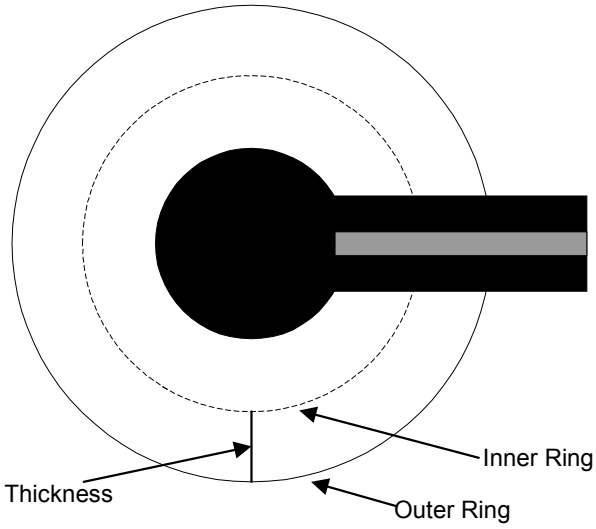


Figure 53: Example part for use with Polar Object

Figure 54 shows what the taught model looks like. It is just a ring that is mostly white with one section of black and gray. When the Polar object is run it will search the search window for this type of pattern where one section is black and gray while the rest of the ring is white.

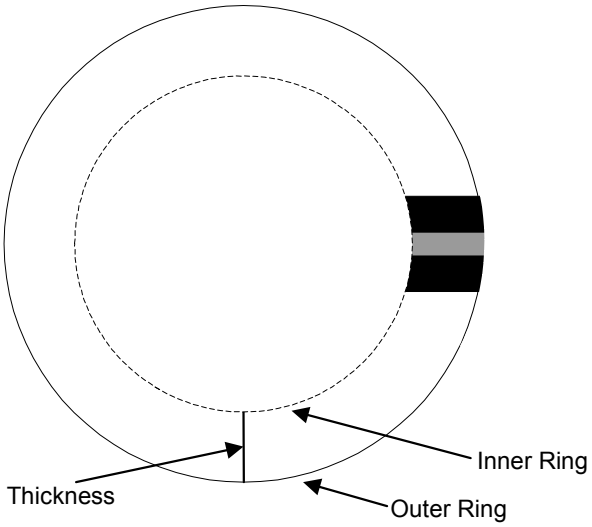


Figure 54: The Polar Model representation of the Part from Figure 53

In Figure 55 we show the original part which has rotated about 90 degrees as compared to the model. The Polar object will calculate the angular displacement between the model and the new position of the object.

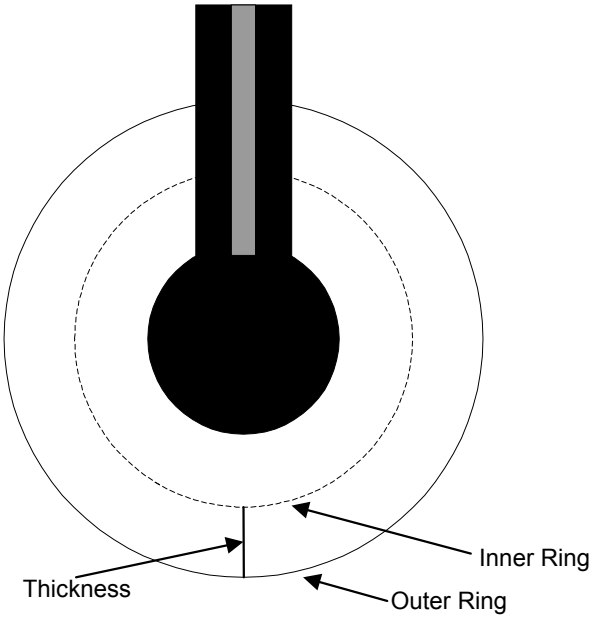


Figure 55: Part rotated 90 degrees from original taught model

Figure 56 shows the original Model taught at 0 degrees on the left and the part that was rotated 90 degrees on the right.

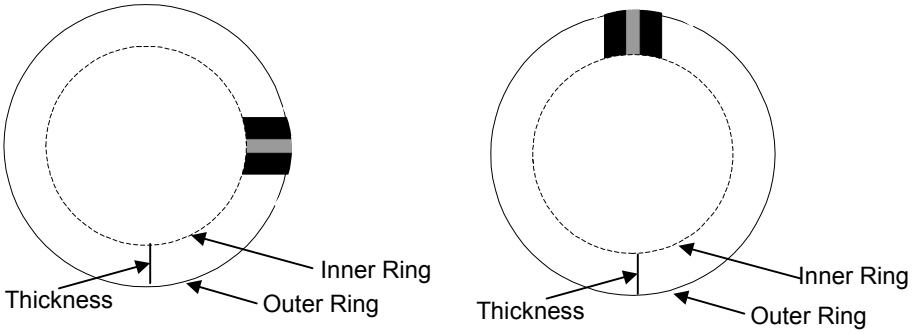


Figure 56: Original Polar Model and rotated part's Polar representation

The Primary Parameters Associated with Polar Objects

As you can probably derive from looking at the Figures in the last section, there are 3 primary parameters associated with Polar objects that are each important to achieve the best results possible. These parameters are defined by the following properties:

- CenterPoint property
- Radius property
- Thickness property
- AngleOffset property

The CenterPoint property defines the center position of the Polar object. As mentioned earlier, the center point of the Polar object must be aligned properly so that the center point of the taught model and the center point of the part you are searching for an angular shift within are perfectly aligned. Otherwise, the translation in XY space will cause the angular results to be inaccurate.

The Radius property defines the distance from the center of the Polar object to the outermost ring of the Polar object. This defines the outer Search Area boundary.

The Thickness property defines distance (in pixel units) from the outer ring to the imaginary inner ring. This is in effect the thickness of the Search Area.

The AngleOffset property provides a mechanism for the user to set the angular value for the graphics line that is used to display the rotational position for Polar objects. This line can be seen at 3 O'clock (the 0 degrees position) after running a Polar object. But since you may want a graphical indicator as to the rotational position of the feature you are most interested in, the Angle Offset property allows you to reposition this graphical indicator according to your needs. (Remember that this AngleOffset property is normally set after Teaching a Model and running the Polar object.)

Determining Object Rotation

A typical application requiring object rotation determination involves an integrated circuit die to be picked up by a robot that must know the die's XY position and its rotational orientation. The die is a different shade of gray from the background, and the surface of the die contains gray level information that indicates its rotation from 0 to 360 degrees.

For this application, a Blob object (called "Blob01") is used to find the center of the die. (Alternatively, you could also have used a Correlation object.) A Polar object is created using the XY position result from the Blob object as the center of the Polar object. (This is done by setting the Polar object's CenterPoint property to "Blob01".) The proper Radius and Thickness property values are then set for the Polar object. A gray level Polar Model is then trained providing a model of the die at 0 degrees.

When searching for a new die at a different orientation, a Polar search window is constructed from the XY position result from the "Blob01" Blob object, and the Radius and Thickness properties. This search window is then searched for the rotational equivalent of the model that was previously taught. The angle at which the model is found is then returned as the Angle result (in Pixels) and RobotU result (in Robot World coordinates).

NOTE

While the RobotU result returned will be the actual rotation of the part in the Robot's coordinate system, remember that your gripper was probably not mounted with exactly 0 degrees of rotation. It is probably off by at least a few degrees. So make sure to adjust for this in your program when moving to the part.

Adjusting the Taught Model's Angle Offset

When a Polar Model is taught, the original circular model is considered as 0 degrees where the 0-degree position is at 3 o'clock. Figure 57 shows a model which is taught where the area of interest is at about 1 o'clock. When this model is taught, the model's 0 degrees position will be at 3 o'clock as with all Polar Models. However, since we want to see a visual indication of the part's actual rotation we must use an Angle Offset to properly adjust the positioning of the polar angle indicator. For example, our part which was taught pointing at about 1 o'clock requires an Angle Offset of about 60 degrees. (The AngleOffset property for the Polar object should be set to 60 degrees.)

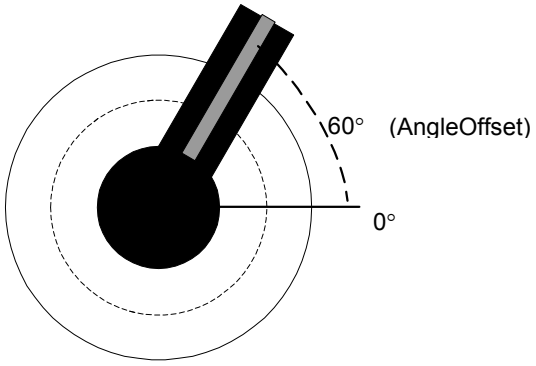


Figure 57: Polar Model where AngleOffset property is set to 60°

Performance Issues for Polar Objects

When teaching Polar objects, the primary concern is that the Polar Model contains enough information to determine the object's orientation. Therefore, the desired accuracy and execution speed of the Polar object determine the size of the Polar Model used for the Polar object search.

If the Polar object's rotational result is to be accurate to 1/2 a degree, then the Polar object need be only 180 pixels wide (2 degrees per pixel or 0.5 degrees per Polar object resolution unit, assuming quarter pixel searching accuracy).

Accuracy of the Polar object is also dependent on the gray level information in the Model.

The thickness of the Polar Model (in pixels) is also chosen to contain enough information that the rotation signature yields reliable results even when there is minor variation in the location of the center point of the image you are searching. Choosing a Thickness property setting of 1 pixel would mean that if the image were misplaced by one pixel, the polar transformation would send completely different pixels to the polar image. To provide some tolerance in the source image location, a Thickness property of 5 is used so that if the source image location is off by one pixel, the corresponding polar image pixel would only be one fifth off. (This is why the minimum value for the Thickness property is 5.)

The choice of the Polar object's Radius and Thickness properties should be based on the amount of gray information in the Model, and on the desired searching speed. Searching speed is proportional to the Radius and Thickness properties. The fastest Search times will result in a small Radius property setting with the Thickness property set to 5. However, in many cases a Thickness property set to 5 may not be enough to accurately find the Polar Model.

Using Polar Objects

The next few sections guide you through how to create and use an Polar object.

- How to create a new Polar object
- Position and Size the search window
- Configure the properties associated with the Polar object
- Test the Polar object & examine the results
- Make adjustments to properties and test again

Prior to starting the steps shown below, you should have already created a new vision sequence or selected a vision sequence to use. If you have no vision sequence to work with, you can create a new vision sequence by clicking on the New Sequence  toolbar button. You can also select a sequence which was created previously by clicking on the Sequence tab in the Vision Guide window and clicking on the drop down list box which is located towards the top of the Sequence tab. See Vision Sequences for details on how to create a new vision sequence or select one which was previously defined.

Step 1: Create a New Polar Object

(1) Click on the New Polar toolbar button  on the Vision Guide Toolbar. (In this case click means to click and release the left mouse button).

(2) Move the mouse over the image display. You will see the mouse pointer change to the Polar object icon.

(3) Continue moving the mouse until the icon is at the desired position in the image display, then click the left mouse button to create the object.

(4) Notice that a name for the object is automatically created. In the example, it is called "Polar01" because this is the first Polar object created for this sequence. (We will explain how to change the name later.).

Step 2: Positioning the Polar Object

You should now see a Polar object similar to the one shown below:

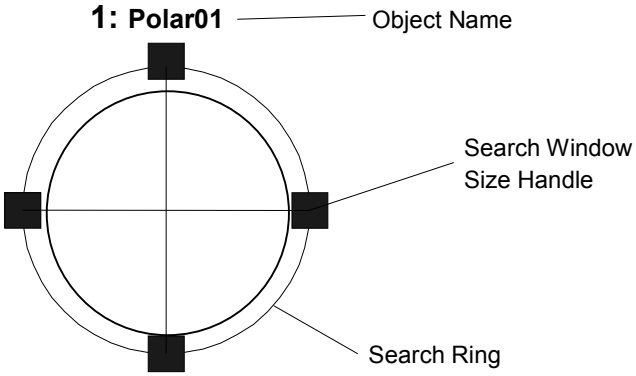


Figure 58: New PolarObject

Polar objects have a circular search window. You can change the position of the entire object or change its radius.

To move the entire object, click on the object name or anywhere along the outer circumference and while holding the left mouse button down drag the entire object to a new location on the screen. When you find the position you like, release the mouse and the Polar object will stay in this new position on the screen.

To change the radius, move the mouse pointer over one of the size handles, press the left mouse button down, then move the mouse to change the size of the radius.

Step 3: Configuring Properties for the Polar Object

We can now set property values for the Polar object. To set any of the properties simply click on the associated property's value field and then either enter a new value or if a drop down list is displayed click on one of the items in the list.

Shown below are some of the more commonly used properties for the Polar object. Explanations for other properties such as AbortSeqOnFail, Graphics, etc. which are used on many of the different vision objects can be seen in the *Vision Properties and Results Reference Manual* or in *Polar Object Properties* found previously in this section.

Name property - The default name given to a newly created Polar object is "Polarxx" where xx is a number which is used to distinguish between multiple Line objects within the same vision sequence. If this is the first Line object for this vision sequence then the default name will be "Polar01". To change the name, click on the Value field of the Name property, type a new name and hit return. You will notice that once the name property is modified, every place where the Polar object's name is displayed is updated to reflect the new name.

CenterPointObject property - Typically you will set this property to one of the objects that occur previously in the sequence. This will determine the center point of the Polar object at runtime.

Thickness property - Typically you will set this property a value large enough to contain sufficient model information to locate the angle of the part.

AngleOffset property - Typically you will set this property to position the final angle result at the desired position. For example, if you were looking for the minute hand of a watch, you would adjust AngleOffset so that the displayed angle matched the minute hand.

Step 4: Running the Polar Object and Examining the Results

To run the Polar object, simply do the following:

Click on the Run Object button located at the bottom left of the Object tab. The CenterPointObject will be run first.

Results for the Polar object will now be displayed. The primary results to examine at this time are:

Angle result - The angle of the model found in degrees.

5.4.7 Frame Object

Frame Object Description

Frame objects provide a type of dynamic positioning reference for vision objects. Once a Frame object is defined other vision objects can be positioned with respect to that Frame. This proves useful for a variety of situations. It also helps reduce application cycle time because once a rough position is found and a Frame defined, the other vision objects that are based on that Frame object need not have large search windows. (Reducing the size of search windows helps reduce vision-processing time.)

Frame objects are best used when there is some type of common reference pattern on a part (such as fiducials on a printed circuit board) which can then be used as a base position on which other vision objects search window locations are based.

Frame Object Layout

The Frame object looks like 2 Line objects that intersect. The user can adjust the position of the Frame object by clicking on the Frame object's name and then dragging the object to a new position. However, in most cases the Frame object position and orientation will be based on other vision object's position.

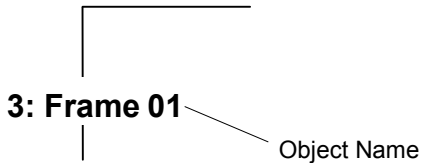


Figure 59: Frame Object Layout

Frame Object Properties

The following list is a summary of properties for the Frame object. The details for each property are explained in the Vision Properties and Results Reference Manual.

Property	Description
AbortSeqOnFail	Allows the user to specify that if the specified object fails (not found), then the entire sequence is aborted at that point and no further objects in the sequence are processed. Default: 0 - False
BackColor	Sets the background color for the object's label. Default: 0 - None
Caption	Used to assign a caption to the Frame object. Default: Empty String
FoundColor	Selects the color for an object when it is found. Default: 1 – Light Green
Graphics	Specifies which graphics to display. Default: 1 - All
Name	Used to assign a unique name to the Frame object. Default: Framexx

Property	Description
OriginAngleEnabled	Enables single point frame usage which causes the frame to rotate with the angle of the origin object instead of based on the rotation of the vector between the OriginPoint and the YaxisPoint as in two point Frames. Default: 0 – False
OriginPntObjResult	Specifies the result to use from the vision object specified in the OriginPoint property. Default: 1 (uses the first result)
OriginPoint	Defines the vision object to be used as the origin point for the Frame object. Default: Screen
YAxisPoint	Defines vision object to be used as the Y-axis point for the frame. (Defines the direction of the Frame object.) Default: Screen
YAxisPntObjResult	Specifies the result to use from the vision object specified in the YAxisPoint property. Default: 1 (uses the first result)

Frame Object Results

The details for each result used with Frame objects are explained in the Vision Properties and Results Reference Manual. However, for convenience we have provided a list of the Frame object results and a brief explanation of each below:

Results	Description
Angle	Returns the amount of rotation of the Frame.
Found	Returns whether or not the frame could be defined.

Two Point Frames

Two point Frame objects require an origin (defined by the OriginPoint property) and a Y-Axis direction (defined by the YAxisPoint property). The combination of Origin and Y Axis direction define what can be thought of as a local coordinate system within which other vision objects can be based. The power of the Frame object is that when the Frame object moves, all the vision objects which are defined within that frame move with it. (i.e. Their search windows adjust based on the XY change of the vision object defined as the OriginPoint and the rotation created with the movement of the YaxisPoint property.) This allows you to keep your search windows small that in turn improves reliability and-processing time.

Defining a Frame Object

Once a new Frame object is created, it requires 2 vision objects to be used as reference positions for the OriginPoint and YAxisPoint of the Frame. These are defined with the OriginPoint property and YAxisPoint property. Any vision object which has XY position results can be used to define a Frame Origin or YAxisPoint. This means that Blob, Correlation, Edge, Polar, and Point objects can all be used to define the Origin or YAxisPoint property for a Frame object.

Single Point Frames

Single point frames are an optional usage method for the Frame object. With this usage the OriginPoint property specifies a vision object to be used as an XY positional origin reference. When the OriginAngleEnabled property is set to FALSE, the frame adjusts position based on the XY position change of the vision object used as the OriginPoint. No rotation is taken into account. This is useful for simple XY shifts where one object (such as a blob or correlation) finds the XY position of a part, and then the rest of the objects in the frame adjust in X and Y accordingly.

In some cases, you may also need to account for rotation within your frame. Assume that a Blob object is used to find a part's X, Y, and U position (XY coordinate + rotation). Then let's assume that a variety of other vision objects are required to find features on the part. The Blob object can be used to define a single point frame including rotation of the frame, and the other objects within the frame will then shift in X,Y and rotate based on the rotation returned by the Blob object. Hence, only one vision object was required to define both the XY shift and rotation of the part. This shows why the YaxisPoint property is not required for Single Point Frames.

Using Frame Objects

The next few sections guide you through how to create and use a Frame object.

- How to create a new Frame object
- Position and Size the search window
- Configure the properties associated with the Frame object
- Test the Frame object & examine the results
- Make adjustments to properties and test again

Prior to starting the steps shown below, you should have already created a new vision sequence or selected a vision sequence to use. If you have no vision sequence to work with, you can create a new vision sequence by clicking on the New Sequence  toolbar button. You can also select a sequence which was created previously by clicking on the Sequence tab in the Vision Guide window and clicking on the drop down list box which is located towards the top of the Sequence tab. See Vision Sequences for details on how to create a new vision sequence or select one which was previously defined.

Step 1: Create a New Frame Object

(1) Click on the New Frame toolbar button  on the Vision Guide Toolbar. (In this case click means to click and release the left mouse button).

(2) Move the mouse over the image display. You will see the mouse pointer change to the Frame object icon.

(3) Continue moving the mouse until the icon is at the desired position in the image display, then click the left mouse button to create the object.

(4) Notice that a name for the object is automatically created. In the example, it is called "Frame01" because this is the first Frame object created for this sequence. (We will explain how to change the name later.).

Step 2: Positioning the Frame Object

You should now see a Frame object similar to the one shown below:

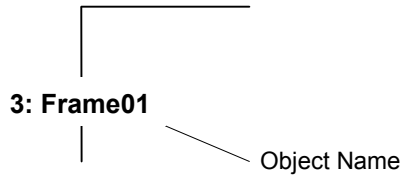


Figure 60: New Frame Object

Frame objects do not have a sizeable window. Click on the name label of the Frame object or anywhere along one of its axes and while holding the mouse down drag the entire Frame object to a new location on the screen. When you find the position you like, release the mouse and the Frame object will stay in this new position on the screen.

Step 3: Configuring Properties for the Frame Object

We can now set property values for the Frame object. To set any of the properties simply click on the associated property's value field and then either enter a new value or if a drop down list is displayed click on one of the items in the list.

Shown below are some of the more commonly used properties for the Frame object. Explanations for other properties such as AbortSeqOnFail, Graphics, etc. which are used on many of the different vision objects can be seen in the Vision Properties and Results Reference Manual or in the Frame Object Properties found previously in this section.

Name property - The default name given to a newly created Frame object is "Framexx" where xx is a number which is used to distinguish between multiple Frame objects within the same vision sequence. If this is the first Frame object for this vision sequence then the default name will be "Frame01". To change the name, click on the Value field of the Name property, type a new name and hit return. You will notice that once the name property is modified, every place where the Frame object's name is displayed is updated to reflect the new name.

OriginPoint property - Typically you will set this property to one of the objects that occur previously in the sequence. This will determine the origin of the frame at runtime.

YAxisPoint property - Typically you will set this property to one of the objects that occur previously in the sequence. This will determine the direction of the Y axis of the frame at runtime.

Step 4: Running the Frame Object and Examining the Results

To run the Frame object, simply do the following:

Click on the Run Object button located at the bottom left of the Object tab. If either the OriginPoint or YAxisPoint properties are not Screen, then the respective objects will be run first. For example, if the OriginPoint is a Blob object, then the blob will be run first to determine the position of the origin of the frame.

Results for the Frame object will now be displayed. The primary results to examine at this time are:

Angle - The angle of the frame.

5.4.8 Line Object

Line Object Description

Line objects are used to define a line between 2 points. The Points can be either based on positions on the screen or on other vision object positions. For example, shown below are just some of the situations where a Line object can be created:

- between 2 Blob objects
- between 2 Correlation objects
- between 2 Point objects
- between a Blob object and a Correlation object
- between 2 results on a single Blob object
- between result 1 on a Blob object and result 3 on a Correlation object
- between a Point object and a Correlation object
- any other variety of combinations between objects that have an XY position associated with them.

Line objects are useful for the following situations:

- To measure the distance between two vision objects (or vision object results when multiple results are used), the distance can also be checked to make sure it falls between a minimum and maximum distance as per your application requirements.
- To calculate the amount of rotation between 2 vision objects (use the angle of the line returned in Robot Coordinates called the RobotU result)
- To create a building block for computing the midpoint of a line or intersection point between 2 lines

Line Object Layout

The Line object layout appears just as you would expect it to. It's just a line with a starting point, an ending point, and an object name. To reposition the Line object, simply click on the name of the Line object (or anywhere on the line) and then drag the line to a new position. To resize a Line object, click on either the starting or ending point (shown by the sizing handles) of the line and then drag it to a new position.

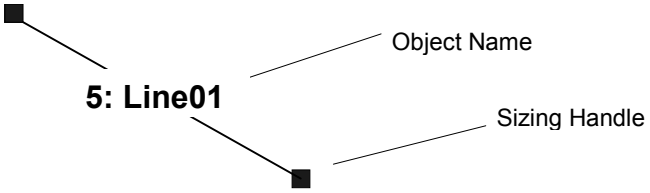


Figure 61: Line Object Layout

Line Object Properties

The following list is a summary of properties for the Line object. The details for each property are explained in the *Vision Properties and Results Reference Manual*.

Property	Description
AbortSeqOnFail	Allows the user to specify that if the specified object fails (not found), then the entire sequence is aborted at that point and no further objects in the sequence are processed. Default: 0 - False
BackColor	Sets the background color for the object's label. Default: 0 - None
Caption	Used to assign a caption to the object. Default: Empty String
EndPntObjResult	Specifies which result to use from the EndPointObject. Default: 1
EndPointObject	Specifies which vision object to use to define the end point of the Line. Default: Screen
EndPointType	Defines the type of end point used to define the end point of a line. Default: 0 – Point
FoundColor	Selects the color for an object when it is found. Default: 1 – Light Green
Frame	Specifies which positioning frame to use. Default: none
Graphics	Specifies which graphics to display. Default: 1 - All
MaxLength	Defines the upper length limit for the Line object. For a Line to be found it must have a Length result below the value set for MaxLength property. Default: 1000
MaxPixelLength	Defines the upper pixel length limit for the Line object. For a Line to be found it must have a PixelLength result below the value set for MaxPixelLength property. Default: 1000
MinLength	Defines the lower length limit for the Line object. For a Line to be found it must have a Length result above the value set for MinLength property. Default: 0
MinPixelLength	Defines the lower length limit for the Line object. For a Line to be found it must have a PixelLength result above the value set for MinPixelLength property. Default: 0
Name	Used to assign a unique name to the Line object. Default: Linexx

Property	Description
ShowExtensions	When set to True, this causes the graphics display of the line to be extended to the end of the image display. Default: 0 - False
StartPntObjResult	Specifies which result to use from the StartPointObject. Default: 1
StartPointObject	Specifies which vision object to use to define the start point of the Line. Default: Screen
StartPointType	Defines the type of start point used to define the start point of a line.
X1	The X coordinate position of the start point of the line.
X2	The X coordinate position of the end point of the line.
Y1	The Y coordinate position of the start point of the line.
Y2	The Y coordinate position of the end point of the line.

Line Object Results

The following list is a summary of the Line object results with brief descriptions. The details for each result are explained in the Vision Guide Properties and Results Reference Manual.

Results	Description
Angle	Returns the angle of the Line formed from StartPoint to EndPoint with respect to the 0 degrees position at 3 O'clock.
CameraX1	Returns the X coordinate position of the starting point of the line in the camera coordinate frame.
CameraX2	Returns the X coordinate position of the ending point of the line in the camera coordinate frame.
CameraY1	Returns the Y coordinate position of the starting point of the line in the camera coordinate frame.
CameraY2	Returns the Y coordinate position of the ending point of the line in the camera coordinate frame.
Found	Returns whether or not the object was found. (i.e. If the vision objects used to define the Line object are not found, then the Line object is not found.)
Length	Returns the length of the line in millimeter units. (The camera must be calibrated or [no cal] will be returned as the length.) If the Line object fails due to the MaxLength and MinLength constraints, the Length result is shown in red in the Results list.
PixelLength	Returns the length of the line in pixels. If the Line object fails due to the MaxPixelLength and MinPixelLength constraints, the PixelLength result is shown in red in the Results list.
PixelLine	Runtime only. Returns the four line coordinates X1, Y1, X2, Y2 in pixels.
PixelX1	Returns the X coordinate position of the starting point of the line in pixels.
PixelX2	Returns the X coordinate position of the ending point of the line in pixels.
PixelY1	Returns the Y coordinate position of the starting point of the line in pixels.
PixelY2	Returns the Y coordinate position of the ending point of the line in pixels.
RobotU	Returns the angle of the Line formed from StartPoint to EndPoint with respect to the Robot Coordinate System.

Using Line Objects

The next few sections guide you through how to create and use an Line object.

- How to create a new Line object
- Position and Size the search window
- Configure the properties associated with the Line object
- Test the Line object & examine the results
- Make adjustments to properties and test again

Prior to starting the steps shown below, you should have already created a new vision sequence or selected a vision sequence to use. If you have no vision sequence to work with, you can create a new vision sequence by clicking on the New Sequence toolbar button. You can also select a sequence which was created previously by clicking on the Sequence tab in the Vision Guide window and clicking on the drop down list box which is located towards the top of the Sequence tab. See Vision Sequences for details on how to create a new vision sequence or select one which was previously defined.

Step 1: Create a New Line Object

(1) Click on the New Line toolbar button on the Vision Guide Toolbar. (In this case click means to click and release the left mouse button).

(2) Move the mouse over the image display. You will see the mouse pointer change to the Line object icon.

(3) Continue moving the mouse until the icon is at the desired position in the image display, then click the left mouse button to create the object.

(4) Notice that a name for the object is automatically created. In the example, it is called "Line01" because this is the first Line object created for this sequence. (We will explain how to change the name later.).

Step 2: Positioning the Line Object

You should now see a Line object similar to the one shown below:

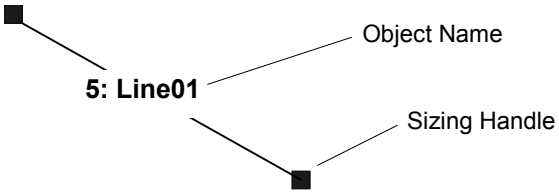


Figure 62: New Line Object

Line objects do not have a window. You can change the length and rotation by clicking down on either size handle, and then dragging that end of the line to a new position. You can also click on the name label of the Line object or anywhere along the line and while holding the mouse down drag the entire Line object to a new location on the screen. When you find the position you like, release the mouse and the Line object will stay in this new position on the screen.

Step 3: Configuring Properties for the Line Object

We can now set property values for the Line object. To set any of the properties simply click on the associated property's value field and then either enter a new value or if a drop down list is displayed click on one of the items in the list.

Shown below are some of the more commonly used properties for the Line object. Explanations for other properties such as AbortSeqOnFail, Graphics, etc. which are used on many of the different vision objects can be seen in the *Vision Properties and Results Reference Manual* or in the *Line Object Properties* found previously in this section.

Name property - The default name given to a newly created Line object is "Linexx" where xx is a number which is used to distinguish between multiple Line objects within the same vision sequence. If this is the first Line object for this vision sequence then the default name will be "Line01". To change the name, click on the Value field of the Name property, type a new name and hit return. You will notice that once the name property is modified, every place where the Line object's name is displayed is updated to reflect the new name.

StartPointObject property - Typically you will set this property to one of the objects that occur previously in the sequence. This will determine the starting point of the line at runtime.

EndPointObject property - Typically you will set this property to one of the objects that occur previously in the sequence. This will determine the end point of the line at runtime.

Step 4: Running the Line Object and Examining the Results

To run the Line object, simply do the following:

Click on the Run Object button located at the bottom left of the Object tab. If either the StartPointObject or EndPointObject properties are not Screen, then the respective objects will be run first. For example, if the StartPointObject is a Blob object, then the blob will be run first to determine the position of the starting point of the line.

Results for the Line object will now be displayed. The primary results to examine at this time are:

Length results - The length of the line in millimeters. There must be a calibration assiciated with the sequence that contains the line for Length to be determined.

PixelX1, PixelY1, PixelX2, PixelY2 results
- The XY position (in pixels) for both ends of the line.

PixelLength results - The length of the line in pixels.

5.4.9 Point Object

Point Object Description

Point objects can be thought of as a type of utility object that is normally used with other vision objects.

Point objects are most useful in defining a position references for Polar and Line objects as follows:

Polar Objects: Point objects can be used to define the CenterPoint property which is used as the center of the Polar object.

Line Objects: Point objects can be used to define the startpoint, midpoint, or endpoint of a single line or the intersection point of 2 lines.

Point Object Layout

The Point object layout appears as a cross hair on the screen. The only real manipulation for a Point object is to change the position. This is done by clicking on the Point object and dragging it to a new position. In most cases Point objects are attached to other objects so the position is calculated based on the position of the associated object.

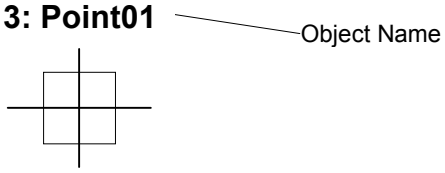


Figure 63: Point Object Layout*:*

Point Object Properties

The following list is a summary of properties for the Point object. The details for each property are explained in the Vision Properties and Results Reference Manual.

Property	Description
AbortSeqOnFail	Allows the user to specify that if the specified object fails (not found), then the entire sequence is aborted at that point and no further objects in the sequence are processed. Default: 0 - False
BackColor	Sets the background color for the object's label. Default: 0 - None
Caption	Used to assign a caption to the object. Default: Empty String
FoundColor	Selects the color for an object when it is found. Default: 1 – Light Green
Frame	Specifies which positioning frame to use. Default: none
Graphics	Specifies which graphics to display. Default: 1 - All
LineObject1	Defines a Line object used to define the position of the point object as the midpoint of the line or as the intersection of 2 lines.

Property	Description
	Default: none
LineObject2	Defines the 2nd Line object used to define the position of the point as the intersection of 2 lines. Default: none
Name	Used to assign a unique name to the Point object. Default: Pointxx
PointType	Defines the position type for the point. Default: Screen
X	The X coordinate position of the Point object in pixels.
Y	The Y coordinate position of the Point object in Pixels.

Point Object Results

The following list is a summary of the Point object results with brief descriptions. The details for each result are explained in the Vision Guide Properties and Results Reference Manual.

Results	Description
CameraX	Returns the X coordinate position of the Point object's position in the camera coordinate frame.
CameraY	Returns the Y coordinate position of the Point object's position in the camera coordinate frame.
CameraXYU	Runtime only. Returns the CameraX, CameraY, and CameraU coordinates of the Point object's position in the camera coordinate frame.
ColorValue	Returns the grayscale value of the pixel at the current location.
Found	Returns whether or not the Point object was found. (i.e. If the vision objects used to define the position of the Point object are not found, then the Line object is not found.)
PixelX	Returns the X coordinate position of the Point object's position in pixels.
PixelY	Returns the Y coordinate position of the Point object's position in pixels.
PixelXYU	Runtime only. Returns the PixelX, PixelY, and PixelU coordinates of the found part's position in pixels.
RobotX	Returns the X coordinate position of the found part's position (referenced by model origin) with respect to the Robot's Coordinate System.
RobotY	Returns the Y coordinate position of the found part's position (referenced by model origin) with respect to the Robot's Coordinate System.
RobotXYU	Runtime only. Returns the RobotX, RobotY, and RobotU coordinates of the Point object's position with respect to the Robot's Coordinate System. (Keep in mind that the U coordinate value has no meaning since a Point object has no rotation.)

Understanding Point Objects

Point objects were created to give users a method to mark a position on the screen or with respect to Line objects so that these positions could be used by other vision objects as reference positions. There are 2 fundamental parts to understanding Point objects:

1. Defining the position of the Point object
2. Using the position of the Point object as a reference position for other objects

Defining the Position of the Point Object

The position of a Point object can either be based upon the position on the screen where you place the Point object, the midpoint of a Line object, or the intersection between 2 Line objects. The PointType property is used to define which item the Point object's position is based upon. The PointType property can be set to Screen, MidPoint, or Intersection.

When the PointType property is Set to Screen

When a Point object is first created the default PointType is Screen which means that the position of the Point object is based upon the position on the screen where the Point object was placed. As long as the PointType is set to Screen, the Point object's position only changes when you move it manually with the mouse or change the values of the X property or Y property.

Setting the PointType property to MidPoint

To define a Point object's position as the Midpoint of a line can be done by setting the PointType property to MidPoint. However, it is important to note that you must first define the line to be used prior to setting PointType to MidPoint. Otherwise, Vision Guide would have no idea about which line to use for computing the midpoint. The LineObject1 property is used to define the Line object to for which the midpoint will be computed and the position of the Point object then set to. Any valid Line object that is ahead of the Point object in the vision sequence Step list can be selected from the drop down list of the LineObject1 property. In fact Vision Guide automatically checks which Line objects are ahead of the Point object in the sequence Step list and only displays these lines in the LineObject1 drop down list. This makes the system easier to use.

If you try to set the PointType property to MidPoint without first specifying the Line object to be used with the LineObject1 property, then an error message will appear telling you that you must first select a line for the LineObject1 property.

Setting the PointType property to Intersection

To define a Point object's position as the intersection point between 2 lines requires that the 2 lines first be defined. This is done with the LineObject1 and LineObject2 properties. The LineObject1 and LineObject2 properties must each define a different Line object. Once a Line object is defined for each, the PointType property can be set to Intersection which indicates that the position of the Point object will be the intersection point of the 2 lines defined by the LineObject1 and LineObject2 properties.

Any valid Line object which is ahead of the Point object in the vision sequence Step list can be selected from the drop down list of the LineObject1 property to be used as the 1st line required for the intersection. Then any valid remaining Line object which is ahead of the Point object in the vision sequence Step list can be selected from the drop down list of the LineObject2 property to be used as the 2nd line required for the intersection. Vision Guide automatically takes care of displaying only those Line objects that are valid Line objects for use as LineObject1 or LineObject2 in the associated drop down lists.

If you try to set the PointType property to Intersection without first specifying the Line objects to use for both LineObject1 and LineObject2, then an error message will appear. The error message will tell you that you must first define the line for LineObject1 or LineObject2 prior to setting the PointType property to Intersection. Depending upon whichever Line object is not defined.

NOTE

Using the intersection of 2 lines to define a position is useful for computing things such as the center of an object. For example, consider a rectangular object. Once you find the 4 corners, you can create 2 diagonal lines that intersect at the center of the rectangle. A Point object positioned on this intersection point is then positioned at the center of the rectangle.

Using the Position of the Point Object as a Reference Position for Other Objects

The Point object's primary purpose is to act as a reference position for other vision objects such as the Line and Polar objects. This means that the position of the Point object is used as a base position for a Line or Polar object. This is powerful because it allows you to create such situations as a Point object which is defined as the intersection of 2 lines to calculate the center of an object, which can then be used as an end point for a Line object which calculates the distance between the centers of 2 objects.

Point Objects used as a Reference Position for Polar Objects

The Polar object requires a CenterPoint position to base its Polar Search upon. When first using the Polar object, you may try using the screen as the reference point for the CenterPoint property but you will soon find that as the feature you are searching moves the Polar object must also move with respect to the CenterPosition of the feature. This is where being able to apply the XY position from another vision object (such as the Point object) to the Polar object becomes very powerful.

There may be times where you want to apply the position of the Point object as the CenterPoint of a Polar object. There are 2 primary situations for which Point objects can be used as the CenterPoint of a Polar object.

1. CenterPoint defined as the midpoint of a line
2. CenterPoint defined as the intersection point of 2 lines

For example, after you find the midpoint of a line, you may want to do a Polar search from this midpoint. You may also want to base a Polar search CenterPoint on the intersection point of 2 lines. This is the more common use for Polar objects with Point objects.

Point Objects used as a Reference Position for Line Objects

A line requires a starting and ending position. Many times the starting and ending positions of lines are based on the XY position results from other vision objects such as the Blob or Correlation object. However, you can also use a Point object position as the reference position of a line. A line can be defined with both it's starting and ending positions being based on Point objects or it can have just one of the endpoints of the line be based on a Point object.

One of the more common uses of the Point object is to use it to reference the intersection point of 2 lines. Figure 64 shows an example of 2 line objects (Line1 and Line2) which intersect at various heights. Line 3 is used to calculate the height of the intersection point. The Point object is shown at the intersection point between Line1 and Line2.)

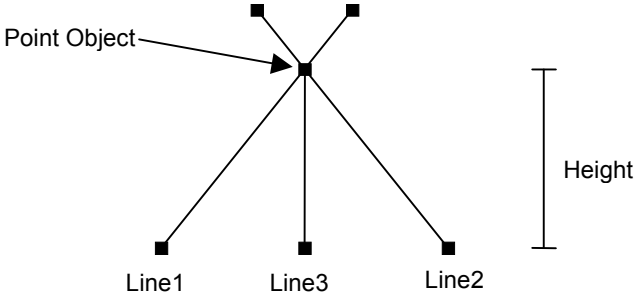


Figure 64: Point object defined as intersection point of Lines 1 and 2

Using Point Objects

The next few pages take you through how to create and use a Point object. We will review the following items

- Create a new Point object
- Positioning the Point object on the screen
- Selecting the Object tab
- Configuring properties associated with the Point object
- Running the Point object and Examining the results

Prior to starting the steps shown below, you should have already created a new vision sequence or selected a vision sequence to use. If you have no vision sequence to work with, you can create a new vision sequence by clicking on the New Sequence toolbar button. You can also select a sequence which was created previously by clicking on the Sequence tab in the Vision Guide window and clicking on the drop down list box which is located towards the top of the Sequence tab. See Vision Sequences for more details on how to create a new vision sequence or select one that was previously defined.

Step 1: Create a New Point Object

(1) Click on the New Point toolbar button on the Vision Guide Toolbar. (In this case click means to click and release the left mouse button)

(2) You will see a point icon appear above the Point object toolbar button

(3) Click on the point icon and drag to the image display of the Vision Guide window

(4) Notice that a name for the object is automatically created. In the example, it is called "Pnt01" because this is the first Point object created for this sequence. (We will explain how to change the name later.)

Step 2: Positioning the Point Object

You should now see a Point object similar to the one shown below:

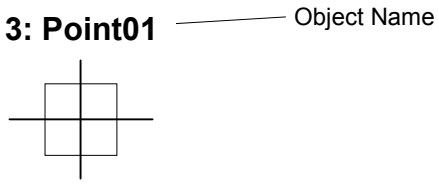


Figure 65: New Point Object Layout

Point objects cannot be resized since they are just a point and have no height or thickness. However, they can be positioned with the mouse or by setting position values into the X and Y properties. Since Point objects are created with a PointType property setting of "0 - Screen", we can move the Point object around a little with the mouse as described below:

(1) Click on the name label of the Point object and while holding the mouse down drag the Point object to a new location on the screen. When you find the position you like, release the mouse and the Point object will stay in this new position on the screen.

Step 3: Configuring Properties for the Point Object

We can now set property values for the Point object. To set any of the properties simply click on the associated property's value field and then either enter a new value or if a drop down list is displayed click on one of the items in the list.

Shown below are some of the more commonly used properties for the Point object. Explanations for other properties such as Allows the user to specify that if the specified object fails (not found), then the entire sequence is aborted at that point and no further objects in the sequence are processed., Graphics, etc. which are used on many of the different vision objects can be seen in the Vision Properties and Results Reference Manual or in the Point Object Layout.

The Point object layout appears as a cross hair on the screen. The only real manipulation for a Point object is to change the position. This is done by clicking on the Point object and dragging it to a new position. In most cases Point objects are attached to other objects so the position is calculated based on the position of the associated object.

Point Object Layout

The Point object layout appears as a cross hair on the screen. The only real manipulation for a Point object is to change the position. This is done by clicking on the Point object and dragging it to a new position. In most cases Point objects are attached to other objects so the position is calculated based on the position of the associated object.

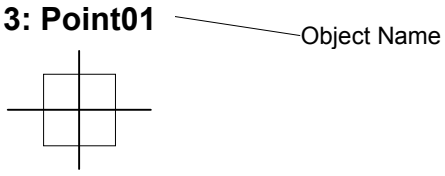


Figure 66: Point Object Layout:

Point Object Properties

Point Object Properties list on page 173 earlier in this chapter. We shouldn't have to set any of these properties to test our Point object because the default values are fine since we are not positioning the Point object at the midpoint of a single line or intersection point of 2 lines. However, you may want to read through this information if you are working with Point objects for the first time.

Name property ("Pntxx") - The default name given to a newly created Point object is "Pntxx" where xx is a number which is used to distinguish between multiple Point objects within the same vision sequence. If this is the first Point object for this vision sequence then the default name will be "Pnt01". To change the name, click on the Value field of the Name property, type a new name and hit return. You will notice that once the name property is modified, every place where the Point object's name is displayed is updated to reflect the new name.

LineObject1 (None) - If you will set the Point object's PointType property to Midpoint (specifying the midpoint of a line), then this property will specify which line to use. (It is also used to specify the 1st of 2 lines required if the PointType will be an intersection point.)

LineObject2 (None) - If you will set the Point object's PointType property to Intersection (specifying the intersection point of 2 lines), then this property will specify the 2nd line to use. (The 1st line for the intersection is specified by the LineObject1 property.)

PointType (Screen) - This property is used to define the position of the Point object. It can be based on the Screen position, midpoint of a line specified by the LineObject1 property, or the intersection point of 2 lines specified by the LineObject1 and LineObject2 properties.

Since there are no Line objects specified in this example, the LineObject1, LineObject2, and PointType properties cannot be changed from their default state. Normally, we would select a Line Object for the LineObject1 property if we want to make this Point the midpoint of a line. Or we would select a Line object for the LineObject1 property and a 2nd Line object for the LineObject2 property if we want to make this Point an intersection point between two lines. This was explained earlier in the section titled *Defining the Position of the Point Object* on page 175.

Step 4: Running the Point Object and Examining the Results

To run the Point object, simply do the following:

(1) Click on the Run Object button located at the bottom left of the Object tab.

Results for the Point object will now be displayed. The primary results to examine at this time are:

PixelX, PixelY results - The XY position (in pixels) of the Point object. If the PointType property for the Point object was set to midpoint, then the PixelX and PixelY results would return the XY position (in Pixels) of the midpoint of the Line object specified by LineObject1.

CameraX, CameraY results
- These define the XY position of the Point object in the Camera's World Coordinate system. The CameraX and CameraY results will only return a value if the camera has been calibrated. If it has not then [No Cal] will be returned.

RobotX, RobotY results - These define the XY position of the Point object in Robot World Coordinates. The robot can be told to move to this XY position. (No other transformation or other steps are required.) The RobotX and RobotY results will only return a value if the camera has been calibrated. If it has not then [No Cal] will be returned.

5.4.10 Working with Multiple Results from a Single Object

Blob and Correlation objects can be used to find more than only 1 feature within a single search window. The properties and results that you will need to understand for working with multi-result vision objects are listed below:

- CurrentResult property
- NumberToFind property
- NumberFound result
- ShowAllResults result

The Default and Maximum Number of Features a Vision Object Can Find

The default property configurations for a Blob or Correlation object cause it to find only 1 feature within a search window. This is because the NumberToFind property is set to 1. However, when you set the NumberToFind property (for the Blob and Correlation objects) to a number larger than 1, the vision object will attempt to find as many features as you specify.

Once the NumberToFind property is set, run your vision object. You will see that all the features in the current image that meet the acceptance criteria will be shown as found (green box with crosshair indicating position returned for found object). You can also see that the feature found which is considered the CurrentResult is highlighted in a lighter shade of green than the other features.

The Sorting Order of the Features That Are Found

When a Blob or Correlation object is first run, the CurrentResult is automatically set to 1 indicating that the 1st result for the multi-result vision object should have its results displayed in the Results list on the Object tab. For a Correlation object, the first result is the one with the highest score (as compared to all the features found for that Correlation object). The second result is the one with the second highest score and so on.

For a Blob object, the first result is the one returned based on the values for the SizeToFind and Sort properties. (For example, if SizeToFind is set to "Largest" then the first result will be the largest Blob found.) See the SizeToFind and Sort properties in the *Vision Guide Properties and Results Manual* for more info regarding sorting blob results.)

Examining the Vision Object's Multiple Results

If you look closely at the Results list heading you can see where it says something like "Result 1 of 10". (Assume for the sake of this discussion that we had set the NumberToFind property to 10 for a Blob object.) This means the CurrentResult is 1 out of 10 features that were searched for (as defined by the NumToFind property.) **It is important to remember that the 2nd number in the Results list heading is the value of the NumToFind property and not the number of features that were actually found.**

You can examine the results for any of the multiple blobs you tried to find by changing the CurrentResult property to reflect the blob you would like to see. This can be done by manually typing a number into the value field of the CurrentResult property or by moving the cursor to the value field of the CurrentResult property and then using the SHIFT+DnArrow or SHIFT+UpArrow keys to move through the results. You will see the results displayed in the Results list. You can always tell which result is displayed in the Results list by either looking at the CurrentResult property or by just looking at the heading of the Results list. It will display “Result 1 of 10” for the 1st result when NumToFind is set to 10 and “Result 2 of 10” for the 2nd result, and “Result 3 of 10” for the 3rd result, etc.

 WARNING	■ When the CurrentResult property is changed, this also changes the VGet result in a program. For example, if a SPEL program uses VGet to get the RobotXYU result from a vision object, and the CurrentResult is set to 3, then the VGet instruction will return the RobotXYU 3rd result. Be careful with this. As you may end up getting the wrong results because the CurrentResult was set wrong for your SPEL+ program.

Suggestion: Always specify the result you want to return from a VGet statement explicitly in your SPEL+ program when specifying the Vision Guide result name when working with multiple results.

Example: VGet seqname.objname.RobotXYU**(1)**, found, X, Y, U

Using the NumberFound Result

The NumberFound result is useful because it displays how many blobs or matching features for a Correlation Model were actually found. This result is also available from the SPEL+ language so you can use a SPEL+ program to make sure that the proper number of results were found, or to count how many were found, or a whole host of other things. See the examples below:

This shows a small section of code to check if the number of results found is less than 5.

```
VGet seqname.objname.NumberFound, numfound
If numfound < 5 Then
     'put code to handle this case here
```

Consider a situation where vision is used to find as many parts in one VRun as it can. The robot will then get each part until all the found parts are moved onto a conveyor. This example shows a small section of code to have the robot pick each of the parts found and then drop them off one by one onto the same position on a moving conveyor.

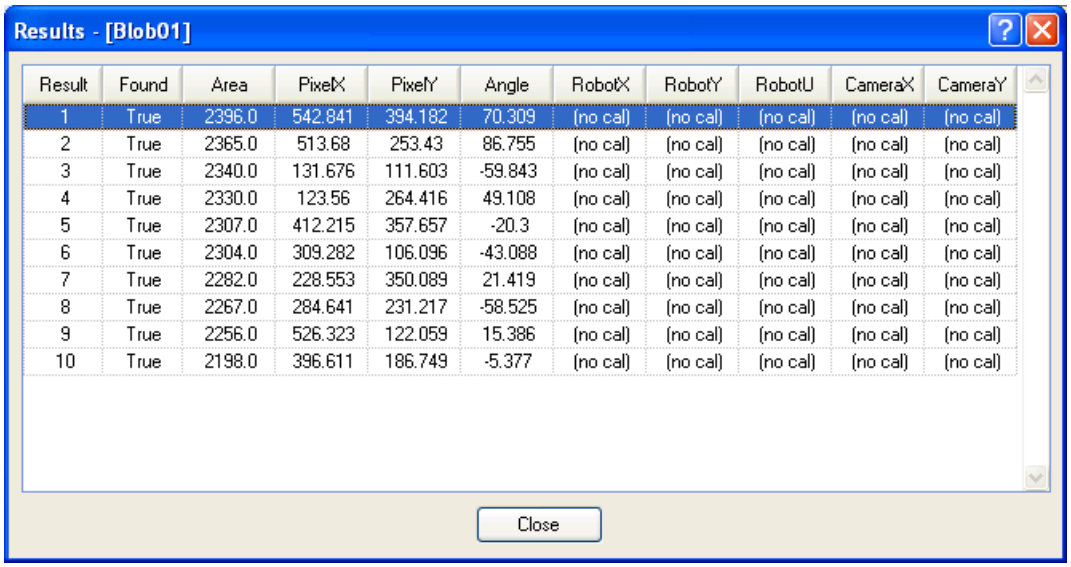
```
VRun seqname
VGet seqname.objname.NumberFound, numfound
For count = 1 to numfound
  VGet seqname.objname.RobotXYU(count), found, X, Y, U
  If found = TRUE Then
    VPick = X, Y, -100.000, U    'set   coordinates   found   from
vision
  Else
    Print "Vision Error: part not found"
  EndIf

  Jump Vpick           'Jump to Vision Pickup position
  On Gripper           'Turn on vacuum
  Wait .1
  Jump Vconvey         'Jump to Conveyor Position to drop part
  Off Gripper          'Turn off vacuum
  Wait .1                'No real need for this wait because Vget
above takes
                       'some   time   but   put   here   anyway   for
consistency
Next count
```

Examining All the Multiple Results at Once

One of the nicest features built in with the multiple results feature for Blob, and Correlation objects is the ShowAllResults result. There may be cases where you want to compare the different results to see the top Score vs. the second Score and so on. For example, maybe there is a big drop off in score after the 3rd feature is found. Using the ShowAllResults result makes is easy to see all results at once.

Clicking on the ShowAllResults result's value field will cause a button to appear. Click on the button and a dialog will be displayed which shows all the results for the current vision object. A sample ShowAllResults dialog is shown in Figure 67.



Result	Found	Area	PixelX	PixelY	Angle	RobotX	RobotY	RobotU	CameraX	CameraY
1	True	2396.0	542.841	394.182	70.309	(no cal)	(no cal)	(no cal)	(no cal)	(no cal)
2	True	2365.0	513.68	253.43	86.755	(no cal)	(no cal)	(no cal)	(no cal)	(no cal)
3	True	2340.0	131.676	111.603	-59.843	(no cal)	(no cal)	(no cal)	(no cal)	(no cal)
4	True	2330.0	123.56	264.416	49.108	(no cal)	(no cal)	(no cal)	(no cal)	(no cal)
5	True	2307.0	412.215	357.657	-20.3	(no cal)	(no cal)	(no cal)	(no cal)	(no cal)
6	True	2304.0	309.282	106.096	-43.088	(no cal)	(no cal)	(no cal)	(no cal)	(no cal)
7	True	2282.0	228.553	350.089	21.419	(no cal)	(no cal)	(no cal)	(no cal)	(no cal)
8	True	2267.0	284.641	231.217	-58.525	(no cal)	(no cal)	(no cal)	(no cal)	(no cal)
9	True	2256.0	526.323	122.059	15.386	(no cal)	(no cal)	(no cal)	(no cal)	(no cal)
10	True	2198.0	396.611	186.749	-5.377	(no cal)	(no cal)	(no cal)	(no cal)	(no cal)

Figure 67: The ShowAllResults dialog

Using the Multiple Results Dialog to Debug Searching Problems

Sometimes the parts which you are working with vary considerably (even within the same production lot) and sometimes there are 2 or more features on a part which are similar. This can make it very difficult to determine a good Accept property value. Just when you think you have set the Accept property to a good value, another part will come in which fools the system. In these cases it can be very difficult to see what is going on.

The Show All Results dialog was created to help solve these and other problems. While you may only be interested in 1 feature on a part, requesting multiple results can help you see why a secondary feature is sometimes being returned by Vision Guide as the primary feature you are interested in. This generally happens a few different ways:

1. When 2 or more features within the search window are very similar and as such have very close Score results.
2. When the Confusion or Accept properties are not set high enough which allow other features with lower scores than the feature you are interested in to meet the Accept property setting .

Both of the situations above can be quite confusing for the beginning Vision Guide user when searching for a single feature within a search window.

If you have a situation where sometimes the feature you are searching for is found and sometimes another feature is found instead, use the Show All Results dialog to home in on the problem. Follow the following steps to get a better view of what is happening:

(1) Set your NumberToFind property to 3 or more.

(2) Run the vision object from the Vision Guide Development Environment.

(3) Click on the ShowAllResults property button to bring up the Show All Results dialog.

(4) Examine the scores of the top 3 or more features that were found.

(5) If only 1 or 2 features were found (*Vision Guide* will only set scores for those features that are considered found) reduce your Accept property so that more than 1 feature will be found and Run the vision object again. (You can change the Accept level back after examining the Show All Results dialog)

(6) Click on the ShowAllResults property button to bring up the Show All Results dialog.

(7) Examine the scores of the top 3 or more features that were found.

Once you examine the scores of the top 3 or more features that were found as described above, it should become clear to you what is happening. In most cases you will see one of these two situations.

1. Each of the features that were found has a score greater than the Accept property setting. If this is the case, simply adjust your Confusion property value up higher to force the best feature to always be found rather that allowing other features to be returned because they meet the Accept threshold. You may also want to adjust the Accept property setting.

2. Each of the features are very close in score. If this is the case, then you will need to do something to differentiate between the feature which you are primarily interested in such as:

   - Readjust the search window so the features that are randomly returning as the found feature are not contained inside.

   - Teach the Model again for the feature that you are most interested in.

   - Adjust the lighting for your application so the feature that you are most interested in gets a much higher score than the other features that are currently fooling the system.

Accessing Multiple Results from the SPEL+ Language

We already explained that the CurrentResult property is used to set which result will be displayed in the Results list. It is also used to determine which result record to return results for. In other words if we want to get the Area results from the 3rd result returned from a Blob object, CurrentResult must be set to 3. You have already seen how this can be done from the Object tab Properties list. Now let's take a look at how to access multiple results from SPEL+.

Multiple result access from SPEL+ treats the result like a type of array where the CurrentResult is referenced with a subscript number next to the result to be obtained. The first example below shows how to get the third Area result and put it in a variable called area from the SPEL+ Language.

```
VGet seqname.objname.Area(3), area
```

The 2nd example below shows how to get that same 3rd Area result but this time assign it as the value of the 3rd element in an array called Area().

```
VGet seqname.objname.Area(3), area(3)
```

Variable names can also be used to represent the element in an array rather than fixed elements as in example #2 above. Notice that the variable called “var” is used as the subscript for the Area result.

```
VGet seqname.objname.Area(var), area(var)
```

The 4th example assumes that you have used a single vision object to find multiple like parts (lets say up to 10). You would now like to pick these parts (let’s say they are pens) with a robot so you will need to store the X, Y, and U coordinates to variables to represent the coordinate values of each of the parts found. The following code could pull these coordinates out of the RobotXYU result and put them into X, Y and U arrays that could later be used for the robot to move to.

```
Function test
  Boolean found(10)
  Integer numfound, i
  Real X(10), Y(10), U(10)

  Jump camshot   'move camera into position snap shot

  VRun seq01     'run the vision sequence to find the pens

  VGet seq01.blob01.NumFound, numfound     'how many found

  For i = 1 to numfound    'get robot coords
    VGet seq01.blob01.RobotXYU(I), found(i), X(i), Y(i), Z(i)
  Next i
  'Add code for robot motion here……
Fend
```

5.4.11 Turning All Vision Object Labels On and Off

Force Labels Off (Vision Toolbar Only)

The Force Labels Off feature is a useful method to reduce screen clutter when working with many vision objects in one Sequence. The Force Labels Off button on the Vision Guide toolbar is a two-position button. When pressed in, the labels for all vision objects are turned off, thus making the objects which are displayed easier to see. When the Force Labels Off is in the out position (not pressed in), then labels are shown for each vision object that is displayed in the image display.

The Force Labels Off feature is sometimes used in conjunction with the Force All Graphics On toolbar button. When the Force all Graphics On toolbar button is pressed, you can still Force Labels Off. This means that even though the Force All Graphics On toolbar button is pressed in, the labels still will not be displayed because the Force Labels Off button is also pressed in.

NOTE

If you are working in a vision sequence where you just turned the Force Labels Off button Off (i.e. it is in the out position), and you still cannot see a specific vision object, chances are that the Graphics property for that vision object is set to None. This means don't display graphics at all for this object and may be your problem.

The Force Labels Off toolbar button is dimmed (made light gray in color and inaccessible) if there are no vision sequences for the current project.

5.4.12 Turning All Vision Object Graphics On

Force All Graphics On (Vision Toolbar Button Only)

The Force All Graphics On toolbar button provides a quick method to turn all graphics (search window, model origin, model window, Lines, and Labels) for all vision objects in the current vision sequence On with one button click. This button overrides the setting of the Graphics property for each individual vision object making it easy to quickly see all vision objects rather than modifying the Graphics property for each vision object individually.

NOTE

The Force Labels Off feature is sometimes used in conjunction with the Force All Graphics On toolbar button. However, it is important to note which button has precedence. When the Force all Graphics On toolbar button is pressed, you can still Force Labels Off. This means that even though the Force All Graphics On toolbar button is pressed in, the labels still will not be displayed because the Force Labels Off button is also pressed in.

It should be noted that the Force All Graphics On toolbar button is dimmed (made light gray in color and inaccessible) if there are no vision sequences for the current project.

6. Histograms and Statistics Tools

6.1 Vision Guide Histograms

Histograms are a very powerful feature when working with machine vision systems. They allow you to view data in a chart format that makes it much easier to see things like why a part isn't found properly. They also give hints for things like how to find a medium gray blob when a dark and light gray blob can also be found within the search window.

The Vision Guide Histogram dialog displays gray levels along the horizontal axis and pixel counts along the vertical axis. This lets us easily see how many pixels for each gray level are shown in the current image. The left side of the Histogram graph (or lower gray level numbers) shows pixels that are approaching darker gray levels (black). Darker pixels are represented with a low value (approaching 0). The right side of the Histogram graph (or higher gray level numbers) shows pixels that are approaching lighter gray levels (white). Lighter pixels are represented with a high value (approaching 255).

6.1.1 Using Histograms

To bring up the Histogram dialog, first select the vision object that you would like to see a Histogram for. (This can be done from the drop down list at the top of the Object tab.) Histograms are supported for the Correlation and Blob objects. Next run the vision object or vision sequence to make sure that the vision object you selected has valid results. Then click on the Histogram toolbar button or select Histogram from the Vision menu. The Histogram dialog will appear as shown in Figure 69 if the vision object is a Correlation object and as shown in Figure 70 if the vision object is a Blob object.

6.1.2 Example Histograms

Figure 68 shows 10 dark rings on a light gray background. The best vision tool for finding these parts will probably be a Blob object since each of these parts will rotate up to 360 degrees. However, we are going to use this same image for showing the Histogram dialog when used with a Correlation object and also when used with a Blob object.

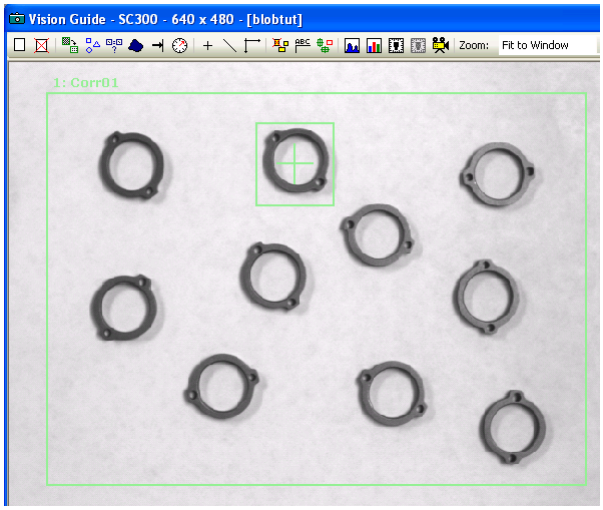


Figure 68: Example Image with 10 Rings using Corr01 object

6.1.3 Histograms with Correlation Objects

There are some important differences in the Histogram dialog when used with a Correlation object vs. using it with a Blob object. Figure 69 shows the Vision Guide Histogram dialog that was generated for the Corr01 object. Notice that it shows the spread of pixels over the various gray levels very nicely so that we can see how the image is dispersed. You can easily see that there are a lot more light pixels than dark pixels which is true since most of the image in Figure 68 is the gray background and not the dark rings.

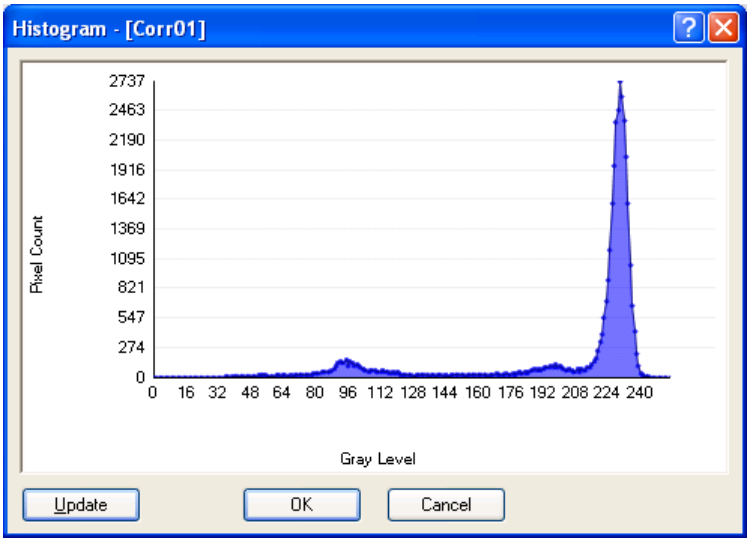


Figure 69: Example Histogram for Correlation object “Corr01”

6.1.4 Histograms with Blob Objects

Figure 70 shows the Histogram for the same image when used with a Blob object. Notice that there are 2 vertical bars with values attached at the base called ThresholdLow and ThresholdHigh. These are Blob object properties that are used to specify which gray levels to include as part of the found blob and which to include as part of the background.

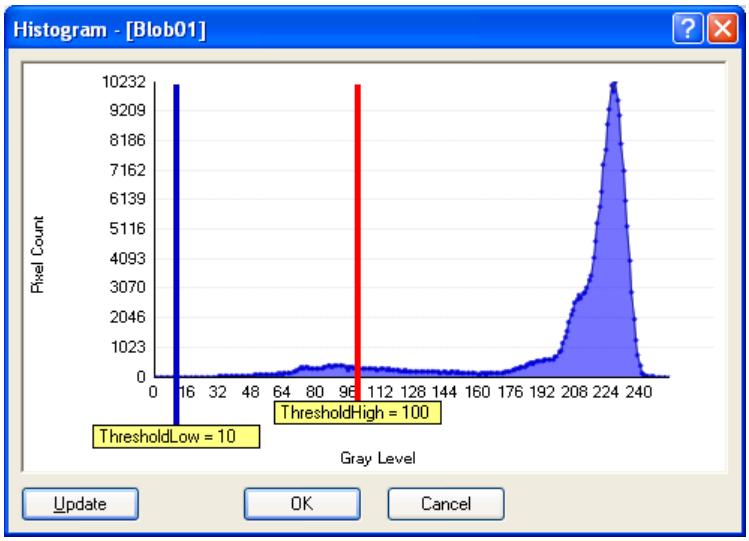


Figure 70: Example Histogram for Blob object “Blob01” (Dark on Light)

The area between the ThresholdLow and ThresholdHigh slider bars is the grouping of pixels that represent the levels of gray we want to define as dark or white pixels, which is set with the ThresholdColor property. The Polarity property defines whether we are looking for dark objects or light objects.

Adjusting the ThresholdLow and ThresholdHigh Properties

Look at Figure 70 again. Notice that the ThresholdLow property is set to 10 and the ThresholdHigh property is set to 100. These are the default values for these Blob object properties. When we first run the Blob object on the Rings image (with NumToFind set to 10) we get a result as shown in Figure 71. Notice that the Extrema for many of the blobs found does not go around the outer parts of the rings and in some cases 2 blobs are found as 1 part (see the arrows in Figure 71 that indicate the problem areas). This is because the ThresholdLow and ThresholdHigh properties must be adjusted based on the Histogram results.

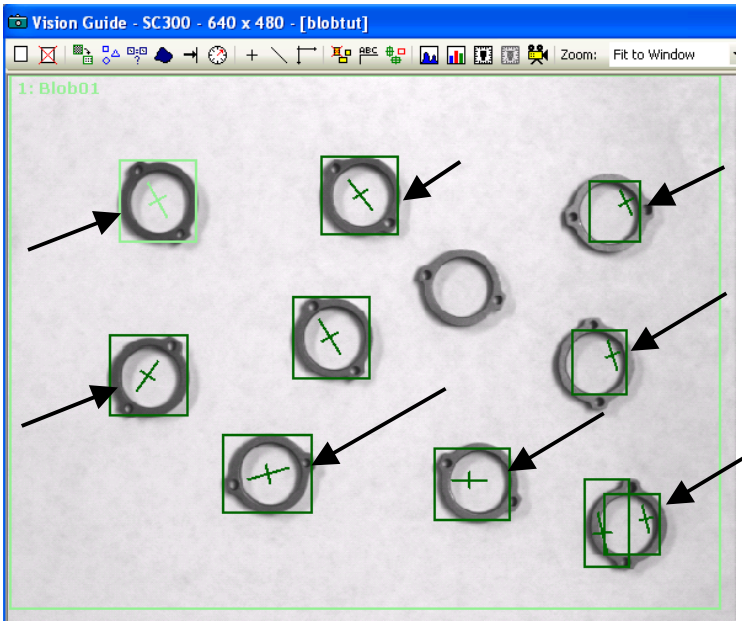


Figure 71: Image of 10 Rings with poor threshold settings

If we use the Histogram dialog to examine the histogram for the rings image we see a large distribution of gray starting at a level around 32. Then another large peak starts at around 160. Since the largest distribution of gray in the rings image is the light background it is easy to see that the distribution of pixels above 160 are for the background. Also, since the other peak in the histogram runs from about 32 till up to 160 this shows the distribution of the darker parts of our image which are the rings (the part we are interested in). The ThresholdLow and ThresholdHigh values can be adjusted so that our found boxes for each blob fall right around the outer bounds of the rings. This is done by clicking on the ThresholdHigh and ThresholdLow bars in the Histogram dialog and dragging them to the positions as shown in Figure 72.

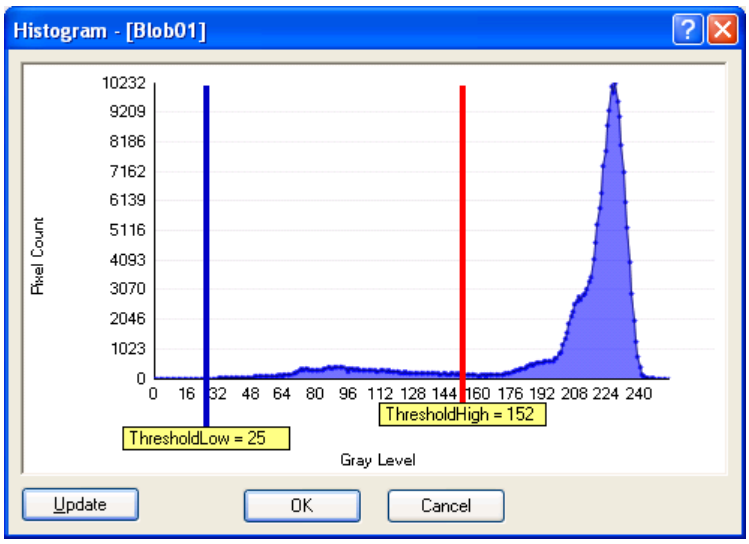


Figure 72: Histogram of Rings Image with better Threshold Settings

If we look at the rings image after running the Blob object with our new ThresholdLow and ThresholdHigh settings, we can see that the returned results are much more in keeping with what we wanted. Each ring is now found with the correct extrema for each blob. (See Figure 73)

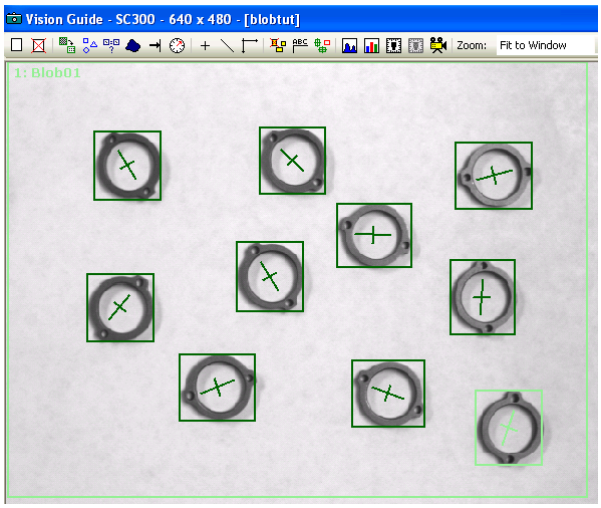


Figure 73: Image of 10 Rings with Improved Threshold Settings

6.2 Using Vision Guide Statistics

Selecting Statistics from the Vision menu or clicking on the Statistics toolbar button opens the Statistics dialog. Statistics calculations are a built-in feature of Vision Guide and can prove to be quite useful.

Vision Guide automatically keeps track of result data for future statistical reference for each vision object every time it is run from the Vision Guide window, Run window, or Operator window. This means that each time you run a vision object (regardless of the method used to initiate the vision object execution) the result data for that instance of execution is stored for use in calculating statistical information about that object.

Since most vision applications can be solved in a variety of ways, many times you may want to try multiple methods and compare the results. This is where the statistics feature is invaluable. It allows you to run reliability tests on vision objects thereby pinpointing which vision solution is best for your application. Best of all, these tests can be initiated from the SPEL+ language or even from the Vision Guide point and click interface where no programming is required.

6.2.1 Dialog Box Options/ Info

Option	Description
Object	Dropdown list for selecting which object to review statistics for.
Found	Displays the number of times the object was found vs. the number of times it was run. For example, 5/6 means the object was run 6 times and found 5.
Reset Current	Reset the statistics calculations for the current vision object. This clears all data for the current object only.
Reset All	Reset the statistics calculations for all vision objects in the sequence.
Close	Closes the Statistics dialog. All results remain in place.
Help	Opens the on-line help for Statistics.

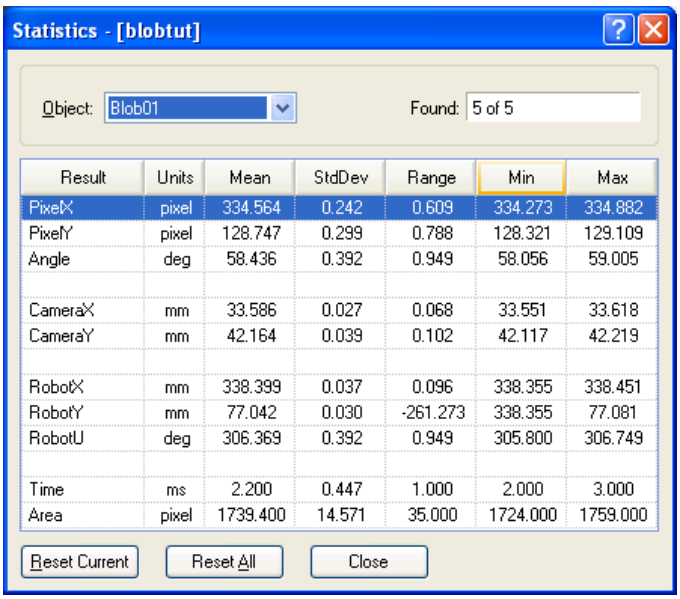


Figure 74: The Statistics dialog

6.2.2 Vision Objects and Statistics Supported

The following list of vision objects are supported for use with statistics:

Blob, Correlation, Geometric, Edge, Line, Point, Polar

The following statistics are available from the Statistics dialog:

Statistic	Description
Mean	The computed average for all found objects of the specified object.
Standard Deviation	The "Sample Standard Deviation" which is basically the square root of ($(\sum (xi - mean)^2$) / n - 1). Only found object results are used for this computation.
Range	The range of the results which is (Min - Max). This is computed for found objects only.
Min	The minimum result of all found objects of the specified object.
Max	The maximum result of all found objects of the specified object.

6.2.3 Vision Object Results Supported

The following list of vision object results are displayed in the Statistics dialog. Not all results are returned for every object type. See the individual result descriptions in the Vision Guide Properties and Results Reference Manual for more information.

Result	Description
PixelX	The X coordinate position of the found part's position in pixel coordinates.
PixelY	The Y coordinate position of the found part's position in pixel coordinates.
Angle	The amount of rotation associated with a part. This shows the amount of rotation a part may have vs. the originally taught position.
Area	The number of connected pixels found for a blob.
CameraX	The X coordinate position of the found part's position in the camera coordinate frame.
CameraY	The Y coordinate position of the found part's position in the camera coordinate frame.
Length	The length of a Line object in millimeters. (Requires that a calibration be done)
PixelLength	The length of a Line object in pixels. (Does not require calibration.)
RobotX	The X coordinate positions of the found part's position with respect to the Robot World Coordinate System.
RobotY	The Y coordinate positions of the found part's position with respect to the Robot World Coordinate System.
RobotU	The U coordinate positions of the found part's position with respect to the Robot World Coordinate System. (Rotational orientation of the part in robot space.)
Time	The amount of time which it took to execute the vision object. (i.e. to find the part.)
Score	The measure of how well a particular part was found. (i.e. how well a correlation object matches it's model, or the strength of transition for an Edge object.)

6.2.4 Vision Object Statistics Available From SPEL+

The following list of vision object Statistics results are available from the SPEL+ Language.

Vision Object	Statistics Results Supported from SPEL+
Geometric	AngleMax, AngleMean, AngleMin, AngleStdDev CameraXMax, CameraXMean, CameraXMin, CameraXStdDev CameraYMax, CameraYMean, CameraYMin, CameraYStdDev PixelXMax, PixelXMean, PixelXMin, PixelXStdDev PixelYMax, PixelYMean, PixelYMin, PixelYStdDev RobotXMax, RobotXMean, RobotXMin, RobotXStdDev RobotYMax, RobotYMean, RobotYMin, RobotYStdDev RobotUMax, RobotUMean, RobotUMin, RobotUStdDev ScoreMax, ScoreMean, ScoreMin, ScoreStdDev TimeMax, TimeMean, TimeMin, TimeStdDev
Blob	AngleMax, AngleMean, AngleMin, AngleStdDev AreaMax, AreaMean, AreaMin, AreaStdDev CameraXMax, CameraXMean, CameraXMin, CameraXStdDev CameraYMax, CameraYMean, CameraYMin, CameraYStdDev PixelXMax, PixelXMean, PixelXMin, PixelXStdDev PixelYMax, PixelYMean, PixelYMin, PixelYStdDev RobotXMax, RobotXMean, RobotXMin, RobotXStdDev RobotYMax, RobotYMean, RobotYMin, RobotYStdDev RobotUMax, RobotUMean, RobotUMin, RobotUStdDev TimeMax, TimeMean, TimeMin, TimeStdDev
Correlation	AngleMax, AngleMean, AngleMin, AngleStdDev CameraXMax, CameraXMean, CameraXMin, CameraXStdDev CameraYMax, CameraYMean, CameraYMin, CameraYStdDev PixelXMax, PixelXMean, PixelXMin, PixelXStdDev PixelYMax, PixelYMean, PixelYMin, PixelYStdDev RobotXMax, RobotXMean, RobotXMin, RobotXStdDev RobotYMax, RobotYMean, RobotYMin, RobotYStdDev RobotUMax, RobotUMean, RobotUMin, RobotUStdDev ScoreMax, ScoreMean, ScoreMin, ScoreStdDev TimeMax, TimeMean, TimeMin, TimeStdDev
Edge	CameraXMax, CameraXMean , CameraXMin, CameraXStdDev CameraYMax, CameraYMean, CameraYMin, CameraYStdDev PixelXMax, PixelXMean, PixelXMin, PixelXStdDev PixelYMax, PixelYMean, PixelYMin, PixelYStdDev RobotXMax, RobotXMean, RobotXMin, RobotXStdDev RobotYMax, RobotYMean, RobotYMin, RobotYStdDev ScoreMax, ScoreMean, ScoreMin, ScoreStdDev TimeMax, TimeMean, TimeMin, TimeStdDev

Vision Object	Statistics Results Supported from SPEL+
Polar	AngleMax, AngleMean, AngleMin, AngleStdDev CameraXMax, CameraXMean, CameraXMin, CameraXStdDev CameraYMax, CameraYMean, CameraYMin, CameraYStdDev PixelXMax, PixelXMean, PixelXMin, PixelXStdDev PixelYMax, PixelYMean, PixelYMin, PixelYStdDev RobotXMax, RobotXMean, RobotXMin, RobotXStdDev RobotYMax, RobotYMean, RobotYMin, RobotYStdDev RobotUMax, RobotUMean, RobotUMin, RobotUStdDev ScoreMax, ScoreMean, ScoreMin, ScoreStdDev TimeMax, TimeMean, TimeMin, TimeStdDev
Line	AngleMax, AngleMean, AngleMin, AngleStdDev LengthMax, LengthMean, LengthMin, LengthStdDev PixelLengthMax, PixelLengthMean, PixelMeanMin, PixelLengthStdDev
Point	CameraXMax, CameraXMean , CameraXMin, CameraXStdDev CameraYMax, CameraYMean, CameraYMin, CameraYStdDev PixelXMax, PixelXMean, PixelXMin, PixelXStdDev PixelYMax, PixelYMean, PixelYMin, PixelYStdDev RobotXMax, RobotXMean, RobotXMin, RobotXStdDev RobotYMax, RobotYMean, RobotYMin, RobotYStdDev

7. Vision Sequence

7.1 Vision Sequence Overview

This chapter will provide the information required to understand and use vision sequences. However, since vision objects are such a fundamental part of vision sequences, it is important to read the vision objects chapter prior to reading about vision sequences. In this chapter we will cover the following items:

- Vision sequence definition and general information.
- The Sequence tab.
- Creating and deleting vision sequences.
- Vision sequence properties and results.
- Running vision sequences from the Vision Guide window.
- Testing and Debugging vision sequences.
- How to run vision sequences from the SPEL$^+$ language.
- Image Acquisition.

7.2 Vision Sequence Definition & General Information

What is a vision sequence?

A vision sequence is a group of vision objects in a specific order that can be executed from the Vision Guide Development Window or from the SPEL+ language. A vision sequence, like vision objects, has specific properties that are used to set up execution. For example, the Camera property defines which camera will be used to acquire an image for this vision sequence and the RuntimeAcquire property defines how the image will be acquired for this vision sequence.

While vision objects can be run and tested individually, most vision applications require a series of vision objects to be executed sequentially to compute the final result. This is where vision sequences come in. Vision sequences are used to run the vision objects in a specific order. So once you have created and tested each of your individual vision objects you will then want to test them executing as a group. Or in other words you will want to test your vision sequence.

How does a vision sequence work?

Vision sequences contain one or more vision objects and the fundamental purpose of the vision sequence is to execute these vision objects in sequential order (as previously defined by the user.) The following steps show what happens when a vision sequence executes:

1. The vision sequence uses its property settings to set up the execution of the sequence. These include things like which camera to use to acquire an image, determining the method to use to acquire an image (Strobed, Stationary, or No acquire at all).
2. The image is acquired and placed in the frame buffer for use by the vision objects that will execute next. (If the RuntimeAcquire property is set to "None", then the image that was already in the frame buffer is used for the sequence. This allows multiple sequences to operate on the same image.)
3. Vision objects are now executed in sequential order using the image acquired in step 2 above. Once all the vision objects have been executed, sequence and object results are available for display on the Sequence tab and Object tab and are also available from the SPEL$^+$ Language.

Vision sequences and EPSON RC+ projects

Remember that EPSON RC+ is based upon the Project concept where each EPSON RC+ Project contains programs, teach points, and all other robot configuration parameters required for a specific application. When Vision Guide is used within a EPSON RC+ project all vision sequences which were created within that project are also saved with the project. This is done so that the next time you open a specific project, all the vision sequences which were created in that project are available for use or modification.

7.3 The Vision Sequence Tab

The Sequence tab is used to:

- Select which vision sequence to work on
- Set properties for the vision sequence
- Examine the results for the entire vision sequence execution
- Manipulate the execution order of the vision objects.

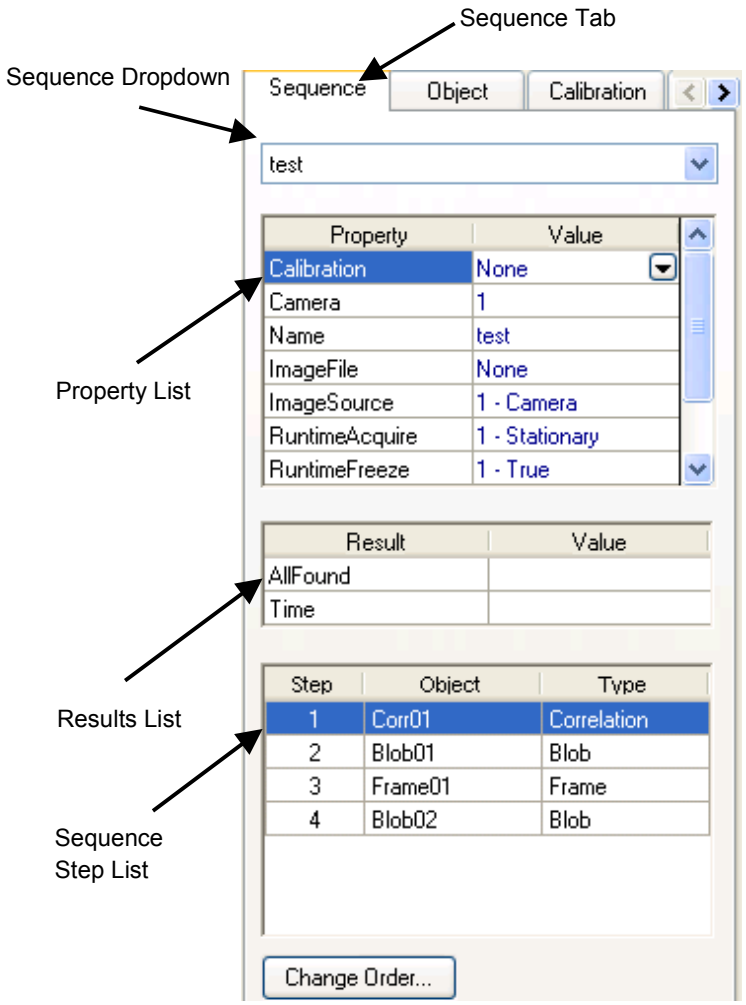


Figure 75: The Sequence Tab

Selecting a Vision Sequence

Notice the drop down list shown at the top of the Sequence tab. Clicking on the down arrow displays a list of all the vision sequences for this project. You can then click on any one of the vision sequences to select which one you want to work on.

Sequence Tab Properties List

The properties for the vision sequence are shown in the Properties list that is similar to a small spreadsheet. The name of the property is on the left side and the value is shown on the right. Vision properties are set by clicking on the Value field for a specific property and then either entering a value or selecting from a list which is displayed. The vision sequence properties are described later in this chapter in the section called *Vision Sequence Properties and Results*.

Sequence Tab Results List

The results for the vision sequence are shown in the Results list that is located just below the Properties list. The Results list will only display values in the Value field after a vision sequence is executed. Prior to execution the result fields will not display anything. The vision sequence results are described later in this chapter in the section called *Vision Sequence Properties and Results*.

Sequence Tab Step List

Located at the bottom of the Sequence tab is the sequence Step list. This list displays all the vision objects that have been defined for this vision sequence. The order they are displayed in is the order in which the vision objects will execute. See *Changing Object* Execution Order later in this chapter for more information.

7.4 Vision Sequence Properties and Results

Vision sequences, like vision objects, have properties and results. The primary difference is that the properties and results for Vision Sequences apply to the entire Vision Sequence vs. vision object property settings that apply to only one vision object.

Vision Sequence Properties

Vision sequence properties are normally used for setting up the execution of the Vision Sequence and to set the proper values for the acquisition of an image for which all the vision objects within the vision sequence will then work from. All vision sequence properties apply to the entire vision sequence. For example, the Calibration property is set at the vision sequence level rather than the vision object Level. This way all vision objects within a vision sequence will use the Calibration specified by the Calibration property and all the results are guaranteed to be with respect to the same calibration.

The following list is a summary of properties for vision sequences. The details for each property are explained in the *Vision Properties and Results Reference Manual*.

Property	Description
Calibration	Specifies the camera calibration to use for the vision sequence. This must be specified in order to return Robot or Camera coordinate results. Default: none
Camera	Specifies which camera to use for the vision sequence. Default: 1
CameraBrightness	Specifies the camera brightness value. Default: 128
CameraContrast	Specifies the camera contrast value. Default: 128
CameraGain	Specifies the camera contrast value that will be assigned to the video signal in the A/D converter in the camera. Default: 80
CameraOffset	Specifies the camera brightness value that will be assigned to the video signal in the A/D converter in the camera. Default: 0
ExposureDelay	Specifies the delay before the exposure is started. (Unit : microsecond) Default: 0
ExposureTime	Specifies the exposure time when used in asynchronous reset mode. (Unit : microsecond) Default: 0
ImageFile	Specifies the name of the file that contains the image on disk that you want to work from. Default: None (no image file)
ImageSource	Specifies the source for the image. Default: 1 – Camera

Property	Description
Name	The name of the vision sequence.
RuntimeAcquire	Defines which method to use to acquire an image for use with the vision sequence. Default: 1 - Stationary
RuntimeFreeze	Tells the vision sequence to freeze the image for display. Default: True
SaveImage	Displays a dialog for saving the current image to disk.
ShowProcessing	Specifies whether image processing will be displayed. Default: True
StrobeDelay	Specifies the delay before the strobe output is turned on. (Unit : microsecond) Default: 0
StrobeTime	Specifies the time that the strobe output is on. (Unit : microsecond) Default: 0
TriggerMode	Specifies the electronic shutter trigger mode. Default: 1- Leading Edge

Vision Sequence Results

Vision sequence results apply to the entire vision sequence. They are useful because they tell you information about the vision sequence as a whole. For example, the AllFound result returns whether or not all vision objects within the vision sequence were found or not.

The following list is a summary of vision sequence results with brief descriptions. The details for each result are explained in the *Vision Guide Properties and Results Reference Manual*.

Result	Description
AllFound	Returns whether or not all vision objects within the sequence were found.
Time	Returns the total execution time for the vision sequence. This includes the cumulative time for all vision objects as well as the time required to acquire an image.

7.5 Creating a New Vision Sequence

Description

Clicking the New Sequence toolbar button or selecting New Sequence from the Vision menu opens a dialog that requests a name to be given for the new sequence. Type a name of up to 16 alphanumeric characters and click OK. You will notice that once a vision sequence is created, almost all the Vision Guide window toolbar icons become enabled indicating that they are now available for use.

When creating a new sequence, you can copy an existing sequence by selecting it from the Copy from existing sequence dropdown list.

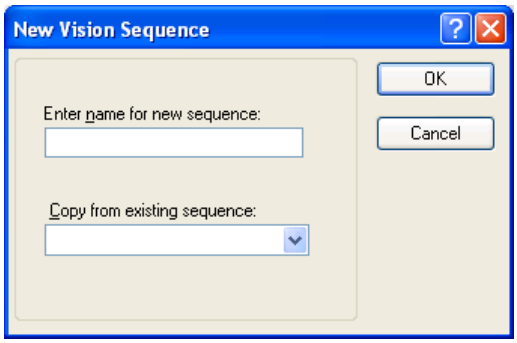
Shortcuts

Toolbar:

Keys: None

Dialog Box Options

Option	Description
OK	Creates the new vision sequence with the associated name.
Cancel	Cancels the new vision sequence operation.
Help	Opens the help description for the New Sequence command.



7.6 Deleting a Vision Sequence

Description

Clicking on the **Delete Sequence** Vision Guide toolbar button or selecting the **Delete Sequence** item from the Vision menu opens a dialog with a list of all vision sequences that have been created for the current project. The user must then use the mouse or arrow keys to select which vision sequence to delete. Once the correct vision sequence name is highlighted, click on the **Delete** button in the Delete Vision Sequence dialog. A confirmation message box will be displayed.

NOTE

Keep in mind that once you delete a vision sequence and save the current project, the sequence is gone forever. If you accidently delete a sequence, you can execute File | Restore to restore the entire vision project to the state of the last save. If you think you may want to use a sequence at a later date be sure to keep it. Just because you keep it with a project doesn't mean you have to use it in your programs.

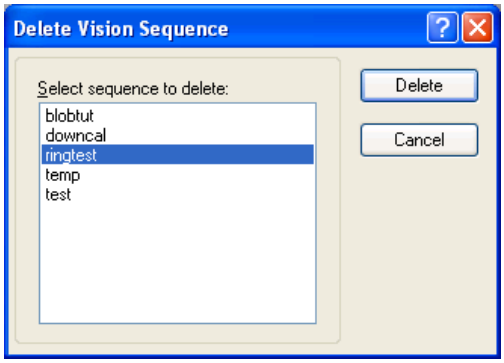
Shortcuts

Toolbar:

Keys: None

Dialog Box Options

Option	Description
Selection List	Used to select which sequence to delete.
Delete	Deletes the highlighted vision sequence.
Cancel	Cancels the delete vision sequence operation.
Help	Opens the help description for Delete Sequence Command.



7.7 Deleting a Vision Object

To delete a vision object, first select the vision object you want to delete. This is done by either clicking on the vision object's name or by selecting one of the vision objects from the vision object drop down list which is located at the top of the Object tab.

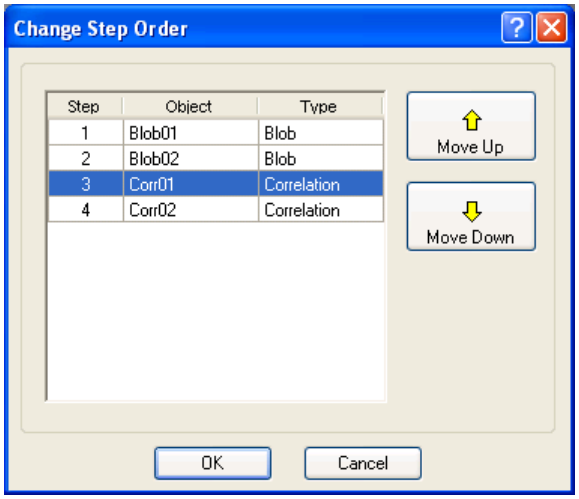
Once the vision object to delete has been selected, select **Delete Object** from the Vision menu or click on the **Delete Object** toolbar button  . A confirmation dialog will appear. Click Yes to delete the object.

NOTE

The **Delete Object** toolbar button is dimmed if there are no vision sequences for the current project or if there are no objects for the current vision sequence.

7.8 Change Order of Sequence Steps

To change the order of sequence steps, click on Change Order button located at the bottom of the Vision Guide window Sequence tab. This opens the Change Step Order dialog. To change the step number for an object, select the object from the list, then click on the Move Up or Move Down buttons. Click OK to accept the changes.



NOTE

When modifying the execution order of the vision objects, pay close attention to vision objects that can be based on other objects, such as a Line object having a StartingPoint as "Corr01" and an EndPoint of "Blob01". For example, if the Line object step is moved above the "Corr01" object but remains below the "Blob01" object, then a warning will appear stating that the Line object's StartingPoint property will be modified to "Screen". There are many possible combinations of vision objects based on other vision objects so pay close attention to which objects require another object to execute properly. Vision Guide should always warn you when you modify the order of an object that was dependent upon another, but it's a good idea that you try to remember the dependencies as well.

7.9 Running Vision Sequences

Vision Sequence Selection

Before you can run a vision sequence you will need to select the vision sequence that you want to run from the drop down list box that is found at the top of the Sequence tab (see Figure 75). This drop down list box makes it easy to change between vision sequences within a project.

Setting Vision Sequence Properties

After you have selected the vision sequence to run, you may need to set some parameters for the sequence. Setting vision sequence parameters is done from the Properties list on the Sequence tab. Properties must be properly set up prior to running a vision sequence. For example, if you want to create and run a sequence using an image which will be acquired from Camera 2, then you must be sure and set the Camera property for that sequence to 2. One of the most common mistakes for new users is to run a vision sequence with the wrong Camera property setting.

NOTE

The Camera property is used to select which camera to use for a particular vision sequence. The cameras will be automatically detected by the system at startup. If you power up a camera while Vision Guide is displaying the video for that camera, you may not see video until you reselect the camera from the sequence Camera property.

Vision Sequence Results

Once you run a vision sequence, you will probably want to see the results for the sequence. The vision sequence results can be seen from the Results list located on the Sequence tab see Figure 75).

Sequence Tab Step List

In the lower half of the Sequence tab there is a list of all the objects in the currently selected sequence called the Step list. The Step list shows the objects in the order they will be executed. When using the Step button to step through a sequence, the next step to execute is high lighted in the Step list.

Running a Vision Sequence

On the Run Panel located at the bottom of the Vision Guide window you can see a button labeled **Run Sequence**. Clicking on this button causes the entire sequence to execute. Once you have created and tested the objects which you want to use for a specific sequence you will want to test the entire vision sequence by clicking on the **Run Sequence** button.

Running a Sequence Multiple Times (Cycles)

When using the **Run Sequence** button, pay close attention to the **Cycles** text box which is located to the right of the **Run Sequence** button. The number that is entered in this box specifies how many times to run the vision sequence. This is useful for testing the reliability of a vision sequence before writing any code to make it work with the robot. By running a vision sequence multiple times and then using the Statistics feature of Vision Guide, you can see the Mean, Standard Deviation, Range, and Minimum and Maximum values for each vision object in the sequence. This helps give you a good perspective on how well your vision sequence is performing. Any number larger than 1 will cause the **Cycles** text box color to change to yellow. This warns you that you are about to execute multiple cycles when you click the **Run Sequence** button.

The Image Display During Vision Sequence Execution

When a vision sequence is executed, the image display will show which vision objects were found and which were not found by a display of color. The vision objects that were found will display a green outline of the search window and found position. The vision objects that were not found will display a red outline of the search window. (Since they are not found there will be no found position displayed.)

NOTE

Keep in mind that if the Graphics property for a specific vision object is set to None, then no graphics will be displayed for that specific vision object. To ensure that all vision objects have their graphics displayed, click the **Force All Graphics On** Vision Guide toolbar button  . This will cause all graphics to be displayed regardless of the individual Graphics property settings.

Aborting Vision Sequence Cycles

When running multiple sequence cycles, you can click the Abort button on the Vision Guide Run Panel to stop sequence execution.

7.10 Testing and Debugging a Vision Sequence

One of the first places to look to see if everything in a vision sequence is functioning properly or not is to look at the status of the vision sequence AllFound property. If any of the vision objects was not found, the AllFound vision sequence property will be set to False. While this is a good first place to look to see if everything is working properly, it is no guarantee that everything is OK. There are certainly cases where all vision objects are found but the net result is still not acceptable. Therefore, eventually you will need to test and debug your vision sequence because something isn't quite working properly. The rest of this section is dedicated to explaining some of the debugging features and techniques used with Vision Guide sequences.

Examine the found color of the vision objects

One of the first things to always do when trying to find the source of a problem is to use the Found/Not Found color codes which are placed on the vision objects on the image display. vision objects that are found have their search windows and found position displayed in green. vision objects that are not found display their search windows in red. If any of the vision objects within your vision sequence display a red search window, this should be your first clue as to which vision object to more closely examine.

NOTE

Keep in mind that if the Graphics property for a specific vision object is set to None, then no graphics will be displayed for that specific vision object. To ensure that all vision objects have their graphics displayed, click on the **Force All Graphics On** Vision Guide toolbar button  . This will cause all graphics to be displayed regardless of the individual Graphics property settings.

Examining Individual Vision Object Results

When running a vision sequence, you can display the vision sequence results on the Sequence tab or you can display an individual vision object's results on the Object tab. If you suspect a specific vision object's performance or just want to closely monitor it after the vision sequence runs, you can do so with just a few clicks.

(1) Click on the Object tab to make it come to the front position

(2) Select the vision object you are interested in

(3) Examine the results for that vision object once the vision sequence is run

You can even switch vision objects or between the Object tab and Sequence tab while a vision sequence is running. This gives you the ability to start a vision sequence running for 1 or more cycles and then examine the results for many of the different vision objects within that vision sequence without having to stop the vision sequence.

Single stepping through a vision sequence

One of the other nice debugging features for the Vision Guide Development Environment is the Single Step feature. Located on the Run Panel is the **Step** button. Clicking on this button causes the vision sequence to single step one vision object at a time. When using the single step feature, you can see which vision object will execute next as the vision object name is highlighted in blue on the Sequence tab Step list. You can also change properties for any of the vision objects prior to their execution during single stepping. The first click of the **Step** button will cause all vision objects to go in an inactive state. They will become blue in color. The next click of the **Step** button will cause the first vision object in the Sequence to execute. It will be green in color if the object is found and red if it was not found. If the Sequence tab is currently displayed you can also see which vision object will execute next. It will be highlighted in blue.

Using the Statistics Feature

The Statistics feature is another debugging tool. When testing a vision sequence for reliability, you may want to set the cycle count to a high number and then let the vision sequence run for a while. Then check the statistical results for all the vision objects from the Statistics dialog. You can see which vision objects performed reliably and which had varying results. (The vision statistics features are explained in detail in *Histograms and Statistics Tools*.)

Switching between the Run and Vision Guide windows

There will be times when you have completed testing and think your system is OK to run from the Run window for final test and in fact something may still need adjustment. For these situations, we made the Vision Guide Development Window accessible while running from the Run window.

Let's take a look at an example. Assume you are running a Vision Guide application from the Run window and you would like to see the Score result for the Correlation object called "pin1" because you have had problems with this object in the past. While the system is running from the Run window, click on the Vision toolbar button  from the EPSON RC+ main toolbar. This will bring up the Vision Guide Development Window while the application is still running. Next, click on the Object tab to bring it forward. Then select the "Pin1" object from the drop down list box at the top of the Object tab. You will now see the results for the object called "Pin1" without actually stopping the program from running. To return back to the Run window simply click on the Run window Toolbar button on the EPSON RC+ main toolbar.

7.11 Running Vision Sequences from SPEL+

One of the primary design goals for Vision Guide was to make sure that whatever was created from the Point and Click Vision Guide Development Environment was fully accessible from the SPEL+ language. We didn't want our users to use the Vision Guide Development Environment as a prototyping environment and then have to rewrite everything again in SPEL+.

Therefore, once you create a vision sequence from the Vision Guide Development Environment, that vision sequence can be executed from VRun command in SPEL+.

The syntax for VRun is just "VRun seqname". Where "seqname" is the name of the vision sequence you want to run. In the code snippet shown below you will see 2 different VRun statements each initiating a different vision sequence.

"VRun findpin" initiates a sequence to find the center position of a pin for the robot to pick up the pin. "VRun housing" is executed later in the program to find the position in the housing to place the pin.

```
VRun findpin
VGet findpin.pincntr.RobotXYU, found, X, Y, U
.
.
VRun housing
VGet housing.setpos.RobotXYU, found, X, Y, U
```

This is just a small sampling of how to use the VRun command. Complete details on how to use the SPEL+ language with Vision Guide Sequences and with the Robot are explained in *Using Vision Guide in SPEL+*.

7.12 Image Acquisition

When are images acquired?

Many vision systems require that you specify a special command or step to acquire an image to do processing with. Vision Guide helps remove this extra step since images are acquired at the start of a vision sequence. All you have to do is specify the proper setting for the RuntimeAcquire property. For most applications you will never need to even set the Runtime Acquire property because it defaults to Stationary. This means that an image will be acquired at the beginning of the vision sequence execution.

Using the same image with multiple vision sequences

If you want to use the same image for 2 different vision sequences all you have to do is set the RuntimeAcquire property for the 1st vision sequence to Stationary and then set the RuntimeAcquire property for the 2nd vision sequence to None. Setting the 2nd vision sequence to None will keep that sequence from acquiring another image so all processing will be done on the image acquired from the 1st vision sequence.

Using strobed image acquistion

Vision Guide supports a strobe input that allows the vision sequence to take acquire an image and search after a strobe input has fired. To use a strobe input, follow these steps:

(1) Wire the strobe circuit to fire the strobe trigger input of the camera. The camera provides an ouput for a strobe lamp.

(2) Set the RuntimeAcquire property for the sequence you want to strobe to Strobed.

(3) In your SPEL$^+$ program, execute VRun as usual, then wait for the AcquireState property to change to the value 3, which indicates that the strobe has fired.

```
Integer state
Boolean found

#define PICTURE_DONE 3
TmReset 0
VRun seq1
Do
  Wait 0.01
  VGet seq1.AcquireState, state
  If Tmr(0) > 10 Then
    Error ER_STROBE_OT  ' User defined overtime error
  EndIf
Loop Until state = PICTURE_DONE
VGet seq1.obj1.Found, found
```

When you run a strobe sequence from the Vision Guide GUI, the system will wait until the strobe has fired. A message will be displayed in the EPSON RC+ status bar indicating this after waiting a few seconds. You can abort by clicking the Abort button.

 CAUTION	■ Ambient lighting and external equipment noise may affect vision sequence image and results. A corrupt image may be acquired and the detected position could be any position in an object's search area. Be sure to create image processing sequences with objects that use search areas that are no larger than necessary.

8. Calibration

8.1 Overview

Calibration is an important part of the Vision Guide system. In order to locate parts in the robot world coordinate system, or to make physical measurements, you must calibrate the system. Vision Guide supports both mobile (mounted to robot) and fixed camera orientations. The software has been designed to make calibration as simple and painless as possible. However, it is important that you review this chapter before creating calibrations for your applications.

The following topics will be discussed in this chapter:

- Calibration definition
- Camera orientation
- Reference points
- Create vision sequences for calibration
- Calibration GUI
- Calibration procedures

8.2 Calibration Definition

Vision Guide supports multiple calibration schemes for each project. Any calibration scheme can be used with one or more vision sequences in the same project. Each calibration scheme includes the following data:

Calibration name	The name of the calibration that is referenced by vision sequences. This can be any alphanumeric name up to 16 characters.
Camera number	The number for the camera being calibrated.
Camera orientation	How the camera is mounted.
Calibration target sequence name	The name of the vision sequence that will be used to calibrate the camera. This can be any sequence in the current project.
Robot calibration speed and acceleration (when required)	The speed and acceleration used during calibration.
Optional robot Arm, Tool, and Local used during calibration	You can use an Arm,Tool, and/or Local during calibration. These must be defined prior to teaching points or calibrating.
Robot Points for reference and camera locations.	These points are saved for each calibration.
Light outputs	These are optional standard outputs that can be used to control lights during calibration.

Each calibration uses nine points that span the field of view of the camera being calibrated. By spanning the field of view, the system can do the best job of compensating for camera tilt and lens inaccuracies.

When do you need to calibrate?

Calibration is required anytime you want to make a physical measurement or locate a position in robot world coordinates. Several results reported for vision objects require calibration. For example, the CameraX, CameraY, RobotX, RobotY, RobotU results require calibration.

8.3 Camera Orientations

Each calibration scheme requires that you select a camera orientation. This tells the system how your camera is mounted. Vision Guide supports the following camera orientations:

Camera Orientation	Description
Mobile J2	Camera is mounted on Joint 2 on SCARA robot or Cartesian robot. It reports robot world coordinates. Uses one reference point.
Mobile J4	Camera is mounted on Joint 4 on SCARA robot or Cartesian robot. It reports robot world coordinates. Uses one reference point.
Mobile J5	Camera is mounted on Joint 5 on 6-Axis robot. It reports robot world coordinates. Uses one reference point.
Mobile J6	Camera is mounted on Joint 6 on 6-Axis robot. It reports robot world coordinates. Uses one reference point.
Fixed downward	Camera does not move and is looking down into robot work envelope. It reports robot world coordinates. Uses nine reference points.
Fixed upward	Camera does not move and is looking up into robot work envelope. It reports robot world coordinates. Does not require a reference point. The calibration target is on the end effector.
Standalone	Camera does not move and can be at any orientation. This scheme does not report robot world coordinates. It is used for making physical measurements only. This includes mounting to robot Z-axis to make inspection measurements.

8.4 Reference Points

Each calibration scheme requires one or more reference points. These points are used to tell the system where a calibration target is during calibration. Except for fixed camera calibration, you teach these points using the robot. For standalone camera calibration, you manually enter the coordinates of the reference points into the system.

Whatever you use for a reference point, the vision system must be able to locate it anywhere in the field of view of the camera being calibrated.

8.4.1 Mobile calibration reference points

This scheme requires one reference point. You can specify a taught point that you manually teach by jogging the robot, or you can specify a point that is found with an upward looking camera. The later is preferred for greatest accuracy and automation.

Here are some examples of taught reference points:

- A part or calibration target in the robot work envelope.
- A hole somewhere in the work envelope that a tool mounted on the robot end effector can be slipped into.

When using an upward camera to find the reference point, there needs to be a thin plate with hole through it that can be seen by both the mobile camera and the upward looking camera. During calibration, the upward camera (which has been previously calibrated) locates the reference hole in the plate. Then the mobile camera is calibrated by searching for the reference hole in nine positions.

Using an upward camera to find the reference point for mobile camera calibration is more accurate because during calibration of the upward camera, the robot tool is rotated 180 degrees for each camera point so that the precise center of the U axis of the robot can be determined. This allows the upward camera to more accurately find the reference hole that the mobile camera will locate during calibration.

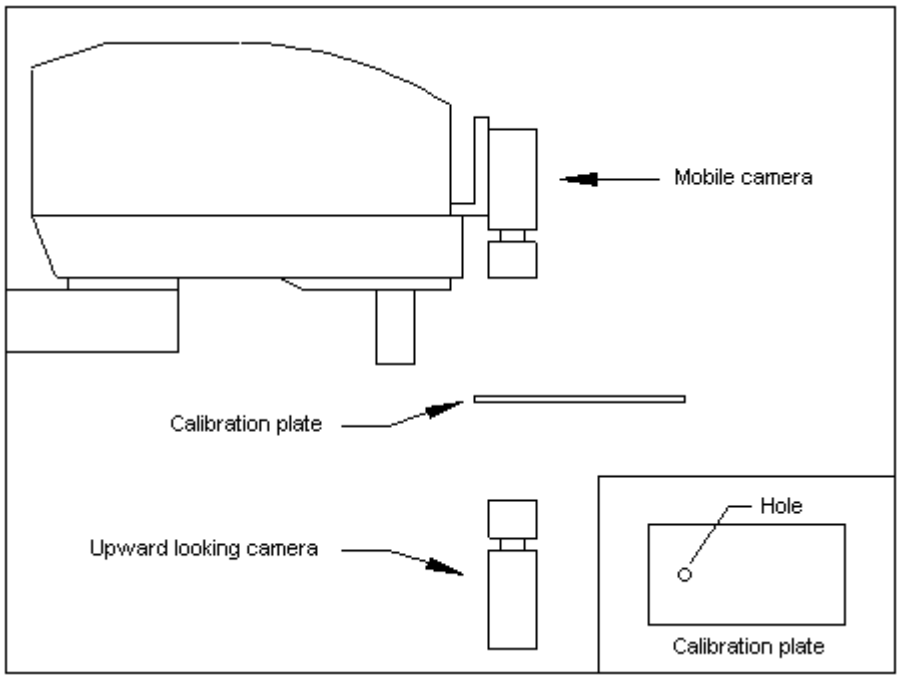


Figure 76: Mobile Camera Calibration using Upward Camera

8.4.2 Fixed Downward and Standalone camera reference points

The 'Fixed Downward' and 'Standalone' calibration schemes require a calibration target plate or sheet that contains nine targets. The figure below shows the pattern for the nine calibration targets.

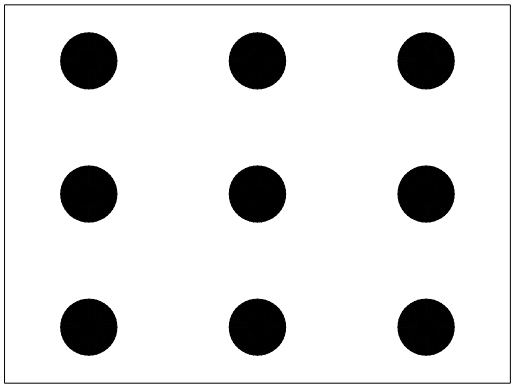


Figure 77: Fixed Camera Calibration Targets

For 'Fixed Downward' calibrations, the targets could be holes in a plate that a rod on the robot end effector can be slipped into. The distances between the targets do not have to be exact. When the points are taught, the calibration software will read the robot positions.

For Standalone cameras, a pattern on paper, mylar, etc. can be used. The horizontal and vertical distances between the targets must be known. These are entered when you teach the points from the Calibration dialog.

8.4.3 Teaching reference points

When teaching reference points for mobile and fixed downward calibration, you will be prompted to teach the point and then teach it again at 180 degrees from the first position. This allows the system to determine the true location of the reference point in the robot world. However, if you are using a robot tool that has been accurately defined and specified in the calibration setup, you can skip teaching the point at 180 degrees. To skip the 180-degree steps, click on the **Next** button to continue to the next step. For details on defining tools, see Defining Tools in *Using Vision Guide with SPEL+*.

8.5 Creating Vision Sequences for Calibration

Before you can calibrate a camera, you must create a vision sequence that can find the calibration target(s).

8.5.1 Mobile and Fixed Upward Calibration Vision Sequences

For 'Mobile' and 'Fixed upward', the sequence will search for one calibration target in the entire field of view. Be sure to define the size of the search window as large as the entire image display (see Figure 78). The X and Y results for the last object in the sequence will be used by the calibration software during calibration.

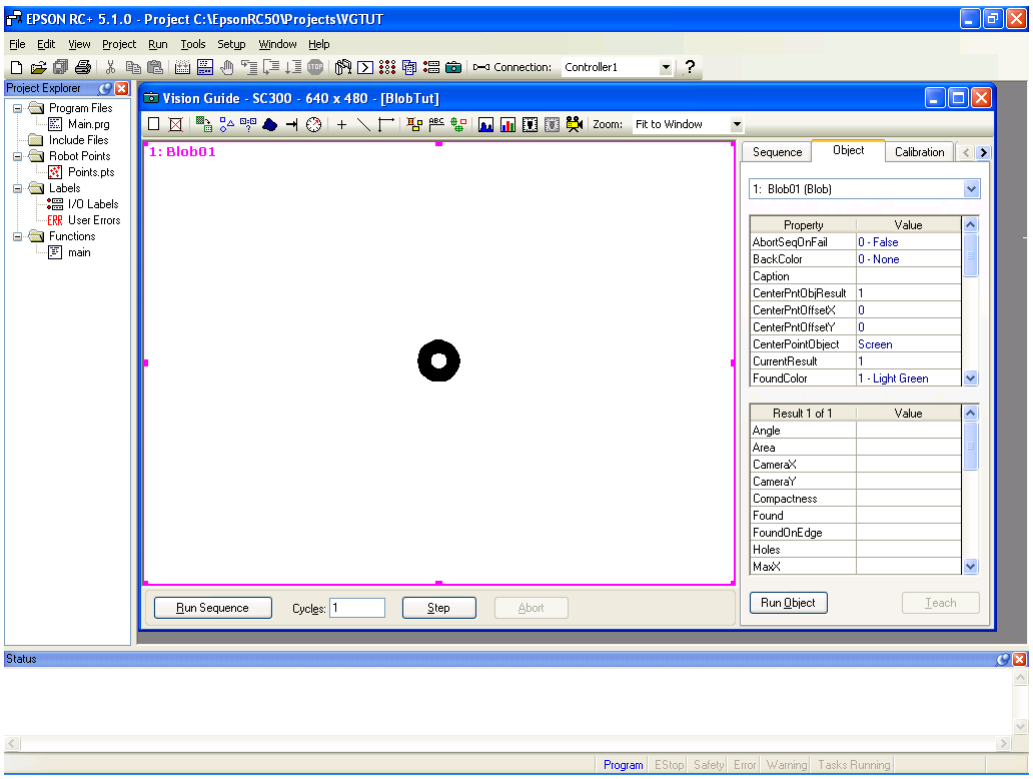


Figure 78: Vision Object search window sized to cover entire image display

8.5.2 Fixed Downward and Standalone Calibration Vision sequences

The 'Fixed Downward' and 'Standalone' calibration vision sequences must locate nine calibration targets. There are two methods to do this.

1. Create an object for each target, for a total of nine objects.
2. Use one of the multi-result objects (Blob or Correlation objects) to return nine results. (See the section titled "Working with Multiple Results from a Single Object" in the chapter *Vision Objects* of this manual.)

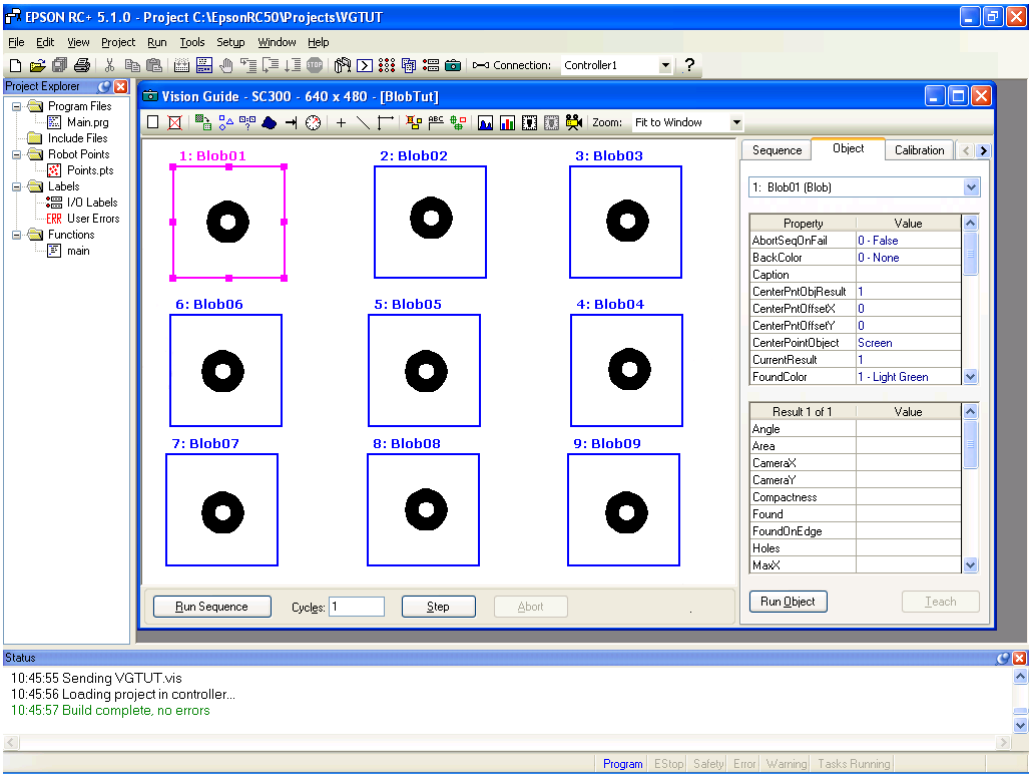


Figure 79: Vision objects placed in order for Standalone Calibration

The objects must be place on the image window in the same order that the targets will be taught. The first object will be in the upper left corner, the second at top center, the third in the upper right corner, etc.

NOTE

For 'Fixed Downward' calibrations, the order for the middle row of objects is in reverse order: 6, 5, 4. This is because moving to the positions in the order 1, 2, 3, 6, 5,4, 7, 8, 9 produces the most efficient motion from the robot and helps make calibration complete faster.

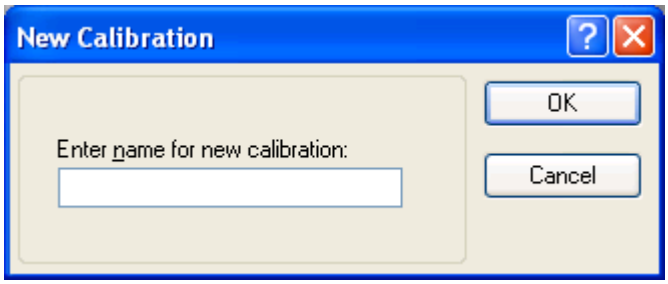
8.6 Calibration GUI

This section describes the GUI used for calibration.

8.6.1 Create a New Calibration

To create a new calibration, click the **New calibration** button  on the Vision Guide toolbar, or select **New Calibration** from the Vision menu.

The following dialog will be displayed:

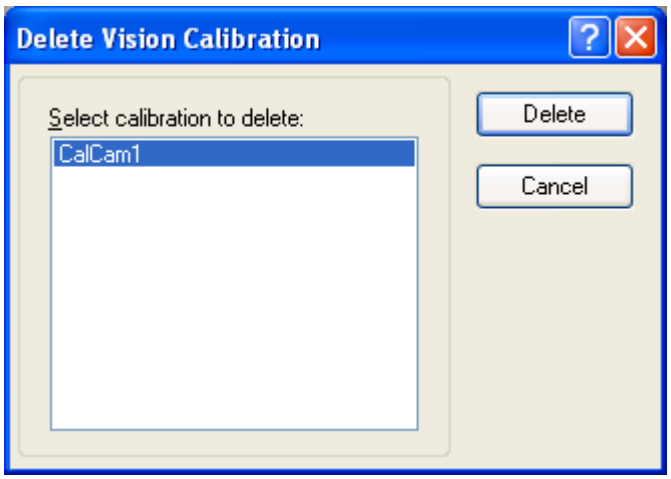


Enter the name for the new calibration and click OK. If the name already exists for another calibration, an error message will be displayed.

8.6.2 Delete a Calibration

To delete a calibration, click the **Delete calibration** button  on the Vision Guide toolbar, or select **Delete Calibration** from the Vision menu.

The following dialog will be displayed:



Select the calibration you want to delete and click the Delete button.

8.6.3 Calibration Properties and Results

After a new calibration has been created, the Vision Guide **Calibration Tab** is used to set calibration properties and view calibration results.

Property	Description
Camera	Specifies the camera for the currently selected calibration.
CameraOrientation	Specifies camera orientation. The choices are as follows: **Mobile J2.** The camera is looking downward and is mounted to joint 2 on SCARA or Cartesian robots. **Mobile J4.** The camera is looking downward and is mounted to joint 4 on SCARA or Cartesian robots. **Mobile J5.** The camera is looking downward and is mounted to joint 5 on 6-Axis robots. **Mobile J6.** The camera is looking downward and is mounted to joint 6 on 6-Axis robots. **Fixed downward.** The camera does not move and reports coordinates in the robot world. **Fixed upward.** The camera does not move and reports coordinates in the robot world. **Standalone.** The camera does not move and does not report coordinates in the robot world.
Lamp	This is an optional output channel that will be turned on automatically when calibration is started.
LampDelay	This is the amount of delay (in seconds) before a lamp is turned on. This allows for time required to turn on fluorescent lamps.
MotionDelay	This is the amount of delay (in milliseconds) after each robot motion during the calibration cycle.
PointsTaught	This property defines whether calibration points have been taught.
ReferenceType	This property specifies the type of reference point used for calibration. There are two types of reference points for mobile camera calibrations: Taught and Upward camera.
RobotArm	This property specifies the arm number to use during calibration. Normally, this is set to zero.
RobotLimZ	This property specifies the LimZ value for the first motion command during a Scara mobile calibration cycle.
RobotLocal	This property specifies the local number to use for the current calibration. The local used must be previously defined.
RobotNumber	This property specifies the robot number to use for the current calibration.
RobotTool	This property specifies the tool number to use during calibration. The tool number used must be previously defined.
RobotSpeed	This property specifies the speed that the robot will move during calibration. For higher speeds, you will need to set higher accelerations.

Property	Description
RobotAccel	This is the acceleration that will be used during calibration.
TargetSequence	This property specifies which vision sequence to use for calibration. This sequence can be any sequence in the current project.
UpwardLamp	This is an optional output channel that will be turned on automatically when the upward camera sequence is run.
UpwardSequence	This property specifies which vision sequence to use for upward looking camera. This list is only active for mobile calibrations that use an upward camera for the reference point.

Result	Description
CalComplete	The completion status of a calibration.
XAvgError	The average calibration error along the camera X axis.
XMaxError	The maximum calibration error along the X axis.
XmmPerPixel	The X millimeters / pixel value.
XTilt	The calibration X tilt result.
YAvgError	The average calibration error along the camera Y axis.
YMaxError	The maximum calibration error along the Y axis.
YmmPerPixel	The Y millimeters / pixel value.
YTilt	The calibration Y tilt result.

8.6.4 Teach Points Button

The Teach Points button opens the Teach Calibration Points dialog so that points can be taught for the currently selected calibration.

8.6.5 Calibrate Button

This button starts the calibration cycle. A confirmation message is displayed before the cycle starts if the robot will be moved.

8.6.6 Teach Calibration Points Dialog

The Teach Calibration Points dialog is used to teach calibration points for the currently selected calibration scheme. You open this dialog by clicking on the teach points button on the Calibration tab.

The Teach Calibration Points dialog is used for the following camera orientations:

- Mobile mounted on robot
- Fixed downward in robot world
- Fixed upward in robot world

Vision Guide uses a nine-point calibration so there are at least nine points required to be taught. In addition, mobile camera calibration requires a reference point if you are not using an upward looking camera to search for the reference. The nine points should span the entire field of view.

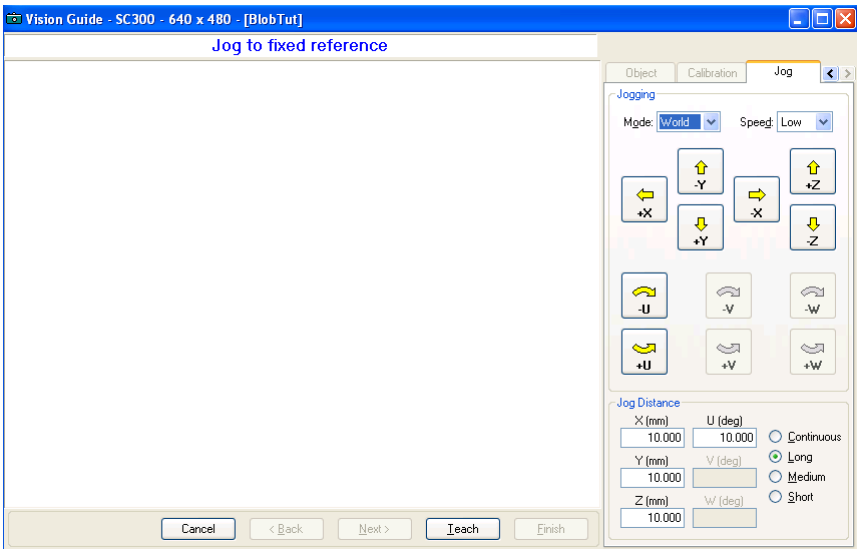


Figure 80: Teach Calibration Points dialog

On the left side of this dialog there is a vision display window. On the right side there are controls for jogging the robot.

At the bottom of the display is a message box that displays instructions. After performing the required instruction, click on the Next button located to the right of the message box to continue to the next step. When all steps have been completed, a message box will appear indicating you are finished.

The instructions displayed depend on the type of calibration you are teaching points for.

Camera Orientation	Points to be Taught
Mobile camera	Teach one reference point if no upward camera is being used. Teach nine camera positions.
Fixed downward	Teach nine reference points in the field of view of the camera.
Fixed upward	Teach nine positions in the field of view of the camera.

Instruction	Description
Jog to fixed reference	This instruction is required for mobile camera calibration that is using a taught point for the reference (as opposed to using an upward looking camera to automatically search for the reference point). Jog the robot until the end effector is lined up with the calibration target.
Jog to fixed reference at 180 degrees	This is an optional step. If you are using a calibration tool that has been accurately defined, you can skip this step by clicking the Next button. If you are not using a calibration tool, then jog the robot 180 degrees from the current position and line up the end effector with the calibration target.
Jog to top left camera position	Jog the robot until the calibration target is in the upper left corner of the vision display area.
Jog to top center camera position	Jog the robot until the calibration target is in the top center of the vision display area.
Jog to top right camera position	Jog the robot until the calibration target is in the upper right corner of the vision display area.
Jog to center right camera position	Jog the robot until the calibration target is in the center of the right side of the vision display area.
Jog to center camera position	Jog the robot until the calibration target is in the center of the vision display area.
Jog to center left camera position	Jog the robot until the calibration target is in the center of the left side of the vision display area.
Jog to bottom left camera position	Jog the robot until the calibration target is in the lower left corner of the vision display area.
Jog to bottom center camera position	Jog the robot until the calibration target is in the bottom center of the vision display area.
Jog to top right camera position	Jog the robot until the calibration target is in the lower right corner of the vision display area.

8.6.7 Calibration Complete dialog

The dialog box in Figure 81 appears after a calibration cycle has been completed. It shows a summary of calibration values for the current calibration and the previous calibration. If this is the first calibration, then the previous calibration values will be blank.

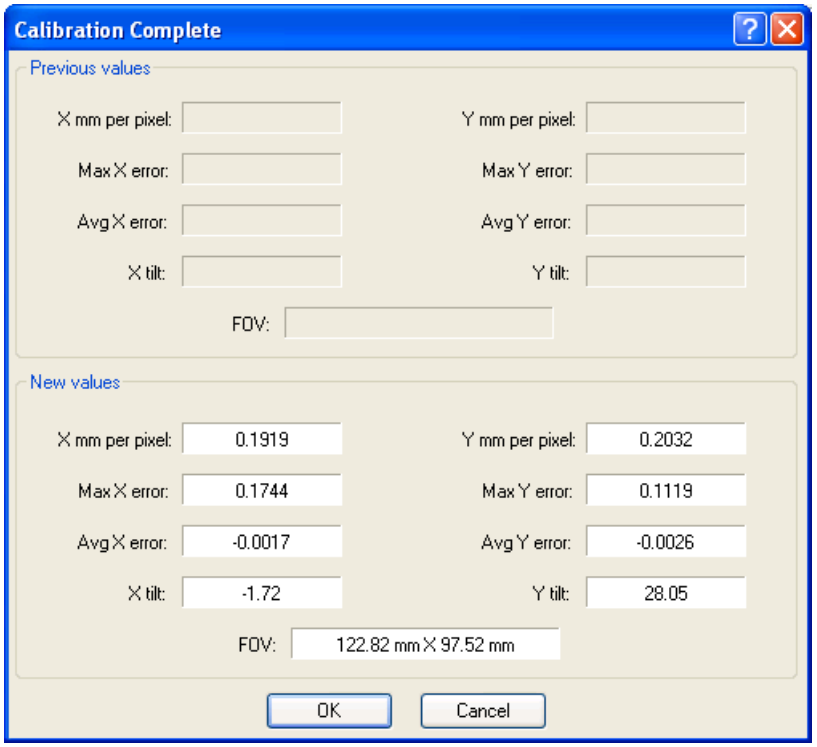


Figure 81: Calibration Status dialog that appears after Calibration is complete

The table below describes the values displayed for the Calibration Complete dialog. After examining the results, click OK to use the new calibration values, or Cancel to abort.

Value	Description
X mm per pixel, Y mm per pixel	This is the resolution of the camera. The average width and height for one pixel.
Max X error, Max Y error	This is the maximum error that occurred during calibration verification. These values should be below the millimeter per pixel values. If they are not, the cause could be that the reference point(s) was not taught properly, or that the vision system is not locating the calibration target consistently, due to improper teaching or lighting.
Avg X error, Avg Y error	This is the average error that occurred during calibration verification.
X tilt, Y tilt	These values are relative indicators of camera tilt. The directions are as viewed from the camera in the image buffer coordinate system (plus x is right and plus y is down). For X tilt, a plus value indicates tilt to the right, minus is tilt to the left. For Y tilt, a plus value indicates tilt down, minus indicates tilt up. Vision Guide compensates for camera tilt. However, it is recommended that you keep the tilt values below 1.0. This will help the vision system to find parts more accurately and repeatably, depending on the application. Sometimes a high tilt value is caused by bad reference points. For example, if you are calibrating a fixed camera, which requires that you type in the coordinates of the calibration targets, inaccurate target location values could cause higher tilt values.
FOV	This is the width and height of the camera's Field of View in millimeters.

8.7 Calibration Procedures

This section contains step by step instructions for calibrating each of the camera orientations.

8.7.1 Calibration Procedure: Mobile

This calibration will allow you to search for objects from a camera mounted on a robot joint and get their coordinates in the robot world.

Step 1: Mount the camera to the robot and connect to the vision board

- Mount the camera to the selected joint of the robot. You can mount the camera at any rotational angle.
- The camera must be approximately aligned with the Z axis of the local coordinate system you will be using.

Step 2: Decide on local coordinate system

By default, the camera is calibrated in local 0, which is the robot base world coordinate system. You can also calibrate the camera in a local coordinate system. The robot coordinates returned from the vision system will be in that local. Generally, you will use locals other than 0 for 6-Axis robots when you want to work in a plane that is not parallel with the robot base plane.

If you want to calibrate in a local coordinate system, you must define the local before you can calibrate the camera. Then set the Calibration RobotLocal property to the number of the local you will be using. See *Local Statement* in the SPEL+ Language Reference manual for details on how to create a local coordinate system.

Step 3: Decide on reference point type

There are two choices for the reference point: a manually taught point, or a point that is found with an upward camera. For greatest accuracy, you should use an upward camera to find the reference target for mobile camera calibration. See the procedure for *Fixed, upward in robot world* later in this chapter.

If you are not using an upward camera to find the reference point, then use one of the following for training the reference point:

- Use a rod mounted to the U axis that extends through the center of the axis.
- Use the tool that is being used to pick up parts if it can be aligned with the calibration reference point.

NOTE

If you define a tool (using TLSet) for the calibration rod of pick up tool, then you will not have to teach zero and 180 degree reference point(s) during calibration.

Step 4: Create vision sequence to find calibration reference target

(1) Create a new vision sequence.

(2) Add one or more objects to locate the calibration target.

The results from the last object in the sequence will be used by the calibration routines to locate the part. This means that if you have multiple vision objects in the vision sequence to be used for Calibration, make sure that the last object in the vision sequence is the one with the correct positional results.

Make sure that the search window for the vision object to be used is set to cover the full screen. This will ensure that the vision object is found as long as the Camera is positioned properly.

Step 5: Create a calibration scheme

(1) Click the New Calibration  button on the Vision Guide tool bar or select New Calibration from the Vision menu. Enter the name for the new scheme.

(2) Set the CameraOrientation property.

(3) Set the ReferenceType property.

(4) Set the TargetSequence property.

(5) Set the UpwardSequence if ReferenceType is Upward Camera.

(6) Set the Lamp property, if required.

(7) Set the RobotArm, RobotAccel, RobotLocal, RobotNumber, RobotSpeed, and RobotTool properties

Step 6: Teach points

NOTE

When using 6-axis robots, the camera must be aligned with the Z axis of the local coordinate system. Normally, the V coordinate for camera view points will be 0 and the W coordinate will be 0 or 180, depending on the orientation of the local. It is recommended that you align the camera to the local before teaching points. You normally should not need to change V and W when teaching the calibration points. This keeps the camera in the same alignment relative to the local for each calibration view position.

(1) On Vision Guide Calibration tab, click the Teach Points button. This will open the Teach Calibration Points dialog.

(2) Follow the instructions displayed in the message box at the bottom of the dialog.

(3) If you are using an upward camera to find the reference point, then skip the next step.

(4) Teach the reference point. This means the robot must be positioned properly such that the tool being used to get the part is positioned directly over the part. (Just like if you were teaching this position for the robot to move to.)

(5) Teach the camera points. There are nine camera positions to be taught. These represent 9 positions of the robot such that the 1st position is seen in the upper left corner of the image display, the 2nd position in the upper center of the image display and so on. For best results, these points should span the field of view of the camera.

Step 7: Calibrate

(1) On Vision Guide Calibration tab, click the Calibrate button to start the calibration cycle.

(2) The robot will move to each camera position and execute the calibration target vision sequence. After moving to all nine positions, the cycle is repeated to verify calibration parameters.

8.7.2 Calibration Procedure: Fixed Downward

This calibration will allow you to search for objects from a fixed camera looking down into the robot work envelope and get their positions in the robot world.

Step 1: Mount the camera

(1) Mount the camera so that it will be looking down into the robot envelope. Make sure that the robot will clear the camera.

Step 2: Make a calibration plate

(1) Make a plate with nine holes or targets that span the field of view of the camera.

Step 3: Create vision sequence to find calibration reference targets

(1) Create a sequence to locate the calibration targets. You can create one object for each target, or you can create one object to find all nine targets, provided that the object can return multiple results (Blob, and Correlation Recognition).

(2) After creating the sequence, test that it finds all nine points.

Step 4: Create a calibration scheme

(1) Click the New Calibration  button on the Vision Guide tool bar or select New Calibration from the Vision menu. Enter the name for the new scheme.

(2) Set the TargetSequence property.

(3) Set the Lamp property, if required.

(4) Set the RobotArm, RobotLocal, RobotNumber, and RobotTool properties

Step 5: Teach points

(1) On Vision Guide Calibration tab, click the Teach Points button. This will open the Teach Calibration Points dialog.

(2) Follow the instructions in the message box near the bottom of the dialog and teach each reference point. You will be prompted to teach the point and then turn the U axis 180 degrees to teach the point again. If you are using a tool, you can skip turning the U axis 180 degrees. Click on the Next button to move to the next step.

Step 6: Calibrate

(1) On Vision Guide Calibration tab, click the Calibrate button to start the calibration cycle. The calibration software will locate the nine targets and then locate them again to verify calibration.

8.7.3 Calibration procedure: Fixed upward

This calibration will allow you to search for objects from a fixed camera looking up and get their positions in the robot world.

Step 1: Mount the camera

(1) Mount the camera so that it will be looking up into the robot envelope.

Step 2: Create vision sequence to find calibration end effector target.

(1) Create a sequence to locate the target on the end effector. Create one or more objects in the sequence to locate the target. The calibration software will use the last step in the sequence to get the location of the target. The last step results for X and Y should be the center of the target.

(2) During calibration, the end effector will be move to nine different points in the field of view of the camera and search for the target. Also, at each position, the calibration software will turn the U axis 180 degrees and search for the target again. This allows the software to determine the center of the U axis for each point. For best results, use a round target.

Step 3: Create a calibration scheme

(1) Click the New Calibration  button on the Vision Guide tool bar or select New Calibration from the Vision menu. Enter the name for the new scheme.

(2) Set the TargetSequence property.

(3) Set the Lamp property, if required.

(4) Set the RobotArm, RobotAccel, RobotLocal, RobotNumber, RobotSpeed, and RobotTool properties. For small fields of view, or if the calibration target is not near the center of the U axis, you must define a tool. This is so that when the calibration software turns the U axis 180 degrees, the target will be near the same position.

Step 4: Teach points

(1) On Vision Guide Calibration tab, click the Teach Points button. This will open the Teach Calibration Points dialog.

(2) Follow the instructions in the message box near the bottom of the dialog and teach each reference point. You will be prompted to teach the point and then turn the U axis 180 degrees to teach the point again. If you are using a tool, you can skip turning the U axis 180 degrees. Click on the Next button to skip to the next step.

Step 5: Calibrate

(1) On Vision Guide Calibration tab, click the Calibrate button to start the calibration cycle. The calibration software will move the robot to each camera position, search for the target, then turn the U axis 180 degrees and search for the target again.

8.7.4 Calibration Procedure: Standalone

This calibration will allow you make physical measurements. Any camera calibrated as a 'Standalone' camera cannot be used to calculate Robot world coordinates. The standalone camera returns CameraX and CameraY values in millimeters.

Step 1: Mount the camera

(1) The camera may be mounted in any orientation as long as the calibration plate is placed perpendicular to the Camera.

Step 2: Make a calibration plate

(1) Make a plate with nine holes or targets that span the field of view of the camera.

Step 3: Create vision sequence to find calibration reference targets

(1) Create a sequence to locate the calibration targets. You can create one object for each target, or you can create one object to find all nine targets, provided that the object can return multiple results (Blob and Correlation Recognition).

(2) After creating the sequence, test that is finds all nine points.

Step 4: Create a calibration scheme

(1) Click the New Calibration  button on the Vision Guide tool bar or select New Calibration from the Vision menu.

(2) Set the TargetSequence property.

(3) Set the Lamp property, if required.

Step 5: Calibrate

(1) On Vision Guide Calibration tab, click the Calibrate button to start the calibration cycle. The calibration software will locate the nine targets and then locate them again to verify calibration.

9. Using Vision Guide in SPEL+

9.1 Overview

Vision Guide was designed to keep vision programming at a minimum, yet allow the application programmer to manipulate vision sequences, objects, and calibrations from SPEL+ programs. In most applications, the vision sequences created with the point and click interface do most of the work. It is a simple matter to run the sequences from SPEL+ and use the results to guide the robot, inspect parts, etc. With this type of design, an applications engineer can create sequences without doing any programming before actually designing the application. This can aid in verifying field of view, accuracy, etc. before quoting a job. Then, at design time, he/she can use the sequences that were created in the actual project.

Vision Guide is supported by commands in the SPEL+ language. After creating vision sequences with the point and click interface, you can run these sequences and get the results from them in your SPEL+ programs. You can also read and set properties for vision sequences and objects.

This chapter contains instructions on how to use Vision Guide commands in SPEL+ programs.

9.2 Vision Guide SPEL+ Commands

Here is a summary of the vision commands available for use in SPEL+ programs.

Basic Vision Commands

VRun	Runs a vision sequence in the current project.
VGet	Gets a result or property for a sequence or object and stores it in one or more variables.
VSet	Sets a property value for a sequence or object.

Before programming with vision commands

Before you begin programming with vision commands, you need to create vision sequences in a EPSON RC+ project. The vision commands you use in your SPEL+ programs will refer to these sequences and the objects created within them.

9.3 Running Vision Sequences from SPEL+: VRun

You can run any vision sequence in the current project from a SPEL+ program using the VRun command. For example:

```
Function VisionTest

  VRun seq1
Fend
```

The simple program above will run the sequence called "seq1" and display the graphical results on the Run window or Operator window, depending on where the program is started from.

The actions taken by VRun depend on the RuntimeAcquire sequence property. This property determines if a picture is taken or not before running the sequence or if a strobe is being used. The table below shows what happens when VRun executes for the different settings of RuntimeAcquire.

RuntimeAcquire	VRun Actions	Usage
0 - None	Do not take picture. Use previous picture. Run each sequence step.	Allows more than one sequence to run on the same picture. Normally, the first sequence takes the picture with RuntimeAcquire set to Stationary. Then the remaining sequences do not take a picture.
1 - Stationary	Acquire new picture. Run each sequence step.	Default. This is the most common way to use VRun. Take a picture and run the sequence.
2 - Strobed	Wait for strobe input. Acquire new picture. Run each sequence step.	Used with strobe input. System will wait for strobe trigger before taking a picture and running the sequence.

9.4 Accessing Properties and Results in SPEL+: VGet, VSet

The VGet and VSet commands can be used to read and set sequence and object properties in SPEL+ programs. For example, you can change a search window size and location, an accept parameter, camera gain, and maximum area to name a few. Nearly all of the properties and results that can be accessed from the Vision Guide point and click interface can also be accessed from a SPEL+ program. There are also some special properties that can only be accessed from using VSet or VGet since they set or return multiple results. (e.g. SearchWin, RobotXYU, and ModelWin to name a few.)

A common syntax is used for both VGet and VSet. Each command must first refer to the name of a sequence. In addition, the name of a vision object must follow the sequence name in order to access properties and results for vision objects. A period is used to separate the sequence, object, and property or result name. If multiple results are used, specific results are selected by appending the result number in parenthesis after the result Name.

For sequence properties and results, the following syntax is used:

VGet *seqName.propName*, *var* *'put property value in variable*

VSet *seqName.propName*, *value* *'set property to value*

For object properties and results, the following syntax is used:

VGet *seqName.objName.resultName*, *var*

VGet *seqName.objName.propertyName*, *var*

VSet *seqName.objName.propertyName*, *value*

For object multiple results, the following syntax is used:

VGet *seqName.objName.resultName(resultnum)*, *var*

The sequence and object names can also be string variables. See *Using Variables for Sequence and Object Names*.

9.4.1 Using VGet

VGet retrieves a property or result and puts it in a SPEL+ variable. You must supply a variable in your program of the proper data type to receive the value from VGet.

Here is an example of using VGet in a SPEL+ program.

```
Function Inspect
  ' Run the vision sequence
  VRun InspectPart
  Integer i, numberFound
  Real area
  VGet inspPart.Part1.NumberFound, numberFound
  For i = 1 to numberFound
    ' Loop through each item that was found
    ' Get the area of the blob result
    VGet inspPart.Part1(i).Area, area
    Print "Area of result ", i, " is ", area
  Next i
Fend
```

9.4.2 Using VSet

VSet sets a property value at runtime. This allows developers to dynamically adjust property settings from a SPEL+ program. In most cases you can set property values from the Vision Guide window and then run vision sequences from SPEL+ programs with no modifications of the properties. However, for those cases that require dynamic adjustments, the VSet SPEL+ command can be used.

Here is an example of using VSet in a SPEL+ program. Notice that the first call to VSet sets a sequence property. The second call to VSet sets an object property called SearchWin that is used to re-define the position and size of a search window before running the sequence.

```
Function findPart

  ' Set camera gain for sequence "findPart"
  VSet findPart.CameraGain, 100

  ' Set search window for object "part"
  VSet findPart.part.SearchWin, 100, 100, 50, 50

  ' Run the sequenced
  VRun findPart
Fend
```

9.5 Using Variables for Sequence and Object Names

String variables can be used for the calibration, sequence, and object name arguments in the VRun, VGet and VSet commands. The example below uses the string variables seq$ and obj$ to specify which vision sequence and which vision object to work with.

```
Function visTest

  #define Z_PICK -100
  String  seq$, obj$
  Boolean found
  Real    x, y, u

  seq$ = "test"
  obj$ = "blob01"
  VSet seq$.Camera, 1
  VSet seq$.Calibration, "CAMCAL1"
  VRun seq$
  VGet seq$.obj$.RobotXYU, found, x, y, u
  If found Then
   pick = XY(x, y, Z_PICK, u)
    Jump pick
    On vacuum
    Wait .1
    Jump place
    Off vacuum
    Wait .1
  EndIf
  Jump park
Fend
```

Arrays can also be used for sequence and object name arguments in the VRun, VGet, and VSet commands. See the example below.

```
Function test
  String obj$(10)
  Integer count

  obj$(0) = "corr01"
  obj$(1) = "corr02"
  obj$(2) = "blob01"
  obj$(3) = "blob02"

  For count = 0 to 3
    VRun seqname
    VGet seqname.obj$(count).Found, found
  Next count
Fend
```

9.6 Using Sequence Results in SPEL+

After running a sequence with the VRun command, there are several results that can be used in your SPEL+ program. The VGet command is used to read results and properties. One of the most common results to use for robot guidance is the RobotXYU result. For example:

```
Function getPart
  #define Z_PICK -100
  Boolean found
  Real x, y, u

  VRun findPart
  VGet findPart.corr01.RobotXYU, found, x, y, u
  If found Then
    pick = XY(x, y, Z_PICK, 0)
    Jump pick
  EndIf
Fend
```

NOTE

Notice in the example above that the found status is checked before moving the robot to the position found by the vision system. It is important to verify that a vision object was found before using its results to move the robot.

9.7 Accessing Multiple Results from the SPEL+ Language

Vision Guide objects such as the Correlation and Blob objects have the ability to find multiple features with one object by using the NumToFind property. The CurrentResult property is used to set which result will be displayed in the Results list if we examine the results in the Object tab. It is also used to determine which result record to return results for. For example, if we want to get the Area result from the 3rd result returned from a Blob object, CurrentResult must be set to 3. You have already seen how this can be done from the Object tab Properties list. Now let's take a look at how to access multiple results from SPEL+.

Multiple result access from SPEL+ treats the result as an array where the result number is the subscript number next to the result to be obtained. The first example below shows how to get the 3rd Area result and put it in a variable called area from the SPEL+ Language.

```
VGet seqname.objname.Area(3), area
```

The 2nd example below shows how to get that same 3rd Area result but this time assign it as the value of the 3rd element in an array called *area*.

```
VGet seqname.objname.Area(3), area(3)
```

Variable names can also be used to represent the element in an array rather than fixed elements as in example #2 above. Notice that the variable called var is used as the subscript for the Area result.

```
VGet seqname.objname.Area(var), area(var)
```

The 4th example assumes that you have used a single vision object to find multiple like parts (lets say up to 10). You would now like to pick these parts (let's say they are pens) with a robot so you will need to store the X, Y, and U coordinates in array variables for the coordinate values of each of the parts found. The code below retrieves these coordinates

using the RobotXYU result and puts them into X, Y and U arrays that could later be used for the robot to move to.

```
Function test
  Boolean found(10)
  Integer numFound, i
  Real x(10), y(10), u(10)

  Jump camshot    'move camera into position snap shot

  VRun seq01      'run the vision sequence to find the pens

  VGet seq01.blob01.NumberFound, numFound  'how many found

  For i = 1 to numFound'get robot coords
    VGet seq01.blob01.RobotXYU(i), found(i), x(i), y(i), z(i)
  Next i

  ' Add code for robot motion here……
Fend
```

9.8 Using Vision Commands with Multitasking

You can execute vision sequences and retrieve results in more than one task in SPEL+. SPEL+ automatically handles executing one vision command at a time internally. When multiple tasks require the vision system, the commands are executed on a first come first serve basis. While one task is using the vision system, the other tasks will block (wait) for the first task to finish the current vision command.

In some cases, you may be running the same sequence from different tasks using different cameras. You will want to run the sequence and get the results in one task while other tasks wait. Use the SyncLock and SyncUnlock commands to lock the vision system for one task, then release it for others.

```
Function VisionTask(camera As Integer)
  Boolean found
  Do
    SyncLock 1 ' Lock vision just for this task
    VSet FindParts.Camera, camera
    VRun FindParts
    VGet FindParts.AllFound, found
    SyncUnlock 1 ' Unlock vision for other tasks
  Loop
Fend
```

9.9 Using Vision with the Robot

Before you can use results from a vision sequence to guide the robot, you need to calibrate the camera that is used by the sequence. See the chapter Calibration for details.

If you attempt to get a result from a sequence object that requires calibration and there is no calibration, then a run time error will occur.

9.9.1 Position results

All position results reported by objects in a vision sequence that uses a robot-calibrated camera are in the specified local coordinate system. Unlike other robot/vision systems, there is no extra step to do a pixel coordinate to robot coordinate transformation function call. This is completely handled by Vision Guide. The X, Y, and U coordinates of the part in the robot local coordinate system can easily be retrieved.

To accurately position an end effector at a position determined by the vision system, you need to define a tool for the end effector. See *Defining a TOOL* in the next section.

The following results can be used for guiding the robot:

RobotXYU Returns x, y, and u along with found status.

RobotX Returns the X coordinate.

RobotY Returns the Y coordinate.

RobotU Returns the U coordinate.

NOTE

The U coordinate zero degree position is along the Y-axis pointing straight out from the robot base.

9.9.2 Defining a Tool

It is important to define tools for the robot end effectors. This tells the robot where the end effector is, so that all position information is with respect to the end effector, not the TOOL 0 position. Use the TLSET command to define a tool in SPEL+. The following are three methods for defining tool offsets.

Define a tool with the Robot Manager Tool Wizard

You can use the Tool Wizard in the Robot Manager to define a tool.

To use the Tool Wizard, perform the following steps:

(1) Open the Robot Manager.

(2) Click the Tools tab.

(3) Click the Tool Wizard button.

Follow the steps in the wizard to create the tool.

Using an upward looking camera to calculate tool offsets

Here is an example of how to use an upward looking camera to calculate tool offsets. The function first runs a sequence to locate the tip of the tool. Then the tool offsets are calculated.

```
Function CalcTool

  Boolean found
  Real x, y, u
  Real x0, y0, u0
  Real xTool, yTool, rTool, theta

  Tool 0
  VRun findTip
  VGet findTip.tip.RobotXYU, found, x, y, u
  If found Then

    ' Get the TOOL 0 position
    x0 = CX(P*)
    y0 = CY(P*)
    u0 = CU(P*)

    ' Calculate initial tool offsets
    ' X and Y distance from tip found with vision
    ' to center of U axis
    xTool = x - x0
    yTool = y - y0

    ' Calculate angle at initial offsets
    theta = Atan2(xTool, yTool)

    ' Calculate angle of tool when U is at zero degrees
    theta = theta - DegToRad(u0)

    ' Calculate tool radius
    rTool = Sqr(xTool * xTool + yTool * yTool)

    ' Calculate final tool offsets
    xTool = Cos(theta) * rTool
    yTool = Sin(theta) * rTool

    ' Set the tool
    TLSet 1, XY(xTool, yTool, 0, 0)
  EndIf
Fend
```

Manually calculating tool offsets

NOTE

In the following steps, you must Jog the robot with the Jog buttons from the Jog & Teach window. You cannot free the axes (using SFREE), and move the robot manually when calculating the tool offsets as shown below.

Perform the following steps to calculate tool offsets.

(1) Position the U axis at zero degrees.

(2) Set tool to zero (TOOL 0).

(3) Jog the end effector over a reference point and align it carefully. Do not change the U axis setting.

(4) Write down the current X and Y coordinates as X1 and Y1.

(5) Jog the U axis to the 180-degree position.

(6) Jog the end effector over the reference point and align it carefully. Do not change the U axis setting.

(7) Write down the current X and Y coordinates as X2 and Y2.

(8) Use the following formula to calculate tool offsets:

```
xTool = (X2 - X1) / 2
yTool = (Y2 - Y1) / 2
```

(9) Define the tool from the Tools page on the Robot Manager or from the Command window.

```
TLSET 1, XY(xTool, yTool, 0, 0)
```

(10) Test the tool settings:

Set the current tool to the tool you defined in the previous step. For example, TOOL 1.
Jog the end effector over the reference point.
Now jog the U axis.
The end effector should remain over the reference point.

9.9.3 Calculating a tool for circuit board in gripper

In this example, Vision Guide is used to calculate a tool for a circuit board that is being held by the robot to be placed. This requires one upward looking camera. You will need to teach the place position one time after calibrating the camera.

To teach the place position:

(1) Pick up the circuit board with the robot.

(2) Call the example function CalcBoardTool to calculate tool 1.

(3) Switch to Tool 1.

(4) Jog board into position.

(5) Teach the place position.

```
Function CalcBoardTool As Boolean

  Boolean found
  Real fidX, fidY, fidU, r
  Real robX, robY, robU
  Real x, y, theta
  Real toolX1, toolY1, toolU
  Real toolX2, toolY2

  CalcBoardTool = FALSE
  Jump Fid1CamPos  ' Locate fiducial 1 over camera
  robX = CX(Fid1CamPos)
  robY = CY(Fid1CamPos)
  robU = CU(Fid1CamPos)
  VRun SearchFid1
  VGet SearchFid1.Corr01.RobotXYU, found, fidX, fidY, fidU
  If found Then
    x = fidX - robX
    y = fidY - robY
    theta = Atan2(x, y) - DegToRad(robU)
    r = Sqr(x ** 2 + y ** 2)
    toolX1 = Cos(theta) * r
    toolY1 = Sin(theta) * r
  Else    'target not found
    Exit Function
  EndIf

  Jump Fid2CamPos  ' Locate fiducial 2 over camera
  robX = CX(Fid2CamPos)
  robY = CY(Fid2CamPos)
  robU = CU(Fid2CamPos)
  VRun SearchFid2
  VGet SearchFid2.Corr01.RobotXYU, found, fidX, fidY, fidU
  If found Then
    x = fidX - robX
    y = fidY - robY
    theta = Atan2(x, y) - DegToRad(robU)
    r = Sqr(x ** 2 + y ** 2)
    toolX2 = Cos(theta) * r
    toolY2 = Sin(theta) * r
  Else    'target not found
    Exit Function
  EndIf
  x = (toolX1 + toolX2) / 2
  y = (toolY1 + toolY2) / 2
  theta = Atan2(toolX1 - toolX2, toolY1 - toolY2)
  toolU = RadToDeg(theta)
  TlSet XY(1, x, y, 0, toolU)
  CalcBoardTool = TRUE
Fend
```

9.9.4 Positioning a camera for a pallet search

When using a J2 or J5 mount camera for pallet search, the camera imaging position is hard to control. However, the camera moves easily to each imaging position on the pallet by defining an Arm for the camera using the Arm command. For details on the Arm command, refer to *EPSON RC+ 5.0 SPEL+ language Reference*.

10. Using the Compact Vision Monitor

The EPSON Compact Vision System supports local monitoring of video and graphics, and basic configuration. To use the monitor, you will need to connect a display monitor. To configure the monitor software, you must temporarily connect a USB mouse and/or keyboard.

10.1 Connecting Monitor, Mouse, Keyboard

To use the monitor feature, you will need to connect a display monitor. To configure the monitor settings or network settings, you will need a mouse and/or keyboard.

10.1.1 Connecting a Display Monitor

Connect a monitor that supports 1024 × 768 or greater resolution to the VGA receptacle on the front panel of the Compact Vision system. The video is output at 1024 × 768 resolution.

10.1.2 Connecting a Mouse and Keyboard

A USB mouse is required to make configuration changes. A USB keyboard can also be connected, but it is normally not needed.

Connect the mouse and/or keyboard to the USB receptacles on the front panel of the Compact Vision system.

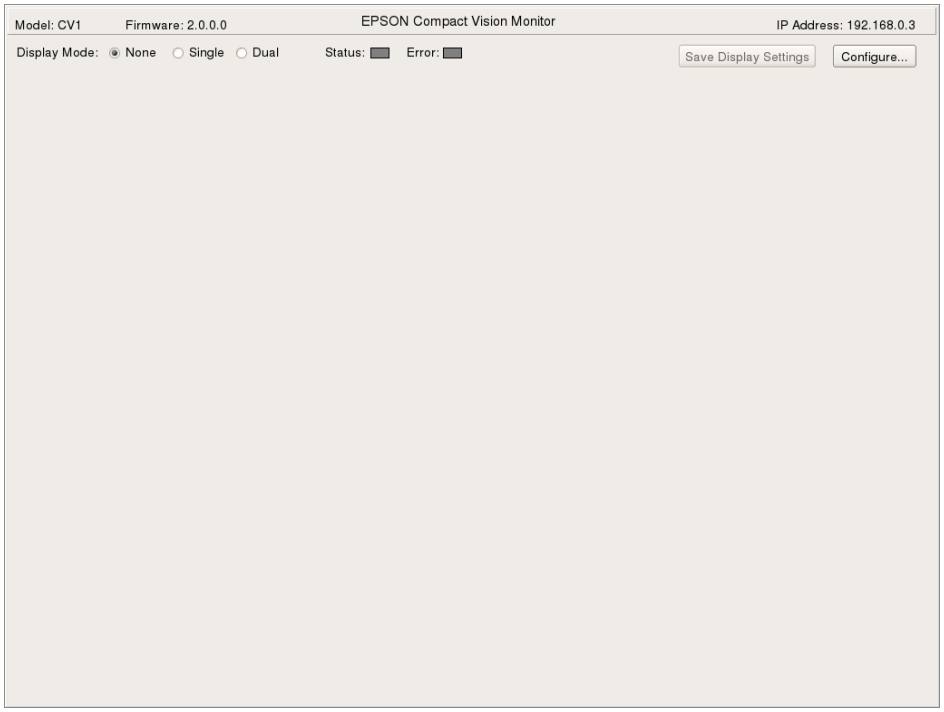
10.2 Monitor Main Screen

This section describes the main screen of the monitor. It includes the following topics:

- Main Screen Layout
- Setting the Display Mode
- Display Settings
- Displaying Video

10.2.1 Main Screen Layout

The main screen is displayed after the system is started and fills the entire monitor display area. The title bar displays the model, firmware version, and IP address. By default, the Display Mode is None.



The states of the STATUS and ERROR LEDs located on the front panel are shown under the title bar.

10.2.2 Setting the Display Mode

You can choose from three display modes:

None : No video is displayed. This is the default setting.

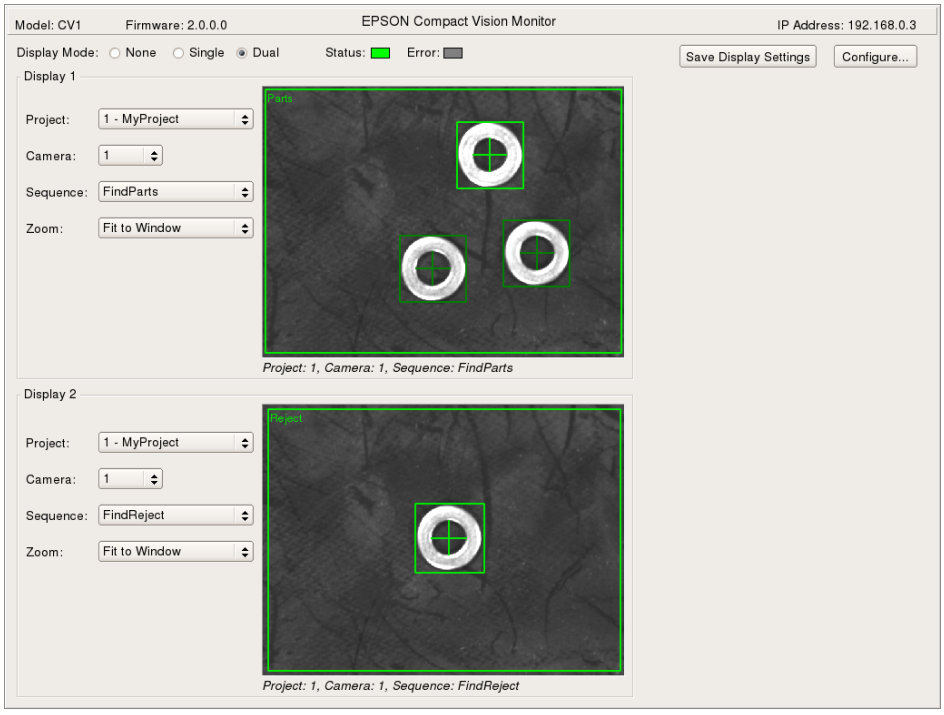
Single : One video display is enabled.

Dual : Two video displays are enabled.

Each video display has settings for Project, Camera, Sequence, and Zoom.

When the Display Mode is Single, the Display 1 video display area is shown.

When the Display Mode is Dual, the Display 1 and Display 2 video display areas are shown.



Under each video display, a caption is shown that includes the project number, camera number, and sequence. When Project is set to None, only the camera is shown in the display captions.

10.2.3 Display Settings

Each display area has the following settings:

Project This is a dropdown list of projects in the system. In addition to each project, you can also select None. When None is selected, you can see live video from the selected camera.

Camera This is a dropdown list of project cameras. Note that the camera numbers are the ones used in the RC+ vision configuration. For example, if the Channel 1 camera is camera 3 in the RC+ vision configuration, then you will see camera 3 in this list. If the Project is set to None, then cameras 1 and 2 are shown in the list, corresponding to channels 1 and 2.

If more than one camera is used in a project, then the choices in the list are Any, Camera<1>, Camera<2>. Only sequences using the current camera will be displayed.

Sequence This is a dropdown list of the sequences in the selected project. If there is more than one sequence, then the Any choice is also added. When Any is selected, then the video and graphics for any sequence in the specified project using the specified camera are displayed.

Zoom This dropdown list has selections for the zoom value. If the selection is not Fit to Window, then scroll bars will appear to allow horizontal and vertical panning.

After making changes, you can click the <SaveDisplay Settings> button to make the changes permanent. If the system is restarted, the settings will be restored to the saved settings.

10.2.4 Displaying Video

When RC+ or a robot controller is connected to the camera, and a project is selected on the monitor main screen, then the video is updated only when a sequence is run from RC+ or the controller. In this case, the video is not live on the monitor unless it is live in RC+. If you want to see live video, select None for the Project, then select the desired camera.

Note that if you show live video on the monitor (with Project set to None) and a client (RC+ and/or a controller) is also using the system, then vision processing will be slower, because the video is being grabbed for both the monitor and the client. Setting Project to None is intended to be used for checking a camera from the monitor without connecting RC+.

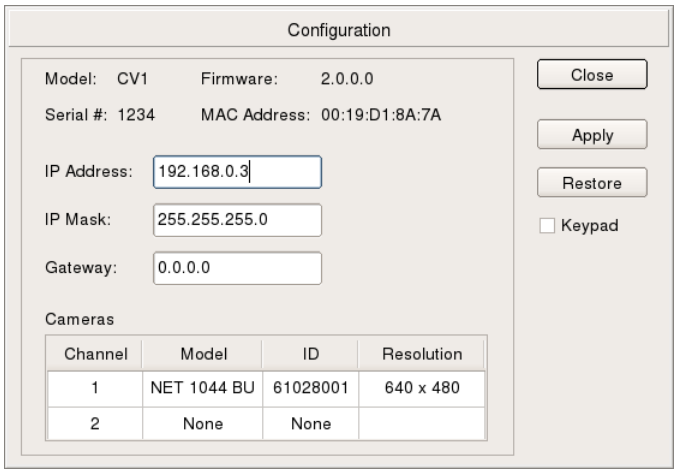
For best performance, select one or two projects that you want to monitor and save the settings. If you will not be using the monitor display, set the display mode to None, then save the settings.

The sequence graphics are displayed in the same way as in RC+. For example, if you run a sequence in the Vision Guide window in RC+, the same graphics will be displayed on the monitor if the associated project, camera, and sequence are selected.

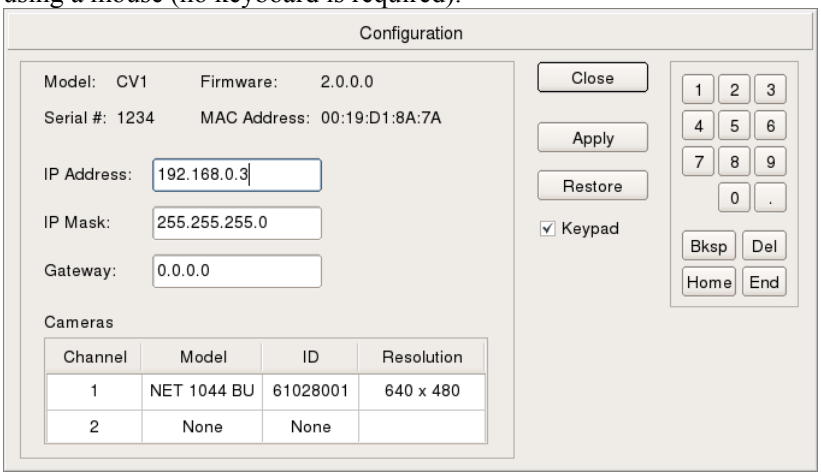
10.3 Configuration Dialog

The [Configuration] dialog displays the current system settings, include model, firmware version, serial number, MAC address, and detected cameras. You can also change the network configuration from this dialog.

To open the [Configuration] dialog, click the <Configure…> button on the main screen. The dialog shown below appears.



Item	Description
IP Address	This is the static IP address for the system.
IP Mask	This is the network mask to specify the subnet.
Gateway	Optional. This is the address of the router for the subnet.
Keypad	Click this checkbox to display a keypad. This allows you to change settings using a mouse (no keyboard is required).



Cameras	This table shows the cameras detected by the system. You cannot change this information.
Apply	Click this button to save changes.
Restore	Click this button to restore changes.
Close	Click this button to close the dialog.

11. Maintenance Parts List

11.1 Maintenance Parts List for Compact Vision

Part Name		Code	Notes
Compact Vision CV1		R12B120342	
Standard USB Camera		R12B120359	
1.3 million pixels USB Camera		R12B120360	
Standard USB Camera Cable (5 m)		R12B020226	
High Flex USB Camera Cable (5 m)		R12B020227	
Standard Trigger Cable (5 m)		R12B020224	
High Flex Trigger Cable (5 m)		R12B020225	
Camera Lens 8 mm		R12R500VIS018	
Camera Lens 12 mm		R12B120361	
Camera Lens 16 mm		R12R500VIS019	
Camera Lens 25 mm		R12R500VIS020	
Camera Lens 50 mm		R12R500VIS021	
Camera Extension Tube Kit		R12R500VIS022	
Camera Mounting Bracket	G3 / LS3 series	R12B031913	
	G6 / LS6 series	R12B031907	
	G10 / G20 series	R12B031908	
	C3 series	R12B031922	
	RS series	R12B021929	
	PS3 series	R12B031904	
	X4 series	R12B031905	
Ethernet Switch		R12B120201	
Ethernet Cable 5 m		R12B020210	
Fan Filter		R13B060512	

11.2 Maintenance Parts List for Smart Camera

Part Name		Code	Notes
Vision Guide 5.0 Fixed Camera		R12B120311	SC300 (Fixed Smart Camera)
Vision Guide 5.0 Mobile Camera		R12B120312	SC300M (Mobile Smart Camera)
C-CS Adapter		R13B120313	
Smart Camera DC Cable		R13B020212	
High Flexible Camera Link Cable 5 m		R12B020211	
Camera Lens 8mm		R12R500VIS018	
Camera Lens 12mm		R12B120361	
Camera Lens 16mm		R12R500VIS019	
Camera Lens 25mm		R12R500VIS020	
Camera Lens 50mm		R12R500VIS021	
Camera Extension Tube Kit		R12R500VIS022	
Camera Mounting Bracket	G3 / LS3 series	R12B031913	
	G6 / LS6 series	R12B031907	
	G10 / G20 series	R12B031908	
	C3 series	R12B031922	
	RS series	R12B021929	
	PS3 series	R12B031904	
	X4 series	R12B031905	
Ethernet Switch		R12B120201	
Ethernet Cable 5 m		R12B020210	

Appendix A: End User License Agreement for Compact Vision

1. Definition

In this Agreement, “Open Source License” shall mean any license to software that as a condition of use, copying, modification or redistribution requires that code and any of its derivative works to be disclosed or distributed in source code form, to be licensed for the purpose of making derivative works, or to be redistributed free of charge, including without limitation software distributed under the GNU General Public License or GNU Lesser/Library GPL.

2. Scope

This Agreement is a legal agreement and constitutes the entire understanding between you (an individual or single entity, referred to hereinafter as "you") and EPSON regarding the following Licensed Software embedded in hardware EPSON provides you with “Hardware”.

Licensed Software : Compact Vision System CV1

3. Copyright and Other Rights

As between the parties, EPSON retains any and all ownership rights, title and interest in the Licensed Software including any and all copies thereof. This Agreement does not constitute a sale of the Licensed Software or any portion thereof. The Licensed Software is protected under Japanese Copyright Laws and International Copyright Treaties, as well as other intellectual property laws and treaties. Except for the rights and licenses expressly granted to you hereunder, you shall not be granted, expressly, impliedly or by estoppel, any rights under any intellectual property rights (including patents, trademarks and copyrights) in and to the Licensed Software.

4. License Grants.

Subject to and conditional upon the terms of this Agreement, EPSON hereby grants to you a non-exclusive, non-sublicensable, personal and non-transferable, royalty-free limited license and right as follows:

(1) to use the Licensed Software only on Hardware; and
(2) to use the Licensed Software only for the purpose of manipulation for EPSON's industrial robot.

5. Open Source Software.

The Licensed Software may be subject to the Open Source License. If there is a conflict between the terms and conditions of this Agreement and any terms or conditions applicable to any Open Source License which covers software contained in the Licensed Software, the terms and conditions of such Open Source License shall prevail and govern. You will not do anything that would subject to an Open Source License any portion of the Licensed Software that is not already subject to an Open Source License.

6. Restrictions

Notwithstanding anything to the contrary herein, you shall not: (a) use the Licensed Software for any purpose except those expressly permitted hereunder; (b) copy the Licensed Software in whole or in part; (c) change or modify the Licensed Software except for linking any other libraries to the Licensed Software, in a manner not to conflict with the Section 5. above, only for your own use under an applicable Open Source License; (d) write the Licensed Software into any other software; (e) combine with the Licensed Software and any other software; (f) develop the software using any part of the Licensed Software (g) disassemble, de-compile or otherwise reverse engineer the Licensed Software downloaded in machine readable form in order to discover or otherwise obtain the source code thereof except for reverse engineering the Licensed Software permitted under an applicable Open Source License only for debugging the modifications you made for your own use under the subsection (c) of this section 6; (h) assign, rent, lease, lend and re-distribute the Licensed Software; (i) remove or change any legend or notice of copyright or other intellectual property right set forth in the Licensed Software as installed; or (j) remove or change any legend or notice of any logo, trade name, and trademark set forth in the Licensed Software as installed.

7. Term

This Agreement shall be effective upon completing the installation of the Licensed Software and shall remain in effect until EPSON terminated in accordance with Section 8.

8. Termination

EPSON may terminate this Agreement at any time with immediate effect and without any notice to you if you are in material breach of this Agreement. You may terminate this Agreement at any time by stopping all use of the Licensed Software and disposing of any and all copies thereof. Upon termination of this Agreement, all licenses granted hereunder shall automatically and simultaneously terminate, and you shall stop all use of the Licensed Software. Except as otherwise provided herein, the terms of this Agreement shall survive termination hereof. Termination is not an exclusive remedy, and any and all other remedies shall be available whether or not this Agreement is terminated.

9. Export Control Laws Compliance

You shall comply with all applicable export laws, restrictions, and/or regulations of Japan, the United States or any other applicable foreign agency or authority. You will not export or re-export, or allow the export or re-export of any product, technology or information you obtain or learn of pursuant to this Agreement (or any direct product thereof) in violation of any such laws, restrictions and/or regulations.

10. Warranty Disclaimers

THE LICENSED SOFTWARE IS PROVIDED "AS IS" AND WITHOUT ANY WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PURPOSE, TITLE, QUIET ENJOYMENT AND NON-INFRINGEMENT. EPSON DOES NOT WARRANT THE ACCURACY OR RELIABILITY OF THE USE OF LICENSED SOFTWARE OR THE RESULTS OF THE USE OF THE LICENSED SOFTWARE, AND THE USE OF THE LICENSED SOFTWARE IS AT YOUR OWN RISK. EPSON DOES NOT WARRANT THAT THE LICENSED SOFTWARE WILL MEET YOUR REQUIREMENTS, OPERATE ERROR FREE, WITHOUT INTERRUPTION, OR IN COMBINATION WITH OTHER SOFTWARE, OR THAT ALL PROGRAM DEFECTS ARE CORRECTABLE.

11. No Assignment

Other than expressly provided herein, you shall NOT assign or otherwise transfer any rights and obligations hereunder, or this Agreement, to any third party without the prior written consent of EPSON.

12. Governing Law

This Agreement shall be construed and governed by the laws of Japan, without reference to conflict of law provisions.

13. Invalidity

If any provision of this Agreement is determined to be invalid under applicable law, such provision shall, to that extent, be deemed omitted. The invalidity of any portion of this Agreement shall not render this Agreement or any other portion hereof invalid.

14. Entire Agreement

This Agreement constitutes the entire understanding and agreement between you and EPSON relating to the subject matter hereof, and supersedes any previous communications, representations, or agreements, whether oral or written, between you and EPSON in respect of the subject matter hereof.

15. Consequential Damages Waiver

IN NO EVENT SHALL EPSON BE LIABLE TO YOU OR ANY THIRD PARTY WITH RESPECT TO ANY SUBJECT MATTER OF THIS AGREEMENT WHETHER ARISING UNDER CONTRACT, TORT (INCLUDING NEGLIGENCE), STRICT LIABILITY, BREACH OF WARRANTY, OR MISREPRESENTATION FOR ANY LOSS OF PROFITS, LOSS OF BUSINESS, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, EVEN IF ADVISED OF SUCH DAMAGES. Some jurisdictions do not allow the exclusion of damages, so the above exclusion may not apply to you.

16. Limitation of Liability

EPSON'S AGGREGATE LIABILITY TO YOU OR ANY THIRD PARTY WITH RESPECT TO ANY SUBJECT MATTER OF THIS AGREEMENT WHETHER ARISING UNDER CONTRACT, TORT (INCLUDING NEGLIGENCE), STRICT LIABILITY, BREACH OF WARRANTY, OR MISREPRESENTATION FOR ANY DAMAGES SHALL NOT EXCEED THE AMOUNTS PAID BY YOU TO EPSON TO USE THE LICENSED SOFTWARE HEREUNDER. Some jurisdictions do not allow the limitation of damages, so the above limitation may not apply to you.

17. U.S. Government End Users

If you are acquiring the Licensed Software on behalf of any unit or agency of the United States Government, the following provisions apply. The Government agrees: (i) if the Licensed Software is supplied to the Department of Defense (DoD), the Licensed Software is classified as "Commercial Computer Software" and the Government is acquiring only "restricted rights" in the Licensed Software and its documentation as that term is defined in Clause 252.227-7013(c)(1) of the DFARS; and (ii) if the Licensed Software is supplied to any unit or agency of the United States Government other than DoD, the Government's rights in the Licensed Software and its documentation will be as defined in Clause 52.227-19(c)(2) of the FAR or, in the case of NASA, in Clause 18-52.227-86(d) of the NASA supplement to the FAR.

Appendix B: OPEN SOURCE SOFTWARE LICENSE for Compact Vision

(1) The Compact Vision product includes open source software programs listed in Section 6) according to the license terms of each open source software program.

(2) We provide the source code of the Open Source Programs (each is defined in Section 6) until five (5) years after the discontinuation of same model of this option product. If you desire to receive the source code above, please contact the "SERVICE CENTER" or "MANUFACTURER & SUPPLIER" in the first pages of the manual. You shall comply with the license terms of each open source software program.

(3) The open source software programs are WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. See the license agreements of each open source software program for more details, which are described on \usr\shared\copyrights in the Installer DVD.

(4) OpenSSL toolkit
The Compact Vision product includes software developed by the OpenSSL project for use in the OpenSSL Toolkit (http://www.openssl.org/).
This product includes cryptographic software written by Eric Young (eay@cryptsoft.com).

(5) The license terms of each open source software program are described on \usr\shared\copyrights in the Installer DVD.

(6) The list of open source software programs which the Compact Vision product includes are as follows.

adduser	base-files	base-passwd	bash
bsdmainutils	bsdutils	busybox	console-common
console-data	console-tools	coreutils	cpio
cpp	cpp-4.3	cron	dbus
dbus-x11	debconf	debconf-i18n	debian-archive-keyring
debianutils	defoma	dhcp3-client	dhcp3-common
diff	dmidecode	dpkg	e2fslibs
e2fsprogs	file	findutils	fontconfig
fontconfig-config	gcc-4.2-base	gcc-4.3-base	gnupg
gpgv	grep	groff-base	grub
grub-common	gzip	hostname	ifupdown
initramfs-tools	initscripts	iproute	iptables
iputils-ping	jwm	klibc-utils	libacl1
libattr1	libaudio2	libblkid1	libbz2-1.0
libc6	libc6-i686	libcomerr2	libconsole
libcwidget3	libdb4.6	libdbus-1-3	libdevmapper1.02.1
libdrm2	libedit2	libexpat1	libfakekey0
libfontconfig1	libfontenc1	libfreetype6	libfs6
libgcc1	libgcrypt11	libgdbm3	libgl1-mesa-dri
libgl1-mesa-glx	libglib2.0-0	libglib2.0-data	libglu1-mesa
libgmp3c2	libgnutls26	libgpg-error0	libhal1
libice6	libjpeg62	libkeyutils1	libklibc
libkrb53	liblcms1	liblocale-gettext-perl	liblockfile1
libmagic1	libmng1	libmpfr1ldbl	libncurses5
libncursesw5	libnewt0.52	libpam0g	libpam-modules
libpam-runtime	libpci3	libpcre3	libpixman-1-0
libpng12-0	libpopt0	libqt4-assistant	libqt4-dbus
libqt4-designer	libqt4-gui	libqt4-network	libqt4-opengl
libqt4-qt3support	libqt4-script	libqt4-sql	libqt4-svg
libqt4-xml	libqtcore4	libqtgui4	libreadline5
libsasl2-2	libselinux1	libsepol1	libsigc++-2.0-0c2a

libslang2	libsm6	libss2	libssl0.9.8
libstdc++6	libtasn1-3	libtext-charwidth-perl	libtext-iconv-perl
libtext-wrapi18n-perl	libtiff4	libusb-0.1-4	libuuid1
libvolume-id0	libwrap0	libx11-6	libx11-data
libxapian15	libxau6	libxaw7	libxcb1
libxcb-xlib0	libxcursor1	libxdamage1	libxdmcp6
libxext6	libxfixes3	libxfont1	libxft2
libxi6	libxinerama1	libxkbfile1	libxmu6
libxmuu1	libxpm4	libxrandr2	libxrender1
libxt6	libxtrap6	libxtst6	libxv1
libxxf86dga1	libxxf86misc1	libxxf86vm1	linux-image-2.6.26-2-cvos1.0.0
locales	lockfile-progs	login	logrotate
lsb-base	lzma	makedev	man-db
manpages	matchbox-keyboard	mawk	mktemp
module-init-tools	mount	ncurses-base	ncurses-bin
netbase	netcat-traditional	net-tools	netusbcam
openssh-blacklist	openssh-blacklist-extra	openssh-client	openssh-server
passwd	perl	perl-base	perl-modules
procps	qt4-qtconfig	readline-common	rsyslog
sed	ssh	sysvinit	sysvinit-utils
sysv-rc	tar	tcpd	traceroute
ttf-dejavu	ttf-dejavu-core	ttf-dejavu-extra	tzdata
ucf	udev	update-inetd	usbmount
usbutils	util-linux	wget	whiptail
x11-apps	x11-common	x11-session-utils	x11-utils
x11-xfs-utils	x11-xkb-utils	x11-xserver-utils	xauth
xbitmaps	xfonts-100dpi	xfonts-75dpi	xfonts-base
xfonts-encodings	xfonts-scalable	xfonts-utils	xinit
xkb-data	xorg	xserver-xorg	xserver-xorg-core
xserver-xorg-input-evdev	xserver-xorg-input-kbd	xserver-xorg-input-mouse	xserver-xorg-input-synaptics
xserver-xorg-video-vesa	xterm	zlib1g	

Appendix C: Hardware Specifications

Compact Vision

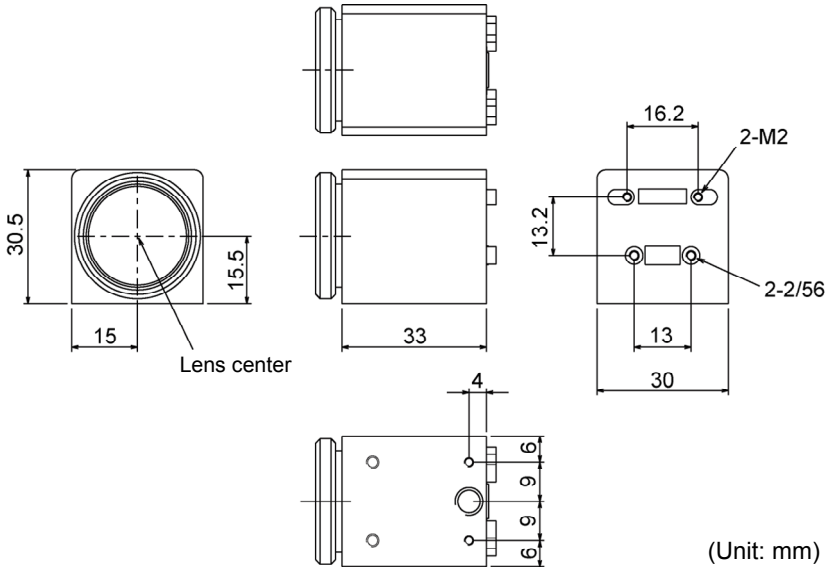
Electrical Specifications

Power Consumption	2A @ 24 VDC or 48 W
USB Camera Strobe Output	Output Voltage: 4V to 24V Output Current: Max. 500 mA
USB Camera Trigger Input	3-24 V

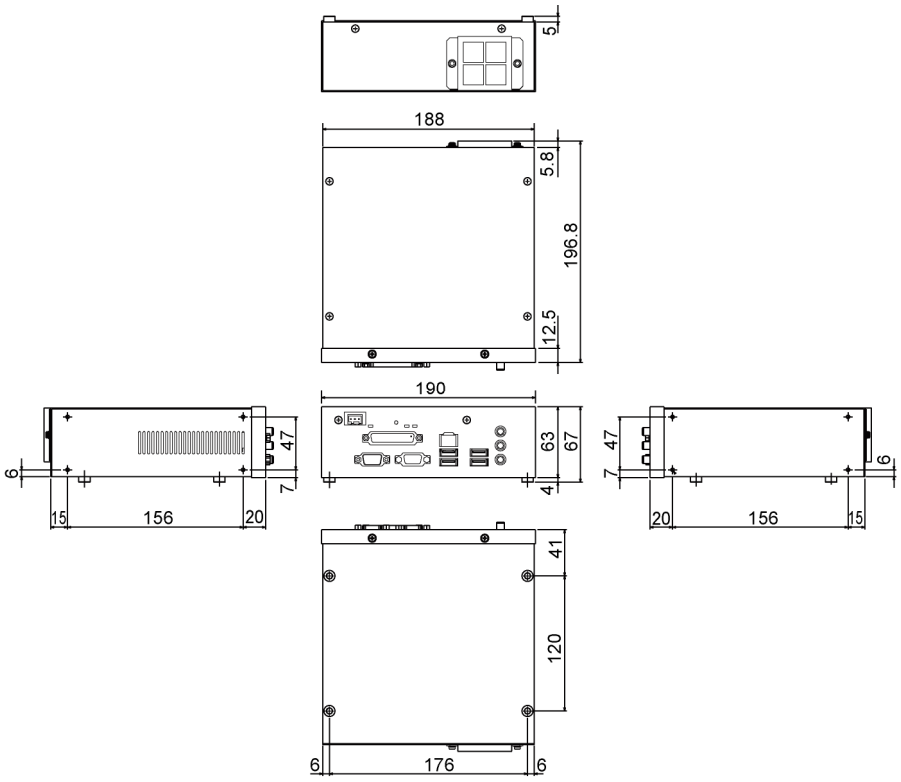
Mechanical Specifications

Connectors	3 pin Power receptacle RJ45 for 10/100/1000 BaseT Ethernet USB connectors: for camera, keyboard, and mouse VGA connector JST BM08B-SRSS-TB 8 pin receptacle for trigger input and strobe ouput
Weight	CV1 Controller Unit: 1.5 kg (52.9 oz) USB Camera Unit: 44 g (1.6 oz)

Camera: NS1044BU / NS4133BU



Compact Vision System: CV1



Environmental Specifications

Operating Temperature	0 °C to 45 °C (32 °F to 113 °F)
Ventilation Requirements	Natural convection
Operating Humidity	20 to 80% (with no condensation)

Smart Camera

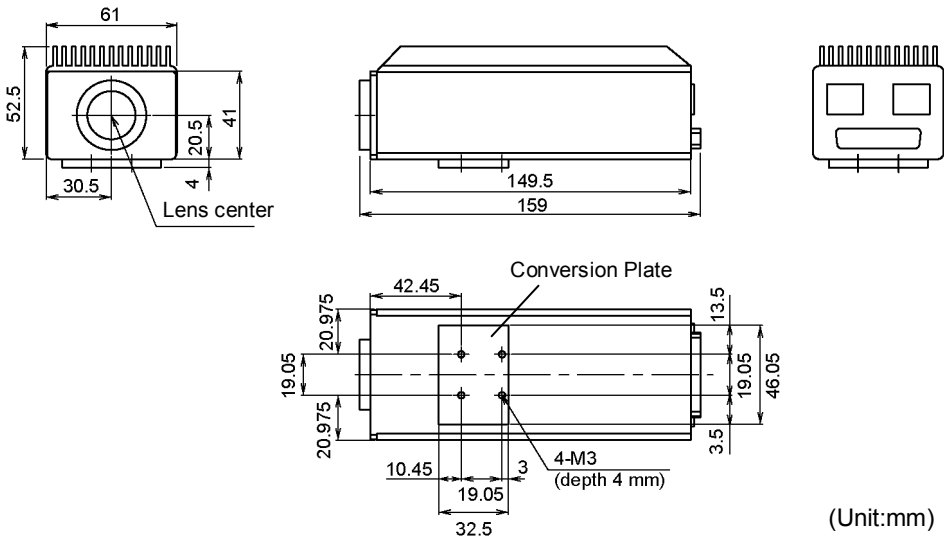
Electrical Specifications

Power Consumption	375 mA @ 24 VDC or 9 W
Strobe Output	Open collector transistor (sink driver) fused @ 100 mA max @ 5 to 24 V DC
Strobe Input	5-24 VDC

Mechanical Specifications

Connectors	RJ45 for Power and RS-232C Diagnostics RJ45 for 10/100 BaseT Ethernet DB-25 for strobe ouput and input
Weight	SC300 / SC1200: 435 g (15.3 oz) + Conversion plate 50 g (1.8 oz) SC300M / SC1200M: Camera 185 g (6.5 oz) + Conversion plate 50 g (1.8 oz) Camera Unit 435 g (15.3 oz) + Conversion plate 50 g (1.8 oz)

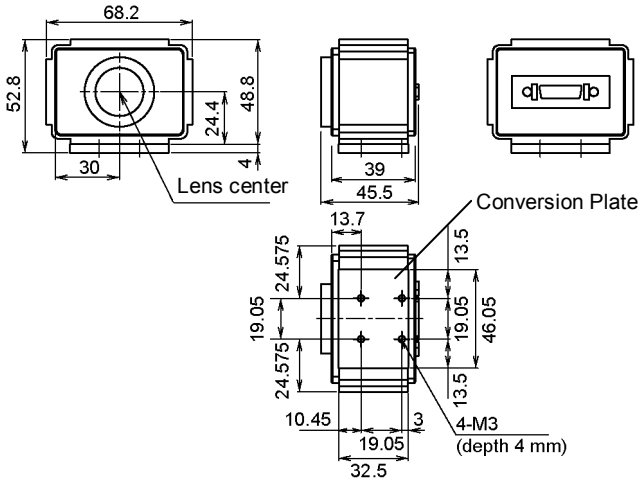
SC300 / SC1200



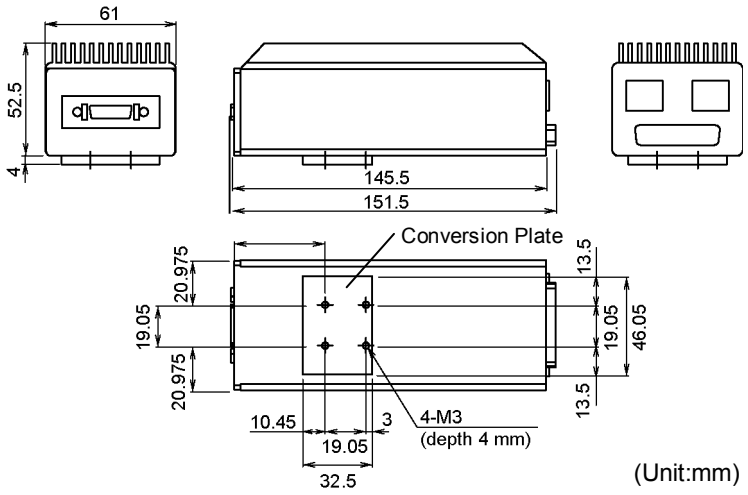
(Unit:mm)

SC300M / SC1200M

Camera



Camera unit



(Unit:mm)

Environmental Specifications

Operating Temperature	0 °C to 45 °C (32 °F to 113 °F)
Ventilation Requirements	Natural convection
Operating Humidity	Up to 95% (non-condensing)

Appendix D: Table of Extension Tubes

Compact Vision

Extension Tube Work Distance

[unit : mm]

		Lens Focal Length				
		f = 8 mm	f = 12 mm	f = 16 mm	f = 25 mm	f = 50 mm
Extension Tubes	0.0 mm	174~1500	258~1500	335~1500	458~1500	907~1500
	0.5 mm	67~134	130~311	207~579	338~1207	775~1500
	1.0 mm	36~55	78~131	139~247	263~607	683~1500
	1.5 mm	23~32	58~84	106~167	219~415	604~1500
	5.0 mm			35~42	102~129	344~502
	10.0 mm			15-16	60~67	224~274
	15.0 mm				43-45	175~198
	20.0 mm					145~160
	40.0 mm					97~101

Extension Tube FOV Table

NS1044BU (Resolution: 640 × 480)

[unit : mm]

		Lens Focal Length				
		f = 8 mm	f = 12 mm	f = 16 mm	f = 25 mm	f = 50 mm
Extension Tubes	0.0 mm	85×64 ~ 736×553	82×62 ~ 468×352	82×62 ~ 358×270	69×52 ~ 229×172	69×52 ~ 116×87
	0.5 mm	38×28 ~ 70×53	45×34 ~ 100×75	51×38 ~ 140×105	51×38 ~ 184×139	59×44 ~ 116×87
	1.0 mm	23×17 ~ 32×24	29×22 ~ 45×34	35×26 ~ 61×46	39×29 ~ 92×69	51×38 ~ 116×87
	1.5 mm	16×12 ~ 21×16	23×17 ~ 31×23	27×20 ~ 42×31	33×25 ~ 63×42	45×34 ~ 116×87
	5.0 mm			11×8 ~ 12×9	15×11 ~ 19×14	24×18 ~ 37×27
	10.0 mm			6×5	8×6 ~ 10×7	14×11 ~ 18×14
	15.0 mm				6×5	10×8 ~ 12×9
	20.0 mm					8×6 ~ 9×7
	40.0 mm					4×3 ~ 5×4

NS4133BU (Resolution: 1280 × 1024)

[unit : mm]

		Lens Focal Length				
		f = 8 mm	f = 12 mm	f = 16 mm	f = 25 mm	f = 50 mm
Extension Tubes	0.0 mm	161×121 ~ 1322×1052	150×114 ~ 828×662	144×109 ~ 632×506	122×92 ~ 406×305	122×97 ~ 206×165
	0.5 mm	68×51 ~ 126×95	80×60 ~ 179×136	91×69 ~ 247×187	90×68 ~ 326×261	103×82 ~ 205×164
	1.0 mm	40×30 ~ 57×42	51×39 ~ 81×61	62×47 ~ 108×82	69×53 ~ 163×123	90×72 ~ 205×164
	1.5 mm	29×21 ~ 37×28	40×30 ~ 54×41	48×37 ~ 74×56	58×44 ~ 111×84	78×60 ~ 205×164
	5.0 mm			19×14 ~ 22×16	26×20 ~ 34×26	42×32 ~ 65×49
	10.0 mm			10×7 ~ 11×8	15×11 ~ 17×13	25×19 ~ 32×25
	15.0 mm				10×8 ~ 11×9	18×14 ~ 22×17
	20.0 mm					14×11 ~ 16×12
	40.0 mm					8×6

The values in these tables are for the camera lens kit of EPSON under LED lights. When the environment is different or using other cameras, values are different. The WD and FOV may change according to the individual variability.

Smart Camera

Extension Tube Work Distance

[unit : mm]

		Lens Focal Length				
		f = 8 mm	f = 12 mm	f = 16 mm	f = 25 mm	f = 50 mm
Extension Tubes *	0.0 mm	259~1500	364~1500	490~1500	544~1500	1030~1500
	0.5 mm	81~195	156~568	232~427	386~926	886~1500
	1.0 mm	47~82	98~168	175~280	294~844	740~1500
	1.5 mm	32~55	72~115	128~182	238~525	647~1500
	5.0 mm			45~57	103~160	345~542
	10.0 mm				61~75	232~288
	15.0 mm				34	178~206
	20.0 mm					147~163
	40.0 mm					99~102

* The C/CS adapter length (5 mm) is not included in the extension tube length.

Extension Tube FOV Table

[unit : mm]

		Lens Focal Length				
		f = 8 mm	f = 12 mm	f = 16 mm	f = 25 mm	f = 50 mm
Extension Tubes *	0.0 mm	123×90 ~ 673×456	110×82 ~ 422×320	110×82 ~ 336×252	66×56 ~ 216×154	73×64 ~ 107×80
	0.5 mm	41×30 ~ 99×69	49×37 ~ 169×123	53×39 ~ 97×72	54×40 ~ 130×95	61×46 ~ 107×80
	1.0 mm	26×19 ~ 42×32	33×25 ~ 52×39	40×30 ~ 64×48	41×30 ~ 119×98	51×28 ~ 107×80
	1.5 mm	19×13 ~ 30×22	23×18 ~ 37×28	30×23 ~ 43×32	33×24 ~ 74×54	44×33 ~ 107×80
	5.0 mm			11×9 ~ 14×11	14×10 ~ 22×16	23×17 ~ 38×28
	10.0 mm			6×5	8×6 ~ 10×7	14×10 ~ 18×13
	15.0 mm				4×3	10×7 ~ 12×9
	20.0 mm					8×5 ~ 9×6
	40.0 mm					4×3 ~ 5×4

The values in this table are for the camera lens kit of EPSON under LED lights. When the environment is different or using other cameras, values are different. The WD and FOV may change according to the individual variability.

* The C/CS adapter length (5 mm) is not included in the extension tube length.

Appendix E: Standard Lens Specifications

Item	Unit	Specification				
Focal length	mm	8	12	16	25	50
Code		R12R500VIS018	R12B120361	R12R500VIS019	R12R500VIS020	R12R500VIS021
Image circle	mm	11φ	8φ	11φ	16φ	11φ
Closest distance*1	m	0.2	0.3	0.4	0.5	1.0
Filter size	mm	M25.5 × P0.5	M27 × P0.5	M27 × P0.5	M27 × P0.5	M30.5 × P0.5
External size	mm	30φ × 34.5	30φ × 34.5	30φ × 24.5	30φ × 24.5	32φ × 37
Weight	g	55	56	40	40	55

P0.5: Screw pitch 0.5 mm

*1: When the extension tube is installed.

Appendix F: Smart Camera RS-232C Diagnostics

If you cannot communicate with your Smart Camera or you do not know the network subnet configured in the camera, you can connect a Windows PC using Hyperterminal to the RS-232C cable that comes with the camera AC adapter and view the boot diagnostics.

NOTE

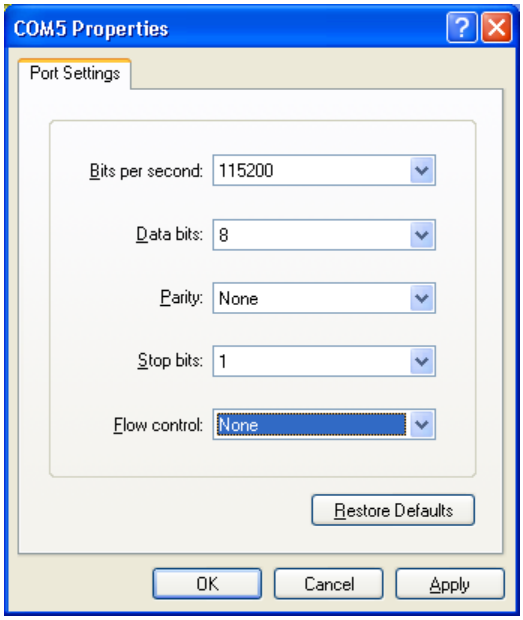
This information applies to the Smart Camera. For an aid in trouble-shooting the Compact Vision system, refer to *10. Using the Compact Vision Monitor*.

(1) Connect an RS-232C cable from an RS-232C port on the PC to the RS-232C power adapter cable.

(2) Open Hyperterminal on the PC. Select Start | All Programs | Accessories | Communications | Hyperterminal.

(3) Create a new connection with any name, such as camera.

(4) Configure the port settings as shown and click OK.



(5) Power the camera off and on. You should see boot diagnostic messages on the Hyperterminal window. If you don't see any information, check your RS-232C cable and com port properties.

(6) After boot is complete, you will see the IP address, IP mask, and IP gateway of the camera.

(7) In some cases, you may be asked to send EPSON a copy of the data displayed in the Hyperterminal window for analysis.